

Clusters

Cluster lifecycle with multicluster engine operator overview

The multicluster engine operator is the cluster lifecycle operator that provides cluster management capabilities for OpenShift Container Platform and Red Hat Advanced Cluster Management hub clusters. From the hub cluster, you can create and manage clusters, as well as destroy any clusters that you created. You can also hibernate, resume, and detach clusters.

The multicluster engine operator is the cluster lifecycle operator that provides cluster management capabilities for OpenShift Container Platform and Red Hat Advanced Cluster Management hub clusters. If you installed Red Hat Advanced Cluster Management, you do not need to install multicluster engine operator, as it is automatically installed. See other important information about Cluster lifecycle with multicluster engine operator:

- Your cluster is created by using the OpenShift Container Platform cluster installer with the Hive resource. See more information about the process of installing OpenShift Container Platform clusters at [Installing and configuring OpenShift Container Platform clusters](#) in the OpenShift Container Platform documentation.
- With your OpenShift Container Platform cluster, you can use multicluster engine operator as a standalone cluster manager for cluster lifecycle function, or you can use it as part of a Red Hat Advanced Cluster Management hub cluster.
- If you are using OpenShift Container Platform only, the operator is included with subscription. Visit [About multicluster engine for Kubernetes operator](#) from the OpenShift Container Platform documentation.
- If you subscribe to Red Hat Advanced Cluster Management, you also receive the operator with installation. You can create, manage, and monitor other Kubernetes clusters with the Red Hat Advanced Cluster Management hub cluster.
- Release images are the version of OpenShift Container Platform that you use when you create a cluster. For clusters that are created using Red Hat Advanced Cluster Management, you can enable automatic upgrading of your release images. For more information about release images in Red Hat Advanced Cluster Management, see *Release images*.
- With hosted control planes for OpenShift Container Platform, you can create control planes as pods on a hosting cluster without the need for dedicated physical machines for each control plane. See the [Hosted control planes overview](#) in the OpenShift Container Platform documentation.
 - [Cluster lifecycle architecture](#)
 - [Release notes for Cluster lifecycle with multicluster engine operator](#)
 - [Installing and upgrading multicluster engine operator](#)
 - [Console overview](#)

- [multicluster engine for Kubernetes operator Role-based access control](#)
- [Network configuration for multicluster engine operator](#)
- [About hosted control planes with multicluster engine operator](#)
- [Managing credentials](#)
- [Cluster lifecycle introduction](#)
- [Release images](#)
- [Discovery service introduction](#)
- [APIs](#)
- [Troubleshooting](#)

Console overview

OpenShift Container Platform console plug-ins are available with the OpenShift Container Platform web console and can be integrated. To use this feature, the console plug-ins must remain enabled. The multicluster engine operator displays certain console features from **Infrastructure** and **Credentials** navigation items. If you install Red Hat Advanced Cluster Management, you see more console capability.

Note: With the plug-ins enabled, you can access Red Hat Advanced Cluster Management within the OpenShift Container Platform console by selecting **Fleet Management** from the perspective selector. For previous OpenShift Container Platform versions, select **All Clusters** from the cluster switcher.

1. To disable the plug-in, be sure you are in the *Administrator* perspective in the OpenShift Container Platform console.
2. Find **Administration** in the navigation and click **Cluster Settings**, then click *Configuration* tab.
3. From the list of *Configuration resources*, click the **Console** resource with the `operator.openshift.io` API group, which contains cluster-wide configuration for the web console.
4. Click on the *Console plug-ins* tab. The `mce` plug-in is listed. **Note:** If Red Hat Advanced Cluster Management is installed, it is also listed as `acm`.
5. Modify plug-in status from the table. In a few moments, you are prompted to refresh the console.

multicluster engine operator role-based access control

RBAC is validated at the console level and at the API level. Actions in the console can be enabled or disabled based on user access role permissions. View the following sections for more information on RBAC for specific lifecycles in the product:

- [Overview of roles](#)
- [Cluster lifecycle RBAC](#)

- [Cluster pools RBAC](#)
- [Console and API RBAC table for cluster lifecycle](#)
- [Credentials role-based access control](#)

Overview of roles

Some product resources are cluster-wide and some are namespace-scoped. You must apply cluster role bindings and namespace role bindings to your users for consistent access controls. View the table list of the following role definitions that are supported:

Table of role definition

Role	Definition
<code>cluster-admin</code>	This is an OpenShift Container Platform default role. A user with cluster binding to the <code>cluster-admin</code> role is an OpenShift Container Platform super user, who has all access.
<code>open-cluster-management:cluster-manager-admin</code>	A user with cluster binding to the <code>open-cluster-management:cluster-manager-admin</code> role is a super user, who has all access. This role allows the user to create a <code>ManagedCluster</code> resource.
<code>open-cluster-management:admin:<managed_cluster_name></code>	A user with cluster binding to the <code>open-cluster-management:admin:<managed_cluster_name></code> role has administrator access to the <code>ManagedCluster</code> resource named, <code><managed_cluster_name></code> . When a user has a managed cluster, this role is automatically created.
<code>open-cluster-management:view:<managed_cluster_name></code>	A user with cluster binding to the <code>open-cluster-management:view:<managed_cluster_name></code> role has view access to the <code>ManagedCluster</code> resource named, <code><managed_cluster_name></code> .

Role	Definition
<code>open-cluster-management:managedclusterset:admin:<managed_clusterset_name></code>	A user with cluster binding to the <code>open-cluster-management:managedclusterset:admin:<managed_clusterset_name></code> role has administrator access to <code>ManagedCluster</code> resource named <code><managed_clusterset_name></code> . The user also has administrator access to <code>managedcluster.cluster.open-cluster-management.io</code> , <code>clusterclaim.hive.openshift.io</code> , <code>clusterdeployment.hive.openshift.io</code> , and <code>clusterpool.hive.openshift.io</code> resources, which has the managed cluster set labels: <code>cluster.open-cluster-management.io</code> and <code>clusterset=<managed_clusterset_name></code> . A role binding is automatically generated when you are using a cluster set. See Creating a ManagedClusterSet to learn how to manage the resource.
<code>open-cluster-management:managedclusterset:view:<managed_clusterset_name></code>	A user with cluster binding to the <code>open-cluster-management:managedclusterset:view:<managed_clusterset_name></code> role has view access to the <code>ManagedCluster</code> resource named, <code><managed_clusterset_name></code> . The user also has view access to <code>managedcluster.cluster.open-cluster-management.io</code> , <code>clusterclaim.hive.openshift.io</code> , <code>clusterdeployment.hive.openshift.io</code> , and <code>clusterpool.hive.openshift.io</code> resources, which has the managed cluster set labels: <code>cluster.open-cluster-management.io</code> , <code>clusterset=<managed_clusterset_name></code> . For more details on how to manage managed cluster set resources, see Creating a ManagedClusterSet .
<code>admin, edit, view</code>	Admin, edit, and view are OpenShift Container Platform default roles. A user with a namespace-scoped binding to these roles has access to <code>open-cluster-management</code> resources in a specific namespace, while cluster-wide binding to the same roles gives access to all of the <code>open-cluster-management</code> resources cluster-wide.

Important:

- Any user can create projects from OpenShift Container Platform, which gives administrator role permissions for the namespace.
- If a user does not have role access to a cluster, the cluster name is not visible. The cluster name is displayed with the following symbol: -.

RBAC is validated at the console level and at the API level. Actions in the console can be enabled or disabled based on user access role permissions. View the following sections for more information on RBAC for specific lifecycles in the product.

Cluster lifecycle RBAC

View the following cluster lifecycle RBAC operations:

- Create and administer cluster role bindings for all managed clusters. For example, create a cluster role binding to the cluster role `open-cluster-management:cluster-manager-admin` by entering the following command:

```
oc create clusterrolebinding <role-binding-name> --clusterrole=open-cluster-management:cluster-manager-admin --user=<username>
```

This role is a super user, which has access to all resources and actions. You can create cluster-scoped `managedcluster` resources, the namespace for the resources that manage the managed cluster, and the resources in the namespace with this role. You might need to add the `username` of the ID that requires the role association to avoid permission errors.

- Run the following command to administer a cluster role binding for a managed cluster named `cluster-name`:

```
oc create clusterrolebinding (role-binding-name) --clusterrole=open-cluster-management:admin:<cluster-name> --user=<username>
```

This role has read and write access to the cluster-scoped `managedcluster` resource. This is needed because the `managedcluster` is a cluster-scoped resource and not a namespace-scoped resource.

- Create a namespace role binding to the cluster role `admin` by entering the following command:

```
oc create rolebinding <role-binding-name> -n <cluster-name> --clusterrole=admin --user=<username>
```

This role has read and write access to the resources in the namespace of the managed cluster.

- Create a cluster role binding for the `open-cluster-management:view:<cluster-name>` cluster role to view a managed cluster named `cluster-name`. Enter the following command:

```
oc create clusterrolebinding <role-binding-name> --clusterrole=open-cluster-management:view:<cluster-name> --user=<username>
```

This role has read access to the cluster-scoped `managedcluster` resource. This is needed because the `managedcluster` is a cluster-scoped resource.

- Create a namespace role binding to the cluster role **view** by entering the following command:

```
oc create rolebinding <role-binding-name> -n <cluster-name> --clusterrole=view
--user=<username>
```

This role has read-only access to the resources in the namespace of the managed cluster.

- View a list of the managed clusters that you can access by entering the following command:

```
oc get managedclusters.clusterview.open-cluster-management.io
```

This command is used by administrators and users without cluster administrator privileges.

- View a list of the managed cluster sets that you can access by entering the following command:

```
oc get managedclustersets.clusterview.open-cluster-management.io
```

This command is used by administrators and users without cluster administrator privileges.

Cluster pools RBAC

View the following cluster pool RBAC operations:

- As a cluster administrator, use cluster pool provision clusters by creating a managed cluster set and grant administrator permission to roles by adding the role to the group. View the following examples:

- Grant **admin** permission to the **server-foundation-cluster-set** managed cluster set with the following command:

```
oc adm policy add-cluster-role-to-group open-cluster-management:clusterset-
admin:server-foundation-cluster-set
server-foundation-team-admin
```

- Grant **view** permission to the **server-foundation-cluster-set** managed cluster set with the following command:

```
oc adm policy add-cluster-role-to-group open-cluster-management:clusterset-
view:server-foundation-cluster-set server-foundation-team-user
```

- Create a namespace for the cluster pool, **server-foundation-clusterpool**. View the following examples to grant role permissions:
 - Grant **admin** permission to **server-foundation-clusterpool** for the **server-foundation-team-admin** by running the following commands:

```
oc adm new-project server-foundation-clusterpool
```

```
oc adm policy add-role-to-group admin server-foundation-team-admin --namespace  
server-foundation-clusterpool
```

- As a team administrator, create a cluster pool named `ocp46-aws-clusterpool` with a cluster set label, `cluster.open-cluster-management.io/clusterset=server-foundation-clusterset` in the cluster pool namespace:
 - The `server-foundation-webhook` checks if the cluster pool has the cluster set label, and if the user has permission to create cluster pools in the cluster set.
 - The `server-foundation-controller` grants `view` permission to the `server-foundation-clusterpool` namespace for `server-foundation-team-user`.
- When a cluster pool is created, the cluster pool creates a `clusterdeployment`. Continue reading for more details:
 - The `server-foundation-controller` grants `admin` permission to the `clusterdeployment` namespace for `server-foundation-team-admin`.
 - The `server-foundation-controller` grants `view` permission `clusterdeployment` namespace for `server-foundation-team-user`.

Note: As a `team-admin` and `team-user`, you have `admin` permission to the `clusterpool`, `clusterdeployment`, and `clusterclaim`.

Console and API RBAC table for cluster lifecycle

View the following console and API RBAC tables for cluster lifecycle:

Table 1. Console RBAC table for cluster lifecycle

Resource	Admin	Edit	View
Clusters	read, update, delete	-	read
Cluster sets	get, update, bind, join	edit role not mentioned	get
Managed clusters	read, update, delete	no edit role mentioned	get
Provider connections	create, read, update, and delete	-	read

Table 2. API RBAC table for cluster lifecycle

API	Admin	Edit	View
managedclusters.cluster.open-cluster-management.io <i>You can use <code>mcl</code> (singular) or <code>mcls</code> (plural) in commands for this API.</i>	create, read, update, delete	read, update	read
managedclusters.view.open-cluster-management.io <i>You can use <code>mcv</code> (singular) or <code>mcvs</code> (plural) in commands for this API.</i>	read	read	read
managedclusters.register.open-cluster-management.io/accept	update	update	
managedclusterset.cluster.open-cluster-management.io <i>You can use <code>mclset</code> (singular) or <code>mclsets</code> (plural) in commands for this API.</i>	create, read, update, delete	read, update	read
managedclustersets.view.open-cluster-management.io	read	read	read
managedclustersetbinding.cluster.open-cluster-management.io <i>You can use <code>mclsetbinding</code> (singular) or <code>mclsetbindings</code> (plural) in commands for this API.</i>	create, read, update, delete	read, update	read
klusterletaddonconfigs.agent.open-cluster-management.io	create, read, update, delete	read, update	read
managedclusteractions.action.open-cluster-management.io	create, read, update, delete	read, update	read

API	Admin	Edit	View
<code>managedclusterviews.view.open-cluster-management.io</code>	create, read, update, delete	read, update	read
<code>managedclusterinfos.internal.open-cluster-management.io</code>	create, read, update, delete	read, update	read
<code>manifestworks.work.open-cluster-management.io</code>	create, read, update, delete	read, update	read
<code>submarinerconfigs.submarineraddn.open-cluster-management.io</code>	create, read, update, delete	read, update	read
<code>placements.cluster.open-cluster-management.io</code>	create, read, update, delete	read, update	read

Credentials role-based access control

The access to credentials is controlled by Kubernetes. Credentials are stored and secured as Kubernetes secrets. The following permissions apply to accessing secrets in Red Hat Advanced Cluster Management for Kubernetes:

- Users with access to create secrets in a namespace can create credentials.
- Users with access to read secrets in a namespace can also view credentials.
- Users with the Kubernetes cluster roles of `admin` and `edit` can create and edit secrets.
- Users with the Kubernetes cluster role of `view` cannot view secrets because reading the contents of secrets enables access to service account credentials.

Network configuration for multicluster engine operator

When you are installing bare metal managed clusters with the Infrastructure Operator, you must configure your network to allow certain connections between the hub cluster and the managed clusters.

Configure your network settings to allow the connections.

Important: The trusted CA bundle is available in the multicluster engine operator namespace, but that enhancement requires changes to your network. The trusted CA bundle ConfigMap uses the default name of `trusted-ca-bundle`. You can change this name by providing it to the operator in an environment variable named `TRUSTED_CA_BUNDLE`. See [Configuring the cluster-wide proxy](#) in the *Networking* section of Red Hat OpenShift Container Platform for more information.

For the multicluster engine operator cluster networking requirements, see the following table:

Table 3. Table list of networking requirements

Direction	Source	Destination	Protocol	Port	Description
Outbound to the managed cluster	Hive and import-controller on the hub cluster	Kubernetes API server of the provisioned managed cluster	HTTPS	6443	The Kubernetes API server of the provisioned managed cluster
Outbound to the managed cluster	Ironic service on the hub cluster	Ironic Python Agent on the managed cluster	TCP	9999	Communication between the bare metal node where the Ironic Python Agent is running and the Ironic conductor service
Inbound from the managed cluster	Baseboard Management Controller (BMC)	HTTP server next to the Ironic service	TCP	6180, 6183	The Baseboard Management Controller (BMC) accesses port 6180 and 6183 for virtual media. The boot firmware accesses port 6180
Inbound from the managed cluster	Ironic Python Agent on the managed cluster	Ironic service on the hub cluster	TCP	6385	Communication between the Ironic Python Agent and the Ironic service on the hub cluster
Inbound from the managed cluster	Cluster proxy add-on agent on the managed cluster	Cluster proxy ANP service	TCP	443	Communication between the cluster proxy add-on agent on the managed cluster and the Cluster Proxy add-on ANP service on the hub cluster

Direction	Source	Destination	Protocol	Port	Description
Inbound from the managed cluster	Klusterlet agent and add-on agents	Kubernetes API server on the hub cluster	HTTPS	6443	The Kubernetes API server of the multicluster engine operator cluster from the managed cluster

Note: If the klusterlet agent on the managed cluster requires proxy settings to access the [apiserver](#) on the hub cluster instead of connecting directly, see [Configuring the proxy between hub cluster and managed cluster](#).

About hosted control planes with multicluster engine operator

Hosted control planes represent a deployment model for OpenShift Container Platform clusters that provides significant infrastructure efficiency. Instead of requiring dedicated virtual or physical machines for each cluster control plane, hosted control planes run as pods in a single namespace on a centralized management cluster.

Standalone

OpenShift Container Platform clusters that use dedicated virtual or physical machines for the control plane components. Each cluster requires individual infrastructure for etcd, API servers, controller managers, and schedulers.

Hosted

A more efficient model where control planes run as pods on a shared management cluster. The hosted model offers the following advantages:

- Reduces infrastructure overhead and costs
- Enables faster cluster provisioning
- Simplifies control plane management and updates
- Allows for better resource utilization

Get started with hosted control plane clusters and multicluster engine operator by following the documentation and guidance.

- See the official OpenShift Container Platform documentation in the [Hosted control planes overview](#) to learn more about hosted control planes.
- Prepare your environment for hosted control planes. See the following documentation:
 - Find the requirements for your environment in [Requirements for hosted control planes](#).
 - Get sizing guidance from [Sizing guidance for hosted control planes](#).

- Learn to create hosted clusters. See [Deploying hosted control planes](#). You can create hosted clusters for the following platforms:
 - [Deploying hosted control planes on AWS](#)
 - [Deploying hosted control planes on bare metal](#)
 - [Deploying hosted control planes on OpenShift Virtualization](#)
- You can manage hosted clusters with multicluster engine operator. See the following documentation:
 - **Optional:** Manually import a hosted cluster. See [Manually importing a hosted cluster](#).
 - Manage hosted clusters. See [Managing hosted control planes](#).
 - Learn about hosted control plane authentication, see [Authentication and authorization for hosted control planes](#).
 - Learn about updating hosted clusters from the [Updating hosted control planes](#) documentation.
 - See [Destroying a hosted cluster](#) when you are ready.
- With Red Hat Advanced Cluster Management enabled, you can discover and manage hosted control plane clusters, while using Red Hat Advanced Cluster Management capability. Learn about hosted control planes with Red Hat Advanced Cluster Management from the following resources, which require Red Hat Advanced Cluster Management:
 - Learn about the integrated Red Hat Advanced Cluster Management and multicluster engine operator hosted control plane fleet management architecture pattern. See [Integrated hosted control plane fleet management architecture pattern](#).
 - Discover hosted control plane clusters. See [Discovering hosted control plane clusters with Red Hat Advanced Cluster Management](#).
 - Set up disaster recovery for hosted control plane clusters with Red Hat Advanced Cluster Management. See [Backup and restore for hosted control planes and hosted clusters](#).

Additional resources

- See the release notes for hosted control planes at [Hosted control planes release notes](#).
- See the Troubleshooting guide at [Troubleshooting hosted control planes](#).

Release notes for Cluster lifecycle with multicluster engine operator

Learn about new features and enhancements, support, deprecations, removals, and fixed issues for Cluster lifecycle with multicluster engine operator version 2.11.

- [New features and enhancements for Cluster lifecycle with multicluster engine operator](#)
- [Fixed issues for Cluster lifecycle with multicluster engine operator](#)
- [Known issues for Cluster lifecycle with multicluster engine operator](#)
- [Deprecations and removals for Cluster lifecycle with multicluster engine operator](#)

Important: OpenShift Container Platform release notes are *not* documented in this product documentation. For your OpenShift Container Platform cluster, see [OpenShift Container Platform release notes](#).

Deprecated: multicluster engine operator 2.4 and earlier versions are no longer supported. The documentation might remain available, but without any errata releases for fixed issues or other updates.

Best practice: Upgrade to the most recent version.

- The documentation references the earliest supported OpenShift Container Platform version, unless a specific component or function is introduced and tested only on a more recent version of OpenShift Container Platform.
- For full support information, see the [multicluster engine operator Support Matrix](#). For lifecycle information, see [Red Hat OpenShift Container Platform Life Cycle policy](#).
- If you experience issues with one of the currently supported releases, or the product documentation, go to [Red Hat Support](#) where you can troubleshoot, view *Knowledgebase* articles, connect with the Support Team, or open a case. You must log in with your credentials.
- You can also learn more about the Customer Portal documentation at [Red Hat Customer Portal FAQ](#).

New features and enhancements for Cluster lifecycle with multicluster engine operator

Learn about multicluster engine for Kubernetes operator 2.11 new features and enhancements, support, deprecations, removals, and fixed issues for errata releases.

multicluster engine operator version 2.11 is released with Red Hat Advanced Cluster Management version 2.16 for Cluster lifecycle management, but you can use stand-alone multicluster engine operator with your OpenShift Container Platform hub cluster, as well.

For full support information, including supported OpenShift Container Platform versions, see the [multicluster engine operator Support Matrix](#). For lifecycle information, see [Red Hat OpenShift Container Platform Life Cycle policy](#).

Use multicluster engine operator features for creating, importing, managing, and destroying Kubernetes clusters across various infrastructure cloud providers, private clouds, and on-premises data centers.

Important:

- Cluster management supports all providers that are certified through the Cloud Native Computing Foundation (CNCF) Kubernetes Conformance Program. Choose a vendor that is recognized by CNCF for your hybrid cloud multicluster management. See the following information about using CNCF providers:
- Learn how CNCF providers are certified at [Certified Kubernetes Conformance](#).
- For Red Hat support information about CNCF third-party providers, see [Red Hat support with third party components](#), or [Contact Red Hat support](#).

- If you bring your own CNCF conformance certified cluster, you need to change the OpenShift Container Platform CLI `oc` command to the Kubernetes CLI command, `kubectl`.

New features and enhancements for each component

Learn more about new features for specific components.

Note: Some features and components are identified and released as [Technology Preview](#).

Learn about new features and enhancements for Cluster lifecycle with multicluster engine operator:

- You can now create an Microsoft Azure Red Hat OpenShift hosted control plane managed cluster by using Cluster API Provider for Azure to reduce costs and setup time. See [Preparing to create Microsoft Azure Red Hat OpenShift hosted control plane clusters by using Cluster API Provider for Azure](#) to learn more.

If you are using multicluster engine operator with Red Hat Advanced Cluster Management installed afterwards, see the [New features and enhancements for Red Hat Advanced Cluster Management](#) this release.

Fixed issues for Cluster lifecycle with multicluster engine operator

By default, fixed issues for *errata* releases are automatically applied when released. The details are published here when the release is available. If no release notes are listed, the product does not have an errata release at this time.

Important: For reference, Jira links and Jira numbers might be added to the content and used internally. Links that require access might not be available for the user. Learn about the types of [errata releases](#) from Red Hat.

Known issues for Cluster lifecycle with multicluster engine operator

Review the Known issues for *Cluster lifecycle with multicluster engine operator* for this release, or known issues that continued from the previous release.

Cluster management known issues and limitations are part of the *Cluster lifecycle with multicluster engine operator* documentation. Known issues for multicluster engine operator integrated with Red Hat Advanced Cluster Management are documented in the [Release notes for Red Hat Advanced Cluster Management](#).

Important: OpenShift Container Platform release notes are not documented in this product documentation. For your OpenShift Container Platform cluster, see [OpenShift Container Platform release notes](#).

- [Installation](#)
- [Cluster management](#)
- [Central infrastructure management](#)

Installation

Learn about known issues and limitations during multicluster engine operator installation.

Unclaimed Hive clusters might be recreated during ClusterPool custom resource definition changes

During a multicluster engine operator upgrade or modification to the `ClusterPool` custom resource definition, Hive might delete and re-create all unclaimed clusters. This is expected behavior and ensures that unclaimed clusters reflect the latest configuration defined in the updated `ClusterPool` resource specification.

[ACM-21686](#)

`installNamespace` field can only have one value

Note: The `spec.installNamespace` field is deprecated. See [Product deprecations and removals](#) to learn more.

When enabling the `managed-serviceaccount` add-on, the `installNamespace` field in the `ManagedClusterAddOn` resource must have `open-cluster-management-agent-addon` as the value. Other values are ignored. The `managed-serviceaccount` add-on agent is always deployed in the `open-cluster-management-agent-addon` namespace on the managed cluster.

[ACM-7523](#)

Cluster management

Learn about Known issues for Cluster lifecycle with multicluster engine operator, such as issues with creating, discovering, importing, and removing clusters, and more cluster management issues for multicluster engine operator.

Cluster API Provider Metal3 certificate generation fails after upgrade

After you upgrade from multicluster engine for Kubernetes operator version 2.10 or 2.10.1 to version 2.11, the automatic generation of certificates for the Cluster API Provider Metal3 webhooks fail due to a naming conflict in the target resources. As a result, webhooks do not validate.

To resolve the issue, manually delete the existing secret and the renamed service. The operator then regenerates the correct secret.

Complete the following steps on your managed cluster:

1. Delete the webhook service. Run the following command:

```
oc delete service capm3-webhook-service -n multicluster-engine
```

2. Delete the Transport Layer Security (TLS) secret. Run the following command:

```
oc delete secret capm3-webhook-service-cert -n multicluster-engine
```

3. Clear the error annotations. Run the following command:

```
oc annotate service mce-capm3-webhook-service -n multicluster-engine \
service.alpha.openshift.io/serving-cert-generation-error- \
service.alpha.openshift.io/serving-cert-generation-error-num- \
service.beta.openshift.io/serving-cert-generation-error- \
service.beta.openshift.io/serving-cert-generation-error-num-
```

4. Delete the stale webhook configurations. Run the following commands:

```
oc delete mutatingwebhookconfiguration capm3-mutating-webhook-configuration
```

```
oc delete validatingwebhookconfiguration capm3-validating-webhook-configuration
```

5. Delete the new TLS secret. Run the following command:

```
oc delete secret mce-capm3-webhook-service-cert -n multicluster-engine
```

6. Restart the pod to regenerate the certificate. Run the following command:

```
oc delete pod mce-capm3-controller-manager-<hash> -n multicluster-engine
```

7. Delete the old **controller-manager** deployment. Run the following command:

```
oc delete deployment capm3-controller-manager -n multicluster-engine
```

MGMT-23257

Cluster API and Cluster API Provider for Azure deployments not creating pods

When you deploy **cluster-api** and **cluster-api-provider-azure** components, the Cluster API controllers might not create pods.

To work around the issue, complete the following steps:

1. Verify that the **cluster-api** component is enabled. Run the following command:

```
oc get mce multiclusterengine -o json | \
jq -r '.spec.overrides.components[] | select(.name | contains("cluster-api"))'
```

2. Check if the **hypershift** component is causing a conflict. Run the following command to check if the **READY** status is **0/1**:


```
oc get deployment -n hypershift
```

3. If the status is **0/1**, force multicluster engine operator reconciliation. Run the following command:

```
oc annotate mce multiclusterengine force-reconcile=$(date +%s) --overwrite
```

ACM-30244

HyperShift conflict when creating Microsoft Azure Red Hat OpenShift hosted control plane clusters by using Cluster API Provider for Azure

You might not be able to create Azure Red Hat OpenShift hosted control plane clusters by using Cluster API Provider for Azure if the **hypershift** and **cluster-api** controllers are running at the same time.

To resolve the issue, complete the following steps:

1. Disable the **hypershift** component. Run the following command:

```
oc patch mce multiclusterengine --type=merge -p
"{\"spec\":{\"overrides\":{\"components\":{\"(oc get mce multiclusterengine -o json |
jq -c '.spec.overrides.components | map(if .name == \"hypershift\" then .enabled =
false else . end)')}}}"
```

2. Disable the **hypershift-local-hosting** component. Run the following command:

```
oc patch mce multiclusterengine --type=merge -p
"{\"spec\":{\"overrides\":{\"components\":{\"(oc get mce multiclusterengine -o json |
jq -c '.spec.overrides.components | map(if .name == \"hypershift-local-hosting\" then
.enabled = false else . end)')}}}"
```

ACM-30244

Microsoft Azure authentication fails when creating Microsoft Azure Red Hat OpenShift hosted control plane clusters by using Cluster API Provider for Azure

When you create Azure Red Hat OpenShift hosted control plane clusters by using Cluster API Provider for Azure, the cluster creation might fail because of Microsoft Azure authentication errors.

To work around the issue, ensure you are connected to Microsoft Azure with your credentials. Complete the following steps:

1. Verify your Microsoft Azure credentials secret. Run the following command. Replace **<cluster-namespace>** with the namespace of your cluster:

```
oc get secret aso-credential -n <cluster-namespace> -o yaml
```

2. Test the Microsoft Azure connection. Run the following command:

```
oc run azure-cli --rm -it --image=mcr.microsoft.com/azure-cli -- bash
```

3. Log in to Microsoft Azure inside the pod. Run the following command. Replace `<client_id>`, `<client_secret>`, and `<tenant_id>` with your Microsoft Azure credentials:

```
az login --service-principal --username <client_id> --password <client_secret> --tenant <tenant_id>
```

[ACM-30244](#)

Resource deletion during cluster distribution updates

When a change occurs in the cluster because of the `Placement` resource, cluster redistribution occurs across multiple `PlacementDecisions` resources. During the sequential update, a cluster is temporarily missing from all `PlacementDecisions` resources.

Controllers that monitor `PlacementDecisions` resources identify this temporary deletion as cluster removal. This sequence leads to unnecessary resource deletion and recreation. For example, the `addon-management-controller` deletes and re-creates `ManagedClusterAddOns`, which cause workload disruption.

[ACM-27799](#)

HyperShift operator and Cluster API components conflict

Red Hat Advanced Cluster Management and multicluster engine operator enable the `hypershift` and `hypershift-local-hosting` components by default.

To manage Red Hat OpenShift Service on AWS (classic) clusters by using the `cluster-api` and `cluster-api-provider-aws` components, you must disable the `hypershift` and `hypershift-local-hosting` components to prevent API conflicts.

When you enable and then disable the `cluster-api` and `cluster-api-provider-aws` components, custom resource definitions of a higher version might remain in the cluster.

You must manually remove the custom resource definitions before you re-enable the `hypershift-local-hosting` component. Use the following command to remove the custom resource definitions:

```
oc delete crd <crd-name>
```

Remove the following custom resource definitions:

- `clusterresourcesetbindings.addons.cluster.x-k8s.io`
- `clusterresourcesets.addons.cluster.x-k8s.io`
- `clusters.cluster.x-k8s.io`
- `clusterclasses.cluster.x-k8s.io`
- `machines.cluster.x-k8s.io`
- `machinesets.cluster.x-k8s.io`
- `machinedeployments.cluster.x-k8s.io`
- `machinepools.cluster.x-k8s.io`
- `machinehealthchecks.cluster.x-k8s.io`
- `machinedrainrules.cluster.x-k8s.io`

HyperShift operator does not receive infrastructure node configuration

When Red Hat Advanced Cluster Management or multicluster engine operator is installed with infrastructure node selection configured, by using `nodeSelector` and `tolerations` for infrastructure node, the HyperShift operator deployment does not receive the settings.

Even if the `multicluster engine` subscription is configured to place pods on infrastructure nodes by using `nodeSelector` and `tolerations`, the HyperShift operator pods are scheduled on regular worker nodes instead of the designated infrastructure nodes.

The HyperShift operator deployment does not automatically propagate the infrastructure node placement configuration from the subscription.

After you install the HyperShift add-on, manually patch the HyperShift operator deployment to add the infrastructure node selector and tolerations. See the following example:

```
nodeSelector:
  node-role.kubernetes.io/infra: ""
tolerations:
- key: node-role.kubernetes.io/infra
  operator: Exists
  effect: NoSchedule
```

To resolve the problem, apply a patch to the operator deployment in the `hypershift` namespace by running the following command with the specifications for your infrastructure nodes:

```
oc patch deployment operator -n hypershift --type=merge -p
'{"spec":{"template":{"spec":{"nodeSelector":{"node-
role.kubernetes.io/infra":""},"tolerations":[{"key":"node-
role.kubernetes.io/infra","operator":"Exists","effect":"NoSchedule"}]}}}}'
```

The manual patch ensures that the operator pods are placed on the correct nodes to comply with your infrastructure requirements.

HyperShift installation fails on Red Hat OpenShift Service on AWS (classic) due to admission webhook error

When you install multicluster engine for Kubernetes operator 2.10 on Red Hat OpenShift Service on AWS (classic), the HyperShift installation job might fail repeatedly with an admission webhook error similar to the following message: `namespace-validation.managed.openshift.io denied the request`. See the following explanations for the error:

- The installation process attempts to create a `hypershift` namespace that includes the `openshift.io/cluster-monitoring: "true"` label.
- The Red Hat OpenShift Service on AWS (classic) managed admission webhooks prevent users from modifying protected labels on namespaces, including the `cluster-monitoring` label.
- The platform monitoring flag defaults to monitoring the operator, which adds the `cluster-monitoring` label to the `hypershift` namespace.
- In managed environments, such as Red Hat OpenShift Service on AWS (classic), the service provider typically handles platform-level monitoring. Therefore, this monitoring is not required and conflicts with namespace protection policies.

To resolve this issue, create a `ConfigMap` in the `local-cluster` namespace to disable platform monitoring by following these steps:

1. Create a YAML file for the `ConfigMap` with the following content:

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: hypershift-operator-install-flags
  namespace: local-cluster
data:
  installFlagsToAdd: "--platform-monitoring=None"
  installFlagsToRemove: ""
```

2. Apply the `ConfigMap` to your cluster. Run the following command, replacing `<filename>` with the name of your YAML file:

```
oc apply -f <filename>.yaml
```

This configuration instructs the HyperShift installer to skip the `cluster-monitoring` label, which avoids the admission webhook conflict.

ClusterCurator status becomes stuck when upgrading the hub cluster

When you use the `ClusterCurator` resource to upgrade the hub cluster, the `ClusterCurator` status becomes stuck, even if the upgrade is successful. During upgrade, the hub cluster reboots, which

stops the `Job` resource pod monitoring.

To work around the issue, manually upgrade the hub cluster. If you already used the cluster curator to upgrade the hub cluster, you must delete the `ClusterCurator` resource hub cluster namespace so that future upgrades appear in the console.

ACM-26863

Kubernetes add-on does not tolerate `node-role.kubernetes.io/infra` taint with `NoExecute` effect

If you use the `node-role.kubernetes.io/infra` taint with the `NoExecute` effect on your infrastructure nodes, your add-on pods might become stuck.

To work around the issue, either use the `NoSchedule` effect instead of `NoExecute` for the `node-role.kubernetes.io/infra` taint, or remove the `node-role.kubernetes.io/infra` taint from your infrastructure nodes.

ACM-15906

Limitation with `nmstate`

Develop quicker by configuring copy and paste features. To configure the `copy-from-mac` feature in the `assisted-installer`, you must add the `mac-address` to the `nmstate` definition interface and the `mac-mapping` interface. The `mac-mapping` interface is provided outside the `nmstate` definition interface. As a result, you must provide the same `mac-address` twice.

ACM-9128

Deleting a managed cluster set does not automatically remove its label

After you delete a `ManagedClusterSet`, the label that is added to each managed cluster that associates the cluster to the cluster set is not automatically removed. Manually remove the label from each of the managed clusters that were included in the deleted managed cluster set. The label resembles the following example: `cluster.open-cluster-management.io/clusterSet:<ManagedClusterSet Name>`.

GH-20727

ClusterClaim error

If you create a Hive `ClusterClaim` against a `ClusterPool` and manually set the `ClusterClaimSpec` lifetime field to an invalid go lang time value, the product stops fulfilling and reconciling all `ClusterClaims`, not just the malformed claim.

You see the following error in the `clusterclaim-controller` pod logs, which is a specific example with the `PoolName` and invalid `Lifetime` included:

```
E0203 07:10:38.266841      1 reflector.go:138] sigs.k8s.io/controller-
runtime/pkg/cache/internal/informers_map.go:224: Failed to watch *v1.ClusterClaim:
failed to list *v1.ClusterClaim: v1.ClusterClaimList.Items: [v1.ClusterClaim:
v1.ClusterClaim.v1.ClusterClaim.Spec: v1.ClusterClaimSpec.Lifetime:
unmarshalerDecoder: time: unknown unit "w" in duration "1w", error found in #10 byte
```

```
of ...|time":"1w"}},{"apiVe|..., bigger context ...|clusterPoolName":"policy-aas-  
hubs","lifetime":"1w"}},{"apiVersion":"hive.openshift.io/v1","kind":"Cl|...
```

You can delete the invalid claim.

If the malformed claim is deleted, claims begin successfully reconciling again without any further interaction.

[GH-19968](#)

The product channel out of sync with provisioned cluster

The `clusterimageset` is in `fast` channel, but the provisioned cluster is in `stable` channel. Currently the product does not sync the `channel` to the provisioned OpenShift Container Platform cluster.

Change to the right channel in the OpenShift Container Platform console. Click **Administration > Cluster Settings > Details Channel**.

[GH-18380](#)

Cluster provision with Ansible automation fails in proxy environment

An Automation template that is configured to automatically provision a managed cluster might fail when both of the following conditions are met:

- The hub cluster has cluster-wide proxy enabled.
- The Ansible Automation Platform can only be reached through the proxy.

[GH-17659](#)

Cannot delete managed cluster namespace manually

You cannot delete the namespace of a managed cluster manually. The managed cluster namespace is automatically deleted after the managed cluster is detached. If you delete the managed cluster namespace manually before the managed cluster is detached, the managed cluster shows a continuous terminating status after you delete the managed cluster. To delete this terminating managed cluster, manually remove the finalizers from the managed cluster that you detached.

[GH-13474](#)

Automatic secret updates for provisioned clusters is not supported

When you change your cloud provider access key on the cloud provider side, you also need to update the corresponding credential for this cloud provider on the console of multicluster engine operator. This is required when your credentials expire on the cloud provider where the managed cluster is hosted and you try to delete the managed cluster.

[GH-3706](#)

Process to destroy a cluster does not complete

When you destroy a managed cluster, the status continues to display **Destroying** after one hour, and the cluster is not destroyed. To resolve this issue complete the following steps:

1. Manually ensure that there are no orphaned resources on your cloud, and that all of the provider resources that are associated with the managed cluster are cleaned up.
2. Open the **ClusterDeployment** information for the managed cluster that is being removed by entering the following command:

```
oc edit clusterdeployment/<mycluster> -n <namespace>
```

Replace **mycluster** with the name of the managed cluster that you are destroying.

Replace **namespace** with the namespace of the managed cluster.

3. Remove the **hive.openshift.io/deprovision** finalizer to forcefully stop the process that is trying to clean up the cluster resources in the cloud.
4. Save your changes and verify that **ClusterDeployment** is gone.
5. Manually remove the namespace of the managed cluster by running the following command:

```
oc delete ns <namespace>
```

Replace **namespace** with the namespace of the managed cluster.

GH-4748

Cannot upgrade OpenShift Container Platform managed clusters on OpenShift Container Platform Dedicated with the console

You cannot use the Red Hat Advanced Cluster Management console to upgrade OpenShift Container Platform managed clusters that are in the OpenShift Container Platform Dedicated environment.

GH-8922

Non-OpenShift Container Platform managed clusters require *ManagedServiceAccount* or *LoadBalancer* for pod logs

The **ManagedServiceAccount** and cluster proxy add-ons are enabled by default in Red Hat Advanced Cluster Management version 2.10 and newer. If the add-ons are disabled after upgrading, you must enable the **ManagedServiceAccount** and cluster proxy add-ons manually to use the pod log feature.

See [ManagedServiceAccount add-on](#) to learn how to enable **ManagedServiceAccount** and see [Configuring cluster proxy add-ons](#) to learn how to enable a cluster proxy add-on.

- To learn how to enable **ManagedServiceAccount** add-on, see [ManagedServiceAccount add-on](#).
- To learn how to enable a cluster proxy add-on, see [Using cluster proxy add-ons](#).

Client cannot reach iPXE script

iPXE is an open source network boot firmware. See [iPXE](#) for more details.

When booting a node, the URL length limitation in some DHCP servers cuts off the `ipxeScript` URL in the `InfraEnv` custom resource definition, resulting in the following error message in the console:

`no bootable devices`

To work around the issue, complete the following steps:

1. Apply the `InfraEnv` custom resource definition when using an assisted installation to expose the `bootArtifacts`, which might resemble the following file:

```
status:
  agentLabelSelector:
    matchLabels:
      infraenvs.agent-install.openshift.io: qe2
  bootArtifacts:
    initrd: https://assisted-image-service-multicloud-engine.redhat.com/images/0000/pxe-initrd?api_key=00000000&arch=x86_64&version=4.11
    ipxeScript: https://assisted-service-multicloud-engine.redhat.com/api/assisted-install/v2/infra-envs/000000/downloads/files?api_key=0000000000&file_name=ipxe-script
    kernel: https://mirror.openshift.com/pub/openshift-v4/x86_64/dependencies/rhcos/4.12/latest/rhcos-live-kernel-x86_64
    rootfs: https://mirror.openshift.com/pub/openshift-v4/x86_64/dependencies/rhcos/4.12/latest/rhcos-live-rootfs.x86_64.img
```

1. Create a proxy server to expose the `bootArtifacts` with short URLs.
2. Copy the `bootArtifacts` and add them to the proxy by running the following commands:

```
for artifact in oc get infraenv qe2 -ojsonpath="{.status.bootArtifacts}" | jq ". | keys[]" | sed "s/\"//g"
do curl -k oc get infraenv qe2 -ojsonpath="{.status.bootArtifacts.${artifact}}" -o $artifact
```

3. Add the `ipxeScript` artifact proxy URL to the `bootp` parameter in `libvirt.xml`.

ManagedClusterSet API specification limitation

The `selectorType: LabelSelector` setting is not supported when using the [Clustersets API](#). The `selectorType: ExclusiveClusterSetLabel` setting is supported.

The Cluster curator does not support OpenShift Container Platform Dedicated clusters

When you upgrade an OpenShift Container Platform Dedicated cluster by using the `ClusterCurator` resource, the upgrade fails because the Cluster curator does not support OpenShift Container Platform Dedicated clusters.

[ACM-10154](#)

Custom ingress domain is not applied correctly

You can specify a custom ingress domain by using the `ClusterDeployment` resource while installing a managed cluster, but the change is only applied after the installation by using the `SyncSet` resource. As a result, the `spec` field in the `clusterdeployment.yaml` file displays the custom ingress domain you specified, but the `status` still displays the default domain.

[ACM-6279](#)

tolerations and nodeSelector settings do not affect the managed-serviceaccount agent

The `tolerations` and `nodeSelector` settings configured on the `MultiClusterEngine` and `MultiClusterHub` resources do not affect the `managed-serviceaccount` agent deployed on the local cluster. The `managed-serviceaccount` add-on is not always required on the local cluster.

If the `managed-serviceaccount` add-on is required, you can work around the issue by completing the following steps:

1. Create the `addonDeploymentConfig` custom resource.
2. Set the `tolerations` and `nodeSelector` values for the local cluster and `managed-serviceaccount` agent.
3. Update the `managed-serviceaccount ManagedClusterAddon` in the local cluster namespace to use the `addonDeploymentConfig` custom resource you created.

To learn more about how to use the `addonDeploymentConfig` custom resource to configure `tolerations` and `nodeSelector` for add-on, see [Configuring klusterlet add-ons](#).

[ACM-7523](#)

Central infrastructure management

Learn about known issues and limitations for central infrastructure management with multicluster engine operator.

A single-node OpenShift cluster installation requires a matching OpenShift Container Platform with infrastructure operator for Red Hat OpenShift

If you want to install a single-node OpenShift cluster with an Red Hat OpenShift Container Platform version before 4.19, your `InfraEnv` custom resource and your booted host must use the same OpenShift Container Platform version that you are using to install the single-node OpenShift cluster. The installation fails if the versions do not match.

To work around the issue, edit your `InfraEnv` resource before you boot a host with the Discovery

ISO, and include the following content. Replace `<4.x>` with the `osImageVersion` that you want to install:

```
apiVersion: agent-install.openshift.io/v1beta1
kind: InfraEnv
spec:
  osImageVersion: "<4.x>"
```

ACM-14943

Cannot use host inventory to boot with the discovery image and add hosts automatically

You cannot use a host inventory, or `InfraEnv` custom resource, to both boot with the discovery image and add hosts automatically. If you used your previous `InfraEnv` resource for the `BareMetalHost` resource, and you want to boot the image yourself, you can work around the issue by creating a new `InfraEnv` resource.

ACM-14719

Nodes shut down after removing `BareMetalHost` resource

If you remove the `BareMetalHost` resource from a hub cluster, the nodes shut down. You can manually power on the nodes again.

ACM-15319

Deprecations and removals for Cluster lifecycle with multicluster engine operator

Learn when parts of the product are deprecated or removed from multicluster engine operator. Consider the alternative actions in the *Recommended action* and details, which display in the tables for the current release and for two prior releases. Tables are removed if no entries are added for that section this release.

Deprecated: multicluster engine operator 2.5 and earlier versions are no longer supported. The documentation might remain available, but without any errata releases for fixed issues or other updates.

Best practice: Upgrade to the most recent version.

Product deprecations and removals

A *deprecated* component, feature, or service is supported, but no longer recommended for use and might become obsolete in future releases. Consider the alternative actions in the *Recommended action* and details that are provided in the following table:

Table 4. Table list of deprecated items for multicluster engine operator

Product or category	Affected item	Version	Recommended action	More details and links
ManagedClusterAddon	The <code>installNamespace</code> field is deprecated in the <code>ManagedClusterAddon</code> spec.	2.10	Use the <code>AddonDeploymentConfig</code> field.	None
Documentation for APIs	The multicluster engine operator <i>API documentation</i>	2.8	View current and supported APIs from the console or the terminal instead of the documentation	None
KlusterletConfig	The <code>hubKubeAPIServerProxyConfig</code> field is deprecated in the <code>KlusterletConfig</code> spec.	2.7	Use the <code>hubKubeAPIServerConfig.proxyURL</code> and <code>hubKubeAPIServerConfig.trustedCABundles</code> fields.	None
KlusterletConfig	The <code>hubKubeAPIServerURL</code> field is deprecated in the <code>KlusterletConfig</code> spec.	2.7	Use the <code>hubKubeAPIServerConfig.url</code> field.	None
KlusterletConfig	The <code>hubKubeAPIServerCABundle</code> field is deprecated in the <code>KlusterletConfig</code> spec	2.7	Use the <code>hubKubeAPIServerConfig.serverVerificationStrategy</code> and <code>hubKubeAPIServerConfig.trustedCABundles</code> fields.	None

A *removed* item is typically function that was deprecated in previous releases and is no longer available in the product. You must use alternatives for the removed function. Consider the alternative actions in the *Recommended action* and details that are provided in the following table:

Table 5. Table list of removed items for multicluster engine operator

Product or category	Affected item	Version	Recommended action	More details and links
Cluster lifecycle	The <code>autoImportRetry</code> field in <code>auto-import-secret.yaml</code>	2.9	None	None

Installing and upgrading multicluster engine operator

The multicluster engine operator is a software operator for managing clusters across clouds and data centers. It supports both OpenShift Container Platform and Kubernetes cluster lifecycle management.

For full support information, see the [multicluster engine operator Support Matrix](#). For life cycle information, see [Red Hat OpenShift Container Platform Life Cycle policy](#).

Important: If you are using Red Hat Advanced Cluster Management, then multicluster engine for Kubernetes operator is already installed on the cluster.

Deprecated: multicluster engine operator 2.4 and earlier versions are no longer supported. The documentation might remain available, but without any errata releases for fixed issues or other updates.

Best practice: Upgrade to the most recent version.

See the following documentation:

- [Installing while connected online](#)
- [Configuring infrastructure nodes for multicluster engine operator](#)
- [Installing on disconnected networks](#)
- [Uninstalling](#)
- [Network configuration](#)
- [Upgrading multicluster engine operator](#)
- [Upgrading disconnected clusters using policies](#)
- [MultiClusterEngine advanced configuration](#)
- [multicluster engine operator with Red Hat Advanced Cluster Management integration](#)

Installing while connected online

The multicluster engine operator is installed with Operator Lifecycle Manager, which manages the installation, upgrade, and removal of the components that encompass the multicluster engine operator.

Required access: Cluster administrator

Important:

- You cannot install multicluster engine operator on a cluster that has a `ManagedCluster` resource configured in an external cluster. You must remove the `ManagedCluster` resource from the external cluster before you can install multicluster engine operator.
- For OpenShift Container Platform Dedicated environment, you must have `cluster-admin` permissions. By default `dedicated-admin` role does not have the required permissions to create namespaces in the OpenShift Container Platform Dedicated environment.

- By default, the multicluster engine operator components are installed on worker nodes of your OpenShift Container Platform cluster without any additional configuration. You can install multicluster engine operator onto worker nodes by using the Red Hat OpenShift Software Catalog console, or by using the OpenShift Container Platform CLI.
- If you configured your OpenShift Container Platform cluster with infrastructure nodes, you can install multicluster engine operator onto those infrastructure nodes by using the OpenShift Container Platform CLI with additional resource parameters. See the *Installing multicluster engine on infrastructure nodes* section for those details.
- If you plan to import Kubernetes clusters that were not created by OpenShift Container Platform or multicluster engine operator, you need to configure an image pull secret. For information about how to configure an image pull secret and other advanced configurations, see options in the [Advanced configuration](#) section of this documentation.
 - [Prerequisites](#)
 - [Confirm your OpenShift Container Platform installation](#)
 - [Installing from the Red Hat OpenShift Software Catalog console](#)
 - [Installing from the OpenShift Container Platform CLI](#)

Prerequisites

Before you install multicluster engine operator, see the following prerequisites:

- Your OpenShift Container Platform cluster must have access to the multicluster engine operator in the Red Hat OpenShift Software Catalog console.
- You need access to the catalog.redhat.com.
- Your cluster does *not* have a **ManagedCluster** resource configured in an external cluster.
- A supported version of OpenShift Container Platform must be deployed in your environment, and you must be logged into with the OpenShift Container Platform CLI. See the following install documentation:
 - [OpenShift Container Platform Installing](#)
- Your OpenShift Container Platform command line interface (CLI) must be configured to run **oc** commands. See [Getting started with the CLI](#) for information about installing and configuring the OpenShift Container Platform CLI.
- Your OpenShift Container Platform permissions must allow you to create a namespace.
- You must have an Internet connection to access the dependencies for the operator.
- To install in a OpenShift Container Platform Dedicated environment, see the following:
 - You must have the OpenShift Container Platform Dedicated environment configured and running.
 - You must have **cluster-admin** authority to the OpenShift Container Platform Dedicated environment where you are installing the engine.
- If you plan to create managed clusters by using the Assisted Installer that is provided with Red Hat OpenShift Container Platform, see [Preparing to install with the Assisted Installer](#) topic in the OpenShift Container Platform documentation for the requirements.

Confirm your OpenShift Container Platform installation

You must have a supported OpenShift Container Platform version, including the registry and storage services, installed and working. For more information about installing OpenShift Container Platform, see the OpenShift Container Platform documentation.

1. Verify that multicluster engine operator is not already installed on your OpenShift Container Platform cluster. The multicluster engine operator allows only one single installation on each OpenShift Container Platform cluster. Continue with the following steps if there is no installation.
2. To ensure that the OpenShift Container Platform cluster is set up correctly, access the OpenShift Container Platform web console with the following command:

```
kubectl -n openshift-console get route console
```

See the following example output:

```
console console-openshift-console.apps.new-coral.purple-chesterfield.com
console https reencrypt/Redirect None
```

3. Open the URL in your browser and check the result. If the console URL displays `console-openshift-console.router.default.svc.cluster.local`, set the value for `openshift_master_default_subdomain` when you install OpenShift Container Platform. See the following example of a URL: <https://console-openshift-console.apps.new-coral.purple-chesterfield.com>.

You can proceed to install multicluster engine operator.

Installing from the Red Hat OpenShift Software Catalog console

Best practice: From the *Administrator* view in your OpenShift Container Platform navigation, install the Red Hat OpenShift Software Catalog console that is provided with OpenShift Container Platform.

1. Go to **Ecosystem > Software Catalog**.
2. Choose the Operator that you want to install.
3. On the *Operator Installation* page, select the options for your installation:
 - Namespace:
 - The multicluster engine operator engine must be installed in its own namespace, or project.
 - By default, the Red Hat OpenShift Software Catalog console installation process creates a namespace titled `multicluster-engine`. **Best practice:** Continue to use the `multicluster-engine` namespace if it is available.
 - If there is already a namespace named `multicluster-engine`, select a different

namespace.

- **Channel:** The channel that you select corresponds to the release that you are installing. When you select the channel, it installs the identified release, and establishes that the future errata updates within that release are obtained.
- **Approval strategy:** The approval strategy identifies the human interaction that is required for applying updates to the channel or release to which you subscribed.
 - Select **Automatic**, which is selected by default, to ensure any updates within that release are automatically applied.
 - Select **Manual** to receive a notification when an update is available. If you have concerns about when the updates are applied, this might be best practice for you.

Note: To upgrade to the next minor release, you must return to the Red Hat OpenShift Software Catalog and select a new channel for the more current release.

4. Select **Install** to apply your changes and create the operator.
5. See the following process to create the *MultiClusterEngine* custom resource.
 - a. In the OpenShift Container Platform console navigation, select **Installed Operators > multicluster engine for Kubernetes**.
 - b. Select the **MultiCluster Engine** tab.
 - c. Select **Create MultiClusterEngine**.
 - d. Update the default values in the YAML file. See options in the *MultiClusterEngine advanced configuration* section of the documentation.
 - The following example shows the default template:

```
apiVersion: multicluster.openshift.io/v1
kind: MultiClusterEngine
metadata:
  name: multiclusterengine
spec: {}
```

6. Select **Create** to initialize the custom resource. It can take up to 10 minutes for the multicluster engine operator engine to build and start.

After the **MultiClusterEngine** resource is created, the status for the resource is **Available** on the **MultiCluster Engine** tab.

Installing from the OpenShift Container Platform CLI

See the following procedure to install from the command-line interface:

1. Create a multicluster engine operator engine namespace where the operator requirements are contained. Run the following command, where **namespace** is the name for your multicluster engine operator namespace. The value for **namespace** might be referred to as *Project* in the OpenShift Container Platform environment:

```
oc create namespace <namespace>
```

2. Switch your project namespace to the one that you created. Replace `namespace` with the name of the namespace that you created in step 1.

```
oc project <namespace>
```

3. Create a YAML file to configure an `OperatorGroup` resource. Each namespace can have only one operator group. Replace `default` with the name of your operator group. Replace `namespace` with the name of your project namespace. See the following example:

```
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: <default>
  namespace: <namespace>
spec:
  targetNamespaces:
  - <namespace>
```

4. Run the following command to create the `OperatorGroup` resource. Replace `operator-group` with the name of the operator group YAML file that you created:

```
oc apply -f <path-to-file>/<operator-group>.yaml
```

5. Create a YAML file to configure an OpenShift Container Platform Subscription. Your file appears similar to the following example, with `<stable-2.x>` changed to represent the supported version you are using:

```
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: multicluster-engine
spec:
  sourceNamespace: openshift-marketplace
  source: redhat-operators
  channel: stable-2.x
  installPlanApproval: Automatic
  name: multicluster-engine
```

Note: To configure infrastructure nodes, see [Configuring infrastructure nodes for multicluster engine operator](#).

6. Run the following command to create the OpenShift Container Platform Subscription. Replace

subscription with the name of the subscription file that you created:

```
oc apply -f <path-to-file>/<subscription>.yaml
```

7. Create a YAML file to configure the **MultiClusterEngine** custom resource. Your default template should look similar to the following example:

```
apiVersion: multicluster.openshift.io/v1
kind: MultiClusterEngine
metadata:
  name: multiclusterengine
spec: {}
```

Note: For installing the multicluster engine operator on infrastructure nodes, see the [MultiClusterEngine custom resource additional configuration](#) section:

8. Run the following command to create the **MultiClusterEngine** custom resource. Replace **custom-resource** with the name of your custom resource file:

```
oc apply -f <path-to-file>/<custom-resource>.yaml
```

If this step fails with the following error, the resources are still being created and applied. Run the command again in a few minutes when the resources are created:

```
error: unable to recognize "./mce.yaml": no matches for kind "MultiClusterEngine"
in version "operator.multicluster-engine.io/v1"
```

9. Run the following command to get the custom resource. It can take up to 10 minutes for the **MultiClusterEngine** custom resource status to display as **Available** in the **status.phase** field after you run the following command:

```
oc get mce -o=jsonpath='{.items[0].status.phase}'
```

If you are reinstalling the multicluster engine operator and the pods do not start, see [Troubleshooting reinstallation failure](#) for steps to work around this problem.

Notes:

- A **ServiceAccount** with a **ClusterRoleBinding** automatically gives cluster administrator privileges to multicluster engine operator and to any user credentials with access to the namespace where you install multicluster engine operator.

You can now configure your OpenShift Container Platform cluster to contain infrastructure nodes to run approved management components. Running components on infrastructure nodes avoids allocating OpenShift Container Platform subscription quota for the nodes that are running those

management components. See [Configuring infrastructure nodes for multicluster engine operator](#) for that procedure.

Configuring infrastructure nodes for multicluster engine operator

Configure your OpenShift Container Platform cluster to contain infrastructure nodes to run approved multicluster engine operator management components. Running components on infrastructure nodes avoids allocating OpenShift Container Platform subscription quota for the nodes that are running multicluster engine operator management components.

After adding infrastructure nodes to your OpenShift Container Platform cluster, follow the [Installing from the OpenShift Container Platform CLI](#) instructions and add the following configurations to the Operator Lifecycle Manager Subscription and `MultiClusterEngine` custom resource.

Configuring infrastructure nodes to the OpenShift Container Platform cluster

Follow the procedures that are described in [Creating infrastructure machine sets](#) in the OpenShift Container Platform documentation. Infrastructure nodes are configured with a Kubernetes `taints` and `labels` to keep non-management workloads from running on them.

For compatibility with the infrastructure node enablement, which is provided by multicluster engine operator, ensure your infrastructure nodes have the following `taints` and `labels` applied:

```
metadata:
  labels:
    node-role.kubernetes.io/infra: ""
spec:
  taints:
  - effect: NoSchedule
    key: node-role.kubernetes.io/infra
```

Operator Lifecycle Manager subscription configuration

Configure your Operator Lifecycle Manager subscription.

1. Add the following additional configuration before applying the Operator Lifecycle Manager Subscription:

```
spec:
  config:
    nodeSelector:
      node-role.kubernetes.io/infra: ""
    tolerations:
    - key: node-role.kubernetes.io/infra
      effect: NoSchedule
      operator: Exists
```

2. Update any add-ons to include the following node selectors and tolerations. You need to use the instructions [Configuring klusterlet add-ons](#):

```
nodeSelector:
  node-role.kubernetes.io/infra: ""
tolerations:
- key: node-role.kubernetes.io/infra
  effect: NoSchedule
  operator: Exists
```

3. If you used Red Hat OpenShift Data Foundation as a storage provisioner, make sure *Container Storage Interface* pods can run on infrastructure nodes. Learn more at [Managing container storage interface \(CSI\) component placements](#) in the Red Hat OpenShift Data Foundation documentation.

MultiClusterEngine custom resource additional configuration

Add the following additional configuration before applying the `MultiClusterEngine` custom resource:

```
spec:
  nodeSelector:
    node-role.kubernetes.io/infra: ""
```

Install on disconnected networks

You might need to install the multicluster engine operator on Red Hat OpenShift Container Platform clusters that are not connected to the Internet. The procedure to install on a disconnected engine requires some of the same steps as the connected installation.

You must download copies of the packages to access them during the installation, rather than accessing them directly from the network during the installation.

- [Prerequisites](#)
- [Confirm your OpenShift Container Platform installation](#)
- [Installing in a disconnected environment](#)

Prerequisites

You must meet the following requirements before you install The multicluster engine operator:

- A supported OpenShift Container Platform version must be deployed in your environment, and you must be logged in with the command line interface (CLI).
- You need access to catalog.redhat.com.

Note: For managing bare metal clusters, you need a supported OpenShift Container Platform version.

See the [OpenShift Container Platform Installing](#).

- Your Red Hat OpenShift Container Platform permissions must allow you to create a namespace.
- You must have a workstation with Internet connection to download the dependencies for the operator.

Confirm your OpenShift Container Platform installation

- You must have a supported OpenShift Container Platform version, including the registry and storage services, installed and working in your cluster. For information about OpenShift Container Platform, see [OpenShift Container Platform documentation](#).
- When and if you are connected, accessing the OpenShift Container Platform web console with the following command to verify:

```
oc -n openshift-console get route console
```

See the following example output:

```
console console-openshift-console.apps.new-name.purple-name.com
console https reencrypt/Redirect None
```

The console URL in this example is: [https:// console-openshift-console.apps.new-coral.purple-chesterfield.com](https://console-openshift-console.apps.new-coral.purple-chesterfield.com). Open the URL in your browser and check the result.

If the console URL displays `console-openshift-console.router.default.svc.cluster.local`, set the value for `openshift_master_default_subdomain` when you install OpenShift Container Platform.

Installing in a disconnected environment

Important: You need to download the required images to a mirroring registry to install the operators in a disconnected environment. Without the download, you might receive `ImagePullBackOff` errors during your deployment.

Follow these steps to install the multicluster engine operator in a disconnected environment:

1. Create a mirror registry. If you do not already have a mirror registry, create one by completing the procedure in the [Mirroring in disconnected environments](#) topic of the Red Hat OpenShift Container Platform documentation.

If you already have a mirror registry, you can configure and use your existing one.

2. **Note:** For bare metal only, you need to provide the certificate information for the disconnected registry in your `install-config.yaml` file. To access the image in a protected disconnected registry, you must provide the certificate information so the multicluster engine operator can access the registry.
 - a. Copy the certificate information from the registry.
 - b. Open the `install-config.yaml` file in an editor.

- c. Find the entry for `additionalTrustBundle: |`.
- d. Add the certificate information after the `additionalTrustBundle` line. The resulting content should look similar to the following example:

```
additionalTrustBundle: |
  -----BEGIN CERTIFICATE-----
  certificate_content
  -----END CERTIFICATE-----
sshKey: >-
```

3. **Optional:** You can specify one or multiple SSH public keys, which allow you to access the hosts after installing. See the following example:

```
sshKey: |
  ssh-rsa <public-key-1>
  ssh-rsa <public-key-2>
  ssh-rsa <public-key-3>
```

4. **Important:** Additional mirrors for disconnected image registries are needed if the following Governance policies are required:

- Container Security Operator policy: Locate the images in the `registry.redhat.io/quay` source.
- Compliance Operator policy: Locate the images in the `registry.redhat.io/compliance` source.
- Gatekeeper Operator policy: Locate the images in the `registry.redhat.io/gatekeeper` source.

See the following example of mirrors lists for all three operators:

```
- mirrors:
  - <your_registry>/rhacm2
  source: registry.redhat.io/rhacm2
- mirrors:
  - <your_registry>/quay
  source: registry.redhat.io/quay
- mirrors:
  - <your_registry>/compliance
  source: registry.redhat.io/compliance
```

5. Save the `install-config.yaml` file.
6. Create a YAML file that contains the `ImageContentSourcePolicy` with the name `mce-policy.yaml`.
Note: If you modify this on a running cluster, it causes a rolling restart of all nodes.

```
apiVersion: operator.openshift.io/v1alpha1
kind: ImageContentSourcePolicy
metadata:
```

```
name: mce-repo
spec:
  repositoryDigestMirrors:
  - mirrors:
    - mirror.registry.com:5000/multicluster-engine
    source: registry.redhat.io/multicluster-engine
```

7. Apply the ImageContentSourcePolicy file by entering the following command:

```
oc apply -f mce-policy.yaml
```

8. Enable the disconnected Operator Lifecycle Manager Red Hat Operators and Community Operators.

the multicluster engine operator is included in the Operator Lifecycle Manager Red Hat Operator catalog.

9. Configure the disconnected Operator Lifecycle Manager for the Red Hat Operator catalog. Follow the steps in the [Using Operator Lifecycle Manager on restricted networks](#) topic of the Red Hat OpenShift Container Platform documentation.

10. Continue to install the multicluster engine operator for Kubernetes from the *Operator Lifecycle Manager* catalog.

See [Installing while connected online](#) for the required steps.

Upgrading multicluster engine for Kubernetes operator

Learn how to upgrade your multicluster engine operator. Control your multicluster engine operator upgrades by using the operator subscription settings in the OpenShift Container Platform console.

Required access: OpenShift Container Platform administrator

Important:

- Upgrades are always supported from the immediate previous version. You can upgrade to the next available feature release.
- If you are on a version with an **Extended Update Support** (EUS) status, you can upgrade to the next version, or you can skip to the next EUS version.
- You can perform skip-level upgrades from `<2.x>` to `<2.x+2>` on even number versions of multicluster engine operator, starting with Red Hat Advanced Cluster Management version 2.11. For instance, when you upgrade multicluster engine operator 2.6, which is an EUS version, you can choose to upgrade to version 2.8, which is also an EUS version.

Procedure

You can upgrade your **MultiClusterEngine** resource from the console. Use the Operator Lifecycle Manager **operatorcondition** to control how versions are upgraded. When you initially deploy multicluster engine operator by using the operator, you make the following selections:

Channel

Channel corresponds to the version of the product that you are installing. The initial channel setting is often the most current channel that was available at the time of installation.

Approval

Approval specifies whether approval is required for updates within the channel, or if they are done automatically.

- If set to **Automatic**, then minor release updates in the selected channel are deployed without administrator intervention.
- If set to **Manual**, then each update to the minor release within the channel requires an administrator to approve the update.

You also use these settings when you upgrade to the latest version of multicluster engine operator by using the operator. Complete the following steps to upgrade your operator:

Important:

Downgrading a version is not supported. You cannot revert back to an earlier version after upgrading to a later version in the channel selection. You must uninstall the operator and reinstall it with the earlier version to use a previous version.

See the following procedure to upgrade your cluster:

1. Log in to your OpenShift Container Platform *Software Catalog*.
2. In the OpenShift Container Platform navigation, select **Operators** > **Installed operators**.
3. Select the **multicluster engine for Kubernetes operator** operator.
4. Select the **Subscription** tab to edit the subscription settings.
5. Ensure that the *Upgrade Status* is labeled **Up to date**. This status indicates that the operator is at the latest level that is available in the selected channel. If the *Upgrade Status* indicates that there is an upgrade pending, complete the following steps to update it to the latest minor release that is available in the channel:
 - a. Click the **Manual** setting in the *Approval* field to edit the value.
 - b. Select **Automatic** to enable automatic updates.
 - c. Select **Save** to commit your change.
 - d. Wait for the automatic updates to be applied to the operator. The updates automatically add the required updates to the latest version in the selected channel. When all of the updated updates are complete, the *Upgrade Status* field indicates an **Up to date** status.

Note: It can take up to 10 minutes for the **MultiClusterEngine** custom resource to finish upgrading.

6. Run the following command to verify the upgrade process is started:

```
oc get mce
```

While in the process of upgrading, the **Status** displays **Updating**. After the upgrade is complete, the **Status** field changes to **Available** status.

7. When the *Upgrade Status* is **Up to date**, click the value in the *Channel* field to edit the channel.
8. Select the channel for the next available feature release, but do not attempt to skip a channel.

The Operator Lifecycle Manager **operatorcondition** resource checks for previous upgrades during the current upgrade process and prevents skipping versions. You can check that same resource status to see if the upgradable status is **true** or **false**.

9. Click **Save** to save your changes.
10. Wait for the automatic upgrade to complete. After the upgrade to the next feature release completes, the updates to the latest patch releases within the channel are deployed.
11. If you have to upgrade to a later feature release, repeat steps 7-9 until your operator is at the latest level of the desired channel. Make sure that all of the patch releases are deployed for your final channel.
12. Optional: You can set your *Approval* setting to **Manual**, if you want your future updates within the channel to require manual approvals.
13. If you performed a skip-level upgrade, see the following sub-steps to verify your upgrade:
 - a. Verify that the **csv** version from the previous release, for instance, **2.6** (EUS), is replaced with the upgraded version, for instance, **2.8** (EUS).
 - b. Verify that the **MultiClusterEngine** instance **status.currentVersion** specification value is set at **2.8** and matches the **desiredVersion** status, which is also set at **2.8**.

For more information about upgrading your operator, see [Operators](#) in the OpenShift Container Platform documentation.

Upgrading disconnected clusters by using policies

If you have the Red Hat Advanced Cluster Management for Kubernetes hub cluster, which uses the **MultiClusterHub** operator to manage, upgrade, and install hub cluster components, you can use OpenShift Update Service with Red Hat Advanced Cluster Management policies to upgrade multiple clusters in a disconnected environment.

OpenShift Update Service is a separate operator and operand that monitors the available versions of your managed clusters and makes them available for upgrading in a disconnected environment. OpenShift Update Service can perform the following actions:

- Monitor when upgrades are available for your disconnected clusters.
- Identify which updates are mirrored to your local site for upgrading by using the graph data file.
- Notify you that an upgrade is available for your cluster by using the console.
- [Prerequisites](#)
- [Prepare your disconnected mirror registry](#)
- [Deploy the operator for OpenShift Update Service](#)

- [Build the graph data init container](#)
- [Configuring the certificate for the mirrored registry](#)
- [Deploy the OpenShift Update Service instance](#)
- [Optional: Deploying a policy to override the default registry](#)
- [Deploying a policy to deploy a disconnected catalog source](#)
- [Deploying a policy to change the managed cluster parameter](#)
- [Viewing available upgrades](#)
- [Selecting a channel](#)
- [Upgrading the cluster](#)
- [Additional resources](#)

See [Configuring additional trust stores for image registry access](#) in the OpenShift Container Platform documentation to learn more about the external registry CA certificate.

Prerequisites

You must have the following prerequisites before you can use OpenShift Update Service to upgrade your disconnected clusters:

- You need to install Red Hat Advanced Cluster Management. See the Red Hat Advanced Cluster Management [Installing and upgrading](#) documentation.
- You need a hub cluster that is running on a supported Red Hat OpenShift Container Platform version with restricted OLM configured. See [Using Operator Lifecycle Manager on restricted networks](#) for details about how to configure restricted OLM. Take note of the catalog source image when you configure restricted OLM.
- You need an OpenShift Container Platform cluster that the hub cluster manages.
- You need access credentials to a local repository where you can mirror the cluster images. See [Mirroring in disconnected environments](#) for more information.

Note: The image for the current version of the cluster that you upgrade must remain available as one of the mirrored images. If an upgrade fails, the cluster reverts back to the version of the cluster when you tried to upgrade.

Preparing your disconnected mirror registry

You must mirror both the image that you want to upgrade to and the current image that you are upgrading from to your local mirror registry.

Complete the following steps to mirror the images:

1. Create a script file with content that resembles the following example. Replace `<pull-secret>` with the path to your OpenShift Container Platform pull secret:

```
UPSTREAM_REGISTRY=quay.io
PRODUCT_REPO=openshift-release-dev
```

```
RELEASE_NAME=ocp-release
OCP_RELEASE=4.17.2-x86_64
LOCAL_REGISTRY=$(hostname):5000
LOCAL_SECRET_JSON=<pull-secret>

oc adm -a ${LOCAL_SECRET_JSON} release mirror \
--from=${UPSTREAM_REGISTRY}/${PRODUCT_REPO}/${RELEASE_NAME}:${OCP_RELEASE} \
--to=${LOCAL_REGISTRY}/ocp4 \
--to-release-image=${LOCAL_REGISTRY}/ocp4/release:${OCP_RELEASE}
```

2. Run the script to mirror the images, configure settings, and separate the release images from the release content.

Deploying the operator for OpenShift Update Service

To deploy the operator for OpenShift Update Service in your OpenShift Container Platform environment, complete the following steps:

1. On your hub cluster, access the Red Hat OpenShift Software Catalog.
2. Deploy the operator by selecting **OpenShift Update Service Operator** and update the default values if needed. The deployment of the operator creates a new project named **openshift-update-service**.
3. Wait for the installation of the operator to finish.

You can check the status of the installation by running the **oc get pods** command. Verify that the operator is in the **running** state.

Building the graph data init container

OpenShift Update Service uses graph data information to find the available upgrades. In a connected environment, OpenShift Update Service pulls the graph data information for available upgrades directly from the [update-service graph data GitHub repository](#).

In a disconnected environment, you must make the graph data available in a local repository by using an **init container**.

Complete the following steps to create a graph data **init container**:

1. Clone the *graph data* Git repository by running the following command:

```
git clone https://github.com/openshift/cincinnati-graph-data
```

2. Create a file that has the information for your graph data **init**. You can find a sample [Dockerfile](#) in the **cincinnati-operator** GitHub repository.
 - The **FROM** value is the external registry where OpenShift Update Service finds the images.
 - The **RUN** commands create the directory and package the upgrade files.
 - The **CMD** command copies the package file to the local repository and extracts the files for an

upgrade.

3. Run the following command to build the **graph data init container**:

```
podman build -f <docker-path> -t <graph-path>:latest
```

Replace **<docker-path>** with the path to the file that you created in the previous step.

Replace **<graph-path>** with the path to your local graph data init container.

4. Run the following command to push the **graph data init container**:

```
podman push <graph-path>:latest --authfile=<pull-secret>.json
```

Replace **<graph-path>** with the path to your local graph data init container.

Replace **<pull-secret>** with the path to your pull secret file.

Optional: If you do not have **podman** installed, replace **podman** with **docker** in step three and four.

Configuring the certificate for the mirrored registry

If you are using a secure external container registry to store your mirrored OpenShift Container Platform release images, OpenShift Update Service requires access to this registry to build an upgrade graph.

Complete the following steps to configure your CA certificate to work with the OpenShift Update Service pod:

1. Find the OpenShift Container Platform external registry API, which is located in **image.config.openshift.io**. This is where the external registry CA certificate is stored. See *Configuring additional trust stores for image registry access* in the additional resources section to learn more.
2. Create a ConfigMap in the **openshift-config** namespace and add your CA certificate in the **updateservice-registry** section. See the following example:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: trusted-ca
data:
  updateservice-registry: |
    -----BEGIN CERTIFICATE-----
    ...
    -----END CERTIFICATE-----
```

3. Edit the **cluster** resource in the **image.config.openshift.io** API to set the **additionalTrustedCA** field to the name of the ConfigMap that you created. Run the following command and replace

<trusted_ca> with the path to your new ConfigMap:

```
oc patch image.config.openshift.io cluster -p
'{"spec":{"additionalTrustedCA":{"name":"<trusted_ca>"}}}' --type merge
```

The OpenShift Update Service Operator watches the `image.config.openshift.io` API and the ConfigMap you created in the `openshift-config` namespace for changes, then restarts the deployment if the CA cert has changed.

Deploying the OpenShift Update Service instance

When you finish deploying the OpenShift Update Service instance on your hub cluster, the instance is located where the images for the cluster upgrades are mirrored and made available to the disconnected managed cluster.

Complete the following steps to deploy the instance:

1. If you do not want to use the default namespace of the operator, navigate to **Administration > Namespaces** in the console to change it.
2. In the *Installed Operators* section of the OpenShift Container Platform console, select **OpenShift Update Service Operator**.
3. Select **Create Instance** in the menu.
4. Paste the contents from your OpenShift Update Service instance. Your YAML instance might resemble the following manifest:

```
apiVersion: update-service.openshift.io/v1beta2
kind: update-service
metadata:
  name: openshift-cincinnati-instance
  namespace: openshift-update-service
spec:
  registry: <registry-host-name>:<port> ①
  replicas: 1
  repository: ${LOCAL_REGISTRY}/ocp4/release
  graphDataImage: '<host-name>:<port>/cincinnati-graph-data-container' ②
```

① Replace with the path to your local disconnected registry for your images.

② Replace with the path to your graph data init container. This is the same value that you used when you ran the `podman push` command to push your graph data init container.

5. Select **Create** to create the instance.
6. From the hub cluster CLI, enter the `oc get pods` command to view the status of the instance creation.

It might take a few minutes. The process is complete when the result of the command shows that the instance and the operator are running.

Optional: Deploying a policy to override the default registry

The following steps only apply if you have mirrored your releases into your mirrored registry.

Deprecated: `PlacementRule`

OpenShift Container Platform has a default image registry value that specifies where it finds the upgrade packages. In a disconnected environment, you can create a policy to replace that value with the path to your local image registry where you mirrored your release images.

Complete the following steps to create the policy:

1. Log in to the OpenShift Container Platform environment of your hub cluster.
2. From the console, select **Governance > Create policy**.
3. Set the **YAML** switch to *On* to view the YAML version of the policy.
4. Delete all of the content in the YAML code.
5. Paste the following YAML content into the window to create a custom policy:

```
apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  name: policy-mirror
  namespace: default
spec:
  disabled: false
  remediationAction: enforce
  policy-templates:
    - objectDefinition:
        apiVersion: policy.open-cluster-management.io/v1
        kind: ConfigurationPolicy
        metadata:
          name: policy-image-content-source-policy
        spec:
          object-templates:
            - complianceType: musthave
              objectDefinition:
                apiVersion: operator.openshift.io/v1alpha1
                kind: ImageContentSourcePolicy
                metadata:
                  name: <your-local-mirror-name> ①
                spec:
                  repositoryDigestMirrors:
                    - mirrors:
                        - <your-registry> ②
                      source: registry.redhat.io
          ---
        apiVersion: policy.open-cluster-management.io/v1
        kind: PlacementBinding
        metadata:
```

```

name: binding-policy-mirror
namespace: default
placementRef:
  name: placement-policy-mirror
  kind: PlacementRule
  apiGroup: apps.open-cluster-management.io
subjects:
- name: policy-mirror
  kind: Policy
  apiGroup: policy.open-cluster-management.io
---
apiVersion: apps.open-cluster-management.io/v1
kind: PlacementRule
metadata:
  name: placement-policy-mirror
  namespace: default
spec:
  clusterSelector:
    matchExpressions:
      [] ③

```

① Replace with your local mirror name.

② Replace with the path to your local mirror repository. You can find the path to your local mirror by running the `oc adm release mirror` command.

③ Selects all clusters if not specified.

6. Select **Enforce if supported**.

7. Select **Create** to create the policy.

Deploying a policy to deploy a disconnected catalog source

You can push the *Catalogsource* policy to the managed cluster to change the default location from a connected location to your disconnected local registry.

Complete the following steps to change the default location:

1. In the console menu, select **Governance > Create policy**.
2. Set the **YAML** switch to *On* to view the YAML version of the policy.
3. Delete all of the content in the **YAML** code.
4. Paste the following **YAML** content into the window to create a custom policy:

```

apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  name: policy-catalog
  namespace: default
spec:
  disabled: false

```

```

remediationAction: enforce
policy-templates:
- objectDefinition:
  apiVersion: policy.open-cluster-management.io/v1
  kind: ConfigurationPolicy
  metadata:
    name: policy-catalog
  spec:
    object-templates:
    - complianceType: musthave
      objectDefinition:
        apiVersion: config.openshift.io/v1
        kind: OperatorHub
        metadata:
          name: cluster
        spec:
          disableAllDefaultSources: true
    - complianceType: musthave
      objectDefinition:
        apiVersion: operators.coreos.com/v1alpha1
        kind: CatalogSource
        metadata:
          name: my-operator-catalog
          namespace: openshift-marketplace
        spec:
          sourceType: grpc
          image: '<registry_host_name>:<port>/olm/redhat-operators:v1' ①
          displayName: My Operator Catalog
          publisher: grpc
---
apiVersion: policy.open-cluster-management.io/v1
kind: PlacementBinding
metadata:
  name: binding-policy-catalog
  namespace: default
placementRef:
  name: placement-policy-catalog
  kind: PlacementRule
  apiGroup: apps.open-cluster-management.io
subjects:
- name: policy-catalog
  kind: Policy
  apiGroup: policy.open-cluster-management.io
---
apiVersion: apps.open-cluster-management.io/v1
kind: PlacementRule
metadata:
  name: placement-policy-catalog
  namespace: default
spec:
  clusterSelector:

```

```
matchExpressions:
  [] ②
```

- ① Replace with the path to your local restricted catalog source image.
- ② Selects all clusters if not specified.

5. Select **Enforce if supported**.
6. Select **Create** to create the policy.

Deploying a policy to change the managed cluster parameter

You can push the *ClusterVersion* policy to the managed cluster to change the default location where it retrieves its upgrades.

Complete the following steps:

1. From the managed cluster, confirm that the *ClusterVersion* upstream parameter is currently the default public OpenShift Update Service operand by running the following command:

```
oc get clusterversion -o yaml
```

2. From the hub cluster, identify the route URL to the OpenShift Update Service operand by running the following command:

```
oc get routes
```

Remember the result for later.

3. In the hub cluster console menu, select **Governance > Create a policy**.
4. Set the **YAML** switch to *On* to view the YAML version of the policy.
5. Delete all of the content in the **YAML** code.
6. Paste the following **YAML** content into the window to create a custom policy:

```
apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  name: policy-cluster-version
  namespace: default
  annotations:
    policy.open-cluster-management.io/standards: null
    policy.open-cluster-management.io/categories: null
    policy.open-cluster-management.io/controls: null
spec:
  disabled: false
  remediationAction: enforce
  policy-templates:
```



```

- objectDefinition:
  apiVersion: policy.open-cluster-management.io/v1
  kind: ConfigurationPolicy
  metadata:
    name: policy-cluster-version
  spec:
    object-templates:
      - complianceType: musthave
        objectDefinition:
          apiVersion: config.openshift.io/v1
          kind: ClusterVersion
          metadata:
            name: version
          spec:
            channel: stable-4.4
            upstream: >-
              https://example-cincinnati-policy-engine-
uri/api/upgrades_info/v1/graph ①

---
apiVersion: policy.open-cluster-management.io/v1
kind: PlacementBinding
metadata:
  name: binding-policy-cluster-version
  namespace: default
placementRef:
  name: placement-policy-cluster-version
  kind: PlacementRule
  apiGroup: apps.open-cluster-management.io
subjects:
- name: policy-cluster-version
  kind: Policy
  apiGroup: policy.open-cluster-management.io
---
apiVersion: apps.open-cluster-management.io/v1
kind: PlacementRule
metadata:
  name: placement-policy-cluster-version
  namespace: default
spec:
  clusterSelector:
    matchExpressions:
      []

```

① Replace with the path to your hub cluster OpenShift Update Service operand. Selects all clusters if not specified.

You can complete the following steps to determine the path to the operand:

- a. Run the `oc get get routes -A` command on the hub cluster.
- b. Find the route to `update-service`.

The path to the operand is the value in the **HOST/PORT** field.

7. Select **Enforce if supported**.
8. Select **Create** to create the policy.
9. In the managed cluster CLI, confirm that the upstream parameter in the **ClusterVersion** is updated with the local hub cluster OpenShift Update Service URL by running the following command:

```
oc get clusterversion -o yaml
```

10. Verify that the results resemble the following content:

```
apiVersion: v1
items:
- apiVersion: config.openshift.io/v1
  kind: ClusterVersion
[...]
```

```
spec:
  channel: stable-4.4
  upstream: https://<hub-cincinnati-uri>/api/upgrades_info/v1/graph
```

Viewing available upgrades

You can view a list of available upgrades for your managed cluster by completing the following steps:

1. From the console, select **Infrastructure > Clusters**.
2. Select a cluster that is in the *Ready* state.
3. From the **Actions** menu, select **Upgrade cluster**.
4. Verify that the optional upgrade paths are available.

Note: No available upgrade versions are shown if the current version is not mirrored into the local image repository.

Selecting a channel

You can use the Red Hat Advanced Cluster Management console to select a channel for your cluster upgrades on OpenShift Container Platform. Those versions must be available on the mirror registry.

Complete the steps in [Selecting a channel](#) to specify a channel for your upgrades.

Upgrading the cluster

After configuring the disconnected registry, Red Hat Advanced Cluster Management and OpenShift Update Service use the disconnected registry to find if upgrades are available.

If no available upgrades are displayed, make sure that you have the release image of the current level of the cluster and at least one later level mirrored in the local repository.

If the release image for the current version of the cluster is not available, no upgrades are available.

Complete the following steps to upgrade:

1. In the console, select **Infrastructure > Clusters**.
2. Find the cluster that you want to choose if there is an available upgrade.
3. If there is an upgrade available, the **Distribution version** column for the cluster shows an upgrade available.
4. Select the *Options* menu for the cluster, and select **Upgrade cluster**.
5. Select the target version for the upgrade, and select **Upgrade**.

If your cluster upgrade fails, the Operator generally retries the upgrade a few times, stops, and reports the status of the failing component. In some cases, the upgrade process continues to cycle through attempts to complete the process. Rolling your cluster back to a previous version after a failed upgrade is not supported. Contact Red Hat support for assistance if your cluster upgrade fails.

Additional resources

See [Configuring additional trust stores for image registry access](#) in the OpenShift Container Platform documentation to learn more about the external registry CA certificate.

Advanced configuration

The multicluster engine operator is installed using an operator that deploys all of the required components. The multicluster engine operator can be further configured during or after installation. Learn more about the advanced configuration options.

Deployed components

Add one or more of the following attributes to the **MultiClusterEngine** custom resource:

Table 6. Table list of the deployed components

Name	Description	Enabled
assisted-service	Installs OpenShift Container Platform with minimal infrastructure prerequisites and comprehensive pre-flight validations	True
cluster-api	Provides capabilities to handle the Cluster API lifecycle from within a managed cluster	False

cluster-api-provider-aws	Provides Kubernetes-style APIs for cluster creation, configuration, and management for AWS	False
cluster-api-provider-openshift-assisted	Provides Kubernetes-style APIs for cluster creation, configuration, and management with Assisted Installer	False
cluster-api-provider-metal3	Provides Kubernetes-style APIs for cluster creation, configuration, and management on bare-metal infrastructure	False
cluster-lifecycle	Provides cluster management capabilities for OpenShift Container Platform and Kubernetes hub clusters	True
cluster-manager	Manages various cluster-related operations within the cluster environment	True
cluster-proxy-addon	Automates the installation of apiserver-network-proxy on both hub and managed clusters using a reverse proxy server	True
console-mce	Enables the multicluster engine operator console plug-in	True
discovery	Discovers and identifies new clusters within the OpenShift Cluster Manager	True
hive	Provisions and performs initial configuration of OpenShift Container Platform clusters	True
hypershift	Hosts OpenShift Container Platform control planes at scale with cost and time efficiency, and cross-cloud portability	True
hypershift-local-hosting	Enables local hosting capabilities for within the local cluster environment	True
image-based-install-operator	Provides site configuration to single-node OpenShift clusters to complete installation	False

local-cluster	Enables the import and self-management of the local hub cluster where the multicluster engine operator is deployed	True
managedserviceaccount	Synchronizes service accounts to managed clusters, and collects tokens as secret resources to give back to the hub cluster	True
server-foundation	Provides foundational services for server-side operations within the multicluster environment	True

When you install multicluster engine operator on to the cluster, not all of the listed components are enabled by default.

You can further configure multicluster engine operator during or after installation by adding one or more attributes to the **MultiClusterEngine** custom resource. Continue reading for information about the attributes that you can add.

Console and component configuration

The following example displays the **spec.overrides** default template that you can use to enable or disable the component:

```
apiVersion: operator.open-cluster-management.io/v1
kind: MultiClusterEngine
metadata:
  name: multiclusterengine
spec:
  overrides:
    components:
      - name: <name> ①
        enabled: true
```

① Replace **name** with the name of the component.

Alternatively, you can run the following command. Replace **namespace** with the name of your project and **name** with the name of the component:

```
oc patch MultiClusterEngine <multiclusterengine-name> --type=json -p='[{"op": "add",
"path": "/spec/overrides/components/-", "value":{"name":"<name>","enabled":true}}]'
```

Local-cluster enablement

By default, the cluster that is running multicluster engine operator is the *local cluster*, which is a

hub cluster that manages itself and is designated as the **local-cluster** in the list of clusters.

In the **MultiClusterEngine** resource, the local cluster name is set from the **spec.localClusterName** value. The **enabled** setting specifies whether the feature is enabled or disabled.

The **enabled: true** setting designates your cluster as a **local-cluster**, and the **enabled: false** setting does not. See the following sample where the **local-cluster** is enabled with the **enabled: true** value.

```
apiVersion: multicluster.openshift.io/v1
kind: MultiClusterEngine
metadata:
  name: multiclusterengine
spec:
  localClusterName: <your-local-cluster-name>
  overrides:
    components:
      - name: local-cluster
        enabled: true
```

You can change the **local-cluster** name if the **enabled** field is set to **false**. You must use 34 or fewer characters for the **<your-local-cluster-name>** value. The **local-cluster** cannot be renamed if it is set as **enabled: true**.

To install multicluster engine operator *without* designating it as a **local-cluster**, change the **spec.overrides.components** settings to **enabled: false**. See the following YAML sample:

```
apiVersion: multicluster.openshift.io/v1
kind: MultiClusterEngine
metadata:
  name: multiclusterengine
spec:
  localClusterName: <your-local-cluster-name>
  overrides:
    components:
      - name: local-cluster
        enabled: false
```

Custom image pull secret

If you plan to import Kubernetes clusters that were not created by OpenShift Container Platform or the multicluster engine operator, generate a secret that contains your OpenShift Container Platform pull secret information to access the entitled content from the distribution registry.

The secret requirements for OpenShift Container Platform clusters are automatically resolved by OpenShift Container Platform and multicluster engine for Kubernetes operator, so you do not have to create the secret if you are not importing other types of Kubernetes clusters to be managed.

Important: These secrets are namespace-specific, so make sure that you are in the namespace that

you use for your engine.

1. Download your OpenShift Container Platform pull secret file from cloud.redhat.com/openshift/install/pull-secret by selecting **Download pull secret**. Your OpenShift Container Platform pull secret is associated with your Red Hat Customer Portal ID, and is the same across all Kubernetes providers.
2. Run the following command to create your secret:

```
oc create secret generic <secret> -n <namespace> --from
-file=.dockerconfigjson=<path-to-pull-secret> --type=kubernetes.io/dockerconfigjson
```

- Replace **secret** with the name of the secret that you want to create.
- Replace **namespace** with your project namespace, as the secrets are namespace-specific.
- Replace **path-to-pull-secret** with the path to your OpenShift Container Platform pull secret that you downloaded.

The following example displays the **spec.imagePullSecret** template to use if you want to use a custom pull secret. Replace **secret** with the name of your pull secret:

```
apiVersion: multicluster.openshift.io/v1
kind: MultiClusterEngine
metadata:
  name: multiclusterengine
spec:
  imagePullSecret: <secret>
```

Target namespace

The operands can be installed in a designated namespace by specifying a location in the **MultiClusterEngine** custom resource. This namespace is created upon application of the **MultiClusterEngine** custom resource.

Important: If no target namespace is specified, the operator will install to the **multicluster-engine** namespace and will set it in the **MultiClusterEngine** custom resource specification.

The following example displays the **spec.targetNamespace** template that you can use to specify a target namespace. Replace **target** with the name of your destination namespace. **Note:** The **target** namespace cannot be the **default** namespace:

```
apiVersion: multicluster.openshift.io/v1
kind: MultiClusterEngine
metadata:
  name: multiclusterengine
spec:
  targetNamespace: <target>
```

availabilityConfig

The hub cluster has two availabilities: **High** and **Basic**. By default, the hub cluster has an availability of **High**, which gives hub cluster components a **replicaCount** of 2. This provides better support in cases of failover but consumes more resources than the **Basic** availability, which gives components a **replicaCount** of 1.

Important: Set **spec.availabilityConfig** to **Basic** if you are using multicluster engine operator on a single-node OpenShift cluster.

The following examples shows the **spec.availabilityConfig** template with **Basic** availability:

```
apiVersion: multicluster.openshift.io/v1
kind: MultiClusterEngine
metadata:
  name: multiclusterengine
spec:
  availabilityConfig: "Basic"
```

nodeSelector

You can define a set of node selectors in the **MultiClusterEngine** to install to specific nodes on your cluster. The following example shows **spec.nodeSelector** to assign pods to nodes with the label **node-role.kubernetes.io/infra**:

```
spec:
  nodeSelector:
    node-role.kubernetes.io/infra: ""
```

To define a set of node selectors for the Red Hat Advanced Cluster Management for Kubernetes hub cluster, see [nodeSelector](#) in the Red Hat Advanced Cluster Management documentation.

tolerations

You can define a list of tolerations to allow the **MultiClusterEngine** to tolerate specific taints defined on the cluster. The following example shows a **spec.tolerations** that matches a **node-role.kubernetes.io/infra** taint:

```
spec:
  tolerations:
  - key: node-role.kubernetes.io/infra
    effect: NoSchedule
    operator: Exists
```

The previous infra-node toleration is set on pods by default without specifying any tolerations in the configuration. Customizing tolerations in the configuration will replace this default behavior.

To define a list of tolerations for the Red Hat Advanced Cluster Management for Kubernetes hub cluster,, see [tolerations](#) in the Red Hat Advanced Cluster Management documentation.

ManagedServiceAccount add-on

The **ManagedServiceAccount** add-on allows you to create or delete a service account on a managed cluster. To install with this add-on enabled, include the following in the **MultiClusterEngine** specification in **spec.overrides**:

```
apiVersion: multicluster.openshift.io/v1
kind: MultiClusterEngine
metadata:
  name: multiclusterengine
spec:
  overrides:
    components:
      - name: managedserviceaccount
        enabled: true
```

The **ManagedServiceAccount** add-on can be enabled after creating **MultiClusterEngine** by editing the resource on the command line and setting the **managedserviceaccount** component to **enabled: true**. Alternatively, you can run the following command and replace <multiclusterengine-name> with the name of your **MultiClusterEngine** resource.

```
oc patch MultiClusterEngine <multiclusterengine-name> --type=json -p='[{"op": "add",
"path": "/spec/overrides/components/-",
"value":{"name":"managedserviceaccount","enabled":true}}]'
```

Uninstalling

When you uninstall multicluster engine for Kubernetes operator, you see two different levels of the process: A *custom resource removal* and a *complete operator uninstall*. It might take up to five minutes to complete the uninstall process.

- The custom resource removal is the most basic type of uninstall that removes the custom resource of the **MultiClusterEngine** instance but leaves other required operator resources. This level of uninstall is helpful if you plan to reinstall using the same settings and components.
- The second level is a more complete uninstall that removes most operator components, excluding components such as custom resource definitions. When you continue with this step, it removes all of the components and subscriptions that were not removed with the custom resource removal. After this uninstall, you must reinstall the operator before reinstalling the custom resource.

Prerequisite: Detach enabled services

Before you uninstall the multicluster engine for Kubernetes operator, you must detach all of the clusters that are managed by that engine. To avoid errors, detach all clusters that are still managed

by the engine, then try to uninstall again.

- If you have managed clusters attached, you might see the following message.

```
Cannot delete MultiClusterEngine resource because ManagedCluster resource(s) exist
```

For more information about detaching clusters, see the *Removing a cluster from management* section by selecting the information for your provider in [Creating clusters](#).

Removing resources by using commands

1. If you have not already, ensure that your OpenShift Container Platform CLI is configured to run **oc** commands. See [Getting started with the OpenShift CLI](#) in the OpenShift Container Platform documentation for more information about how to configure the **oc** commands.
2. Change to your project namespace by entering the following command. Replace *namespace* with the name of your project namespace:

```
oc project <namespace>
```

3. Enter the following command to remove the **MultiClusterEngine** custom resource:

```
oc delete multiclusterengine --all
```

You can view the progress by entering the following command:

```
oc get multiclusterengine -o yaml
```

4. Enter the following commands to delete the multicluster-engine **ClusterServiceVersion** in the namespace where it is installed and the version replacing the **v2.x.0** variables:

```
❯ oc get csv
NAME                                DISPLAY                                VERSION  REPLACES
PHASE
multicluster-engine.v2.x.0  multicluster engine for Kubernetes  2.x.0
Succeeded

❯ oc delete clusterserviceversion multicluster-engine.v2.x.0
❯ oc delete sub multicluster-engine
```

The CSV version shown here may be different.

Deleting the components by using the console

When you use the RedHat OpenShift Container Platform console to uninstall, you remove the

operator. Complete the following steps to uninstall by using the console:

1. In the OpenShift Container Platform console navigation, select **Operators > Installed Operators > multicluster engine for Kubernetes**.
2. Remove the **MultiClusterEngine** custom resource.
 - a. Select the tab for *Multiclusterengine*.
 - b. Select the *Options* menu for the MultiClusterEngine custom resource.
 - c. Select **Delete MultiClusterEngine**.
3. Run the clean-up script according to the procedure in the following section.

Tip: If you plan to reinstall the same multicluster engine for Kubernetes operator version, you can skip the rest of the steps in this procedure and reinstall the custom resource.

4. Navigate to **Installed Operators**.
5. Remove the `_ multicluster engine for Kubernetes_` operator by selecting the *Options* menu and selecting **Uninstall operator**.

Troubleshooting Uninstall

If the multicluster engine custom resource is not being removed, remove any potential remaining artifacts by running the clean-up script.

- a. Copy the following script into a file:

```
#!/bin/bash
oc delete apiservice v1.admission.cluster.open-cluster-management.io
v1.admission.work.open-cluster-management.io
oc delete validatingwebhookconfiguration
multiclusterengines.multicluster.openshift.io
oc delete mce --all
```

See [Mirroring in disconnected environments](#) for more information.

Managing credentials

A *credential* is required to create and manage a Red Hat OpenShift Container Platform cluster on a cloud service provider with multicluster engine operator. The credential stores the access information for a cloud provider. Each provider account requires its own credential, as does each domain on a single provider.

You can create and manage your cluster credentials. Credentials are stored as Kubernetes secrets. Secrets are copied to the namespace of a managed cluster so that the controllers for the managed cluster can access the secrets. When a credential is updated, the copies of the secret are automatically updated in the managed cluster namespaces.

Note: Changes to the pull secret, SSH keys, or base domain of the cloud provider credentials are not

reflected for existing managed clusters, as they have already been provisioned using the original credentials.

Required access: Edit

- [Creating a credential for Amazon Web Services](#)
- [Creating a credential for Microsoft Azure](#)
- [Creating a credential for Google Cloud Platform](#)
- [Creating a credential for VMware vSphere](#)
- [Creating a credential for Red Hat OpenStack Platform](#)
- [Creating a credential for Red Hat OpenShift Cluster Manager](#)
- [Creating a credential for Ansible Automation Platform](#)
- [Creating a credential for an on-premises environment](#)

Creating a credential for Amazon Web Services

You need a credential to use multicluster engine operator console to deploy and manage an Red Hat OpenShift Container Platform cluster on Amazon Web Services (AWS).

Required access: Edit

Note: This procedure must be done before you can create a cluster with multicluster engine operator.

Prerequisites

You must have the following prerequisites before creating a credential:

- A deployed multicluster engine operator hub cluster
- Internet access for your multicluster engine operator hub cluster so it can create the Kubernetes cluster on Amazon Web Services (AWS)
- AWS login credentials, which include access key ID and secret access key. See *Understanding and getting your security credentials*.
- Account permissions that allow installing clusters on AWS. See *Configuring an AWS account* for instructions on how to configure an AWS account.

Managing a credential by using the console

To create a credential from the multicluster engine operator console, complete the steps in the console.

Start at the navigation menu. Click **Credentials** to choose from existing credential options. **Tip:** Create a namespace specifically to host your credentials, both for convenience and added security.

You can optionally add a *Base DNS domain* for your credential. If you add the base DNS domain to the credential, it is automatically populated in the correct field when you create a cluster with this

credential. See the following steps:

1. Add your *AWS access key ID* for your AWS account. See *Log in to AWS* to find your ID.
2. Provide the contents for your new *AWS Secret Access Key*.
3. If you want to enable a proxy, enter the proxy information:
 - HTTP proxy URL: The URL that should be used as a proxy for **HTTP** traffic.
 - HTTPS proxy URL: The secure proxy URL that should be used for **HTTPS** traffic. If no value is provided, the same value as the **HTTP Proxy URL** is used for both **HTTP** and **HTTPS**.
 - No proxy domains: A comma-separated list of domains that should bypass the proxy. Begin a domain name with a period **.** to include all of the subdomains that are in that domain. Add an asterisk ***** to bypass the proxy for all destinations.
 - Additional trust bundle: One or more additional CA certificates that are required for proxying HTTPS connections.
4. Enter your Red Hat OpenShift pull secret. See *Download your Red Hat OpenShift pull secret* to download your pull secret.
5. Add your *SSH private key* and *SSH public key*, which allows you to connect to the cluster. You can use an existing key pair, or create a new one with key generation program.

You can create a cluster that uses this credential by completing the steps in *Creating a cluster on Amazon Web Services* or *Creating a cluster on Amazon Web Services GovCloud*.

You can edit your credential in the console. If the cluster was created by using this provider connection, then the `<cluster-name>-aws-creds` secret from `<cluster-namespace>` will get updated with the new credentials.

Note: Updating credentials does not work for cluster pool claimed clusters.

When you are no longer managing a cluster that is using a credential, delete the credential to protect the information in the credential. Select **Actions** to delete in bulk, or select the options menu beside the credential that you want to delete.

Creating an S3 secret

To create an Amazon Simple Storage Service (S3) secret, complete the following task from the console:

1. Click **Add credential** > **AWS** > **S3 Bucket**. If you click **For Hosted Control Plane**, the name and namespace are provided.
2. Enter information for the following fields that are provided:
 - **bucket name**: Add the name of the S3 bucket.
 - **aws_access_key_id**: Add your AWS access key ID for your AWS account. Log in to AWS to find your ID.
 - **aws_secret_access_key**: Provide the contents for your new AWS Secret Access Key.
 - **Region**: Enter your AWS region.

Creating an opaque secret by using the API

To create an opaque secret for Amazon Web Services by using the API, apply YAML content in the YAML preview window that is similar to the following example:

```
kind: Secret
metadata:
  name: <managed-cluster-name>-aws-creds
  namespace: <managed-cluster-namespace>
type: Opaque
data:
  aws_access_key_id: $(echo -n "${AWS_KEY}" | base64 -w0)
  aws_secret_access_key: $(echo -n "${AWS_SECRET}" | base64 -w0)
```

Notes:

- Opaque secrets are not visible in the console.
- Opaque secrets are created in the managed cluster namespace you chose. Hive uses the opaque secret to provision the cluster. When provisioning the cluster by using the Red Hat Advanced Cluster Management console, the credentials you previously created are copied to the managed cluster namespace as the opaque secret.
- Add labels to your credentials to view your secret in the console. For example, the following AWS S3 Bucket `oc label secret` is appended with `type=awss3` and `credentials --from-file=...`:

```
oc label secret hypershift-operator-oidc-provider-s3-credentials -n <your-local-cluster-name> "cluster.open-cluster-management.io/type=awss3"
oc label secret hypershift-operator-oidc-provider-s3-credentials -n <your-local-cluster-name> "cluster.open-cluster-management.io/credentials=credentials="
```

Additional resources

- See [Understanding and getting your security credentials](#).
- See [Installing on AWS](#).
- [Log in to AWS](#).
- [Download your Red Hat OpenShift pull secret](#).
- See [Installation configuration parameters for AWS](#) for more information about how to generate a key.
- See [Creating a cluster on Amazon Web Services](#).
- See [Creating a cluster on Amazon Web Services GovCloud](#).
- Return to [Creating a credential for Amazon Web Services](#).

Creating a credential for Microsoft Azure

You need a credential to use multicluster engine operator console to create and manage a Red Hat

OpenShift Container Platform cluster on Microsoft Azure or on Microsoft Azure Government.

Required access: Edit

Note: This procedure is a prerequisite for creating a cluster with multicluster engine operator.

Prerequisites

You must have the following prerequisites before creating a credential:

- A deployed multicluster engine operator hub cluster.
- Internet access for your multicluster engine operator hub cluster so that it can create the Kubernetes cluster on Azure.
- Azure login credentials, which include your Base Domain Resource Group and Azure Service Principal JSON. See *Microsoft Azure portal* to get your login credentials.
- Account permissions that allow installing clusters on Azure. See *How to configure Cloud Services* and *Configuring an Azure account* for more information.

Managing a credential by using the console

To create a credential from the multicluster engine operator console, complete the steps in the console. Start at the navigation menu. Click **Credentials** to choose from existing credential options.

Tip: Create a namespace specifically to host your credentials, both for convenience and added security.

1. **Optional:** Add a *Base DNS domain* for your credential. If you add the base DNS domain to the credential, it is automatically populated in the correct field when you create a cluster with this credential.
2. Select whether the environment for your cluster is **AzurePublicCloud** or **AzureUSGovernmentCloud**. The settings are different for the Azure Government environment, so ensure that this is set correctly.
3. Add your *Base domain resource group name* for your Azure account. This entry is the resource name that you created with your Azure account. You can find your Base Domain Resource Group Name by selecting **Home** > **DNS Zones** in the Azure interface. See *Create an Azure service principal with the Azure CLI* to find your base domain resource group name.
4. Provide the contents for your *Client ID*. This value is generated as the **appId** property when you create a service principal with the following command:

```
az ad sp create-for-rbac --role Contributor --name <service_principal> --scopes  
<subscription_path>
```

Replace *service_principal* with the name of your service principal.

5. Add your *Client Secret*. This value is generated as the **password** property when you create a service principal with the following command:

```
az ad sp create-for-rbac --role Contributor --name <service_principal> --scopes
<subscription_path>
```

Replace *service_principal* with the name of your service principal.

6. Add your *Subscription ID*. This value is the **id** property in the output of the following command:

```
az account show
```

7. Add your *Tenant ID*. This value is the **tenantId** property in the output of the following command:

```
az account show
```

8. If you want to enable a proxy, enter the proxy information:

- HTTP proxy URL: The URL that should be used as a proxy for **HTTP** traffic.
- HTTPS proxy URL: The secure proxy URL that should be used for **HTTPS** traffic. If no value is provided, the same value as the **HTTP Proxy URL** is used for both **HTTP** and **HTTPS**.
- No proxy domains: A comma-separated list of domains that should bypass the proxy. Begin a domain name with a period **.** to include all of the subdomains that are in that domain. Add an asterisk ***** to bypass the proxy for all destinations.
- Additional trust bundle: One or more additional CA certificates that are required for proxying HTTPS connections.

9. Enter your *Red Hat OpenShift pull secret*. See *Download your Red Hat OpenShift pull secret* to download your pull secret.

10. Add your *SSH private key* and *SSH public key* to use to connect to the cluster. You can use an existing key pair, or create a new pair using a key generation program.

You can create a cluster that uses this credential by completing the steps in *Creating a cluster on Microsoft Azure*.

You can edit your credential in the console.

When you are no longer managing a cluster that is using a credential, delete the credential to protect the information in the credential. Select **Actions** to delete in bulk, or select the options menu beside the credential that you want to delete.

Creating an opaque secret by using the API

To create an opaque secret for Microsoft Azure by using the API instead of the console, apply YAML content in the YAML preview window that is similar to the following example:

```
kind: Secret
metadata:
  name: <managed-cluster-name>-azure-creds
```



```
namespace: <managed-cluster-namespace>
type: Opaque
data:
  baseDomainResourceGroupName: $(echo -n "${azure_resource_group_name}" | base64
-w0)
  osServicePrincipal.json: $(base64 -w0 "${AZURE_CRED_JSON}")
```

Notes:

- Opaque secrets are not visible in the console.
- Opaque secrets are created in the managed cluster namespace you chose. Hive uses the opaque secret to provision the cluster. When provisioning the cluster by using the Red Hat Advanced Cluster Management console, the credentials you previously created are copied to the managed cluster namespace as the opaque secret.

Additional resources

- See [Microsoft Azure portal](#).
- See [How to configure Cloud Services](#).
- See [Configuring an Azure account](#).
- See [Create an Azure service principal with the Azure CLI](#) to find your base domain resource group name.
- [Download your Red Hat OpenShift pull secret](#).
- See [Generating a key pair for cluster node SSH access](#) for more information about how to generate a key.
- See [Creating a cluster on Microsoft Azure](#).
- Return to [Creating a credential for Microsoft Azure](#).

Creating a credential for Google Cloud Platform

You need a credential to use multicluster engine operator console to create and manage a Red Hat OpenShift Container Platform cluster on Google Cloud Platform (GCP).

Required access: Edit

Note: This procedure is a prerequisite for creating a cluster with multicluster engine operator.

Prerequisites

You must have the following prerequisites before creating a credential:

- A deployed multicluster engine operator hub cluster
- Internet access for your multicluster engine operator hub cluster so it can create the Kubernetes cluster on GCP
- GCP login credentials, which include user Google Cloud Platform Project ID and Google Cloud

Platform service account JSON key. See *Creating and managing projects*.

- Account permissions that allow installing clusters on GCP. See *Configuring a GCP project* for instructions on how to configure an account.

Managing a credential by using the console

To create a credential from the multicluster engine operator console, complete the steps in the console.

Start at the navigation menu. Click **Credentials** to choose from existing credential options. **Tip:** Create a namespace specifically to host your credentials, for both convenience and security.

You can optionally add a *Base DNS domain* for your credential. If you add the base DNS domain to the credential, it is automatically populated in the correct field when you create a cluster with this credential. See the following steps:

1. Add your *Google Cloud Platform project ID* for your GCP account. See *Log in to GCP* to retrieve your settings.
2. Add your *Google Cloud Platform service account JSON key*. See the *Create service accounts* documentation to create your service account JSON key. Follow the steps for the GCP console.
3. Provide the contents for your new *Google Cloud Platform service account JSON key*.
4. If you want to enable a proxy, enter the proxy information:
 - HTTP proxy URL: The URL that should be used as a proxy for **HTTP** traffic.
 - HTTPS proxy URL: The secure proxy URL that should be used for **HTTPS** traffic. If no value is provided, the same value as the **HTTP Proxy URL** is used for both **HTTP** and **HTTPS**.
 - No proxy domains: A comma-separated list of domains that should bypass the proxy. Begin a domain name with a period . to include all of the subdomains that are in that domain. Add and asterisk * to bypass the proxy for all destinations.
 - Additional trust bundle: One or more additional CA certificates that are required for proxying HTTPS connections.
5. Enter your Red Hat OpenShift pull secret. See *Download your Red Hat OpenShift pull secret* to download your pull secret.
6. Add your *SSH private key* and *SSH public key* so you can access the cluster. You can use an existing key pair, or create a new pair using a key generation program.

You can use this connection when you create a cluster by completing the steps in *Creating a cluster on Google Cloud Platform*.

You can edit your credential in the console.

When you are no longer managing a cluster that is using a credential, delete the credential to protect the information in the credential. Select **Actions** to delete in bulk, or select the options menu beside the credential that you want to delete.

Creating an opaque secret by using the API

To create an opaque secret for Google Cloud Platform by using the API instead of the console, apply YAML content in the YAML preview window that is similar to the following example:

```
kind: Secret
metadata:
  name: <managed-cluster-name>-gcp-creds
  namespace: <managed-cluster-namespace>
type: Opaque
data:
  osServiceAccount.json: $(base64 -w0 "${GCP_CRED_JSON}")
```

Notes:

- Opaque secrets are not visible in the console.
- Opaque secrets are created in the managed cluster namespace you chose. Hive uses the opaque secret to provision the cluster. When provisioning the cluster by using the Red Hat Advanced Cluster Management console, the credentials you previously created are copied to the managed cluster namespace as the opaque secret.

Additional resources

- See [Creating and managing projects](#).
- See [Configuring a GCP project](#).
- [Log in to GCP](#).
- See the [Create service accounts](#) to create your service account JSON key.
- [Download your Red Hat OpenShift pull secret](#).
- See [Generating a key pair for cluster node SSH access](#) for more information about how to generate a key.
- See [Creating a cluster on Google Cloud Platform](#).

Return to [Creating a credential for Google Cloud Platform](#).

Creating a credential for VMware vSphere

You need a credential to use multicluster engine operator console to deploy and manage a Red Hat OpenShift Container Platform cluster on VMware vSphere.

Required access: Edit

Prerequisites

You must have the following prerequisites before you create a credential:

- You must create a credential for VMware vSphere before you can create a cluster with multicluster engine operator.

- A deployed hub cluster on a supported OpenShift Container Platform version.
- Internet access for your hub cluster so it can create the Kubernetes cluster on VMware vSphere.
- VMware vSphere login credentials and vCenter requirements configured for OpenShift Container Platform when using installer-provisioned infrastructure. See *Installing a cluster on vSphere with customizations*. These credentials include the following information:
 - vCenter account privileges.
 - Cluster resources.
 - DHCP available.
 - ESXi hosts have time synchronized (for example, NTP).

Managing a credential by using the console

To create a credential from the multicluster engine operator console, complete the steps in the console.

Start at the navigation menu. Click **Credentials** to choose from existing credential options. **Tip:** Create a namespace specifically to host your credentials, both for convenience and added security.

You can optionally add a *Base DNS domain* for your credential. If you add the base DNS domain to the credential, it is automatically populated in the correct field when you create a cluster with this credential. See the following steps:

1. Add your *VMware vCenter server fully-qualified host name or IP address*. The value must be defined in the vCenter server root CA certificate. If possible, use the fully-qualified host name.
2. Add your *VMware vCenter username*.
3. Add your *VMware vCenter password*.
4. Add your *VMware vCenter root CA certificate*.
 - a. You can download your certificate in the `download.zip` package with the certificate from your VMware vCenter server at: https://<vCenter_address>/certs/download.zip. Replace `vCenter_address` with the address to your vCenter server.
 - b. Unpackage the `download.zip`.
 - c. Use the certificates from the `certs/<platform>` directory that have a `.0` extension.

Tip: You can use the `ls certs/<platform>` command to list all of the available certificates for your platform.

Replace `<platform>` with the abbreviation for your platform: `lin`, `mac`, or `win`.

For example: `certs/lin/3a343545.0`

Best practice: Link together multiple certificates with a `.0` extension by running the `cat certs/lin/*.0 > ca.crt` command.

- d. Add your *VMware vSphere cluster name*.
- e. Add your *VMware vSphere datacenter*.

- f. Add your *VMware vSphere default datastore*.
 - g. Add your *VMware vSphere disk type*.
 - h. Add your *VMware vSphere folder*.
 - i. Add your *VMware vSphere resource pool*.
5. For disconnected installations only: Complete the fields in the **Configuration for disconnected installation** subsection with the required information:
- *Cluster OS image*: This value contains the URL to the image to use for Red Hat OpenShift Container Platform cluster machines.
 - *Image content source*: This value contains the disconnected registry path. The path contains the hostname, port, and repository path to all of the installation images for disconnected installations. Example: `repository.com:5000/openshift/ocp-release`.

The path creates an image content source policy mapping in the `install-config.yaml` to the Red Hat OpenShift Container Platform release images. As an example, `repository.com:5000` produces this `imageContentSource` content:

```
- mirrors:
  - registry.example.com:5000/ocp4
    source: quay.io/openshift-release-dev/ocp-release-nightly
- mirrors:
  - registry.example.com:5000/ocp4
    source: quay.io/openshift-release-dev/ocp-release
- mirrors:
  - registry.example.com:5000/ocp4
    source: quay.io/openshift-release-dev/ocp-v4.0-art-dev
```

- *Additional trust bundle*: This value provides the contents of the certificate file that is required to access the mirror registry.

Note: If you are deploying managed clusters from a hub that is in a disconnected environment, and want them to be automatically imported post install, add an Image Content Source Policy to the `install-config.yaml` file by using the **YAML** editor. A sample entry is shown in the following example:

```
- mirrors:
  - registry.example.com:5000/rhacm2
    source: registry.redhat.io/rhacm2
```

6. If you want to enable a proxy, enter the proxy information:
- **HTTP proxy URL**: The URL that should be used as a proxy for **HTTP** traffic.
 - **HTTPS proxy URL**: The secure proxy URL that should be used for **HTTPS** traffic. If no value is provided, the same value as the **HTTP Proxy URL** is used for both **HTTP** and **HTTPS**.
 - **No proxy domains**: A comma-separated list of domains that should bypass the proxy. Begin a domain name with a period `.` to include all of the subdomains that are in that domain. Add

and asterisk ***** to bypass the proxy for all destinations.

- Additional trust bundle: One or more additional CA certificates that are required for proxying HTTPS connections.
- 7. Enter your Red Hat OpenShift pull secret. See *Download your Red Hat OpenShift pull secret* to download your pull secret.
- 8. Add your *SSH private key* and *SSH public key*, which allows you to connect to the cluster.

You can use an existing key pair, or create a new one with key generation program.

You can create a cluster that uses this credential by completing the steps in *Creating a cluster on VMware vSphere*.

You can edit your credential in the console.

When you are no longer managing a cluster that is using a credential, delete the credential to protect the information in the credential. Select **Actions** to delete in bulk, or select the options menu beside the credential that you want to delete.

Creating an opaque secret by using the API

To create an opaque secret for VMware vSphere by using the API instead of the console, apply YAML content in the YAML preview window that is similar to the following example:

```
kind: Secret
metadata:
  name: <managed-cluster-name>-vsphere-creds
  namespace: <managed-cluster-namespace>
type: Opaque
data:
  username: $(echo -n "${VMW_USERNAME}" | base64 -w0)
  password.json: $(base64 -w0 "${VMW_PASSWORD}")
```

Notes:

- Opaque secrets are not visible in the console.
- Opaque secrets are created in the managed cluster namespace you chose. Hive uses the opaque secret to provision the cluster. When provisioning the cluster by using the Red Hat Advanced Cluster Management console, the credentials you previously created are copied to the managed cluster namespace as the opaque secret.

Additional resources

- See [Installing a cluster on vSphere with customizations](#).
- [Download your Red Hat OpenShift pull secret](#).
- See [Generating a key pair for cluster node SSH access](#) for more information.
- See [Creating a cluster on VMware vSphere](#).

- Return to [Creating a credential for VMware vSphere](#).

Creating a credential for Red Hat OpenStack

You need a credential to use multicluster engine operator console to deploy and manage a supported Red Hat OpenShift Container Platform cluster on Red Hat OpenStack Platform.

Notes: You must create a credential for Red Hat OpenStack Platform before you can create a cluster with multicluster engine operator.

Prerequisites

You must have the following prerequisites before you create a credential:

- A deployed hub cluster on a supported OpenShift Container Platform version.
- Internet access for your hub cluster so it can create the Kubernetes cluster on Red Hat OpenStack Platform.
- Red Hat OpenStack Platform login credentials and Red Hat OpenStack Platform requirements configured for OpenShift Container Platform when using installer-provisioned infrastructure. See *Installing a cluster on OpenStack with customizations*.
- Download or create a `clouds.yaml` file for accessing the CloudStack API. Within the `clouds.yaml` file:
 - Determine the cloud auth section name to use.
 - Add a line for the **password**, immediately following the **username** line.

Managing a credential by using the console

To create a credential from the multicluster engine operator console, complete the steps in the console.

Start at the navigation menu. Click **Credentials** to choose from existing credential options. To enhance security and convenience, you can create a namespace specifically to host your credentials.

1. **Optional:** You can add a Base DNS domain for your credential. If you add the base DNS domain, it is automatically populated in the correct field when you create a cluster with this credential.
2. Add your Red Hat OpenStack Platform `clouds.yaml` file contents. The contents of the `clouds.yaml` file, including the password, provide the required information for connecting to the Red Hat OpenStack Platform server. The file contents must include the password, which you add to a new line immediately after the `username`.
3. Add your Red Hat OpenStack Platform cloud name. This entry is the name specified in the cloud section of the `clouds.yaml` to use for establishing communication to the Red Hat OpenStack Platform server.
4. **Optional:** For configurations that use an internal certificate authority, enter your certificate in the **Internal CA certificate** field to automatically update your `clouds.yaml` with the certificate information.

5. For disconnected installations only: Complete the fields in the **Configuration for disconnected installation** subsection with the required information:

- *Cluster OS image*: This value contains the URL to the image to use for Red Hat OpenShift Container Platform cluster machines.
- *Image content sources*: This value contains the disconnected registry path. The path contains the hostname, port, and repository path to all of the installation images for disconnected installations. Example: `repository.com:5000/openshift/ocp-release`.

The path creates an image content source policy mapping in the `install-config.yaml` to the Red Hat OpenShift Container Platform release images. As an example, `repository.com:5000` produces this `imageContentSource` content:

```
- mirrors:
  - registry.example.com:5000/ocp4
    source: quay.io/openshift-release-dev/ocp-release-nightly
- mirrors:
  - registry.example.com:5000/ocp4
    source: quay.io/openshift-release-dev/ocp-release
- mirrors:
  - registry.example.com:5000/ocp4
    source: quay.io/openshift-release-dev/ocp-v4.0-art-dev
```

- *Additional trust bundle*: This value provides the contents of the certificate file that is required to access the mirror registry.

Note: If you are deploying managed clusters from a hub that is in a disconnected environment, and want them to be automatically imported post install, add an Image Content Source Policy to the `install-config.yaml` file by using the **YAML** editor. A sample entry is shown in the following example:

```
- mirrors:
  - registry.example.com:5000/rhacm2
    source: registry.redhat.io/rhacm2
```

6. If you want to enable a proxy, enter the proxy information:

- **HTTP proxy URL**: The URL that should be used as a proxy for **HTTP** traffic.
- **HTTPS proxy URL**: The secure proxy URL that should be used for **HTTPS** traffic. If no value is provided, the same value as the **HTTP Proxy URL** is used for both **HTTP** and **HTTPS**.
- **No proxy domains**: A comma-separated list of domains that should bypass the proxy. Begin a domain name with a period `.` to include all of the subdomains that are in that domain. Add an asterisk `*` to bypass the proxy for all destinations.
- **Additional trust bundle**: One or more additional CA certificates that are required for proxying HTTPS connections.

7. Enter your Red Hat OpenShift pull secret. See *Download your Red Hat OpenShift pull secret* to

download your pull secret.

8. Add your SSH Private Key and SSH Public Key, which allows you to connect to the cluster. You can use an existing key pair, or create a new one with key generation program.
9. Click **Create**.
10. Review the new credential information, then click **Add**. When you add the credential, it is added to the list of credentials.

You can create a cluster that uses this credential by completing the steps in *Creating a cluster on Red Hat OpenStack Platform*.

You can edit your credential in the console.

When you are no longer managing a cluster that is using a credential, delete the credential to protect the information in the credential. Select **Actions** to delete in bulk, or select the options menu beside the credential that you want to delete.

Creating an opaque secret by using the API

To create an opaque secret for Red Hat OpenStack Platform by using the API instead of the console, apply YAML content in the YAML preview window that is similar to the following example:

```
kind: Secret
metadata:
  name: <managed-cluster-name>-osp-creds
  namespace: <managed-cluster-namespace>
type: Opaque
data:
  clouds.yaml: $(base64 -w0 "${OSP_CRED_YAML}") cloud: $(echo -n "openstack" |
base64 -w0)
```

Notes:

- Opaque secrets are not visible in the console.
- Opaque secrets are created in the managed cluster namespace you chose. Hive uses the opaque secret to provision the cluster. When provisioning the cluster by using the Red Hat Advanced Cluster Management console, the credentials you previously created are copied to the managed cluster namespace as the opaque secret.

Additional resources

- See [Installing a cluster on OpenStack with customizations](#).
- [Download your Red Hat OpenShift pull secret](#).
- See [Generating a key pair for cluster node SSH access](#) for more information.
- See [Creating a cluster on Red Hat OpenStack Platform](#).
- Return to [Creating a credential for Red Hat OpenStack](#).

Creating a credential for Red Hat OpenShift Cluster Manager

Add an OpenShift Cluster Manager credential so that you can discover clusters.

Required access: Administrator

Prerequisites

You need an API token for the OpenShift Cluster Manager account, or you can use a separate Service Account.

- To obtain an API token, see [Downloading the OpenShift Cluster Manager API token](#).
- To use a Service Account, you must obtain the client ID and client secret when you are creating the Service Account. Enter the credentials to create the OpenShift Cluster Manager credential on your multicluster engine for Kubernetes operator. See [Creating and managing a service account](#).

Adding a credential by using the console

You need to add your credential to discover clusters. To create a credential from the multicluster engine operator console, complete the steps in the console:

1. Log in to your cluster.
2. Click **Credentials** > **Credential type** to choose from existing credential options.
3. Create a namespace specifically to host your credentials, both for convenience and added security.
4. Click **Add credential**.
5. Select the Red Hat OpenShift Cluster Manager option.
6. Select one of the authentication methods.

Notes:

- When you are no longer managing a cluster that is using a credential, delete the credential to protect the information in the credential.
- If your credential is removed, or your OpenShift Cluster Manager API token expires or is revoked, then the associated discovered clusters are removed.

Creating a credential for Ansible Automation Platform

You need a credential to use multicluster engine operator console to deploy and manage an Red Hat OpenShift Container Platform cluster that is using Red Hat Ansible Automation Platform.

Required access: Edit

Note: This procedure must be done before you can create an Automation template to enable automation on a cluster.

Prerequisites

You must have the following prerequisites before creating a credential:

- A deployed multicluster engine operator hub cluster
- Internet access for your multicluster engine operator hub cluster
- Ansible login credentials, which includes Ansible Automation Platform hostname and OAuth token; see [Credentials for Ansible Automation Platform](#).
- Account permissions that allow you to install hub clusters and work with Ansible. Learn more about [Ansible users](#).

Managing a credential by using the console

To create a credential from the multicluster engine operator console, complete the steps in the console.

Start at the navigation menu. Click **Credentials** to choose from existing credential options. **Tip:** Create a namespace specifically to host your credentials, both for convenience and added security.

The Ansible Token and host URL that you provide when you create your Ansible credential are automatically updated for the automations that use that credential when you edit the credential. The updates are copied to any automations that use that Ansible credential, including those related to cluster lifecycle, governance, and application management automations. This ensures that the automations continue to run after the credential is updated.

You can edit your credential in the console. Ansible credentials are automatically updated in your automation that use that credential when you update them in the credential.

You can create an Ansible Job that uses this credential by completing the steps in [Configuring Ansible Automation Platform tasks to run on managed clusters](#).

When you are no longer managing a cluster that is using a credential, delete the credential to protect the information in the credential. Select **Actions** to delete in bulk, or select the options menu beside the credential that you want to delete.

Creating a credential for an on-premises environment

You need a credential to use the console to deploy and manage a Red Hat OpenShift Container Platform cluster in an on-premises environment. The credential specifies the connections that are used for the cluster.

Required access: Edit

- [Prerequisites](#)
- [Managing a credential by using the console](#)

Prerequisites

You need the following prerequisites before creating a credential:

- A hub cluster that is deployed.
- Internet access for your hub cluster so it can create the Kubernetes cluster on your infrastructure environment.
- For a disconnected environment, you must have a configured mirror registry where you can copy the release images for your cluster creation. See [Mirroring in disconnected environments](#) in the OpenShift Container Platform documentation for more information.
- Account permissions that support installing clusters on the on-premises environment.

Managing a credential by using the console

To create a credential from the console, complete the steps in the console.

Start at the navigation menu. Click **Credentials** to choose from existing credential options. **Tip:** Create a namespace specifically to host your credentials, both for convenience and added security.

1. Select **Host inventory** for your credential type.
2. You can optionally add a *Base DNS domain* for your credential. If you add the base DNS domain to the credential, it is automatically populated in the correct field when you create a cluster with this credential. If you do not add the DNS domain, you can add it when you create your cluster.
3. Enter your *Red Hat OpenShift pull secret*. This pull secret is automatically entered when you create a cluster and specify this credential. You can download your pull secret from [Pull secret](#). See [Using image pull secrets](#) for more information about pull secrets.
4. Enter your **SSH public key**. This **SSH public key** is also automatically entered when you create a cluster and specify this credential.
5. Select **Add** to create your credential.

You can create a cluster that uses this credential by completing the steps in [Creating a cluster in an on-premises environment](#).

When you are no longer managing a cluster that is using a credential, delete the credential to protect the information in the credential. Select **Actions** to delete in bulk, or select the options menu beside the credential that you want to delete.

Cluster lifecycle introduction

The multicluster engine operator is the cluster lifecycle operator that provides cluster management capabilities for OpenShift Container Platform and Red Hat Advanced Cluster Management hub clusters. The multicluster engine operator is a software operator that enhances cluster fleet management and supports OpenShift Container Platform cluster lifecycle management across clouds and data centers. You can use multicluster engine operator with or without Red Hat Advanced Cluster Management. Red Hat Advanced Cluster Management also installs multicluster engine operator automatically and offers further multicluster capabilities.

See the following documentation:

- [Cluster lifecycle architecture](#)
- [Managing credentials overview](#)
- [Release images](#)
- [Creating clusters](#)
- [Cluster import](#)
- [Accessing your cluster](#)
- [Scaling managed clusters](#)
- [Hibernating a created cluster](#)
- [Upgrading your cluster](#)
- [Enabling cluster proxy add-ons](#)
- [Configuring Ansible Automation Platform tasks to run on managed clusters](#)
- [ClusterClaims](#)
- [ManagedClusterSets](#)
- [Placement](#)
- [Managing cluster pools \(Technology Preview\)](#)
- [Enabling ManagedServiceAccount](#)
- [Cluster lifecycle advanced configuration](#)
- [Removing a cluster from management](#)

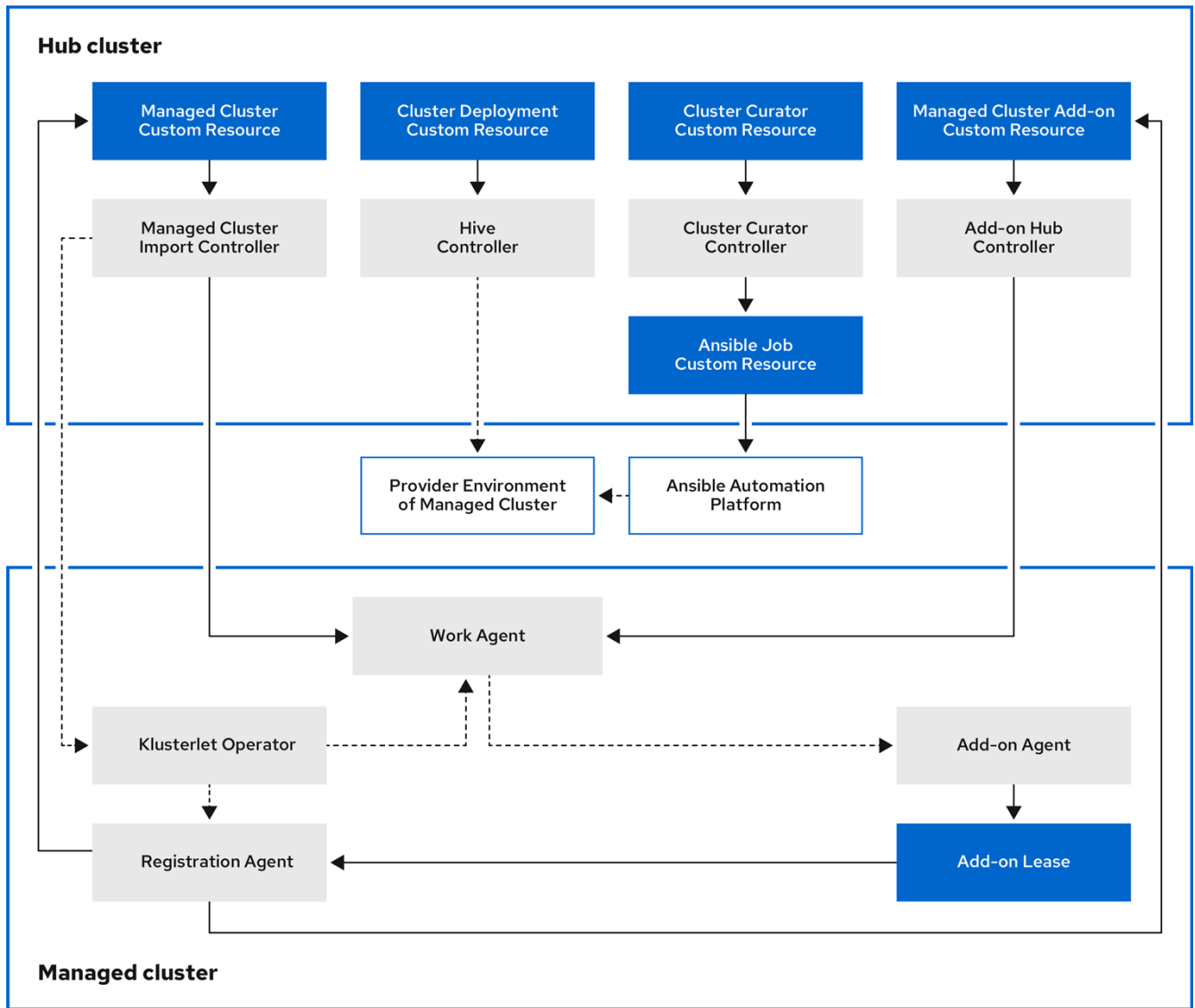
Cluster lifecycle architecture

Cluster lifecycle requires two types of clusters: *hub clusters* and *managed clusters*.

The hub cluster is the OpenShift Container Platform (or Red Hat Advanced Cluster Management) main cluster with the multicluster engine operator automatically installed. You can create, manage, and monitor other Kubernetes clusters with the hub cluster. You can create clusters by using the hub cluster, while you can also import existing clusters to be managed by the hub cluster.

When you create a managed cluster, the cluster is created using the Red Hat OpenShift Container Platform cluster installer with the Hive resource. See more information about the process of installing clusters with the OpenShift Container Platform installer by reading [Installing and configuring OpenShift Container Platform clusters](#) in the OpenShift Container Platform documentation.

The following diagram shows the components that are installed with the multicluster engine for Kubernetes operator for cluster management:



303_RHACM_0223

The components of the cluster lifecycle management architecture include the following items:

Hub cluster

- The *managed cluster import controller* deploys the *kuberneteslet operator* to the managed clusters.
- The *Hive controller* provisions the clusters that you create by using the multicuster engine for Kubernetes operator. The *Hive Controller* also destroys managed clusters that were created by the multicuster engine for Kubernetes operator.
- The *cluster curator controller* creates the Ansible jobs as the pre-hook or post-hook to configure the cluster infrastructure environment when creating or upgrading managed clusters.
- When a managed cluster add-on is enabled on the hub cluster, its *add-on hub controller* is deployed on the hub cluster. The *add-on hub controller* deploys the *add-on agent* to the managed clusters.

Managed cluster

- The *kuberneteslet operator* deploys the *kuberneteslet agent* on the managed cluster.

- The *klusterlet agent* handles both cluster registration and work management functions. It registers the managed cluster and the managed cluster add-ons with the hub cluster, maintains the status of the managed cluster and the managed cluster add-ons, and applies add-on agents to the managed cluster. The following permissions are automatically created within the Clusterrole to allow the managed cluster to access the hub cluster:
 - Allows the agent to get or update its owned cluster that the hub cluster manages
 - Allows the agent to update the status of its owned cluster that the hub cluster manages
 - Allows the agent to rotate its certificate
 - Allows the agent to **get** or **update** the **coordination.k8s.io** lease
 - Allows the agent to **get** its managed cluster add-ons
 - Allows the agent to update the status of its managed cluster add-ons
 - Allows the agent to send events to the hub cluster

To continue adding and managing clusters, see the [Cluster lifecycle introduction](#).

Release images

When you build your cluster, use the version of Red Hat OpenShift Container Platform that the release image specifies. By default, OpenShift Container Platform uses the **clusterImageSets** resources to get the list of supported release images.

Continue reading to learn more about release images:

- [Specifying release images](#)
- [Maintaining a custom list of release images while connected](#)
- [Maintaining a custom list of release images while disconnected](#)
- [Synchronizing available release images](#)

Specifying release images

When you create a cluster on a provider by using multicluster engine for Kubernetes operator, specify a release image to use for your new cluster. To specify a release image, see the following topics:

- [Locating *ClusterImageSets*](#)
- [Configuring *ClusterImageSets*](#)
- [Creating a release image to deploy a cluster on a different architecture](#)

Locating *ClusterImageSets*

The YAML files referencing the release images are maintained in the *acm-hive-openshift-releases* GitHub repository. The files are used to create the list of the available release images in the console. This includes the latest fast channel images from OpenShift Container Platform.

The console only displays the latest release images for the three latest versions of OpenShift

Container Platform. For example, you might see the following release image displayed in the console options, with <4.x> changed to the proper release version that you are using:

`quay.io/openshift-release-dev/ocp-release:<4.x>.1-x86_64`

The console displays the latest versions to help you create a cluster with the latest release images. If you need to create a cluster that is a specific version, older release image versions are also available.

Note: You can only select images with the `visible: 'true'` label when creating clusters in the console. An example of this label in a `ClusterImageSet` resource is provided in the following content. Replace `4.x.1` with the current version of the product:

```
apiVersion: hive.openshift.io/v1
kind: ClusterImageSet
metadata:
  labels:
    channel: fast
    visible: 'true'
  name: img4.x.1-x86-64-appsub
spec:
  releaseImage: quay.io/openshift-release-dev/ocp-release:4.x.1-x86_64
```

Additional release images are stored, but are not visible in the console. To view all of the available release images, run the following command:

```
oc get clusterimageset
```

The repository has the `clusterImageSets` directory, which is the directory that you use when working with the release images. The `clusterImageSets` directory has the following directories:

- **Fast:** Contains files that reference the latest versions of the release images for each supported OpenShift Container Platform version. The release images in this folder are tested, verified, and supported.
- **Releases:** Contains files that reference all of the release images for each OpenShift Container Platform version (stable, fast, and candidate channels)

Note: These releases have not all been tested and determined to be stable.

- **Stable:** Contains files that reference the latest two stable versions of the release images for each supported OpenShift Container Platform version..

Note: By default, the current list of release images updates one time every hour. After upgrading the product, it might take up to one hour for the list to reflect the recommended release image versions for the new version of the product.

Configuring *ClusterImageSets*

You can configure your *ClusterImageSets* with the following options:

- Option 1: To create a cluster in the console, specify the image reference for the specific *ClusterImageSet* that you want to use. Each new entry you specify persists and is available for all future cluster provisions. See the following example entry:

```
quay.io/openshift-release-dev/ocp-release:4.6.8-x86_64
```

- Option 2: Manually create and apply a *ClusterImageSets* YAML file from the [acm-hive-openshift-releases](#) GitHub repository.
- Option 3: To enable automatic updates of *ClusterImageSets* from a forked GitHub repository, follow the [README.md](#) in the *cluster-image-set-controller* GitHub repository.

Creating a release image to deploy a cluster on a different architecture

You can create a cluster on an architecture that is different from the architecture of the hub cluster by manually creating a release image that has the files for both architectures.

For example, you might need to create an *x86_64* cluster from a hub cluster that is running on the *ppc64le*, *aarch64*, or *s390x* architecture. If you create the release image with both sets of files, the cluster creation succeeds because the new release image enables the OpenShift Container Platform release registry to provide a multi-architecture image manifest.

OpenShift Container Platform supports multiple architectures by default. You can use the following *clusterImageSet* to provision a cluster. Replace *4.x.0* with the current supported version:

```
apiVersion: hive.openshift.io/v1
kind: ClusterImageSet
metadata:
  labels:
    channel: fast
    visible: 'true'
    name: img4.x.0-multi-appsub
spec:
  releaseImage: quay.io/openshift-release-dev/ocp-release:4.x.0-multi
```

To create the release image for OpenShift Container Platform images that do not support multiple architectures, complete steps similar to the following example for your architecture type:

1. From the [OpenShift Container Platform release registry](#), create a [manifest list](#) that includes *x86_64*, *s390x*, *aarch64*, and *ppc64le* release images.
 - a. Pull the manifest lists for both architectures in your environment from the [Quay repository](#) by running the following example commands. Replace *4.x.1* with the current version of the product:

```
podman pull quay.io/openshift-release-dev/ocp-release:4.x.1-x86_64
podman pull quay.io/openshift-release-dev/ocp-release:4.x.1-ppc64le
podman pull quay.io/openshift-release-dev/ocp-release:4.x.1-s390x
podman pull quay.io/openshift-release-dev/ocp-release:4.x.1-aarch64
```

- b. Log in to your private repository where you maintain your images by running the following command. Replace `<private-repo>` with the path to your repository:

```
podman login <private-repo>
```

- c. Add the release image manifest to your private repository by running the following commands that apply to your environment. Replace `4.x.1` with the current version of the product. Replace `<private-repo>` with the path to your repository:

```
podman push quay.io/openshift-release-dev/ocp-release:4.x.1-x86_64 <private-repo>/ocp-release:4.x.1-x86_64
podman push quay.io/openshift-release-dev/ocp-release:4.x.1-ppc64le <private-repo>/ocp-release:4.x.1-ppc64le
podman push quay.io/openshift-release-dev/ocp-release:4.x.1-s390x <private-repo>/ocp-release:4.x.1-s390x
podman push quay.io/openshift-release-dev/ocp-release:4.x.1-aarch64 <private-repo>/ocp-release:4.x.1-aarch64
```

- d. Create a manifest for the new information by running the following command:

```
podman manifest create mymanifest
```

- e. Add references to both release images to the manifest list by running the following commands. Replace `4.x.1` with the current version of the product. Replace `<private-repo>` with the path to your repository:

```
podman manifest add mymanifest <private-repo>/ocp-release:4.x.1-x86_64
podman manifest add mymanifest <private-repo>/ocp-release:4.x.1-ppc64le
podman manifest add mymanifest <private-repo>/ocp-release:4.x.1-s390x
podman manifest add mymanifest <private-repo>/ocp-release:4.x.1-aarch64
```

- f. Merge the list in your manifest list with the existing manifest by running the following command. Replace `<private-repo>` with the path to your repository. Replace `4.x.1` with the current version:

```
podman manifest push mymanifest docker://<private-repo>/ocp-release:4.x.1
```

2. On the hub cluster, create a release image that references the manifest in your repository.

- a. Create a YAML file that contains information that is similar to the following example. Replace `<private-repo>` with the path to your repository. Replace `4.x.1` with the current version:

```
apiVersion: hive.openshift.io/v1
kind: ClusterImageSet
metadata:
  labels:
    channel: fast
    visible: "true"
    name: img4.x.1-appsub
spec:
  releaseImage: <private-repo>/ocp-release:4.x.1
```

- b. Run the following command on your hub cluster to apply the changes. Replace `<file-name>` with the name of the YAML file that you created in the previous step:

```
oc apply -f <file-name>.yaml
```

3. Select the new release image when you create your OpenShift Container Platform cluster.
4. If you deploy the managed cluster by using the Red Hat Advanced Cluster Management console, specify the architecture for the managed cluster in the *Architecture* field during the cluster creation process.

The creation process uses the merged release images to create the cluster.

Additional resources

- See the [acm-hive-openshift-releases](#) GitHub repository for the YAML files that reference the release images.
- See the [cluster-image-set-controller GitHub repository](#) to learn how to enable automatic updates of `ClusterImageSets` resources from a forked GitHub repository.

Maintaining a custom list of release images when connected

You might want to use the same release image for all of your clusters. To simplify, create your own custom list of release images that are available when creating a cluster. Complete the following steps to manage your available release images:

1. Fork the [acm-hive-openshift-releases](#) GitHub.
2. Add the YAML files for the images that you want available when you create a cluster. Add the images to the `./clusterImageSets/stable/` or `./clusterImageSets/fast/` directory by using the Git console or the terminal.
3. Create a `ConfigMap` file in the `multicluster-engine` namespace that is named `cluster-image-set-git-repo`. See the following example, but replace `mce-version` with your current version of multicluster engine operator, for example, `2.11`:

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: cluster-image-set-git-repo
  namespace: multicloud-engine
data:
  gitRepoUrl: ❶
  gitRepoBranch: backplane-<mce-version>
  gitRepoPath: clusterImageSets
  channel: ❷
  insecureSkipVerify: ❸
  caCerts: ❹

```

- ❶ Use the URL that you received when you forked the [acm-hive-openshift-releases](#) repository.
- ❷ Choose to use either of the supported channels values, [fast](#) or [stable](#).
- ❸ Set the [insecureSkipVerify](#) specification to [true](#) to build the insecure connection. The supported values are [true](#) or [false](#).
- ❹ List the certified authority (CA) certificates to build a secure HTTPS connection.

You can retrieve the available YAML files from the main repository by merging changes in to your forked repository with the following procedure:

1. Commit and merge your changes to your forked repository.
2. To synchronize your list of fast release images after you clone the [acm-hive-openshift-releases](#) repository, update the value of channel field in the [cluster-image-set-git-repo](#) ConfigMap to [fast](#).
3. To synchronize and display the stable release images, update the value of channel field in the [cluster-image-set-git-repo](#) ConfigMap to [stable](#).

After updating the [ConfigMap](#), the list of available stable release images updates with the currently available images in about one minute.

1. You can use the following commands to list what is available and remove the defaults. Replace [clusterImageSet_NAME](#) with the correct name:

```

oc get clusterImageSets
oc delete clusterImageSet <clusterImageSet_NAME>

```

2. View the list of currently available release images in the console when you are creating a cluster.

For information regarding other fields available through the [ConfigMap](#), view the [cluster-image-set-controller GitHub repository README](#).

Maintaining a custom list of release images while disconnected

In some cases, you need to maintain a custom list of release images when the hub cluster has no

Internet connection. You can create your own custom list of release images that are available when creating a cluster. Complete the following steps to manage your available release images while disconnected:

1. When you are on a connected system, go to the [acm-hive-openshift-releases GitHub repository](#) to access the available cluster image sets.
2. Copy the `clusterImageSets` directory to a system that can access the disconnected multicluster engine operator cluster.
3. Add the mapping between the managed cluster and the disconnected repository with your cluster image sets by completing the following steps that fits your managed cluster:
 - For an OpenShift Container Platform managed cluster, see [Configuring image registry repository mirroring](#) for information about using your `ImageContentSourcePolicy` object to complete the mapping.
 - For a managed cluster that is not an OpenShift Container Platform cluster, use the `ManageClusterImageRegistry` custom resource definition to override the location of the image sets. See [Specifying registry images on managed clusters for import](#) for information about how to override the cluster for the mapping.
4. Add the YAML files for the images that you want available when you create a cluster by using the console or CLI to manually add the `clusterImageSet` YAML content.
5. Modify the `clusterImageSet` YAML files for the remaining OpenShift Container Platform release images to reference the correct offline repository where you store the images. Your updates resemble the following example where `spec.releaseImage` uses your offline image registry of the release image, and the release image is referenced by digest:

```
apiVersion: hive.openshift.io/v1
kind: ClusterImageSet
metadata:
  labels:
    channel: fast
    name: img<4.x.x>-x86-64-appsub
spec:
  releaseImage: IMAGE_REGISTRY_IPADDRESS_or__DNSNAME/REPO_PATH/ocp-
  release@sha256:073a4e46289be25e2a05f5264c8f1d697410db66b960c9ceeddeb1c61e58717
```

6. Ensure that the images are loaded in the offline image registry that is referenced in the YAML file.
7. Obtain the image digest by running the following command:

```
oc adm release info <tagged_openshift_release_image> | grep "Pull From"
```

Replace `<tagged_openshift_release_image>` with the tagged image for the supported OpenShift Container Platform version. See the following example output:

```
Pull From: quay.io/openshift-release-dev/ocp-
```

```
release@sha256:69d1292f64a2b67227c5592c1a7d499c7d00376e498634ff8e1946bc9ccdddfc
```

To learn more about the image tag and digest, see [Referencing images in imagestreams](#).

8. Create each of the `clusterImageSets` by entering the following command for each YAML file:

```
oc create -f <clusterImageSet_FILE>
```

Replace `clusterImageSet_FILE` with the name of the cluster image set file. For example:

```
oc create -f img4.11.9-x86_64.yaml
```

After running this command for each resource you want to add, the list of available release images are available.

9. Alternately you can paste the image URL directly in the create cluster console. Adding the image URL creates new `clusterImageSets` if they do not exist.
10. View the list of currently available release images in the console when you are creating a cluster.

Synchronizing available release images

If you have the Red Hat Advanced Cluster Management hub cluster, which uses the `MultiClusterHub` operator to manage, upgrade, and install hub cluster components, you can synchronize the list of release images to ensure that you can select the latest available versions. Release images are available in the [acm-hive-openshift-releases](#) repository and are updated frequently.

Stability levels

There are three levels of stability of the release images, as displayed in the following table:

Table 7. Stability levels of release images

Category	Description
candidate	The most current images, which are not tested and might have some bugs.
fast	Images that are partially tested, but likely less stable than a stable version.
stable	These fully-tested images are confirmed to install and build clusters correctly.

Refreshing the release images list

Complete the following steps to refresh and synchronize the list of images by using a Linux or Mac operating system:

1. If the installer-managed `acm-hive-openshift-releases` subscription is enabled, disable the

subscription by setting the value of `disableUpdateClusterImageSets` to `true` in the `MultiClusterHub` resource.

2. Clone the [acm-hive-openshift-releases](#) GitHub repository.
3. Remove the subscription by running the following command:

```
oc delete -f subscribe/subscription-fast
```

4. To synchronize and display the `candidate` release images, run the following command by using a Linux or Mac operating system:

```
make subscribe-candidate
```

After about one minute, the latest list of `candidate` release images is available.

5. To synchronize and display the `fast` release images, run the following command:

```
make subscribe-fast
```

After about one minute, the latest list of `fast` release images is available.

6. Connect to the `stable` release images and synchronize your Red Hat Advanced Cluster Management hub cluster. Run the following command using a Linux or Mac operating system:

```
make subscribe-stable
```

After about one minute, the list of available `candidate`, `fast`, and `stable` release images updates with the currently available images.

7. View the list of currently available release images in the Red Hat Advanced Cluster Management console when you are creating a cluster.
8. Unsubscribe from any of these channels to stop viewing the updates by running the following command:

```
oc delete -f subscribe/subscription-fast
```

Creating clusters

Learn how to create Red Hat OpenShift Container Platform clusters across cloud providers with multicluster engine operator.

multicluster engine operator uses the Hive operator that is provided with OpenShift Container Platform to provision clusters for all providers except the on-premises clusters and hosted control planes. When provisioning the on-premises clusters, multicluster engine operator uses the central

infrastructure management and Assisted Installer function that are provided with OpenShift Container Platform. The hosted clusters for hosted control planes are provisioned by using the HyperShift operator.

- [Configuring additional manifests during cluster creation](#)
- [Creating a cluster on Amazon Web Services](#)
- [Creating a cluster on Amazon Web Services GovCloud](#)
- [Creating a cluster on Microsoft Azure](#)
- [Creating a cluster on Google Cloud Platform](#)
- [Creating a cluster on VMware vSphere](#)
- [Creating a cluster on Red Hat OpenStack Platform](#)
- [Creating a cluster in an on-premises environment](#)
- [Creating a cluster in a proxy environment](#)
- [Configuring *AgentClusterInstall* proxy](#)

Creating a cluster with the CLI

The multicluster engine for Kubernetes operator uses internal Hive components to create Red Hat OpenShift Container Platform clusters. See the following information to learn how to create clusters.

- [Prerequisites](#)
- [Create a cluster with ClusterDeployment](#)
- [Create a cluster with cluster pool](#)

Prerequisites

- Make sure the ClusterImageSet resources are available on the hub cluster by running the following command:

```
oc get clusterimageset -A
```

The output might resemble the following example:

```
img4.18.4-multi-appsub    quay.io/openshift-release-dev/ocp-release:4.18.4-multi
img4.18.4-x86-64-appsub   quay.io/openshift-release-dev/ocp-release:4.18.4-x86_64
```

Note: Each ClusterImageSet references an OpenShift Container Platform release image. The ClusterImageSet resources are populated by the `cluster-image-set-controller` in multicluster engine for Kubernetes operator.

- If you use the Nutanix platform, use the `x86_64` architecture for the `releaseImage` in the `ClusterImageSet` resource and set the `visible` label value to `'true'`. See the following example:


```
apiVersion: hive.openshift.io/v1
kind: ClusterImageSet
metadata:
  labels:
    channel: stable
    visible: 'true'
  name: img4.x.47-x86-64-appsub
spec:
  releaseImage: quay.io/openshift-release-dev/ocp-release:4.x.47-x86_64
```

- Review the hub cluster **KubeAPIServer** certificate verification strategy and update the strategy if needed. To learn what strategy to use for your setup, see [Configuring the hub cluster KubeAPIServer verification strategy](#).

Create a cluster with ClusterDeployment

A **ClusterDeployment** is a Hive custom resource that is used to control the lifecycle of a cluster.

Follow the [Using Hive](#) documentation to create the **ClusterDeployment** custom resource and create an individual cluster.

Create a cluster with ClusterPool

A **ClusterPool** is also a Hive custom resource that is used to create multiple clusters.

Follow the [Cluster Pools](#) documentation to create a cluster with the Hive **ClusterPool** API.

Configuring additional manifests during cluster creation

You can configure additional Kubernetes resource manifests during the installation process of creating your cluster. This can help if you need to configure additional manifests for scenarios such as configuring networking or setting up a load balancer.

Prerequisites

Add a reference to the **ClusterDeployment** resource that specifies a config map resource that contains the additional resource manifests.

Note: The **ClusterDeployment** resource and the config map must be in the same namespace.

Configuring additional manifests during cluster creation by using examples

If you want to configure additional manifests by using a config map with resource manifests, complete the following steps:

1. Create a YAML file and add the following example content:

```
kind: ConfigMap
apiVersion: v1
metadata:
```

```

name: <my-baremetal-cluster-install-manifests>
namespace: <mynamespace>
data:
  99_metal3-config.yaml: |
    kind: ConfigMap
    apiVersion: v1
    metadata:
      name: metal3-config
      namespace: openshift-machine-api
    data:
      http_port: "6180"
      provisioning_interface: "enp1s0"
      provisioning_ip: "172.00.0.3/24"
      dhcp_range: "172.00.0.10,172.00.0.100"
      deploy_kernel_url: "http://172.00.0.3:6180/images/ironic-python-agent.kernel"
      deploy_ramdisk_url: "http://172.00.0.3:6180/images/ironic-python-
agent.initramfs"
      ironic_endpoint: "http://172.00.0.3:6385/v1/"
      ironic_inspector_endpoint: "http://172.00.0.3:5150/v1/"
      cache_url: "http://192.168.111.1/images"
      rhcos_image_url: "https://releases-art-
rhcos.svc.ci.openshift.org/art/storage/releases/rhcos-
4.3/43.81.201911192044.0/x86_64/rhcos-43.81.201911192044.0-
openstack.x86_64.qcow2.gz"

```

Note: The example `ConfigMap` contains a manifest with another `ConfigMap` resource. The resource manifest `ConfigMap` can contain multiple keys with resource configurations added in the following pattern, `data.<resource_name>\.yaml`.

2. Apply the file by running the following command:

```
oc apply -f <filename>.yaml
```

If you want to configure additional manifests by using a `ClusterDeployment` by referencing a resource manifest `ConfigMap`, complete the following steps:

3. Create a YAML file and add the following example content. The resource manifest `ConfigMap` is referenced in `spec.provisioning.manifestsConfigMapRef`:

```

apiVersion: hive.openshift.io/v1
kind: ClusterDeployment
metadata:
  name: <my-baremetal-cluster>
  namespace: <mynamespace>
  annotations:
    hive.openshift.io/try-install-once: "true"
spec:
  baseDomain: test.example.com
  clusterName: <my-baremetal-cluster>

```

```

controlPlaneConfig:
  servingCertificates: {}
platform:
  baremetal:
    libvirtSSHPrivateKeySecretRef:
      name: provisioning-host-ssh-private-key
  provisioning:
    installConfigSecretRef:
      name: <my-baremetal-cluster-install-config>
    sshPrivateKeySecretRef:
      name: <my-baremetal-hosts-ssh-private-key>
    manifestsConfigMapRef:
      name: <my-baremetal-cluster-install-manifests>
    imageSetRef:
      name: <my-clusterimageset>
    sshKnownHosts:
      - "10.1.8.90 ecdsa-sha2-nistp256
AAAAE2VjZHNhLXvVVVKUYVkuYvkuYgkuyTCYTYtftkufTYAAAAIbmlzdHAyNTYAAABBBKWjJRzeUVuZs4yxS
y4eu45xiANFIbWE3e1aPzGD58x/NX7Yf+S8eFKq4RrsfSaK2hVJyJjvVIhUsU9z2sBJP8="
    pullSecretRef:
      name: <my-baremetal-cluster-pull-secret>

```

4. Apply the file by running the following command:

```
oc apply -f <filename>.yaml
```

Creating a cluster on Amazon Web Services

You can use the multicluster engine operator console to create a Red Hat OpenShift Container Platform cluster on Amazon Web Services (AWS).

When you create a cluster, the creation process uses the OpenShift Container Platform installer with the Hive resource. If you have questions about cluster creation after completing this procedure, see [Installing on AWS](#) in the OpenShift Container Platform documentation for more information about the process.

- [Prerequisites](#)
- [Creating your AWS cluster](#)
- [Creating your cluster with the console](#)

Prerequisites

See the following prerequisites before creating a cluster on AWS:

- You must have a deployed hub cluster.
- You need an AWS credential. See [Creating a credential for Amazon Web Services](#) for more information.

- You need a configured domain in AWS. See [Installing on AWS](#) for instructions on how to configure a domain.
- You must have Amazon Web Services (AWS) login credentials, which include user name, password, access key ID, and secret access key. See [Understanding and Getting Your Security Credentials](#).
- You must have an OpenShift Container Platform image pull secret. See [Using image pull secrets](#).

Note: If you change your cloud provider access key on the cloud provider, you also need to manually update the corresponding credential for the cloud provider on the console. This is required when your credentials expire on the cloud provider where the managed cluster is hosted and you try to delete the managed cluster.

Creating your AWS cluster

See the following important information about creating an AWS cluster:

- When you review your information and optionally customize it before creating the cluster, you can select **YAML: On** to view the `install-config.yaml` file content in the panel. You can edit the YAML file with your custom settings, if you have any updates.
- When you create a cluster, the controller creates a namespace for the cluster and the resources. Ensure that you include only resources for that cluster instance in that namespace.
- *Destroying* the cluster deletes the namespace and all of the resources in it.
- If you want to add your cluster to an existing cluster set, you must have the correct permissions on the cluster set to add it. If you do not have `cluster-admin` privileges when you are creating the cluster, you must select a cluster set on which you have `clusterset-admin` permissions.
- If you do not have the correct permissions on the specified cluster set, the cluster creation fails. Contact your cluster administrator to provide you with `clusterset-admin` permissions to a cluster set if you do not have any cluster set options to select.
- Every managed cluster must be associated with a managed cluster set. If you do not assign the managed cluster to a `ManagedClusterSet`, it is automatically added to the `default` managed cluster set.
- If there is already a base DNS domain that is associated with the selected credential that you configured with your AWS account, that value is populated in the field. You can change the value by overwriting it. This name is used in the hostname of the cluster.
- The release image identifies the version of the OpenShift Container Platform image that is used to create the cluster. Select the image from the list of images that are available. If the image that you want to use is not available, you can enter the URL to the image that you want to use.
- The node pools include the control plane pool and the worker pools. The control plane nodes share the management of the cluster activity. The information includes the following fields:
 - Region: Specify the region where you want the node pool.
 - CPU architecture: If the architecture type of the managed cluster is not the same as the architecture of your hub cluster, enter a value for the instruction set architecture of the machines in the pool. Valid values are `amd64`, `ppc64le`, `s390x`, and `arm64`.
 - Zones: Specify where you want to run your control plane pools. You can select multiple

zones within the region for a more distributed group of control plane nodes. A closer zone might provide faster performance, but a more distant zone might be more distributed.

- Instance type: Specify the instance type for your control plane node. You can change the type and size of your instance after it is created.
- Root storage: Specify the amount of root storage to allocate for the cluster.
- You can create zero or more worker nodes in a worker pool to run the container workloads for the cluster. This can be in a single worker pool, or distributed across multiple worker pools. If zero worker nodes are specified, the control plane nodes also function as worker nodes. The optional information includes the following fields:
 - Zones: Specify where you want to run your worker pools. You can select multiple zones within the region for a more distributed group of nodes. A closer zone might provide faster performance, but a more distant zone might be more distributed.
 - Instance type: Specify the instance type of your worker pools. You can change the type and size of your instance after it is created.
 - Node count: Specify the node count of your worker pool. This setting is required when you define a worker pool.
 - Root storage: Specify the amount of root storage allocated for your worker pool. This setting is required when you define a worker pool.
- Networking details are required for your cluster, and multiple networks are required for using IPv6. You can add an additional network by clicking **Add network**.
- Proxy information that is provided in the credential is automatically added to the proxy fields. You can use the information as it is, overwrite it, or add the information if you want to enable a proxy. The following list contains the required information for creating a proxy:
 - HTTP proxy: Specify the URL that should be used as a proxy for **HTTP** traffic.
 - HTTPS proxy: Specify the secure proxy URL that should be used for **HTTPS** traffic. If no value is provided, the same value as the **HTTP Proxy URL** is used for both **HTTP** and **HTTPS**.
 - No proxy sites: A comma-separated list of sites that should bypass the proxy. Begin a domain name with a period **.** to include all of the subdomains that are in that domain. Add an asterisk ***** to bypass the proxy for all destinations.
 - Additional trust bundle: One or more additional CA certificates that are required for proxying HTTPS connections.

Creating your cluster with the console

To create a new cluster, see the following procedure. If you have an existing cluster that you want to *import* instead, see [Cluster import](#).

Note: You do not have to run the **oc** command that is provided with the cluster details to import the cluster. When you create the cluster, it is automatically configured under the management of multicluster engine operator.

1. Navigate to **Infrastructure > Clusters**.
2. On the *Clusters* page. Click **Cluster > Create cluster** and complete the steps in the console.

3. **Optional:** Select **YAML: On** to view content updates as you enter the information in the console.

If you need to create a credential, see [Creating a credential for Amazon Web Services](#) for more information.

The name of the cluster is used in the hostname of the cluster.

If you are using Red Hat Advanced Cluster Management for Kubernetes and want to configure your managed cluster `klusterlet` to run on specific nodes, see [Optional: Configuring the `klusterlet` to run on specific nodes](#) for the required steps.

Additional resources

- The AWS private configuration information is used when you are creating an AWS GovCloud cluster. See [Creating a cluster on Amazon Web Services GovCloud](#) for information about creating a cluster in that environment.
- See [Installing on AWS](#) for more information.
- See [Release images](#) for more information about release images.
- Find more information about supported instant types by visiting your cloud provider sites, such as [AWS General purpose instances](#).

Creating a cluster on Amazon Web Services GovCloud

You can use the console to create a Red Hat OpenShift Container Platform cluster on Amazon Web Services (AWS) or on AWS GovCloud. This procedure explains how to create a cluster on AWS GovCloud. See [Creating a cluster on Amazon Web Services](#) for the instructions for creating a cluster on AWS.

AWS GovCloud provides cloud services that meet additional requirements that are necessary to store government documents on the cloud. When you create a cluster on AWS GovCloud, you must complete additional steps to prepare your environment.

When you create a cluster, the creation process uses the OpenShift Container Platform installer with the Hive resource. If you have questions about cluster creation after completing this procedure, see [Installing a cluster on AWS into a government region](#) in the OpenShift Container Platform documentation for more information about the process. The following sections provide the steps for creating a cluster on AWS GovCloud:

- [Prerequisites](#)
- [Configure Hive to deploy on AWS GovCloud](#)
- [Creating your cluster with the console](#)

Prerequisites

You must have the following prerequisites before creating an AWS GovCloud cluster:

- You must have AWS login credentials, which include user name, password, access key ID, and secret access key. See [Understanding and Getting Your Security Credentials](#).

- You need an AWS credential. See [Creating a credential for Amazon Web Services](#) for more information.
- You need a configured domain in AWS. See [Installing on AWS](#) for instructions on how to configure a domain.
- You must have an OpenShift Container Platform image pull secret. See [Using image pull secrets](#).
- You must have an Amazon Virtual Private Cloud (VPC) with an existing Red Hat OpenShift Container Platform cluster for the hub cluster. This VPC must be different from the VPCs that are used for the managed cluster resources or the managed cluster service endpoints.
- You need a VPC where the managed cluster resources are deployed. This cannot be the same as the VPCs that are used for the hub cluster or the managed cluster service endpoints.
- You need one or more VPCs that provide the managed cluster service endpoints. This cannot be the same as the VPCs that are used for the hub cluster or the managed cluster resources.
- Ensure that the IP addresses of the VPCs that are specified by Classless Inter-Domain Routing (CIDR) do not overlap.
- You need a **HiveConfig** custom resource that references a credential within the Hive namespace. This custom resource must have access to create resources on the VPC that you created for the managed cluster service endpoints.

Note: If you change your cloud provider access key on the cloud provider, you also need to manually update the corresponding credential for the cloud provider on the multicluster engine operator console. This is required when your credentials expire on the cloud provider where the managed cluster is hosted and you try to delete the managed cluster.

Configure Hive to deploy on AWS GovCloud

While creating a cluster on AWS GovCloud is almost identical to creating a cluster on standard AWS, you have to complete some additional steps to prepare an AWS PrivateLink for the cluster on AWS GovCloud.

Create the VPCs for resources and endpoints

As listed in the prerequisites, two VPCs are required in addition to the VPC that contains the hub cluster. See [Create a VPC](#) in the Amazon Web Services documentation for specific steps for creating a VPC.

1. Create a VPC for the managed cluster with private subnets.
2. Create one or more VPCs for the managed cluster service endpoints with private subnets. Each VPC in a region has a limit of 255 VPC endpoints, so you need multiple VPCs to support more than 255 clusters in that region.
3. For each VPC, create subnets in all of the supported availability zones of the region. Each subnet must have at least 255 usable IP addresses because of the controller requirements.

The following example shows how you might structure subnets for VPCs that have 6 availability zones in the **us-gov-east-1** region:


```
vpc-1 (us-gov-east-1) : 10.0.0.0/20
  subnet-11 (us-gov-east-1a): 10.0.0.0/23
  subnet-12 (us-gov-east-1b): 10.0.2.0/23
  subnet-13 (us-gov-east-1c): 10.0.4.0/23
  subnet-12 (us-gov-east-1d): 10.0.8.0/23
  subnet-12 (us-gov-east-1e): 10.0.10.0/23
  subnet-12 (us-gov-east-1f): 10.0.12.0/2
```

```
vpc-2 (us-gov-east-1) : 10.0.16.0/20
  subnet-21 (us-gov-east-1a): 10.0.16.0/23
  subnet-22 (us-gov-east-1b): 10.0.18.0/23
  subnet-23 (us-gov-east-1c): 10.0.20.0/23
  subnet-24 (us-gov-east-1d): 10.0.22.0/23
  subnet-25 (us-gov-east-1e): 10.0.24.0/23
  subnet-26 (us-gov-east-1f): 10.0.28.0/23
```

4. Ensure that all of the hub environments (hub cluster VPCs) have network connectivity to the VPCs that you created for VPC endpoints that use peering, transit gateways, and that all DNS settings are enabled.
5. Collect a list of VPCs that are needed to resolve the DNS setup for the AWS PrivateLink, which is required for the AWS GovCloud connectivity. This includes at least the VPC of the multicluster engine operator instance that you are configuring, and can include the list of all of the VPCs where various Hive controllers exist.

Configure the security groups for the VPC endpoints

Each VPC endpoint in AWS has a security group attached to control access to the endpoint. When Hive creates a VPC endpoint, it does not specify a security group. The default security group of the VPC is attached to the VPC endpoint. The default security group of the VPC must have rules to allow traffic where VPC endpoints are created from the Hive installer pods. See [Control access to VPC endpoints using endpoint policies](#) in the AWS documentation for details.

For example, if Hive is running in `hive-vpc(10.1.0.0/16)`, there must be a rule in the default security group of the VPC where the VPC endpoint is created that allows ingress from `10.1.0.0/16`.

Set permissions for AWS PrivateLink

You need multiple credentials to configure the AWS PrivateLink. The required permissions for these credentials depend on the type of credential.

- The credentials for ClusterDeployment require the following permissions:

```
ec2:CreateVpcEndpointServiceConfiguration
ec2:DescribeVpcEndpointServiceConfigurations
ec2:ModifyVpcEndpointServiceConfiguration
ec2:DescribeVpcEndpointServicePermissions
ec2:ModifyVpcEndpointServicePermissions
```



```
ec2:DeleteVpcEndpointServiceConfigurations
```

- The credentials for HiveConfig for endpoint VPCs account `.spec.awsPrivateLink.credentialsSecretRef` require the following permissions:

```
ec2:DescribeVpcEndpointServices
ec2:DescribeVpcEndpoints
ec2:CreateVpcEndpoint
ec2:CreateTags
ec2:DescribeNetworkInterfaces
ec2:DescribeVPCs

ec2:DeleteVpcEndpoints

route53:CreateHostedZone
route53:GetHostedZone
route53:ListHostedZonesByVPC
route53:AssociateVPCWithHostedZone
route53:DisassociateVPCFromHostedZone
route53:CreateVPCAssociationAuthorization
route53>DeleteVPCAssociationAuthorization
route53:ListResourceRecordSets
route53:ChangeResourceRecordSets

route53>DeleteHostedZone
```

- The credentials specified in the `HiveConfig` custom resource for associating VPCs to the private hosted zone (`.spec.awsPrivateLink.associatedVPCs[$idx].credentialsSecretRef`). The account where the VPC is located requires the following permissions:

```
route53:AssociateVPCWithHostedZone
route53:DisassociateVPCFromHostedZone
ec2:DescribeVPCs
```

Ensure that there is a credential secret within the Hive namespace on the hub cluster.

The `HiveConfig` custom resource needs to reference a credential within the Hive namespace that has permissions to create resources in a specific provided VPC. If the credential that you are using to provision an AWS cluster in AWS GovCloud is already in the Hive namespace, then you do not need to create another one. If the credential that you are using to provision an AWS cluster in AWS GovCloud is not already in the Hive namespace, you can either replace your current credential or create an additional credential in the Hive namespace.

The `HiveConfig` custom resource needs to include the following content:

- An AWS GovCloud credential that has the required permissions to provision resources for the given VPC.

- The addresses of the VPCs for the OpenShift Container Platform cluster installation, as well as the service endpoints for the managed cluster.

Best practice: Use different VPCs for the OpenShift Container Platform cluster installation and the service endpoints.

The following example shows the credential content:

```
spec:
  awsPrivateLink:
    ## The list of inventory of VPCs that can be used to create VPC
    ## endpoints by the controller.
    endpointVPCInventory:
      - region: us-east-1
        vpcID: vpc-1
        subnets:
          - availabilityZone: us-east-1a
            subnetID: subnet-11
          - availabilityZone: us-east-1b
            subnetID: subnet-12
          - availabilityZone: us-east-1c
            subnetID: subnet-13
          - availabilityZone: us-east-1d
            subnetID: subnet-14
          - availabilityZone: us-east-1e
            subnetID: subnet-15
          - availabilityZone: us-east-1f
            subnetID: subnet-16
      - region: us-east-1
        vpcID: vpc-2
        subnets:
          - availabilityZone: us-east-1a
            subnetID: subnet-21
          - availabilityZone: us-east-1b
            subnetID: subnet-22
          - availabilityZone: us-east-1c
            subnetID: subnet-23
          - availabilityZone: us-east-1d
            subnetID: subnet-24
          - availabilityZone: us-east-1e
            subnetID: subnet-25
          - availabilityZone: us-east-1f
            subnetID: subnet-26
    ## The credentialsSecretRef references a secret with permissions to create.
    ## The resources in the account where the inventory of VPCs exist.
    credentialsSecretRef:
      name: <hub-account-credentials-secret-name>

    ## A list of VPC where various mce clusters exists.
    associatedVPCs:
```

```
- region: region-mce1
  vpcID: vpc-mce1
  credentialsSecretRef:
    name: <credentials-that-have-access-to-account-where-MCE1-VPC-exists>
- region: region-mce2
  vpcID: vpc-mce2
  credentialsSecretRef:
    name: <credentials-that-have-access-to-account-where-MCE2-VPC-exists>
```

You can include a VPC from all the regions where AWS PrivateLink is supported in the [endpointVPCInventory](#) list. The controller selects a VPC that meets the requirements for the `ClusterDeployment`.

For more information, refer to the [Hive documentation](#).

Creating your cluster with the console

To create a cluster from the console, navigate to **Infrastructure > Clusters > Create cluster AWS > Standalone** and complete the steps in the console.

Note: This procedure is for creating a cluster. If you have an existing cluster that you want to import, see [Cluster import](#) for those steps.

The credential that you select must have access to the resources in an AWS GovCloud region, if you create an AWS GovCloud cluster. You can use an AWS GovCloud secret that is already in the Hive namespace if it has the required permissions to deploy a cluster. Existing credentials are displayed in the console. If you need to create a credential, see [Creating a credential for Amazon Web Services](#) for more information.

The name of the cluster is used in the hostname of the cluster.

Important: When you create a cluster, the controller creates a namespace for the cluster and its resources. Ensure that you include only resources for that cluster instance in that namespace. Destroying the cluster deletes the namespace and all of the resources in it.

Tip: Select **YAML: On** to view content updates as you enter the information in the console.

If you want to add your cluster to an existing cluster set, you must have the correct permissions on the cluster set to add it. If you do not have `cluster-admin` privileges when you are creating the cluster, you must select a cluster set on which you have `cluster-set-admin` permissions. If you do not have the correct permissions on the specified cluster set, the cluster creation fails. Contact your cluster administrator to provide you with `cluster-set-admin` permissions to a cluster set if you do not have any cluster set options to select.

Every managed cluster must be associated with a managed cluster set. If you do not assign the managed cluster to a `ManagedClusterSet`, it is automatically added to the `default` managed cluster set.

If there is already a base DNS domain that is associated with the selected credential that you configured with your AWS or AWS GovCloud account, that value is populated in the field. You can

change the value by overwriting it. This name is used in the hostname of the cluster. See [Installing on AWS](#) for more information.

The release image identifies the version of the OpenShift Container Platform image that is used to create the cluster. If the version that you want to use is available, you can select the image from the list of images. If the image that you want to use is not a standard image, you can enter the URL to the image that you want to use. See [Release images](#) for more information about release images.

The node pools include the control plane pool and the worker pools. The control plane nodes share the management of the cluster activity. The information includes the following fields:

- **Region:** The region where you create your cluster resources. If you are creating a cluster on an AWS GovCloud provider, you must include an AWS GovCloud region for your node pools. For example, `us-gov-west-1`.
- **CPU architecture:** If the architecture type of the managed cluster is not the same as the architecture of your hub cluster, enter a value for the instruction set architecture of the machines in the pool. Valid values are `amd64`, `ppc64le`, `s390x`, and `arm64`.
- **Zones:** Specify where you want to run your control plane pools. You can select multiple zones within the region for a more distributed group of control plane nodes. A closer zone might provide faster performance, but a more distant zone might be more distributed.
- **Instance type:** Specify the instance type for your control plane node, which must be the same as the *CPU architecture* that you previously indicated. You can change the type and size of your instance after it is created.
- **Root storage:** Specify the amount of root storage to allocate for the cluster.

You can create zero or more worker nodes in a worker pool to run the container workloads for the cluster. They can be in a single worker pool, or distributed across multiple worker pools. If zero worker nodes are specified, the control plane nodes also function as worker nodes. The optional information includes the following fields:

- **Pool name:** Provide a unique name for your pool.
- **Zones:** Specify where you want to run your worker pools. You can select multiple zones within the region for a more distributed group of nodes. A closer zone might provide faster performance, but a more distant zone might be more distributed.
- **Instance type:** Specify the instance type of your worker pools. You can change the type and size of your instance after it is created.
- **Node count:** Specify the node count of your worker pool. This setting is required when you define a worker pool.
- **Root storage:** Specify the amount of root storage allocated for your worker pool. This setting is required when you define a worker pool.

Networking details are required for your cluster, and multiple networks are required for using IPv6. For an AWS GovCloud cluster, enter the values of the block of addresses of the Hive VPC in the *Machine CIDR* field. You can add an additional network by clicking **Add network**.

Proxy information that is provided in the credential is automatically added to the proxy fields. You

can use the information as it is, overwrite it, or add the information if you want to enable a proxy. The following list contains the required information for creating a proxy:

- **HTTP proxy URL:** Specify the URL that should be used as a proxy for **HTTP** traffic.
- **HTTPS proxy URL:** Specify the secure proxy URL that should be used for **HTTPS** traffic. If no value is provided, the same value as the **HTTP Proxy URL** is used for both **HTTP** and **HTTPS**.
- **No proxy domains:** A comma-separated list of domains that should bypass the proxy. Begin a domain name with a period **.** to include all of the subdomains that are in that domain. Add an asterisk ***** to bypass the proxy for all destinations.
- **Additional trust bundle:** One or more additional CA certificates that are required for proxying **HTTPS** connections.

When creating an AWS GovCloud cluster or using a private environment, complete the fields on the *AWS private configuration* page with the AMI ID and the subnet values. Ensure that the value of `spec:platform:aws:privateLink:enabled` is set to `true` in the `ClusterDeployment.yaml` file, which is automatically set when you select **Use private configuration**.

When you review your information and optionally customize it before creating the cluster, you can select **YAML: On** to view the `install-config.yaml` file content in the panel. You can edit the YAML file with your custom settings, if you have any updates.

Note: You do not have to run the `oc` command that is provided with the cluster details to import the cluster. When you create the cluster, it is automatically configured under the management of multicluster engine for Kubernetes operator.

If you are using Red Hat Advanced Cluster Management for Kubernetes and want to configure your managed cluster kubernetes to run on specific nodes, see [Optional: Configuring the kubernetes to run on specific nodes](#) for the required steps.

Continue with [Accessing your cluster](#) for instructions for accessing your cluster.

Creating a cluster on Microsoft Azure

You can use the multicluster engine operator console to deploy a Red Hat OpenShift Container Platform cluster on Microsoft Azure or on Microsoft Azure Government.

When you create a cluster, the creation process uses the OpenShift Container Platform installer with the Hive resource. If you have questions about cluster creation after completing this procedure, see [Installing on Azure](#) in the OpenShift Container Platform documentation for more information about the process.

- [Prerequisites](#)
- [Creating your cluster with the console](#)

Prerequisites

See the following prerequisites before creating a cluster on Azure:

- You must have a deployed hub cluster.

- You need an Azure credential. See [Creating a credential for Microsoft Azure](#) for more information.
- You need a configured domain in Azure or Azure Government. See [Configuring a custom domain name for an Azure cloud service](#) for instructions on how to configure a domain.
- You need Azure login credentials, which include user name and password. See the [Microsoft Azure Portal](#).
- You need Azure service principals, which include `clientId`, `clientSecret`, and `tenantId`. See [azure.microsoft.com](#).
- You need an OpenShift Container Platform image pull secret. See [Using image pull secrets](#).

Note: If you change your cloud provider access key on the cloud provider, you also need to manually update the corresponding credential for the cloud provider on the console of multicluster engine operator. This is required when your credentials expire on the cloud provider where the managed cluster is hosted and you try to delete the managed cluster.

Creating your cluster with the console

To create a cluster from the multicluster engine operator console, navigate to **Infrastructure > Clusters**. On the *Clusters* page, click **Create cluster** and complete the steps in the console.

Note: This procedure is for creating a cluster. If you have an existing cluster that you want to import, see [Cluster import](#) for those steps.

If you need to create a credential, see [Creating a credential for Microsoft Azure](#) for more information.

The name of the cluster is used in the hostname of the cluster.

Important: When you create a cluster, the controller creates a namespace for the cluster and its resources. Ensure that you include only resources for that cluster instance in that namespace. Destroying the cluster deletes the namespace and all of the resources in it.

Tip: Select **YAML: On** to view content updates as you enter the information in the console.

If you want to add your cluster to an existing cluster set, you must have the correct permissions on the cluster set to add it. If you do not have `cluster-admin` privileges when you are creating the cluster, you must select a cluster set on which you have `clusterset-admin` permissions. If you do not have the correct permissions on the specified cluster set, the cluster creation fails. Contact your cluster administrator to provide you with `clusterset-admin` permissions to a cluster set if you do not have any cluster set options to select.

Every managed cluster must be associated with a managed cluster set. If you do not assign the managed cluster to a `ManagedClusterSet`, it is automatically added to the `default` managed cluster set.

If there is already a base DNS domain that is associated with the selected credential that you configured for your Azure account, that value is populated in that field. You can change the value by overwriting it. See [Configuring a custom domain name for an Azure cloud service](#) for more information. This name is used in the hostname of the cluster.

The release image identifies the version of the OpenShift Container Platform image that is used to create the cluster. If the version that you want to use is available, you can select the image from the list of images. If the image that you want to use is not a standard image, you can enter the URL to the image that you want to use. See [Release images](#) for more information about release images.

The Node pools include the control plane pool and the worker pools. The control plane nodes share the management of the cluster activity. The information includes the following optional fields:

- **Region:** Specify a region where you want to run your node pools. You can select multiple zones within the region for a more distributed group of control plane nodes. A closer zone might provide faster performance, but a more distant zone might be more distributed.
- **CPU architecture:** If the architecture type of the managed cluster is not the same as the architecture of your hub cluster, enter a value for the instruction set architecture of the machines in the pool. Valid values are *amd64*, *ppc64le*, *s390x*, and *arm64*.

You can change the type and size of the Instance type and Root storage allocation (required) of your control plane pool after your cluster is created.

You can create one or more worker nodes in a worker pool to run the container workloads for the cluster. They can be in a single worker pool, or distributed across multiple worker pools. If zero worker nodes are specified, the control plane nodes also function as worker nodes. The information includes the following fields:

- **Zones:** Specifies here you want to run your worker pools. You can select multiple zones within the region for a more distributed group of nodes. A closer zone might provide faster performance, but a more distant zone might be more distributed.
- **Instance type:** You can change the type and size of your instance after it is created.

You can add an additional network by clicking **Add network**. You must have more than one network if you are using IPv6 addresses.

Proxy information that is provided in the credential is automatically added to the proxy fields. You can use the information as it is, overwrite it, or add the information if you want to enable a proxy. The following list contains the required information for creating a proxy:

- **HTTP proxy:** The URL that should be used as a proxy for **HTTP** traffic.
- **HTTPS proxy:** The secure proxy URL that should be used for **HTTPS** traffic. If no value is provided, the same value as the **HTTP Proxy URL** is used for both **HTTP** and **HTTPS**.
- **No proxy:** A comma-separated list of domains that should bypass the proxy. Begin a domain name with a period . to include all of the subdomains that are in that domain. Add an asterisk * to bypass the proxy for all destinations.
- **Additional trust bundle:** One or more additional CA certificates that are required for proxying HTTPS connections.

When you review your information and optionally customize it before creating the cluster, you can click the **YAML** switch **On** to view the `install-config.yaml` file content in the panel. You can edit the YAML file with your custom settings, if you have any updates.

If you are using Red Hat Advanced Cluster Management for Kubernetes and want to configure your managed cluster `klusterlet` to run on specific nodes, see [Optional: Configuring the `klusterlet` to run on specific nodes](#) for the required steps.

Note: You do not have to run the `oc` command that is provided with the cluster details to import the cluster. When you create the cluster, it is automatically configured under the management of multicluster engine operator.

Continue with [Accessing your cluster](#) for instructions for accessing your cluster.

Creating a cluster on Google Cloud Platform

Follow the procedure to create a Red Hat OpenShift Container Platform cluster on Google Cloud Platform (GCP). For more information about GCP, see [Google Cloud Platform](#).

When you create a cluster, the creation process uses the OpenShift Container Platform installer with the Hive resource. If you have questions about cluster creation after completing this procedure, see [Installing on GCP](#) in the OpenShift Container Platform documentation for more information about the process.

- [Prerequisites](#)
- [Creating your cluster with the console](#)

Prerequisites

See the following prerequisites before creating a cluster on GCP:

- You must have a deployed hub cluster.
- You must have a GCP credential. See [Creating a credential for Google Cloud Platform](#) for more information.
- You must have a configured domain in GCP. See [Setting up a custom domain](#) for instructions on how to configure a domain.
- You need your GCP login credentials, which include user name and password.
- You must have an OpenShift Container Platform image pull secret. See [Using image pull secrets](#).

Note: If you change your cloud provider access key on the cloud provider, you also need to manually update the corresponding credential for the cloud provider on the console of multicluster engine operator. This is required when your credentials expire on the cloud provider where the managed cluster is hosted and you try to delete the managed cluster.

Creating your cluster with the console

To create clusters from the multicluster engine operator console, navigate to **Infrastructure > Clusters**. On the *Clusters* page, click **Create cluster** and complete the steps in the console.

Note: This procedure is for creating a cluster. If you have an existing cluster that you want to import, see [Cluster import](#) for those steps.

If you need to create a credential, see [Creating a credential for Google Cloud Platform](#) for more

information.

The name of your cluster is used in the hostname of the cluster. There are some restrictions that apply to naming your GCP cluster. These restrictions include not beginning the name with `goog` or containing a group of letters and numbers that resemble `google` anywhere in the name. See [Bucket naming guidelines](#) for the complete list of restrictions.

Important: When you create a cluster, the controller creates a namespace for the cluster and its resources. Ensure that you include only resources for that cluster instance in that namespace. Destroying the cluster deletes the namespace and all of the resources in it.

Tip: Select **YAML: On** to view content updates as you enter the information in the console.

If you want to add your cluster to an existing cluster set, you must have the correct permissions on the cluster set to add it. If you do not have `cluster-admin` privileges when you are creating the cluster, you must select a cluster set on which you have `cluster-set-admin` permissions. If you do not have the correct permissions on the specified cluster set, the cluster creation fails. Contact your cluster administrator to provide you with `cluster-set-admin` permissions to a cluster set if you do not have any cluster set options to select.

Every managed cluster must be associated with a managed cluster set. If you do not assign the managed cluster to a `ManagedClusterSet`, it is automatically added to the `default` managed cluster set.

If there is already a base DNS domain that is associated with the selected credential for your GCP account, that value is populated in the field. You can change the value by overwriting it. See [Setting up a custom domain](#) for more information. This name is used in the hostname of the cluster.

The release image identifies the version of the OpenShift Container Platform image that is used to create the cluster. If the version that you want to use is available, you can select the image from the list of images. If the image that you want to use is not a standard image, you can enter the URL to the image that you want to use. See [Release images](#) for more information about release images.

The Node pools include the control plane pool and the worker pools. The control plane nodes share the management of the cluster activity. The information includes the following fields:

- **Region:** Specify a region where you want to run your control plane pools. A closer region might provide faster performance, but a more distant region might be more distributed.
- **CPU architecture:** If the architecture type of the managed cluster is not the same as the architecture of your hub cluster, enter a value for the instruction set architecture of the machines in the pool. Valid values are `amd64`, `ppc64le`, `s390x`, and `arm64`.

You can specify the instance type of your control plane pool. You can change the type and size of your instance after it is created.

You can create one or more worker nodes in a worker pool to run the container workloads for the cluster. They can be in a single worker pool, or distributed across multiple worker pools. If zero worker nodes are specified, the control plane nodes also function as worker nodes. The information includes the following fields:

- Instance type: You can change the type and size of your instance after it is created.
- Node count: This setting is required when you define a worker pool.

The networking details are required, and multiple networks are required for using IPv6 addresses. You can add an additional network by clicking **Add network**.

Proxy information that is provided in the credential is automatically added to the proxy fields. You can use the information as it is, overwrite it, or add the information if you want to enable a proxy. The following list contains the required information for creating a proxy:

- HTTP proxy: The URL that should be used as a proxy for **HTTP** traffic.
- HTTPS proxy: The secure proxy URL that should be used for **HTTPS** traffic. If no value is provided, the same value as the **HTTP Proxy URL** is used for both **HTTP** and **HTTPS**.
- No proxy sites: A comma-separated list of sites that should bypass the proxy. Begin a domain name with a period **.** to include all of the subdomains that are in that domain. Add an asterisk ***** to bypass the proxy for all destinations.
- Additional trust bundle: One or more additional CA certificates that are required for proxying HTTPS connections.

When you review your information and optionally customize it before creating the cluster, you can select **YAML: On** to view the **install-config.yaml** file content in the panel. You can edit the YAML file with your custom settings, if you have any updates.

If you are using Red Hat Advanced Cluster Management for Kubernetes and want to configure your managed cluster klusterlet to run on specific nodes, see [Optional: Configuring the klusterlet to run on specific nodes](#) for the required steps.

Note: You do not have to run the **oc** command that is provided with the cluster details to import the cluster. When you create the cluster, it is automatically configured under the management of multicluster engine operator.

Continue with [Accessing your cluster](#) for instructions for accessing your cluster.

Creating a cluster on VMware vSphere

You can use the multicluster engine operator console to deploy a Red Hat OpenShift Container Platform cluster on VMware vSphere.

When you create a cluster, the creation process uses the OpenShift Container Platform installer with the Hive resource. To learn more about cluster creation after completing this procedure, see [Installing on VMware vSphere](#) in the OpenShift Container Platform documentation.

- [Prerequisites](#)
- [Creating your cluster with the console](#)

Prerequisites

See the following prerequisites before creating a cluster on vSphere:

- You must have a hub cluster that is deployed on a supported OpenShift Container Platform version.
- You need a vSphere credential. See [Creating a credential for VMware vSphere](#) for more information.
- You need an OpenShift Container Platform image pull secret. See [Using image pull secrets](#).
- You must have the following information for the VMware instance where you are deploying:
 - Required static IP addresses for API and Ingress instances
 - DNS records for:
 - The following API base domain must point to the static API VIP:

```
api.<cluster_name>.<base_domain>
```

- The following application base domain must point to the static IP address for Ingress VIP: `*.apps.<cluster_name>.<base_domain>`

Creating your cluster with the console

To create a cluster from the multicluster engine operator console, navigate to **Infrastructure > Clusters**. On the *Clusters* page, click **Create cluster** and complete the steps in the console.

Note: This procedure is for creating a cluster. If you have an existing cluster that you want to import, see [Cluster import](#) for those steps.

If you need to create a credential, see [Creating a credential for VMware vSphere](#) for more information about creating a credential.

The name of your cluster is used in the hostname of the cluster.

Important: When you create a cluster, the controller creates a namespace for the cluster and its resources. Ensure that you include only resources for that cluster instance in that namespace. Destroying the cluster deletes the namespace and all of the resources in it.

Tip: Select **YAML: On** to view content updates as you enter the information in the console.

If you want to add your cluster to an existing cluster set, you must have the correct permissions on the cluster set to add it. If you do not have `cluster-admin` privileges when you are creating the cluster, you must select a cluster set on which you have `cluster-set-admin` permissions. If you do not have the correct permissions on the specified cluster set, the cluster creation fails. Contact your cluster administrator to provide you with `cluster-set-admin` permissions to a cluster set if you do not have any cluster set options to select.

Every managed cluster must be associated with a managed cluster set. If you do not assign the managed cluster to a `ManagedClusterSet`, it is automatically added to the `default` managed cluster set.

If there is already a base domain associated with the selected credential that you configured for your vSphere account, that value is populated in the field. You can change the value by overwriting

it. See [Installing a cluster on vSphere with customizations](#) for more information. This value must match the name that you used to create the DNS records listed in the prerequisites section. This name is used in the hostname of the cluster.

The release image identifies the version of the OpenShift Container Platform image that is used to create the cluster. If the version that you want to use is available, you can select the image from the list of images. If the image that you want to use is not a standard image, you can enter the URL to the image that you want to use. See [Release images](#) for more information about release images.

Note: Choose release images for supported OpenShift Container Platform versions.

The node pools include the control plane pool and the worker pools. The control plane nodes share the management of the cluster activity. The information includes the *CPU architecture* field. View the following field description:

- **CPU architecture:** If the architecture type of the managed cluster is not the same as the architecture of your hub cluster, enter a value for the instruction set architecture of the machines in the pool. Valid values are *amd64*, *ppc64le*, *s390x*, and *arm64*.

You can create one or more worker nodes in a worker pool to run the container workloads for the cluster. They can be in a single worker pool, or distributed across multiple worker pools. If zero worker nodes are specified, the control plane nodes also function as worker nodes. The information includes *Cores per socket*, *CPUs*, *Memory_min MiB*, *_Disk size* in GiB, and *Node count*.

Networking information is required. Multiple networks are required for using IPv6. Some of the required networking information is included the following fields:

- **vSphere network name:** Specify the VMware vSphere network name.
- **API VIP:** Specify the IP address to use for internal API communication.

Note: This value must match the name that you used to create the DNS records listed in the prerequisites section. If not provided, the DNS must be pre-configured so that `api.` resolves correctly.

- **Ingress VIP:** Specify the IP address to use for ingress traffic.

Note: This value must match the name that you used to create the DNS records listed in the prerequisites section. If not provided, the DNS must be pre-configured so that `test.apps.` resolves correctly.

You can add an additional network by clicking **Add network**. You must have more than one network if you are using IPv6 addresses.

Proxy information that is provided in the credential is automatically added to the proxy fields. You can use the information as it is, overwrite it, or add the information if you want to enable a proxy. The following list contains the required information for creating a proxy:

- **HTTP proxy:** Specify the URL that should be used as a proxy for **HTTP** traffic.
- **HTTPS proxy:** Specify the secure proxy URL that should be used for **HTTPS** traffic. If no value is provided, the same value as the **HTTP Proxy URL** is used for both **HTTP** and **HTTPS**.

- No proxy sites: Provide a comma-separated list of sites that should bypass the proxy. Begin a domain name with a period . to include all of the subdomains that are in that domain. Add an asterisk * to bypass the proxy for all destinations.
- Additional trust bundle: One or more additional CA certificates that are required for proxying HTTPS connections.

Click **Disconnected installation** to define the disconnected installation. When creating a cluster by using VMware vSphere provider and disconnected installation, if a certificate is required to access the mirror registry, you must enter it in the *Additional trust bundle* field in the *Configuration for disconnected installation* section when configuring your credential, or the *Disconnected installation* section when creating a cluster.

You can click **Add automation template** to create a template.

When you review your information and optionally customize it before creating the cluster, you can click the **YAML** switch **On** to view the `install-config.yaml` file content in the panel. You can edit the YAML file with your custom settings, if you have any updates.

If you are using Red Hat Advanced Cluster Management for Kubernetes and want to configure your managed cluster kubernetes to run on specific nodes, see [Optional: Configuring the kubernetes to run on specific nodes](#) for the required steps.

Note: You do not have to run the `oc` command that is provided with the cluster details to import the cluster. When you create the cluster, it is automatically configured under the management of multicluster engine operator.

Continue with [Accessing your cluster](#) for instructions for accessing your cluster.

Creating a cluster on Red Hat OpenStack Platform

You can use the multicluster engine operator console to deploy a Red Hat OpenShift Container Platform cluster on Red Hat OpenStack Platform.

When you create a cluster, the creation process uses the OpenShift Container Platform installer with the Hive resource. If you have questions about cluster creation after completing this procedure, see [Installing on OpenStack](#) in the OpenShift Container Platform documentation for more information about the process.

- [Prerequisites](#)
- [Creating your cluster with the console](#)

Prerequisites

See the following prerequisites before creating a cluster on Red Hat OpenStack Platform:

- You must have a hub cluster that is deployed on OpenShift Container Platform version 4.6 or later.
- You must have a Red Hat OpenStack Platform credential. See [Creating a credential for Red Hat OpenStack Platform](#) for more information.

- You need an OpenShift Container Platform image pull secret. See [Using image pull secrets](#).
- You need the following information for the Red Hat OpenStack Platform instance where you are deploying:
 - Flavor name for the control plane and worker instances; for example, `m1.xlarge`
 - Network name for the external network to provide the floating IP addresses
 - Required floating IP addresses for API and ingress instances
 - DNS records for:
 - The following API base domain must point to the floating IP address for the API:

```
api.<cluster_name>.<base_domain>
```

- The following application base domain must point to the floating IP address for ingress:app-name:

```
*.apps.<cluster_name>.<base_domain>
```

Creating your cluster with the console

To create a cluster from the multicluster engine operator console, navigate to **Infrastructure > Clusters**. On the *Clusters* page, click **Create cluster** and complete the steps in the console.

Note: This procedure is for creating a cluster. If you have an existing cluster that you want to import, see [Cluster import](#) for those steps.

If you need to create a credential, see [Creating a credential for Red Hat OpenStack Platform](#) for more information.

The name of the cluster is used in the hostname of the cluster. The name must contain fewer than 15 characters. This value must match the name that you used to create the DNS records listed in the credential prerequisites section.

Important: When you create a cluster, the controller creates a namespace for the cluster and its resources. Ensure that you include only resources for that cluster instance in that namespace. Destroying the cluster deletes the namespace and all of the resources in it.

Tip: Select **YAML: On** to view content updates as you enter the information in the console.

If you want to add your cluster to an existing cluster set, you must have the correct permissions on the cluster set to add it. If you do not have `cluster-admin` privileges when you are creating the cluster, you must select a cluster set on which you have `clusterset-admin` permissions. If you do not have the correct permissions on the specified cluster set, the cluster creation fails. Contact your cluster administrator to provide you with `clusterset-admin` permissions to a cluster set if you do not have any cluster set options to select.

Every managed cluster must be associated with a managed cluster set. If you do not assign the

managed cluster to a `ManagedClusterSet`, it is automatically added to the `default` managed cluster set.

If there is already a base DNS domain that is associated with the selected credential that you configured for your Red Hat OpenStack Platform account, that value is populated in the field. You can change the value by overwriting it. See [Managing domains](#) in the Red Hat OpenStack Platform documentation for more information. This name is used in the hostname of the cluster.

The release image identifies the version of the OpenShift Container Platform image that is used to create the cluster. If the version that you want to use is available, you can select the image from the list of images. If the image that you want to use is not a standard image, you can enter the URL to the image that you want to use. See [Release images](#) for more information about release images. Only release images for OpenShift Container Platform versions 4.6.x and higher are supported.

The node pools include the control plane pool and the worker pools. The control plane nodes share the management of the cluster activity. If the architecture type of the managed cluster is not the same as the architecture of your hub cluster, enter a value for the instruction set architecture of the machines in the pool. Valid values are *amd64*, *ppc64le*, *s390x*, and *arm64*.

You must add an instance type for your control plane pool, but you can change the type and size of your instance after it is created.

You can create one or more worker nodes in a worker pool to run the container workloads for the cluster. They can be in a single worker pool, or distributed across multiple worker pools. If zero worker nodes are specified, the control plane nodes also function as worker nodes. The information includes the following fields:

- Instance type: You can change the type and size of your instance after it is created.
- Node count: Specify the node count for your worker pool. This setting is required when you define a worker pool.

Networking details are required for your cluster. You must provide the values for one or more networks for an IPv4 network. For an IPv6 network, you must define more than one network.

You can add an additional network by clicking **Add network**. You must have more than one network if you are using IPv6 addresses.

Proxy information that is provided in the credential is automatically added to the proxy fields. You can use the information as it is, overwrite it, or add the information if you want to enable a proxy. The following list contains the required information for creating a proxy:

- HTTP proxy: Specify the URL that should be used as a proxy for **HTTP** traffic.
- HTTPS proxy: The secure proxy URL that should be used for **HTTPS** traffic. If no value is provided, the same value as the **HTTP Proxy** is used for both **HTTP** and **HTTPS**.
- No proxy: Define a comma-separated list of sites that should bypass the proxy. Begin a domain name with a period `.` to include all of the subdomains that are in that domain. Add an asterisk `*` to bypass the proxy for all destinations.
- Additional trust bundle: One or more additional CA certificates that are required for proxying

HTTPS connections.

You can define the disconnected installation image by clicking **Disconnected installation**. When creating a cluster by using Red Hat OpenStack Platform provider and disconnected installation, if a certificate is required to access the mirror registry, you must enter it in the *Additional trust bundle* field in the *Configuration for disconnected installation* section when configuring your credential or the *Disconnected installation* section when creating a cluster.

When you review your information and optionally customize it before creating the cluster, you can click the **YAML** switch **On** to view the `install-config.yaml` file content in the panel. You can edit the YAML file with your custom settings, if you have any updates.

When creating a cluster that uses an internal certificate authority (CA), you need to customize the YAML file for your cluster by completing the following steps:

1. With the **YAML** switch on at the review step, insert a **Secret** object at the top of the list with the CA certificate bundle. **Note:** If the Red Hat OpenStack Platform environment provides services using certificates signed by multiple authorities, the bundle must include the certificates to validate all of the required endpoints. The addition for a cluster named `ocp3` resembles the following example:

```
apiVersion: v1
kind: Secret
type: Opaque
metadata:
  name: ocp3-openstack-trust
  namespace: ocp3
stringData:
  ca.crt: |
    -----BEGIN CERTIFICATE-----
    <Base64 certificate contents here>
    -----END CERTIFICATE-----
    -----BEGIN CERTIFICATE-----
    <Base64 certificate contents here>
    -----END CERTIFICATE-----
```

2. Modify the Hive **ClusterDeployment** object to specify the value of `certificatesSecretRef` in `spec.platform.openstack`, similar to the following example:

```
platform:
  openstack:
    certificatesSecretRef:
      name: ocp3-openstack-trust
    credentialsSecretRef:
      name: ocp3-openstack-creds
    cloud: openstack
```

The previous example assumes that the cloud name in the `clouds.yaml` file is `openstack`.

If you are using Red Hat Advanced Cluster Management for Kubernetes and want to configure your managed cluster `klusterlet` to run on specific nodes, see [Optional: Configuring the `klusterlet` to run on specific nodes](#) for the required steps.

Note: You do not have to run the `oc` command that is provided with the cluster details to import the cluster. When you create the cluster, it is automatically configured under the management of multicluster engine operator.

Continue with [Accessing your cluster](#) for instructions for accessing your cluster.

Creating a cluster in an on-premises environment

You can use the console to create on-premises Red Hat OpenShift Container Platform clusters. You can create single-node OpenShift clusters, multi-node clusters, and compact three-node clusters. You can create these clusters on VMware vSphere, Red Hat OpenStack, Nutanix, External, or in a bare metal environment.

Since the platform value is set to `platform=none`, there is no platform integration with the platform where you install the cluster. A single-node OpenShift cluster has only a single node, which hosts the control plane services and the user workloads. Use this configuration when you want to minimize the resource footprint of the cluster.

You can also provision multiple single-node OpenShift clusters on edge resources by using the zero touch provisioning feature, which is a feature that is available with Red Hat OpenShift Container Platform. For more information about zero touch provisioning, see *Clusters at the network far edge* in the OpenShift Container Platform documentation.

- [Prerequisites](#)
- [Creating your cluster with the console](#)
- [Creating your cluster with the command line](#)

Prerequisites

See the following prerequisites before creating a cluster in an on-premises environment:

- You must have a deployed hub cluster on a supported OpenShift Container Platform version.
- You need a configured infrastructure environment with a host inventory of configured hosts.
- You must have internet access for your hub cluster (connected), or a connection to an internal or mirror registry that has a connection to the internet (disconnected) to retrieve the required images for creating the cluster.
- You need a configured on-premises credential.
- You need an OpenShift Container Platform image pull secret. See *Using image pull secrets*.
- You need the following DNS records:
 - The following API base domain must point to the static API VIP:

`api.<cluster_name>.<base_domain>`

- The following application base domain must point to the static IP address for Ingress VIP:

`*.apps.<cluster_name>.<base_domain>`

- Review the hub cluster `KubeAPIServer` certificate verification strategy and update the strategy if needed. To learn what strategy to use for your setup, see [Configuring the hub cluster KubeAPIServer verification strategy](#).

Creating your cluster with the console

To create a cluster from the console, complete the following steps:

1. Navigate to **Infrastructure > Clusters**.
2. On the *Clusters* page, click **Create cluster** and complete the steps in the console.
3. Select **Host inventory** as the type of cluster.

The following options are available for your assisted installation:

- **Use existing discovered hosts:** Select your hosts from a list of hosts that are in an existing host inventory.
- **Discover new hosts:** Discover hosts that are not already in an existing infrastructure environment. Discover your own hosts, rather than using one that is already in an infrastructure environment.

If you need to create a credential, see *Creating a credential for an on-premises environment* for more information.

The name for your cluster is used in the hostname of the cluster.

Important: When you create a cluster, the controller creates a namespace for the cluster and its resources. Ensure that you include only resources for that cluster instance in that namespace. Destroying the cluster deletes the namespace and all of the resources in it.

Note: Select **YAML: On** to view content updates as you enter the information in the console.

If you want to add your cluster to an existing cluster set, you must have the correct permissions on the cluster set to add it. If you do not have `cluster-admin` privileges when you are creating the cluster, you must select a cluster set on which you have `clusterset-admin` permissions. If you do not have the correct permissions on the specified cluster set, the cluster creation fails. Contact your cluster administrator to provide you with `clusterset-admin` permissions to a cluster set if you do not have any cluster set options to select.

Every managed cluster must be associated with a managed cluster set. If you do not assign the managed cluster to a `ManagedClusterSet`, it is automatically added to the `default` managed cluster set.

If there is already a base DNS domain that is associated with the selected credential that you configured for your provider account, that value is populated in that field. You can change the value by overwriting it, but this setting cannot be changed after the cluster is created. The base domain of your provider is used to create routes to your Red Hat OpenShift Container Platform

cluster components. It is configured in the DNS of your cluster provider as a Start of Authority (SOA) record.

The **OpenShift version** identifies the version of the OpenShift Container Platform image that is used to create the cluster. If the version that you want to use is available, you can select the image from the list of images. If the image that you want to use is not a standard image, you can enter the URL to the image that you want to use. See *Release images* to learn more.

When you select a supported OpenShift Container Platform version, an option to select **Install single-node OpenShift** is displayed. A single-node OpenShift cluster contains a single node which hosts the control plane services and the user workloads. See *Scaling hosts to an infrastructure environment* to learn more about adding nodes to a single-node OpenShift cluster after it is created.

If you want your cluster to be a single-node OpenShift cluster, select the **single-node OpenShift** option. You can add additional workers to single-node OpenShift clusters by completing the following steps:

1. From the console, navigate to **Infrastructure > Clusters** and select the name of the cluster that you created or want to access.
2. Select **Actions > Add hosts** to add additional workers.

Note: The single-node OpenShift control plane requires 8 CPU cores, while a control plane node for a multinode control plane cluster only requires 4 CPU cores.

After you review and save the cluster, your cluster is saved as a draft cluster. You can close the creation process and finish the process later by selecting the cluster name on the *Clusters* page.

If you are using existing hosts, select whether you want to select the hosts yourself, or if you want them to be selected automatically. The number of hosts is based on the number of nodes that you selected. For example, a single-node OpenShift cluster only requires one host, while a standard three-node cluster requires three hosts.

The locations of the available hosts that meet the requirements for this cluster are displayed in the list of *Host locations*. For distribution of the hosts and a more high-availability configuration, select multiple locations.

If you are discovering new hosts with no existing infrastructure environment, complete the steps in *Adding hosts to the host inventory by using the Discovery Image*.

After the hosts are bound, and the validations pass, complete the networking information for your cluster by adding the following IP addresses:

- **API VIP:** Specifies the IP address to use for internal API communication.

Note: This value must match the name that you used to create the DNS records listed in the prerequisites section. If not provided, the DNS must be pre-configured so that **api.** resolves correctly.

- **Ingress VIP:** Specifies the IP address to use for ingress traffic.

Note: This value must match the name that you used to create the DNS records listed in the

prerequisites section. If not provided, the DNS must be pre-configured so that `test.apps.` resolves correctly.

If you are using Red Hat Advanced Cluster Management for Kubernetes and want to configure your managed cluster `klusterlet` to run on specific nodes, see [Optional: Configuring the klusterlet to run on specific nodes](#) for the required steps.

You can view the status of the installation on the *Clusters* navigation page.

Continue with [Accessing your cluster](#) for instructions for accessing your cluster.

Creating your cluster with the command line

You can also create a cluster without the console by using the Assisted Installer feature within the central infrastructure management component. After you complete this procedure, you can boot the host from the discovery image that is generated. The order of the procedures is generally not important, but is noted when there is a required order.

Create the namespace

You need a namespace for your resources. It is more convenient to keep all of the resources in a shared namespace. This example uses `sample-namespace` for the name of the namespace, but you can use any name except `assisted-installer`. Create a namespace by creating and applying the following file:

```
apiVersion: v1
kind: Namespace
metadata:
  name: sample-namespace
```

Add the pull secret to the namespace

Add your [pull secret](#) to your namespace by creating and applying the following custom resource:

```
apiVersion: v1
kind: Secret
type: kubernetes.io/dockerconfigjson
metadata:
  name: <pull-secret>
  namespace: sample-namespace
stringData:
  .dockerconfigjson: 'your-pull-secret-json' ①
```

① Add the content of the pull secret. For example, this can include a `cloud.openshift.com`, `quay.io`, or `registry.redhat.io` authentication.

Generate a ClusterImageSet

Generate a `CustomImageSet` to specify the version of OpenShift Container Platform for your cluster

by creating and applying the following custom resource. Replace `<4.x.0>` with the supported version that you are using for `ClusterImageSet` for both the `name` and the `releaseImage`:

```
apiVersion: hive.openshift.io/v1
kind: ClusterImageSet
metadata:
  name: openshift-v<4.x.0>
spec:
  releaseImage: quay.io/openshift-release-dev/ocp-release:<4.x.0>-rc.0-x86_64
```

Note: You need to create a multi-architecture `ClusterImageSet` if you install a managed cluster that has a different architecture than the hub cluster. To learn more, see [Creating a release image to deploy a cluster on a different architecture](#).

Create the `ClusterDeployment` custom resource

The `ClusterDeployment` custom resource definition is an API that controls the lifecycle of the cluster. It references the `AgentClusterInstall` custom resource in the `spec.ClusterInstallRef` setting which defines the cluster parameters.

Create and apply a `ClusterDeployment` custom resource based on the following example:

```
apiVersion: hive.openshift.io/v1
kind: ClusterDeployment
metadata:
  name: single-node
  namespace: demo-worker4
spec:
  baseDomain: hive.example.com
  clusterInstallRef:
    group: extensions.hive.openshift.io
    kind: AgentClusterInstall
    name: test-agent-cluster-install ①
    version: v1beta1
  clusterName: test-cluster
  controlPlaneConfig:
    servingCertificates: {}
  platform:
    agentBareMetal:
      agentSelector:
        matchLabels:
          location: internal
  pullSecretRef:
    name: <pull-secret> ②
```

① Use the name of your `AgentClusterInstall` resource.

② Use the pull secret that you downloaded in [Add the pull secret to the namespace](#).

Create the AgentClusterInstall custom resource

In the `AgentClusterInstall` custom resource, you can specify many of the requirements for the clusters. For example, you can specify the cluster network settings, platform, number of control planes, and worker nodes.

Create and add the a custom resource that resembles the following example, replacing `<4.x.0>` with the version of OpenShift Container Platform that you are using:

```
apiVersion: extensions.hive.openshift.io/v1beta1
kind: AgentClusterInstall
metadata:
  name: test-agent-cluster-install
  namespace: demo-worker4
spec:
  platformType: BareMetal ①
  clusterDeploymentRef:
    name: single-node ②
  imageSetRef:
    name: openshift-v4.x.0 ③
  networking:
    clusterNetwork:
      - cidr: 10.128.0.0/14
        hostPrefix: 23
    machineNetwork:
      - cidr: 192.168.111.0/24
    serviceNetwork:
      - 172.30.0.0/16
  provisionRequirements:
    controlPlaneAgents: 1 ④
  sshPublicKey: | ⑤
    ssh-rsa <public-key-1>
    ssh-rsa <public-key-2>
    ssh-rsa <public-key-3>
```

- ① Specify the platform type of the environment where the cluster is created. Possible values are: `BareMetal`, `None`, `VSphere`, `Nutanix`, or `External`.
- ② Use the same name that you used for your `ClusterDeployment` resource.
- ③ Use the `ClusterImageSet` that you generated in [Generate a ClusterImageSet](#).
- ④ Possible values are `1` and `3`. If you are using OpenShift Container Platform version 4.18 or newer, the `platformType` field is set to `BareMetal` or `None`, and your CPU architecture is `x86_64`, you can also use `4` and `5`. Control plane nodes manage workloads, maintain cluster state, and ensure stability. You can increase fault tolerance and availability, and reduce downtime during failures, if you use more than three nodes.
- ⑤ You can specify one or multiple SSH public keys, which allow you to access the host after installing.

If you set `platformType` to `External`, you must add the `external` spec. See the following example:

```

apiVersion: extensions.hive.openshift.io/v1beta1
kind: AgentClusterInstall
metadata:
  name: test-agent-cluster-install
  namespace: demo-worker4
spec:
  platformType: External
  external:
    platformName: "myplatform" ①
    cloudControllerManager: External ②

```

① Add your platform name.

② If you set the field to empty, {}, it disables the Cloud Controller Manager during installation. If you set it to **External**, you must add custom manifests to enable the Cloud Controller Manager.

Optional: Create the NMStateConfig custom resource

The **NMStateConfig** custom resource is only required if you have a host-level network configuration, such as static IP addresses. If you create the **NMStateConfig** custom resource, you must complete this step before you create an **InfraEnv** custom resource. The values for **spec.nmStateConfigLabelSelector** in the **InfraEnv** custom resource refer to the **NMStateConfig** custom resource.

Create and apply your **NMStateConfig** custom resource, which resembles the following example. Replace values where needed:

```

apiVersion: agent-install.openshift.io/v1beta1
kind: NMStateConfig
metadata:
  name: <mynmstateconfig>
  namespace: <demo-worker4>
  labels:
    demo-nmstate-label: <value>
spec:
  config:
    interfaces:
      - name: eth0
        type: ethernet
        state: up
        mac-address: 02:00:00:80:12:14
        ipv4:
          enabled: true
          address:
            - ip: 192.168.111.30
              prefix-length: 24
          dhcp: false
      - name: eth1
        type: ethernet
        state: up
        mac-address: 02:00:00:80:12:15

```

```

    ipv4:
      enabled: true
      address:
        - ip: 192.168.140.30
          prefix-length: 24
      dhcp: false
  dns-resolver:
    config:
      server:
        - 192.168.126.1
  routes:
    config:
      - destination: 0.0.0.0/0
        next-hop-address: 192.168.111.1
        next-hop-interface: eth1
        table-id: 254
      - destination: 0.0.0.0/0
        next-hop-address: 192.168.140.1
        next-hop-interface: eth1
        table-id: 254
  interfaces:
    - name: "eth0"
      macAddress: "02:00:00:80:12:14"
    - name: "eth1"
      macAddress: "02:00:00:80:12:15"

```

Important:

- You must include the `demo-nmstate-label` label name and value in the `spec.nmStateConfigLabelSelector.matchLabels` field of the `InfraEnv` custom resource.
- You must add a MAC address when using the DHCP or static network protocol.

To see additional examples for NMState that you can add to the `spec.config` field, see [Additional NMState configuration examples](#) in the Assisted Installer for OpenShift Container Platform documentation.

Create the InfraEnv custom resource

The `InfraEnv` custom resource provides the configuration to create the discovery ISO. Within this custom resource, you identify values for proxy settings, ignition overrides, and specify `NMState` labels. The value of `spec.nmStateConfigLabelSelector` in this custom resource references the `NMStateConfig` custom resource.

Note: If you plan to include the optional `NMStateConfig` custom resource, you must reference it in the `InfraEnv` custom resource. If you create the `InfraEnv` custom resource before you create the `NMStateConfig` custom resource edit the `InfraEnv` custom resource to reference the `NMStateConfig` custom resource and download the ISO after the reference is added.

Create and apply the following custom resource:


```

apiVersion: agent-install.openshift.io/v1beta1
kind: InfraEnv
metadata:
  name: myinfraenv
  namespace: demo-worker4
spec:
  clusterRef:
    name: single-node ①
    namespace: demo-worker4 ②
  pullSecretRef:
    name: pull-secret
  sshAuthorizedKey: <your_public_key_here>
  nmStateConfigLabelSelector:
    matchLabels:
      demo-nmstate-label: value
  proxy:
    httpProxy: http://USERNAME:PASSWORD@proxy.example.com:PORT
    httpsProxy: https://USERNAME:PASSWORD@proxy.example.com:PORT
    noProxy: .example.com,172.22.0.0/24,10.10.0.0/24

```

- ① Replace the `clusterDeployment` resource name from [Create the ClusterDeployment](#).
- ② Replace the `clusterDeployment` resource namespace from [Create the ClusterDeployment](#).

Table 8. *InfraEnv* field table

Field	Optional or required	Description
<code>sshAuthorizedKey</code>	Optional	You can specify your SSH public key, which enables you to access the host when it is booted from the discovery ISO image.

<code>nmStateConfigLabelSelector</code>	Optional	Consolidates advanced network configuration such as static IPs, bridges, and bonds for the hosts. The host network configuration is specified in one or more <code>NMStateConfig</code> resources with labels you choose. The <code>nmStateConfigLabelSelector</code> property is a Kubernetes label selector that matches your chosen labels. The network configuration for all <code>NMStateConfig</code> labels that match this label selector is included in the Discovery Image. When you boot, each host compares each configuration to its network interfaces and applies the appropriate configuration.
<code>proxy</code>	Optional	You can specify proxy settings required by the host during discovery in the proxy section.

Note: When provisioning with IPv6, you cannot define a CIDR address block in the `noProxy` settings. You must define each address separately.

Boot the host from the discovery image

The remaining steps explain how to boot the host from the discovery ISO image that results from the previous procedures.

1. Download the discovery image from the namespace by running the following command:

```
curl --insecure -o image.iso $(kubectl -n sample-namespace get infraenvs.agent-install.openshift.io myinfraenv -o=jsonpath="{.status.isoDownloadURL}")
```

2. Move the discovery image to virtual media, a USB drive, or another storage location and boot the host from the discovery image that you downloaded.
3. The `Agent` resource is created automatically. It is registered to the cluster and represents a host that booted from a discovery image. Approve the `Agent` custom resource and start the installation by running the following command:

```
oc -n sample-namespace patch agents.agent-install.openshift.io 07e80ea9-200c-4f82-aff4-4932acb773d4 -p '{"spec":{"approved":true}}' --type merge
```

Replace the agent name and UUID with your values.

You can confirm that it was approved when the output of the previous command includes an entry for the target cluster that includes a value of `true` for the `APPROVED` parameter.

Additional resources

- For additional steps that are required when creating a cluster on the Nutanix platform with the CLI, see [Adding hosts on Nutanix with the API](#) and [Nutanix post-installation configuration](#) in the Red Hat OpenShift Container Platform documentation.
- For additional information about zero touch provisioning, see [Clusters at the network far edge](#) in the OpenShift Container Platform documentation.
- See [Using image pull secrets](#)
- See [Creating a credential for an on-premises environment](#)
- See [Release images](#)
- See [Adding hosts to the host inventory by using the Discovery Image](#)

Creating a cluster in a proxy environment

You can create a Red Hat OpenShift Container Platform cluster when your hub cluster is connected through a proxy server. One of the following situations must be true for the cluster creation to succeed:

- multicluster engine operator has a private network connection with the managed cluster that you are creating, with managed cluster access to the Internet by using a proxy.
- The managed cluster is on an infrastructure provider, but the firewall ports enable communication from the managed cluster to the hub cluster.

To create a cluster that is configured with a proxy, complete the following steps:

1. Configure the `cluster-wide-proxy` setting on the hub cluster by adding the following information to your `install-config` YAML that is stored in your Secret:

```
apiVersion: v1
kind: Proxy
baseDomain: <domain>
proxy:
  httpProxy: http://<username>:<password>@<proxy.example.com>:<port>
  httpsProxy: https://<username>:<password>@<proxy.example.com>:<port>
  noProxy: <wildcard-of-domain>,<provisioning-network/CIDR>,<BMC-address-
range/CIDR>
```

Replace `username` with the username for your proxy server.

Replace `password` with the password to access your proxy server.

Replace `proxy.example.com` with the path of your proxy server.

Replace `port` with the communication port with the proxy server.

Replace `wildcard-of-domain` with an entry for domains that should bypass the proxy.

Replace `provisioning-network/CIDR` with the IP address of the provisioning network and the number of assigned IP addresses, in CIDR notation.

Replace `BMC-address-range/CIDR` with the BMC address and the number of addresses, in CIDR notation.

2. Provision the cluster by completing the procedure for creating a cluster. See *Creating a cluster* to select your provider.

Note: You can only use `install-config` YAML when deploying your cluster. After deploying your cluster, any new changes you make to `install-config` YAML do not apply. To update the configuration after deployment, you must use policies. See *Pod policy* for more information.

Additional resources

- See [Creating clusters](#) to select your provider.
- See [Pod policy](#) to learn how to make configuration changes after deploying your cluster.

Configuring `AgentClusterInstall` proxy

The `AgentClusterInstall` proxy fields determine the proxy settings during installation, and are used to create the cluster-wide proxy resource in the created cluster.

Configuring `AgentClusterInstall`

To configure the `AgentClusterInstall` proxy, add the `proxy` settings to the `AgentClusterInstall` resource. See the following YAML sample with `httpProxy`, `httpsProxy`, and `noProxy`:

```
apiVersion: extensions.hive.openshift.io/v1beta1
kind: AgentClusterInstall
spec:
  proxy:
    httpProxy: http://<username>:<password>@<proxy.example.com>:<port> ①
    httpsProxy: https://<username>:<password>@<proxy.example.com>:<port> ②
    noProxy: <wildcard-of-domain>,<provisioning-network/CIDR>,<BMC-address-range/CIDR> ③
```

① `httpProxy` is the URL of the proxy for HTTP requests. Replace the username and password values with your credentials for your proxy server. Replace `proxy.example.com` with the path of your proxy server.

② `httpsProxy` is the URL of the proxy for HTTPS requests. Replace the values with your credentials. Replace `port` with the communication port with the proxy server.

③ `noProxy` is a comma-separated list of domains and CIDRs for which the proxy should not be used. Replace `wildcard-of-domain` with an entry for domains that should bypass the proxy. Replace `provisioning-network/CIDR` with the IP address of the provisioning network and the number of

assigned IP addresses, in CIDR notation. Replace `BMC-address-range/CIDR` with the BMC address and the number of addresses, in CIDR notation.

Additional resources

- [Enabling the central infrastructure management service](#)

Cluster import

You can import clusters from different Kubernetes cloud providers. After you import, the target cluster becomes a managed cluster for the multicluster engine operator hub cluster. You can generally complete the import tasks anywhere that you can access the hub cluster and the target managed cluster, unless otherwise specified.

- A hub cluster cannot manage *any* other hub cluster, but can manage itself. The hub cluster is configured to automatically be imported and self-managed. You do not need to manually import the hub cluster.
- If you remove a hub cluster and try to import it again, you must add the `local-cluster:true` label to the `ManagedCluster` resource.

Important: Cluster lifecycle now supports all providers that are certified through the Cloud Native Computing Foundation (CNCF) Kubernetes Conformance Program. Choose a vendor that is recognized by CNCF for your hybrid cloud multicluster management.

See the following information about using CNCF providers:

- Learn how CNCF providers are certified at [Certified Kubernetes Conformance](#).
- For Red Hat support information about CNCF third-party providers, see [Red Hat support with third party components](#), or [Contact Red Hat support](#).
- If you bring your own CNCF conformance certified cluster, you need to change the OpenShift Container Platform CLI `oc` command to the Kubernetes CLI command, `kubectl`.

Read the following topics to learn more about importing a cluster so that you can manage it:

Required user type or access level: Cluster administrator

- [Importing an existing cluster by using the console](#)
- [Importing a managed cluster by using the CLI](#)
- [Importing a managed cluster by using agent registration](#)
- [Importing an on-premises Red Hat OpenShift Container Platform cluster](#)

Importing a managed cluster by using the console

After you install multicluster engine for Kubernetes operator, you are ready to import a cluster to manage. Continue reading the following topics learn how to import a managed cluster by using the console:

- [Prerequisites](#)

- [Creating a new pull secret](#)
- [Importing a cluster](#)
- [Optional: Configuring the cluster API address](#)
- [Removing a cluster](#)

Prerequisites

- A deployed hub cluster. If you are importing bare metal clusters, the hub cluster must be installed on a supported Red Hat OpenShift Container Platform version.
- A cluster you want to manage.
- The `base64` command line tool.
- A defined `multiclusterhub.spec.imagePullSecret` if you are importing a cluster that was not created by OpenShift Container Platform. This secret might have been created when multicluster engine for Kubernetes operator was installed. See *Custom image pull secret* for more information about how to define this secret.
- Review the hub cluster `KubeAPIServer` certificate verification strategy and update the strategy if needed. To learn what strategy to use for your setup, see [Configuring the hub cluster KubeAPIServer verification strategy](#).

Required user type or access level: Cluster administrator

Creating a new pull secret

If you need to create a new pull secret, complete the following steps:

1. Download your Kubernetes pull secret from cloud.redhat.com.
2. Add the pull secret to the namespace of your hub cluster.
3. Run the following command to create a new secret in the `open-cluster-management` namespace:

```
oc create secret generic pull-secret -n <open-cluster-management> --from
-file=.dockerconfigjson=<path-to-pull-secret> --type=kubernetes.io/dockerconfigjson
```

Replace `open-cluster-management` with the name of the namespace of your hub cluster. The default namespace of the hub cluster is `open-cluster-management`.

Replace `path-to-pull-secret` with the path to the pull secret that you downloaded.

The secret is automatically copied to the managed cluster when it is imported.

- Ensure that a previously installed agent is deleted from the cluster that you want to import. You must remove the `open-cluster-management-agent` and `open-cluster-management-agent-addon` namespaces to avoid errors.
- For importing in a Red Hat OpenShift Dedicated environment, see the following notes:
 - You must have the hub cluster deployed in a Red Hat OpenShift Dedicated environment.

- The default permission in Red Hat OpenShift Dedicated is `dedicated-admin`, but that does not contain all of the permissions to create a namespace. You must have `cluster-admin` permissions to import and manage a cluster with multicluster engine operator.

Importing a cluster

You can import existing clusters from the console for each of the available cloud providers.

Notes:

- A hub cluster cannot manage a different hub cluster. A hub cluster is set up to automatically import and manage itself as a `local-cluster`. You do not have to manually import a hub cluster to manage.
- The namespace for the imported cluster must match the name of the cluster.

Important: When you create a cluster, the controller creates a namespace for the cluster and its resources. Ensure that you include only resources for that cluster instance in that namespace. Destroying the cluster deletes the namespace and all of the resources in it.

Every managed cluster must be associated with a managed cluster set. If you do not assign the managed cluster to a `ManagedClusterSet`, the cluster is automatically added to the `default` managed cluster set.

If you want to add the cluster to a different cluster set, you must have `cluster-set-admin` privileges to the cluster set. If you do not have `cluster-admin` privileges when you are importing the cluster, you must select a cluster set on which you have `cluster-set-admin` permissions. If you do not have the correct permissions on the specified cluster set, the cluster importing fails. Contact your cluster administrator to provide you with `cluster-set-admin` permissions to a cluster set if you do not have cluster set options to select.

If you import a OpenShift Container Platform Dedicated cluster and do not specify a vendor by adding a label for `vendor=OpenShiftDedicated`, or if you add a label for `vendor=auto-detect`, a `managed-by=platform` label is automatically added to the cluster. You can use this added label to identify the cluster as a OpenShift Container Platform Dedicated cluster and retrieve the OpenShift Container Platform Dedicated clusters as a group.

The following table provides the available options for *import mode*, which specifies the method for importing the cluster:

Table 9. Options for import mode

Run import commands manually	After completing and submitting the information in the console, including any Red Hat Ansible Automation Platform templates, run the provided command on the target cluster to import the cluster.
Enter your server URL and API token for the existing cluster	Provide the server URL and API token of the cluster that you are importing. You can specify a Red Hat Ansible Automation Platform template to run when the cluster is upgraded.

Provide the <code>kubeconfig</code> file	Copy and paste the contents of the <code>kubeconfig</code> file of the cluster that you are importing. You can specify a Red Hat Ansible Automation Platform template to run when the cluster is upgraded.
--	--

Note: You must have the Red Hat Ansible Automation Platform Resource Operator installed from the Red Hat OpenShift Software Catalog to create and run an Ansible Automation Platform job.

To configure a cluster API address, see [Optional: Configuring the cluster API address](#).

To configure your managed cluster klusterlet to run on specific nodes, see [Optional: Configuring the klusterlet to run on specific nodes](#).

Optional: Configuring the cluster API address

Complete the following steps to optionally configure the **Cluster API address** that is on the cluster details page by configuring the URL that is displayed in the table when you run the `oc get managedcluster` command:

1. Log in to your hub cluster with an ID that has `cluster-admin` permissions.
2. Configure a `kubeconfig` file for your targeted managed cluster.
3. Edit the managed cluster entry for the cluster that you are importing by running the following command, replacing `cluster-name` with the name of the managed cluster:

```
oc edit managedcluster <cluster-name>
```

4. Add the `ManagedClusterClientConfigs` section to the `ManagedCluster` spec in the YAML file, as shown in the following example:

```
spec:
  hubAcceptsClient: true
  managedClusterClientConfigs:
    - url: <https://api.new-managed.dev.redhat.com> ①
```

- ① Replace the value of the URL with the URL that provides external access to the managed cluster that you are importing.

Optional: Configuring the klusterlet to run on specific nodes

You can specify which nodes you want the managed cluster klusterlet to run on by configuring the `nodeSelector` and `tolerations` annotation for the managed cluster. Complete the following steps to configure these settings:

1. Select the managed cluster that you want to update from the clusters page in the console.
2. Set the YAML switch to **On** to view the YAML content.

Note: The YAML editor is only available when importing or creating a cluster. To edit the managed cluster YAML definition after importing or creating, you must use the OpenShift Container Platform command-line interface or the Red Hat Advanced Cluster Management search feature.

3. Add the `nodeSelector` annotation to the managed cluster YAML definition. The key for this annotation is: `open-cluster-management/nodeSelector`. The value of this annotation is a string map with JSON formatting.
4. Add the `tolerations` entry to the managed cluster YAML definition. The key of this annotation is: `open-cluster-management/tolerations`. The value of this annotation represents a `toleration` list with JSON formatting. The resulting YAML might resemble the following example:

```
apiVersion: cluster.open-cluster-management.io/v1
kind: ManagedCluster
metadata:
  annotations:
    open-cluster-management/nodeSelector: '{"dedicated":"acm"}'
    open-cluster-management/tolerations:
      '[{"key":"dedicated","operator":"Equal","value":"acm","effect":"NoSchedule"}]'
```

You can also use a `KlusterletConfig` to configure the `nodeSelector` and `tolerations` for the managed cluster. Complete the following steps to configure these settings:

Note: If you use a `KlusterletConfig`, the managed cluster uses the configuration in the `KlusterletConfig` settings instead of the settings in the managed cluster annotation.

1. Apply the following sample YAML content. Replace value where needed:

```
apiVersion: config.open-cluster-management.io/v1alpha1
kind: KlusterletConfig
metadata:
  name: <klusterletconfigName>
spec:
  nodePlacement:
    nodeSelector:
      dedicated: acm
    tolerations:
      - key: dedicated
        operator: Equal
        value: acm
        effect: NoSchedule
```

2. Add the `agent.open-cluster-management.io/klusterlet-config: <klusterletconfigName>` annotation to the managed cluster, replacing `<klusterletconfigName>` with the name of your `KlusterletConfig`.

Removing an imported cluster

Complete the following procedure to remove an imported cluster and the `open-cluster-management-agent-addon` that was created on the managed cluster.

On the *Clusters* page, click **Actions** > **Detach cluster** to remove your cluster from management.

Important: If you attempt to detach the hub cluster, which is named `<your-local-cluster-name>`, be aware that the default setting of `disableHubSelfManagement` is `false` and the hub cluster is the `local-cluster`, which manages itself. This setting causes the hub cluster to reimport itself and manage itself when it is detached and it reconciles the `MultiClusterHub` controller. It might take hours for the hub cluster to complete the detachment process and reimport. If you want to reimport the hub cluster without waiting for the processes to finish, you can run the following command to restart the `multiclusterhub-operator` pod and reimport faster:

```
oc -n open-cluster-management delete -l name=multiclusterhub-operator pods
```

You can change the value of the hub cluster to not import automatically by changing the `disableHubSelfManagement` value to `true`. For more information, see the *disableHubSelfManagement* topic.

By default, the cluster namespace is deleted after you detach a managed cluster.

To prevent multicluster engine operator from deleting the namespace after detaching a managed cluster, complete the following step:

1. Add the `open-cluster-management.io/retain-namespace` annotation to the managed cluster namespace.

If you remove the `open-cluster-management.io/retain-namespace` annotation after detaching a managed cluster, multicluster engine operator deletes the cluster namespace.

The `cluster.open-cluster-management.io/managedCluster` and `open-cluster-management.io/cluster-name` labels identify the annotation as a cluster namespace. If you remove the annotation and labels at the same time, multicluster engine operator does not manage the cluster namespace anymore.

Additional resources

- See [Custom image pull secret](#) for more information about how to define a custom image pull secret.
- See the [disableHubSelfManagement](#) topic.

Importing a managed cluster by using the CLI

After you install multicluster engine for Kubernetes operator, you are ready to import a cluster and manage it by using the Red Hat OpenShift Container Platform CLI. Continue reading the following topics to learn how to import a managed cluster with the CLI by using the auto import secret, or by using manual commands.

- [Prerequisites](#)
- [Supported architectures](#)
- [Preparing cluster import](#)
- [Importing a cluster by using the auto import secret](#)
- [Importing a cluster manually](#)
- [Importing the klusterlet add-on](#)
- [Removing an imported cluster by using the CLI](#)

Important: A hub cluster cannot manage a different hub cluster. A hub cluster is set up to automatically import and manage itself as a *local cluster*. You do not have to manually import a hub cluster to manage itself. If you remove a hub cluster and try to import it again, you need to add the `local-cluster:true` label.

Prerequisites

- A deployed hub cluster. If you are importing bare metal clusters, the hub cluster must be installed on a supported OpenShift Container Platform version.
- A separate cluster you want to manage.
- The OpenShift Container Platform CLI. See [Getting started with the OpenShift CLI](#) for information about installing and configuring the OpenShift Container Platform CLI.
- A defined `multiclusterhub.spec.imagePullSecret` if you are importing a cluster that was not created by OpenShift Container Platform. This secret might have been created when multicluster engine for Kubernetes operator was installed. See [Custom image pull secret](#) for more information about how to define this secret.
- Review the hub cluster `KubeAPIServer` certificate verification strategy and update the strategy if needed. To learn what strategy to use for your setup, see [Configuring the hub cluster KubeAPIServer verification strategy](#).

Supported architectures

- Linux (x86_64, s390x, ppc64le)
- macOS

Preparing for cluster import

Before importing a managed cluster by using the CLI, you must complete the following steps:

1. Log in to your hub cluster by running the following command:

```
oc login
```

2. Run the following command on the hub cluster to create the project and namespace. The cluster name that is defined in `<cluster_name>` is also used as the cluster namespace in the YAML file and commands:

```
oc new-project <cluster_name>
```

Important: The `cluster.open-cluster-management.io/managedCluster` label is automatically added to and removed from a managed cluster namespace. Do not manually add it to or remove it from a managed cluster namespace.

3. Create a file named `managed-cluster.yaml` with the following example content:

```
apiVersion: cluster.open-cluster-management.io/v1
kind: ManagedCluster
metadata:
  name: <cluster_name>
  labels:
    cloud: auto-detect
    vendor: auto-detect
spec:
  hubAcceptsClient: true
```

When the values for `cloud` and `vendor` are set to `auto-detect`, Red Hat Advanced Cluster Management detects the cloud and vendor types automatically from the cluster that you are importing. You can optionally replace the values for `auto-detect` with the cloud and vendor values for your cluster. See the following example:

```
cloud: Amazon
vendor: OpenShift
```

4. Apply the YAML file to the `ManagedCluster` resource by running the following command:

```
oc apply -f managed-cluster.yaml
```

You can now continue with either [Importing the cluster by using the auto import secret](#) or [Importing the cluster manually](#).

Importing a cluster by using the auto import secret

To import a managed cluster by using the auto import secret, you must create a secret that contains either a reference to the `kubeconfig` file of the cluster, or the kube API server and token pair of the cluster. Complete the following steps to import a cluster by using the auto import secret:

1. Retrieve the `kubeconfig` file, or the kube API server and token, of the managed cluster that you want to import. See the documentation for your Kubernetes cluster to learn where to locate your `kubeconfig` file or your kube API server and token.
2. Create the `auto-import-secret.yaml` file in the `${CLUSTER_NAME}` namespace.
 - a. Create a YAML file named `auto-import-secret.yaml` by using content that is similar to the following template:

```

apiVersion: v1
kind: Secret
metadata:
  name: auto-import-secret
  namespace: <cluster_name> ❶
stringData:
  kubeconfig: |- <kubeconfig_file> ❷
  token: <token_cluster> ❸
  server: <cluster_api_url> ❸
type: Opaque

```

- ❶ Add your cluster name.
- ❷ If you are using the **kubeconfig** file, add the value for the **kubeconfig** file that has context set to the cluster you want to import.
- ❸ If you are using the token and server pair, add the cluster access token and cluster API URL instead of the **kubeconfig** file.
 - a. Apply the YAML file in the <cluster_name> namespace by running the following command:

```
oc apply -f auto-import-secret.yaml
```

Note: By default, the auto import secret is used one time and deleted when the import process completes. If you want to keep the auto import secret, add **managedcluster-import-controller.open-cluster-management.io/keeping-auto-import-secret** to the secret. You can add it by running the following command:

```
oc -n <cluster_name> annotate secrets auto-import-secret managedcluster-import-controller.open-cluster-management.io/keeping-auto-import-secret=""
```

1. Validate the **JOINED** and **AVAILABLE** status for your imported cluster. Run the following command from the hub cluster:

```
oc get managedcluster <cluster_name>
```

2. Log in to the managed cluster by running the following command on the cluster:

```
oc login
```

3. You can validate the pod status on the cluster that you are importing by running the following command:

```
oc get pod -n open-cluster-management-agent
```

You can now continue with [Importing the klusterlet add-on](#).

Importing a cluster manually

Important: The import command contains pull secret information that is copied to each of the imported managed clusters. Anyone who can access the imported clusters can also view the pull secret information.

Complete the following steps to import a managed cluster manually:

1. Obtain the `klusterlet-crd.yaml` file that was generated by the import controller on your hub cluster by running the following command:

```
oc get secret <cluster_name>-import -n <cluster_name> -o
jsonpath={.data.crd.yaml} | base64 --decode > klusterlet-crd.yaml
```

2. Obtain the `import.yaml` file that was generated by the import controller on your hub cluster by running the following command:

```
oc get secret <cluster_name>-import -n <cluster_name> -o
jsonpath={.data.import.yaml} | base64 --decode > import.yaml
```

Proceed with the following steps in the cluster that you are importing:

3. Log in to the managed cluster that you are importing by entering the following command:

```
oc login
```

4. Apply the `klusterlet-crd.yaml` that you generated in step 1 by running the following command:

```
oc apply -f klusterlet-crd.yaml
```

5. Apply the `import.yaml` file that you previously generated by running the following command:

```
oc apply -f import.yaml
```

6. You can validate the `JOINED` and `AVAILABLE` status for the managed cluster that you are importing by running the following command from the hub cluster:

```
oc get managedcluster <cluster_name>
```

You can now continue with [Importing the klusterlet add-on](#).

Importing the klusterlet add-on

Implement the `KlusterletAddonConfig` klusterlet add-on configuration to enable other add-ons on your managed clusters. Create and apply the configuration file by completing the following steps:

1. Create a YAML file that is similar to the following example:

```
apiVersion: agent.open-cluster-management.io/v1
kind: KlusterletAddonConfig
metadata:
  name: <cluster_name>
  namespace: <cluster_name>
spec:
  applicationManager:
    enabled: true
  certPolicyController:
    enabled: true
  policyController:
    enabled: true
  searchCollector:
    enabled: true
```

2. Save the file as `klusterlet-addon-config.yaml`.
3. Apply the YAML by running the following command:

```
oc apply -f klusterlet-addon-config.yaml
```

Add-ons are installed after the managed cluster status you are importing is **AVAILABLE**.

4. You can validate the pod status of add-ons on the cluster you are importing by running the following command:

```
oc get pod -n open-cluster-management-agent-addon
```

Removing an imported cluster by using the command line interface

To remove a managed cluster by using the command line interface, run the following command:

```
oc delete managedcluster <cluster_name>
```

Replace `<cluster_name>` with the name of the cluster.

Importing a managed cluster by using agent registration

After you install multicluster engine for Kubernetes operator, you are ready to import a cluster and manage it by using the agent registration endpoint. Continue reading the following topics to learn

how to import a managed cluster by using the agent registration endpoint.

- [Prerequisites](#)
- [Supported architectures](#)
- [Importing a cluster](#)

Prerequisites

- A deployed hub cluster. If you are importing bare metal clusters, the hub cluster must be installed on a supported OpenShift Container Platform version.
- A cluster you want to manage.
- The `base64` command line tool.
- A defined `multiclustertokenhub.spec.imagePullSecret` if you are importing a cluster that was not created by OpenShift Container Platform. This secret might have been created when multiclustertoken engine for Kubernetes operator was installed. See [Custom image pull secret](#) for more information about how to define this secret.

If you need to create a new secret, see [Creating a new pull secret](#).

Supported architectures

- Linux (x86_64, s390x, ppc64le)
- macOS

Importing a cluster

To import a managed cluster by using the agent registration endpoint, complete the following steps:

1. Get the agent registration server URL by running the following command on the hub cluster:

```
export agent_registration_host=$(oc get route -n multiclustertoken-engine agent-  
registration -o=jsonpath="{.spec.host}")
```

Note: If your hub cluster is using a cluster-wide-proxy, make sure that you are using the URL that managed cluster can access.

2. Get the cacert by running the following command:

```
oc get configmap -n kube-system kube-root-ca.crt -o=jsonpath="{.data['ca\.crt']}" >  
ca.crt_
```

Note: If you are not using the `kube-root-ca` issued endpoint, use the public `agent-registration` API endpoint CA instead of the `kube-root-ca` CA.

3. Get the token for the agent registration sever to authorize by applying the following YAML content:


```

apiVersion: v1
kind: ServiceAccount
metadata:
  name: managed-cluster-import-agent-registration-sa
  namespace: multicluster-engine
---
apiVersion: v1
kind: Secret
type: kubernetes.io/service-account-token
metadata:
  name: managed-cluster-import-agent-registration-sa-token
  namespace: multicluster-engine
  annotations:
    kubernetes.io/service-account.name: "managed-cluster-import-agent-registration-sa"
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: managedcluster-import-controller-agent-registration-client
rules:
- nonResourceURLs: ["/agent-registration/*"]
  verbs: ["get"]
---
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: managed-cluster-import-agent-registration
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: managedcluster-import-controller-agent-registration-client
subjects:
- kind: ServiceAccount
  name: managed-cluster-import-agent-registration-sa
  namespace: multicluster-engine

```

4. Run the following command to export the token:

```

export token=$(oc get secret -n multicluster-engine managed-cluster-import-agent-registration-sa-token -o=jsonpath='{.data.token}' | base64 -d)

```

5. Enable the automatic approval and patch the content to `cluster-manager` by running the following command:

```

oc patch clustermanager cluster-manager --type=merge -p
'{"spec":{"registrationConfiguration":{"featureGates":[
{"feature": "ManagedClusterAutoApproval", "mode": "Enable"}]},

```

```
"autoApproveUsers":["system:serviceaccount:multicluster-engine:agent-registration-bootstrap"]}]}'
```

Note: You can also disable automatic approval and manually approve certificate signing requests from managed clusters.

6. Switch to your managed cluster and get the cacert by running the following command:

```
curl --cacert ca.crt -H "Authorization: Bearer $token"  
https://$agent_registration_host/agent-registration/crds/v1 | oc apply -f -
```

7. Run the following command to import the managed cluster to the hub cluster. Replace `<clusterName>` with the name of your cluster. Replace `<duration>` with a time value. For example, `4h`:

Optional: Replace `<kubernetesletconfigName>` with the name of your `KubernetesletConfig`.

```
curl --cacert ca.crt -H "Authorization: Bearer $token"  
https://$agent_registration_host/agent-  
registration/manifests/<clusterName>?kubernetesletconfig=<kubernetesletconfigName>&durati  
on=<duration> | oc apply -f -
```

Note: The `kubeconfig` bootstrap in the `kuberneteslet` manifest does not expire if you do not set a duration.

Importing an on-premises Red Hat OpenShift Container Platform cluster manually by using central infrastructure management

After you install multicluster engine for Kubernetes operator, you are ready to import a managed cluster. You can import an existing OpenShift Container Platform cluster so that you can add additional nodes. Continue reading the following topics to learn more:

- [Prerequisites](#)
- [Importing a cluster](#)
- [Importing cluster resources](#)

Prerequisites

- Enable the central infrastructure management feature.

Importing a cluster

Complete the following steps to import an OpenShift Container Platform cluster manually, without a static network or a bare metal host, and prepare it for adding nodes:

1. Create a namespace for the OpenShift Container Platform cluster that you want to import by applying the following YAML content:

```
apiVersion: v1
kind: Namespace
metadata:
  name: managed-cluster
```

2. Make sure that a **ClusterImageSet** matching the OpenShift Container Platform cluster you are importing exists by applying the following YAML content. Replace **<4.x.0>** with the version of OpenShift Container Platform that you are using: :

```
apiVersion: hive.openshift.io/v1
kind: ClusterImageSet
metadata:
  name: openshift-v4.x
spec:
  releaseImage: quay.io/openshift-release-dev/ocp-
release@sha256:22e149142517dfccb47be828f012659b1ccf71d26620e6f62468c264a7ce7863
```

3. Add your pull secret to access the image by applying the following YAML content:

```
apiVersion: v1
kind: Secret
type: kubernetes.io/dockerconfigjson
metadata:
  name: pull-secret
  namespace: managed-cluster
stringData:
  .dockerconfigjson: <pull-secret-json> ①
```

① Replace **<pull-secret-json>** with your pull secret JSON.

4. Copy the **kubeconfig** from your OpenShift Container Platform cluster to the hub cluster.
 - a. Get the **kubeconfig** from your OpenShift Container Platform cluster by running the following command. Make sure that **kubeconfig** is set as the cluster being imported:

```
oc get secret -n openshift-kube-apiserver node-kubeconfigs -ojson | jq
'.data["lb-ext.kubeconfig"]' --raw-output | base64 -d > /tmp/kubeconfig.some-
other-cluster
```

Note: If your cluster API is accessed through a custom domain, you must first edit this **kubeconfig** by adding your custom certificates in the **certificate-authority-data** field and by changing the **server** field to match your custom domain.

- b. Copy the **kubeconfig** to the hub cluster by running the following command. Make sure that **kubeconfig** is set as your hub cluster:

```
oc -n managed-cluster create secret generic some-other-cluster-admin-kubeconfig
--from-file=kubeconfig=/tmp/kubeconfig.some-other-cluster
```

5. Create an **AgentClusterInstall** custom resource by applying the following YAML content. Replace values where needed:

```
apiVersion: extensions.hive.openshift.io/v1beta1
kind: AgentClusterInstall
metadata:
  name: <your-cluster-name> ①
  namespace: <managed-cluster>
spec:
  networking:
    userManagedNetworking: true
  clusterDeploymentRef:
    name: <your-cluster>
  imageSetRef:
    name: openshift-v4.x.z
  provisionRequirements:
    controlPlaneAgents: ②
  sshPublicKey: <""> ③
```

① Choose a name for your cluster.

② Use 1 if you are using a single-node OpenShift cluster. Use 3 if you are using a multinode cluster.

③ Add the optional **sshPublicKey** field to log in to nodes for troubleshooting.

6. Create a **ClusterDeployment** by applying the following YAML content. Replace values where needed:

```
apiVersion: hive.openshift.io/v1
kind: ClusterDeployment
metadata:
  name: <your-cluster-name> ①
  namespace: managed-cluster
spec:
  baseDomain: <redhat.com> ②
  installed: <true> ③
  clusterMetadata:
    adminKubeconfigSecretRef:
      name: <your-cluster-name-admin-kubeconfig> ④
    clusterID: <""> ⑤
    infraID: <""> ⑤
  clusterInstallRef:
    group: extensions.hive.openshift.io
    kind: AgentClusterInstall
    name: your-cluster-name-install
```

```

version: v1beta1
clusterName: your-cluster-name
platform:
  agentBareMetal:
pullSecretRef:
  name: pull-secret

```

- ① Choose a name for your cluster.
 - ② Make sure **baseDomain** matches the domain you are using for your OpenShift Container Platform cluster.
 - ③ Set to **true** to automatically import your OpenShift Container Platform cluster as a production environment cluster.
 - ④ Reference the **kubeconfig** you created in step 4.
 - ⑤ Leave **clusterID** and **infraID** empty in production environments.
7. Add an **InfraEnv** custom resource to discover new hosts to add to your cluster by applying the following YAML content. Replace values where needed:

Note: The following example might require additional configuration if you are not using a static IP address.

```

apiVersion: agent-install.openshift.io/v1beta1
kind: InfraEnv
metadata:
  name: your-infraenv
  namespace: managed-cluster
spec:
  clusterRef:
    name: your-cluster-name
    namespace: managed-cluster
  pullSecretRef:
    name: pull-secret
  sshAuthorizedKey: ""

```

Table 10. *InfraEnv* field table

Field	Optional or required	Description
clusterRef	Optional	The clusterRef field is optional if you are using late binding. If you are not using late binding, you must add the clusterRef .
sshAuthorizedKey	Optional	Add the optional sshAuthorizedKey field to log in to nodes for troubleshooting.

1. If the import is successful, a URL to download an ISO file appears. Download the ISO file by running the following command, replacing **<url>** with the URL that appears:

You can automate host discovery by using bare metal host.

```
oc get infraenv -n managed-cluster some-other-infraenv -ojson | jq ".status.<url>"  
--raw-output | xargs curl -k -o /storage0/isos/some-other.iso
```

- Optional:** If you want to use Red Hat Advanced Cluster Management features, such as policies, on your OpenShift Container Platform cluster, create a **ManagedCluster** resource. Make sure that the name of your **ManagedCluster** resource matches the name of your **ClusterDeployment** resource. If you are missing the **ManagedCluster** resource, your cluster status is **detached** in the console.

Importing cluster resources

If your OpenShift Container Platform managed cluster was installed by the Assisted Installer, you can move the managed cluster and its resources from one hub cluster to another hub cluster.

You can manage a cluster from a new hub cluster by saving a copy of the original resources, applying them to the new hub cluster, and deleting the original resources. You can then scale down or scale up your managed cluster from the new hub cluster.

Important: You can only scale down imported OpenShift Container Platform managed clusters if they were installed by the Assisted Installer.

You can import the following resources and continue to manage your cluster with them:

Table 11. Managed cluster resource table

Resource	Optional or required	Description
Agent	Required	
AgentClassification	Optional	Required if you want to classify Agents with a filter query.
AgentClusterInstall	Required	
BareMetalHost	Optional	Required if you are using the baremetal platform.
ClusterDeployment	Required	
InfraEnv	Required	
NMStateConfig	Optional	Required if you want to apply your network configuration on the hosts.
ManagedCluster	Required	
Secret	Required	The admin-kubeconfig secret is required. The bmc-secret secret is only required if you are using BareMetalHosts .

Saving and applying managed cluster resources

To save a copy of your managed cluster resources and apply them to a new hub cluster, complete the following steps:

1. Get your resources from your source hub cluster by running the following command. Replace values where needed:

```
oc kubectl get <source_hub_kubeconfig> -n <managed_cluster_name> get  
<resource_name> <cluster_provisioning_namespace> -oyaml > <resource_name>.yaml
```

- a. Repeat the command for every resource you want to import by replacing `<resource_name>` with the name of the resource.
2. Remove the `ownerReferences` property from the following resources by running the following commands:
 - a. `AgentClusterInstall`

```
yq --in-place -y 'del(.metadata.ownerReferences)' AgentClusterInstall.yaml
```

- b. `Secret (admin-kubeconfig)`

```
yq --in-place -y 'del(.metadata.ownerReferences)' AdminKubeconfigSecret.yaml
```

3. Detach the managed cluster from the source hub cluster by running the following command. Replace values where needed:

```
oc kubectl delete <target_hub_kubeconfig> delete ManagedCluster <cluster_name>
```

4. Create a namespace on the target hub cluster for the managed cluster. Use a similar name as the source hub cluster.
5. Apply your stored resources on the target hub cluster individually by running the following command. Replace values where needed:

Note: Replace `<resource_name>.yaml` with `.` if you want to apply all the resources as a group instead of individually.

```
oc kubectl apply <target_hub_kubeconfig> apply -f <resource_name>.yaml
```

Removing the managed cluster from the source hub cluster

After importing your cluster resources, remove your managed cluster from the source hub cluster by completing the following steps:

1. Set the `spec.preserveOnDelete` parameter to `true` in the `ClusterDeployment` custom resource to

prevent destroying the managed cluster.

2. Complete the steps in [Removing a cluster from management](#).

Customizing the automatic import strategy

Set the automatic import behavior of a managed cluster to be either a one-time process or a continuous process. The following two automatic import strategies exist:

ImportOnly

If a managed cluster is missing the `ManagedClusterImportSucceeded` condition, or the condition is set to `False`, the hub cluster controller applies the klusterlet manifests to the managed cluster to initiate the import.

After the managed cluster joins the hub cluster and the `ManagedClusterImportSucceeded` condition is set to `True`, the hub cluster controller stops applying klusterlet manifests and skips automatic import operations for the cluster. From this point, only the klusterlet agent on the managed cluster synchronizes the hub cluster configuration.

If the klusterlet agent fails, the manifests can become outdated, and might cause the managed cluster to appear in an `Unknown` state on the hub cluster. You must then [import the cluster manually](#).

Use the `ImportOnly` strategy for a hub cluster disaster recovery scenario where you want to prevent the initial hub cluster from recovering the managed clusters, if the initial hub cluster restarts unexpectedly after the managed clusters were moved to a different hub cluster.

ImportAndSync

The hub cluster controller applies the klusterlet manifests until the managed cluster joins. After joining, both the hub cluster controller and the klusterlet agent continuously synchronize the klusterlet manifests with the hub cluster configuration, ensuring consistency even if one component stops working.

If you are using multicluster engine for Kubernetes operator 2.9 or later, the default automatic import strategy is `ImportOnly`. If you upgraded your hub cluster from an earlier release, the automatic import strategy is set to `ImportAndSync` to preserve the existing automatic import behavior.

To change the import strategy, use a `ConfigMap` object, named `import-controller-config`, in the multicluster engine operator install namespace. Complete the following steps:

1. To set a custom strategy, run the following command. Replace `<mce_install_namespace>` with the default namespace where multicluster engine operator is installed, `multicluster-engine`:

```
oc -n <mce_install_namespace> create configmap import-controller-config \
  --from-literal=autoImportStrategy=<ImportOnly|ImportAndSync>
```

2. If you want to include the updated strategy in the cluster backup, add the `backup=true` label to the `ConfigMap` by running the following command:


```
oc -n <mce_install_namespace> label configmap import-controller-config \
cluster.open-cluster-management.io/backup=true
```

Customizing the hub cluster API server URL and CA bundle during import

You might not be able to register a managed cluster on your multicluster engine operator hub cluster if intermediate components exist between the managed cluster and the hub cluster. Example intermediate components include a Virtual IP, load balancer, reverse proxy, or API gateway. If you have an intermediate component, you must use a custom server URL and CA bundle for the hub cluster API server when importing a managed cluster.

Required access: Cluster administrator

Prerequisites

- You must configure the intermediate component so that the hub cluster API server is accessible for the managed cluster.
- If the intermediate component terminates the SSL connections between the managed cluster and hub cluster API server, you must bridge the SSL connections and pass the authentication information from the original requests to the back end of the hub cluster API server. You can use the User Impersonation feature of the Kubernetes API server to bridge the SSL connections.

The intermediate component extracts the client certificate from the original requests, adds Common Name (CN) and Organization (O) of the certificate subject as impersonation headers, and then forwards the modified impersonation requests to the back end of the hub cluster API server.

Note: If you bridge the SSL connections, the cluster proxy add-on does not work.

Procedure

To use a custom hub cluster API server URL and CA bundle when importing a managed cluster, complete the following steps:

1. Create a **KlusterConfig** resource with the custom hub cluster API server URL and CA bundle. See the following example:

```
apiVersion: config.open-cluster-management.io/v1alpha1
kind: KlusterletConfig
metadata:
  name: <name> ①
spec:
  hubKubeAPIServerConfig:
    url: "https://api.example.com:6443" ②
    serverVerificationStrategy: UseCustomCABundles
    trustedCABundles:
      - name: <custom-ca-bundle> ③
        caBundle:
          name: <custom-ca-bundle-configmap> ④
```

```
namespace: <multiclustere-engine> ⑤
```

- ① Add your klusterlet config name.
 - ② Add your custom server URL.
 - ③ Add your custom CA bundle name. You can use any value except `auto-detected`, which is reserved for internal use.
 - ④ Add your name of the CA bundle config map. You can create the config map by running the following command: `oc create -n <configmap-namespace> configmap <configmap-name> --from-file=ca.crt=/path/to/ca/file`
 - ⑤ Add your namespace of the CA bundle config map.
2. Select the `KlusterletConfig` resource when creating a managed cluster by adding an annotation that refers to the resource. See the following example:

```
apiVersion: cluster.open-cluster-management.io/v1
kind: ManagedCluster
metadata:
  annotations:
    agent.open-cluster-management.io/klusterlet-config: ①
  name: ②
spec:
  hubAcceptsClient: true
  leaseDurationSeconds: 60
```

- ① Add your klusterlet config name.
- ② Add your cluster name.

Notes:

- If you use the console, you might need to enable the YAML view to add the annotation to the `ManagedCluster` resource.
- You can use a global `KlusterletConfig` to enable the configuration on every managed cluster without using an annotation for binding.

Configuring the global `KlusterletConfig`

If you create a `KlusterletConfig` resource and set the name to `global`, the configurations in the global `KlusterletConfig` are automatically applied on every managed cluster.

In an environment that has a global `KlusterletConfig`, create a cluster-specific `KlusterletConfig` resource and bind it with a managed cluster by adding the `agent.open-cluster-management.io/klusterlet-config: <klusterletconfig-name>` annotation to the `ManagedCluster` resource. The value of the cluster-specific `KlusterletConfig` specification overrides the global `KlusterletConfig` value if you set different values for the same field.

See the following example where the `hubKubeAPIServerURL` field has different values set in your `KlusterletConfig` and the global `KlusterletConfig`. The `https://api.example.test.com:6443` value overrides the `https://api.example.global.com:6443` value:

Deprecation: The `hubKubeAPIServerURL` field is deprecated.

```
apiVersion: config.open-cluster-management.io/v1alpha1
kind: KlusterletConfig
metadata:
  name: test
spec:
  hubKubeAPIServerConfig:
    url: "https://api.example.test.com:6443"
---
apiVersion: config.open-cluster-management.io/v1alpha1
kind: KlusterletConfig
metadata:
  name: global
spec:
  hubKubeAPIServerConfig:
    url: "https://api.example.global.com:6443"
```

The value of the global `KlusterletConfig` is used if there is no cluster-specific `KlusterletConfig` bound to a managed cluster, or the same field is missing or does not have a value in the cluster-specific `KlusterletConfig`.

See the following example, where the `"example.global.com"` value in the `hubKubeAPIServerURL` field of the global `KlusterletConfig` overrides your `KlusterletConfig`:

```
apiVersion: config.open-cluster-management.io/v1alpha1
kind: KlusterletConfig
metadata:
  name: test
spec:
  hubKubeAPIServerURL: ""
---
apiVersion: config.open-cluster-management.io/v1alpha1
kind: KlusterletConfig
metadata:
  name: global
spec:
  hubKubeAPIServerURL: "example.global.com"
```

See the following example, where the `"example.global.com"` value in the `hubKubeAPIServerURL` field of the global `KlusterletConfig` also overrides your `KlusterletConfig`:

```
apiVersion: config.open-cluster-management.io/v1alpha1
kind: KlusterletConfig
metadata:
  name: test
---
apiVersion: config.open-cluster-management.io/v1alpha1
```

```
kind: KlusterletConfig
metadata:
  name: global
spec:
  hubKubeAPIServerURL: "example.global.com"
```

Specifying image registry on managed clusters for import

You might need to override the image registry on the managed clusters that you are importing. You can do this by creating a `ManagedClusterImageRegistry` custom resource definition.

The `ManagedClusterImageRegistry` custom resource definition is a namespace-scoped resource.

The `ManagedClusterImageRegistry` custom resource definition specifies a set of managed clusters for a Placement to select, but needs different images from the custom image registry. After the managed clusters are updated with the new images, the following label is added to each managed cluster for identification: `open-cluster-management.io/image-registry=<namespace>.<managedClusterImageRegistryName>`.

The following example shows a `ManagedClusterImageRegistry` custom resource definition:

```
apiVersion: imageregistry.open-cluster-management.io/v1alpha1
kind: ManagedClusterImageRegistry
metadata:
  name: <imageRegistryName>
  namespace: <namespace>
spec:
  placementRef:
    group: cluster.open-cluster-management.io
    resource: placements
    name: <placementName> ①
  pullSecret:
    name: <pullSecretName> ②
  registries: ③
  - mirror: <mirrored-image-registry-address>
    source: <image-registry-address>
  - mirror: <mirrored-image-registry-address>
    source: <image-registry-address>
```

- ① Replace with the name of a Placement in the same namespace that selects a set of managed clusters.
- ② Replace with the name of the pull secret that is used to pull images from the custom image registry.
- ③ List the values for each of the `source` and `mirror` registries. Replace the `mirrored-image-registry-address` and `image-registry-address` with the value for each of the `mirror` and `source` values of the registries.
 - Example 1: To replace the source image registry named `registry.redhat.io/rhacm2` with `localhost:5000/rhacm2`, and `registry.redhat.io/multicluster-engine` with

`localhost:5000/multicluster-engine`, use the following example:

```
registries:
- mirror: localhost:5000/rhacm2/
  source: registry.redhat.io/rhacm2
- mirror: localhost:5000/multicluster-engine
  source: registry.redhat.io/multicluster-engine
```

- Example 2: To replace the source image, `registry.redhat.io/rhacm2/registration-rhel8-operator` with `localhost:5000/rhacm2-registration-rhel8-operator`, use the following example:

```
registries:
- mirror: localhost:5000/rhacm2-registration-rhel8-operator
  source: registry.redhat.io/rhacm2/registration-rhel8-operator
```

Important: If you are importing a managed cluster by using agent registration, you must create a `KlusterletConfig` that contains image registries. See the following example. Replace values where needed:

```
apiVersion: config.open-cluster-management.io/v1alpha1
kind: KlusterletConfig
metadata:
  name: <klusterletconfigName>
spec:
  pullSecret:
    namespace: <pullSecretNamespace>
    name: <pullSecretName>
  registries:
    - mirror: <mirrored-image-registry-address>
      source: <image-registry-address>
    - mirror: <mirrored-image-registry-address>
      source: <image-registry-address>
```

See [Importing a managed cluster by using the agent registration endpoint](#) to learn more.

Importing a cluster that has a *ManagedClusterImageRegistry*

Complete the following steps to import a cluster that is customized with a `ManagedClusterImageRegistry` custom resource definition:

1. Create a pull secret in the namespace where you want your cluster to be imported. For these steps, the namespace is `myNamespace`.

```
$ kubectl create secret docker-registry myPullSecret \
--docker-server=<your-registry-server> \
--docker-username=<my-name> \
```

```
--docker-password=<my-password>
```

2. Create a Placement in the namespace that you created.

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: myPlacement
  namespace: myNamespace
spec:
  clusterSets:
  - myClusterSet
  tolerations:
  - key: "cluster.open-cluster-management.io/unreachable"
    operator: Exists
```

Note: The `unreachable` toleration is required for the Placement to be able to select the cluster.

3. Create a `ManagedClusterSet` resource and bind it to your namespace.

```
apiVersion: cluster.open-cluster-management.io/v1beta2
kind: ManagedClusterSet
metadata:
  name: myClusterSet

---
apiVersion: cluster.open-cluster-management.io/v1beta2
kind: ManagedClusterSetBinding
metadata:
  name: myClusterSet
  namespace: myNamespace
spec:
  clusterSet: myClusterSet
```

4. Create the `ManagedClusterImageRegistry` custom resource definition in your namespace.

```
apiVersion: imageregistry.open-cluster-management.io/v1alpha1
kind: ManagedClusterImageRegistry
metadata:
  name: myImageRegistry
  namespace: myNamespace
spec:
  placementRef:
    group: cluster.open-cluster-management.io
    resource: placements
    name: myPlacement
  pullSecret:
    name: myPullSecret
```

```
registry: myRegistryAddress
```

5. Import a managed cluster from the console and add it to a managed cluster set.
6. Copy and run the import commands on the managed cluster after the `open-cluster-management.io/image-registry=myNamespace.myImageRegistry` label is added to the managed cluster.

Accessing your cluster

To access an Red Hat OpenShift Container Platform cluster that was created and is managed, complete the following steps:

1. From the console, navigate to **Infrastructure** > **Clusters** and select the name of the cluster that you created or want to access.
2. Select **Reveal credentials** to view the user name and password for the cluster. Note these values to use when you log in to the cluster.

Note: The **Reveal credentials** option is not available for imported clusters.

3. Select **Console URL** to link to the cluster.
4. Log in to the cluster by using the user ID and password that you found in step three.

Scaling managed clusters

For clusters that you created, you can customize and resize your managed cluster specifications, such as virtual machine sizes and number of nodes. See the following option if you are using installer-provisioned infrastructure for cluster deployment:

- [Scaling with MachinePool](#)

See the following options if you are using central infrastructure management for cluster deployment:

- [Adding worker nodes to OpenShift Container Platform clusters](#)
- [Adding control plane nodes to managed clusters](#)

Scaling with MachinePool

For clusters that you provision with multicluster engine operator, a `MachinePool` resource is automatically created for you. You can further customize and resize your managed cluster specifications, such as virtual machine sizes and number of nodes, by using `MachinePool`.

- Using the `MachinePool` resource is not supported for bare metal clusters.
- A `MachinePool` resource is a Kubernetes resource on the hub cluster that groups the `MachineSet` resources together on the managed cluster.
- The `MachinePool` resource uniformly configures a set of machine resources, including zone configurations, instance type, and root storage.

- With **MachinePool**, you can manually configure the desired number of nodes or configure autoscaling of nodes on the managed cluster.

Configure autoscaling

Configuring autoscaling provides the flexibility of your cluster to scale as needed to lower your cost of resources by scaling down when traffic is low, and by scaling up to ensure that there are enough resources when there is a higher demand for resources.

- To enable autoscaling on your **MachinePool** resources using the console, complete the following steps:
 1. In the navigation, select **Infrastructure > Clusters**.
 2. Click the name of your target cluster and select the *Machine pools* tab.
 3. From the machine pools page, select **Enable autoscale** from the *Options* menu for the target machine pool.
 4. Select the minimum and maximum number of machine set replicas. A machine set replica maps directly to a node on the cluster.

The changes might take several minutes to reflect on the console after you click **Scale**. You can view the status of the scaling operation by clicking **View machines** in the notification of the *Machine pools* tab.

- To enable autoscaling on your **MachinePool** resources using the command line, complete the following steps:
 1. Enter the following command to view your list of machine pools, replacing **managed-cluster-namespace** with the namespace of your target managed cluster.

```
oc get machinepools -n <managed-cluster-namespace>
```

2. Enter the following command to edit the YAML file for the machine pool:

```
oc edit machinepool <MachinePool-resource-name> -n <managed-cluster-namespace>
```

- Replace **MachinePool-resource-name** with the name of your **MachinePool** resource.
 - Replace **managed-cluster-namespace** with the name of the namespace of your managed cluster.
3. Delete the **spec.replicas** field from the YAML file.
 4. Add the **spec.autoscaling.minReplicas** setting and **spec.autoscaling.maxReplicas** fields to the resource YAML.
 5. Add the minimum number of replicas to the **minReplicas** setting.
 6. Add the maximum number of replicas into the **maxReplicas** setting.
 7. Save the file to submit the changes.

Disabling autoscaling

You can disable autoscaling by using the console or the command line.

- To disable autoscaling by using the console, complete the following steps:
 1. In the navigation, select **Infrastructure** > **Clusters**.
 2. Click the name of your target cluster and select the *Machine pools* tab.
 3. From the machine pools page, select **Disable autoscale** from the *Options* menu for the target machine pool.
 4. Select the number of machine set replicas that you want. A machine set replica maps directly with a node on the cluster.

It might take several minutes to display in the console after you click **Scale**. You can view the status of the scaling by clicking **View machines** in the notification on the *Machine pools* tab.

- To disable autoscaling by using the command line, complete the following steps:
 1. Enter the following command to view your list of machine pools:

```
oc get machinepools -n <managed-cluster-namespace>
```

Replace **managed-cluster-namespace** with the namespace of your target managed cluster.

2. Enter the following command to edit the YAML file for the machine pool:

```
oc edit machinepool <name-of-MachinePool-resource> -n <namespace-of-managed-cluster>
```

Replace **name-of-MachinePool-resource** with the name of your **MachinePool** resource.

Replace **namespace-of-managed-cluster** with the name of the namespace of your managed cluster.

3. Delete the **spec.autoscaling** field from the YAML file.
4. Add the **spec.replicas** field to the resource YAML.
5. Add the number of replicas to the **replicas** setting.
6. Save the file to submit the changes.

Enabling manual scaling

You can scale manually from the console and from the command line.

Enabling manual scaling with the console

To scale your **MachinePool** resources using the console, complete the following steps:

1. Disable autoscaling for your **MachinePool** if it is enabled. See the previous steps.

2. From the console, click **Infrastructure** > **Clusters**.
3. Click the name of your target cluster and select the *Machine pools* tab.
4. From the machine pools page, select **Scale machine pool** from the *Options* menu for the targeted machine pool.
5. Select the number of machine set replicas that you want. A machine set replica maps directly with a node on the cluster. Changes might take several minutes to reflect on the console after you click **Scale**. You can view the status of the scaling operation by clicking **View machines** from the notification of the Machine pools tab.

Enabling manual scaling with the command line

To scale your **MachinePool** resources by using the command line, complete the following steps:

1. Enter the following command to view your list of machine pools, replacing `<managed-cluster-namespace>` with the namespace of your target managed cluster namespace:

```
oc get machinepools.hive.openshift.io -n <managed-cluster-namespace>
```

2. Enter the following command to edit the YAML file for the machine pool:

```
oc edit machinepool.hive.openshift.io <MachinePool-resource-name> -n <managed-cluster-namespace>
```

- Replace `MachinePool-resource-name` with the name of your **MachinePool** resource.
 - Replace `managed-cluster-namespace` with the name of the namespace of your managed cluster.
3. Delete the `spec.autoscaling` field from the YAML file.
 4. Modify the `spec.replicas` field in the YAML file with the number of replicas you want.
 5. Save the file to submit the changes.

Adding worker nodes to OpenShift Container Platform clusters

If you are using central infrastructure management, you can customize your OpenShift Container Platform clusters by adding additional production environment nodes.

Required access: Administrator

- [Prerequisites](#)
- [Creating a valid kubeconfig](#)
- [Adding worker nodes](#)

Prerequisites

You must have the new CA certificates required to trust the managed cluster API.

Creating a valid `kubeconfig`

Before adding production environment worker nodes to OpenShift Container Platform clusters, you must check if you have a valid `kubeconfig`.

If the API certificates in your managed cluster changed, complete the following steps to update the `kubeconfig` with new CA certificates:

1. Check if the `kubeconfig` for your `clusterDeployment` is valid by running the following commands. Replace `<kubeconfig_name>` with the name of your current `kubeconfig` and replace `<cluster_name>` with the name of your cluster:

```
export <kubeconfig_name>=$(oc get cd $<cluster_name> -o
"jsonpath={.spec.clusterMetadata.adminKubeconfigSecretRef.name}")
oc extract secret/$<kubeconfig_name> --keys=kubeconfig --to=- > original-kubeconfig
oc --kubeconfig=original-kubeconfig get node
```

2. If you receive the following error message, you must update your `kubeconfig` secret. If you receive no error message, continue to [Adding worker nodes](#):

```
Unable to connect to the server: tls: failed to verify certificate: x509:
certificate signed by unknown authority
```

3. Get the `base64` encoded certificate bundle from your `kubeconfig certificate-authority-data` field and decode it by running the following command:

```
echo <base64 encoded blob> | base64 --decode > decoded-existing-certs.pem
```

4. Create an updated `kubeconfig` file by copying your original file. Run the following command and replace `<new_kubeconfig_name>` with the name of your new `kubeconfig` file:

```
cp original-kubeconfig <new_kubeconfig_name>
```

5. Append new certificates to the decoded pem by running the following command:

```
cat decoded-existing-certs.pem new-ca-certificate.pem | openssl base64 -A
```

6. Add the `base64` output from the previous command as the value of the `certificate-authority-data` key in your new `kubeconfig` file by using a text editor.
7. Check if the new `kubeconfig` is valid by querying the API with the new `kubeconfig`. Run the following command. Replace `<new_kubeconfig_name>` with the name of your new `kubeconfig` file:

```
KUBECONFIG=<new_kubeconfig_name> oc get nodes
```

If you receive a successful output, the `kubeconfig` is valid.

8. Update the `kubeconfig` secret in the Red Hat Advanced Cluster Management hub cluster by running the following command. Replace `<new_kubeconfig_name>` with the name of your new `kubeconfig` file:

```
oc patch secret $original-kubeconfig --type=json -p="[{ 'op': 'replace', 'path': '/data/kubeconfig', 'value': '$(openssl base64 -A -in <new_kubeconfig_name>)' }, { 'op': 'replace', 'path': '/data/raw-kubeconfig', 'value': '$(openssl base64 -A -in <new_kubeconfig_name>)' } ]"
```

Adding worker nodes

If you have a valid `kubeconfig`, complete the following steps to add production environment worker nodes to OpenShift Container Platform clusters:

1. Boot the machine that you want to use as a worker node from the ISO you previously downloaded.

Note: Make sure that the worker node meets the requirements for an OpenShift Container Platform worker node.

2. Wait for an agent to register after running the following command:

```
watch -n 5 "oc get agent -n managed-cluster"
```

3. If the agent registration is successful, an agent is listed. Approve the agent for installation. This can take a few minutes.

Note: If the agent is not listed, exit the `watch` command by pressing Ctrl and C, then log in to the worker node to troubleshoot.

4. If you are using late binding, run the following command to associate pending unbound agents with your OpenShift Container Platform cluster. Skip to step 5 if you are not using late binding:

```
oc get agent -n managed-cluster -ojson | jq -r '.items[] | select(.spec.approved==false) | select(.spec.clusterDeploymentName==null) | .metadata.name' | xargs oc -n managed-cluster patch -p '{"spec":{"clusterDeploymentName":{"name":"some-other-cluster"},"namespace":"managed-cluster"}}' --type merge agent
```

5. Approve any pending agents for installation by running the following command:

```
oc get agent -n managed-cluster -ojson | jq -r '.items[] | select(.spec.approved==false) | .metadata.name' | xargs oc -n managed-cluster patch -p '{"spec":{"approved":true}}' --type merge agent
```

Wait for the installation of the worker node. When the worker node installation is complete, the worker node contacts the managed cluster with a Certificate Signing Request (CSR) to start the joining process. The CSR is automatically signed.

Adding control plane nodes to managed clusters

You can replace a failing control plane by adding control plane nodes to healthy or unhealthy managed clusters.

Required access: Administrator

Adding control plane nodes to healthy managed clusters

Complete the following steps to add control plane nodes to healthy managed clusters:

1. Complete the steps in [Adding worker nodes to OpenShift Container Platform clusters](#) to add the new control plane node.
2. If you are using the Discovery ISO to add a node, set the agent to **master** before you approve the agent. Run the following command:

```
oc patch agent <agent-name> -p '{"spec":{"role": "master"}}' --type=merge
```

Note: Certificate Signing Requests (CSRs) are not automatically approved.

3. If you are using a **BareMetalHost** to add a node, add the following line to your **BareMetalHost** annotations when creating the **BareMetalHost** resource:

```
bmac.agent-install.openshift.io/role: master
```

4. Follow the steps in [Replacing a control plane node in a healthy cluster](#) in the Assisted Installer for OpenShift Container Platform documentation.

Adding control plane nodes to healthy managed clusters by using the API

Complete the following steps to add control plane nodes to healthy managed clusters by using the API:

1. Log in to the hub cluster by running the following command. Add the name of your cluster:

```
oc login --token=sha256~value--server=https://api.<cluster-name>.basedomain:6443
```

2. Boot the machine that you want to use as a control plane node from your Discovery ISO.

Note: Alternatively, apply the new host to your **InfraEnv** by applying the **BareMetalHost** YAML.

3. Check the agent state to ensure that the agent is ready to bind to the cluster again. Run the following commands and replace variables where needed:

- a. Run the following command to switch your inventory. Replace `<host-inventory-namespace>` with your namespace:

```
oc project <host-inventory-namespace>
```

- b. Run the following command to get the state of your agent. Replace `<agent-name>` with the name of your agent:

```
oc get agent <agent-name> -oyaml | grep state
```

- c. In the output, check the `stateInfo` line to make sure that the host is ready to bind to a cluster.

4. Set the role of your host to `master`:

- a. If you are using the Discovery ISO to add a node, approve the agent and set the role to `master`. Run the following command:

```
oc patch agent <agent-name> -p '{"spec":{"approved":true,"role":"master"}}' --type=merge
```

- b. If you are using a `BareMetalHost` to add a node, add the following line to your `BareMetalHost` annotations when creating the `BareMetalHost` resource:

```
bmac.agent-install.openshift.io/role: master
```

5. Verify the patch by running the following command:

```
oc get agent
```

In the output, ensure that each line in the `APPROVED` column says `true` and that the `ROLE` is `master`. See the following examples:

NAME	CLUSTER	APPROVED	ROLE	STAGE
c6e00642-0e11-1689-a79d-0e131d2c6ecf		true	master	

6. Add the agent to the cluster by running the following command:

```
oc patch agent <agent name> -p '{"spec":{"clusterDeploymentName":{"name":"<managed-cluster-name>"},"namespace":"<managed-cluster-namespace>"}}' --type=merge
```

7. Confirm that the agent is bound to the cluster by running the following command:

```
oc get agent
```

See the following example output:

NAME	CLUSTER	APPROVED	ROLE	STAGE
c6e00642-0e11-1689-a79d-0e131d2c6ecf	hcpagent	true	master	

8. Verify the agent status by the running the following command. While the agent is installing and joining the cluster, you might need to wait a few minutes:

```
oc get agent -w
```

Note: Certificate Signing Requests (CSRs) are not automatically approved.

9. After the agent reaches the **Rebooting** stage, you must manually approve the CSRs. Complete the following steps:
- Get the managed cluster **kubeconfig** file by running the following command:

```
kubeconfig_secret_name=$(oc -n ${cluster_name} get clusterdeployments  
${cluster_name}  
-ojsonpath='{.spec.clusterMetadata.adminKubeconfigSecretRef.name}')  
oc -n ${cluster_name} get secret ${kubeconfig_secret_name}  
-ojsonpath={.data.kubeconfig} | base64 -d > managed-cluster.kubeconfig
```

- Find any pending CSRs on the managed cluster by running the following command:

```
oc --kubeconfig managed-cluster.kubeconfig get csr | grep Pending
```

- Approve any pending CSRs for the agent in the managed cluster. Run the following command and replace **<csr-name>** with your CSR name:

```
oc --kubeconfig managed-cluster.kubeconfig adm certificate approve <csr-name>
```

The host is installed when the **STAGE** shows **Done**. See the following output example:

NAME	CLUSTER	APPROVED	ROLE	STAGE
c6e00642-0e11-1689-a79d-0e131d2c6ecf	hcpagent	true	master	Done

Adding control plane nodes to unhealthy managed clusters

Complete the following steps to add control plane nodes to unhealthy managed clusters:

1. Remove the agent for unhealthy control plane nodes.
2. If you used the zero-touch provisioning flow for deployment, remove the bare metal host.
3. Complete the steps in [Adding worker nodes to OpenShift Container Platform clusters](#) for your the new control plane node.
4. Set the agent to **master** before you approve the agent by running the following command:

```
oc patch agent <AGENT-NAME> -p '{"spec":{"role": "master"}}' --type=merge
```

Note: CSRs are not automatically approved.

5. Follow the steps in [Installing a primary control plane node on an unhealthy cluster](#) in the Assisted Installer for OpenShift Container Platform documentation.

Hibernating a created cluster

You can hibernate a cluster that was created using multicluster engine operator to conserve resources. A hibernating cluster requires significantly fewer resources than one that is running, so you can potentially lower your provider costs by moving clusters in and out of a hibernating state. This feature only applies to clusters that were created by multicluster engine operator in the following environments:

- Amazon Web Services
- Microsoft Azure
- Google Cloud Platform

Hibernate a cluster by using the console

To use the console to hibernate a cluster that was created by multicluster engine operator, complete the following steps:

1. From the navigation menu, select **Infrastructure** > **Clusters**. Ensure that the *Manage clusters* tab is selected.
2. Select **Hibernate cluster** from the *Options* menu for the cluster. **Note:** If the *Hibernate cluster* option is not available, you cannot hibernate the cluster. This can happen when the cluster is imported, and not created by multicluster engine operator.

The status for the cluster on the *Clusters* page is **Hibernating** when the process completes.

Tip: You can hibernate multiple clusters by selecting the clusters that you want to hibernate on the *Clusters* page, and selecting **Actions** > **Hibernate clusters**.

Your selected cluster is hibernating.

Hibernate a cluster by using the CLI

To use the CLI to hibernate a cluster that was created by multicluster engine operator, complete the following steps:

1. Enter the following command to edit the settings for the cluster that you want to hibernate:

```
oc edit clusterdeployment <name-of-cluster> -n <namespace-of-cluster>
```

Replace **name-of-cluster** with the name of the cluster that you want to hibernate.

Replace **namespace-of-cluster** with the namespace of the cluster that you want to hibernate.

2. Change the value for **spec.powerState** to **Hibernating**.
3. Enter the following command to view the status of the cluster:

```
oc get clusterdeployment <name-of-cluster> -n <namespace-of-cluster> -o yaml
```

Replace **name-of-cluster** with the name of the cluster that you want to hibernate.

Replace **namespace-of-cluster** with the namespace of the cluster that you want to hibernate.

When the process of hibernating the cluster is complete, the value of the type for the cluster is **type=Hibernating**.

Your selected cluster is hibernating.

Resuming normal operation of a hibernating cluster by using the console

To resume normal operation of a hibernating cluster by using the console, complete the following steps:

1. From the navigation menu, select **Infrastructure > Clusters**. Ensure that the *Manage clusters* tab is selected.
2. Select **Resume cluster** from the *Options* menu for the cluster that you want to resume.

The status for the cluster on the *Clusters* page is **Ready** when the process completes.

Tip: You can resume multiple clusters by selecting the clusters that you want to resume on the *Clusters* page, and selecting **Actions > Resume clusters**.

Your selected cluster is resuming normal operation.

Resuming normal operation of a hibernating cluster by using the CLI

To resume normal operation of a hibernating cluster by using the CLI, complete the following steps:

1. Enter the following command to edit the settings for the cluster:

```
oc edit clusterdeployment <name-of-cluster> -n <namespace-of-cluster>
```

Replace **name-of-cluster** with the name of the cluster that you want to hibernate.

Replace `namespace-of-cluster` with the namespace of the cluster that you want to hibernate.

2. Change the value for `spec.powerState` to `Running`.
3. Enter the following command to view the status of the cluster:

```
oc get clusterdeployment <name-of-cluster> -n <namespace-of-cluster> -o yaml
```

Replace `name-of-cluster` with the name of the cluster that you want to hibernate.

Replace `namespace-of-cluster` with the namespace of the cluster that you want to hibernate.

When the process of resuming the cluster is complete, the value of the type for the cluster is `type=Running`.

Your selected cluster is resuming normal operation.

Upgrading your cluster

After you create Red Hat OpenShift Container Platform clusters that you want to manage with multicluster engine operator, you can use the multicluster engine operator console to upgrade those clusters to the latest minor version that is available in the version channel that the managed cluster uses.

In a connected environment, the updates are automatically identified with notifications provided for each cluster that requires an upgrade in the console.

Prerequisites

- Verify that you meet all of the prerequisites for upgrading to that version. You must update the version channel on the managed cluster before you can upgrade the cluster with the console.

Note: After you update the version channel on the managed cluster, the multicluster engine operator console displays the latest versions that are available for the upgrade.

- Your OpenShift Container Platform managed clusters must be in a *Ready* state.

Important: You cannot upgrade Red Hat OpenShift Kubernetes Service managed clusters or OpenShift Container Platform managed clusters on Red Hat OpenShift Dedicated by using the multicluster engine operator console.

Upgrading your cluster in a connected environment

To upgrade your cluster in a connected environment, complete the following steps:

1. From the navigation menu, go to **Infrastructure** > **Clusters**. If an upgrade is available, it appears in the *Distribution version* column.
2. Select the clusters in *Ready* state that you want to upgrade. You can only upgrade OpenShift Container Platform clusters in the console.
3. Select **Upgrade**.

4. Select the new version of each cluster.
5. Select **Upgrade**.

If your cluster upgrade fails, the Operator generally retries the upgrade a few times, stops, and reports the status of the failing component. In some cases, the upgrade process continues to cycle through attempts to complete the process. Rolling your cluster back to a previous version following a failed upgrade is not supported. Contact Red Hat support for assistance if your cluster upgrade fails.

Selecting a channel

You can use the console to select a channel for your cluster upgrades on OpenShift Container Platform. After selecting a channel, you are automatically reminded of cluster upgrades that are available for both errata versions and release versions.

To select a channel for your cluster, complete the following steps:

1. From the navigation, select **Infrastructure > Clusters**.
2. Select the name of the cluster that you want to change to view the *Cluster details* page. If a different channel is available for the cluster, an edit icon is displayed in the *Channel* field.
3. Click the **Edit** icon to change the setting in the field.
4. Select a channel in the *New channel* field.

You can find the reminders for the available channel updates in the *Cluster details* page of the cluster.

Upgrading a disconnected cluster

You can use OpenShift Update Service with multicluster engine operator to upgrade clusters in a disconnected environment.

In some cases, security concerns prevent you from connecting clusters directly to the internet. Configuring OpenShift Update Service to know when upgrades are available, even when your managed cluster is not connected to the internet.

OpenShift Update Service is a separate operator that monitors the available versions of your managed clusters in a disconnected environment, and makes them available for upgrading your managed clusters. After you configure OpenShift Update Service, it can perform the following actions:

- Monitor when upgrades are available for your disconnected clusters.
- Identify which updates are mirrored to your local site for upgrading by using the graph data file.
- Notify you that an upgrade is available for your cluster by using the console.

Follow the steps in [Updating a cluster using the CLI](#) in the Red Hat OpenShift Container Platform documentation on your managed cluster to upgrade your cluster in a disconnected environment.

Enabling cluster proxy add-ons

In some environments, a managed cluster is behind a firewall and that you cannot access directly through the hub cluster. To gain access, you can set up a proxy add-on to access the `kube-apiserver` of the managed cluster with a more secure connection.

Required access: Editor

Configuring cluster proxy add-ons for hub clusters and managed clusters

To configure a cluster proxy add-on for a hub cluster or managed cluster, complete the following steps:

1. Configure the `kubeconfig` file to access the managed cluster `kube-apiserver` by completing the following steps:
 - a. Provide a valid access token for the managed cluster.

Note: : You can use the corresponding token of the service account. You can also use the default service account that is in the default namespace.

- i. Export the `kubeconfig` file of the managed cluster by running the following command:

```
export KUBECONFIG=<managed-cluster-kubeconfig>
```

- ii. Add a role to your service account that allows it to access pods by running the following commands:

```
oc create role -n default test-role --verb=list,get --resource=pods
oc create rolebinding -n default test-rolebinding
--serviceaccount=default:default --role=test-role
```

- iii. Run the following command to locate the secret of the service account token:

```
oc get secret -n default | grep <default-token>
```

Replace `default-token` with the name of your secret.

- iv. Run the following command to copy the token:

```
export MANAGED_CLUSTER_TOKEN=$(kubectl -n default get secret <default-token>
-o jsonpath={.data.token} | base64 -d)
```

Replace `default-token` with the name of your secret.

- b. Configure the `kubeconfig` file on the Red Hat Advanced Cluster Management hub cluster.
 - i. Export the current `kubeconfig` file on the hub cluster by running the following command:

```
oc config view --minify --raw=true > cluster-proxy.kubeconfig
```

- ii. Modify the **server** file with your editor. This example uses commands when using **sed**. Run **alias sed=gsed**, if you are using OSX.

```
export TARGET_MANAGED_CLUSTER=<managed-cluster-name>

export NEW_SERVER=https://$(oc get route -n multicloud-engine cluster-
proxy-addon-user -o=jsonpath='{.spec.host}')/$TARGET_MANAGED_CLUSTER

sed -i'' -e '/server:/c\    server: "$NEW_SERVER"' cluster-
proxy.kubeconfig

export CADATA=$(oc get configmap -n openshift-service-ca kube-root-ca.crt
-o=go-template='{{index .data "ca.crt"}}' | base64)

sed -i'' -e '/certificate-authority-data:/c\    certificate-authority-data:
"$CADATA"' cluster-proxy.kubeconfig
```

- iii. Delete the original user credentials by entering the following commands:

```
sed -i'' -e '/client-certificate-data/d' cluster-proxy.kubeconfig
sed -i'' -e '/client-key-data/d' cluster-proxy.kubeconfig
sed -i'' -e '/token/d' cluster-proxy.kubeconfig
```

- iv. Add the token of the service account:

```
sed -i'' -e '$a\    token: "$MANAGED_CLUSTER_TOKEN"' cluster-
proxy.kubeconfig
```

2. List all of the pods on the target namespace of the target managed cluster by running the following command:

```
oc get pods --kubeconfig=cluster-proxy.kubeconfig -n <default>
```

Replace the **default** namespace with the namespace that you want to use.

3. Access other services on the managed cluster. This feature is available when the managed cluster is a Red Hat OpenShift Container Platform cluster. The service must use **service-serving-certificate** to generate server certificates:

- From the managed cluster, use the following service account token:

```
export PROMETHEUS_TOKEN=$(kubectl get secret -n openshift-monitoring $(kubectl
get serviceaccount -n openshift-monitoring prometheus-k8s
```

```
-o=jsonpath='{.secrets[0].name}') -o=jsonpath='{.data.token}' | base64 -d)
```

- From the hub cluster, convert the certificate authority to a file by running the following command:

```
oc get configmap kube-root-ca.crt -o=jsonpath='{.data.ca\.crt}' > hub-ca.crt
```

4. Get Prometheus metrics of the managed cluster by using the following commands:

```
export SERVICE_NAMESPACE=openshift-monitoring
export SERVICE_NAME=prometheus-k8s
export SERVICE_PORT=9091
export SERVICE_PATH="api/v1/query?query=machine_cpu_sockets"
curl --cacert hub-ca.crt
$NEW_SERVER/api/v1/namespaces/$SERVICE_NAMESPACE/services/$SERVICE_NAME:$SERVICE_PORT/proxy-service/$SERVICE_PATH -H "Authorization: Bearer $PROMETHEUS_TOKEN"
```

Configuring proxy settings for cluster proxy add-ons

You can configure the proxy settings for cluster proxy add-ons to allow a managed cluster to communicate with the hub cluster through a HTTP and HTTPS proxy server. You might need to configure the proxy settings if the cluster proxy add-on agent requires access to the hub cluster through the proxy server.

To configure the proxy settings for the cluster proxy add-on, complete the following steps:

1. Create an `AddOnDeploymentConfig` resource on your hub cluster and add the `spec.proxyConfig` parameter. See the following example:

```
apiVersion: addon.open-cluster-management.io/v1alpha1
kind: AddOnDeploymentConfig
metadata:
  name: <name> ①
  namespace: <namespace> ②
spec:
  agentInstallNamespace: open-cluster-management-agent-addon
  proxyConfig:
    httpsProxy: "http://<username>:<password>@<ip>:<port>" ③
    noProxy: ".cluster.local,.svc,172.30.0.1" ④
    caBundle: <value> ⑤
```

- ① Add your add-on deployment config name.
- ② Add your managed cluster name.
- ③ Specify either a HTTP proxy or a HTTPS proxy.
- ④ Add the IP address of the `kube-apiserver`. To get the IP address, run following command on your managed cluster: `oc -n default describe svc kubernetes | grep IP:`

⑤ If you specify a HTTPS proxy in the `httpsProxy` field, set the proxy server CA bundle.

Note: The `spec.installNamespace` field is deprecated. See [Product deprecations and removals](#) to learn more.

2. Update the `ManagedClusterAddOn` resource by referencing the `AddOnDeploymentConfig` resource that you created. See the following example:

```
apiVersion: addon.open-cluster-management.io/v1alpha1
kind: ManagedClusterAddOn
metadata:
  name: cluster-proxy
  namespace: <namespace> ①
spec:
  installNamespace: open-cluster-management-addon
  configs:
    group: addon.open-cluster-management.io
    resource: addondeploymentconfigs
    name: <name> ②
    namespace: <namespace> ③
```

① Add your managed cluster name.

② Add your add-on deployment config name.

③ Add your managed cluster name.

Note: The `spec.installNamespace` field is deprecated. See [Product deprecations and removals](#) to learn more.

3. Verify the proxy settings by checking if the cluster proxy agent pod in the `open-cluster-management-addon` namespace has `HTTPS_PROXY` or `NO_PROXY` environment variables on the managed cluster.

Customizing the hub cluster *KubeAPIServer* certificates

The managed clusters communicate with the hub cluster through a mutual connection with the Red Hat OpenShift Container Platform *KubeAPIServer* external load balancer. The default OpenShift Container Platform *KubeAPIServer* certificate is issued by an internal OpenShift Container Platform cluster certificate authority (CA) when OpenShift Container Platform is installed. If necessary, you can add or change certificates.

Changing the API server certificate might impact the communication between the managed cluster and the hub cluster. When you add the named certificate before installing the product, you can avoid an issue that might leave your managed clusters in an offline state.

The following list contains some examples of when you might need to update your certificates:

- You want to replace the default API server certificate for the external load balancer with your own certificate. By following the guidance in [Adding API server certificates](#) in the OpenShift Container Platform documentation, you can add a named certificate with host name

`api.<cluster_name>.<base_domain>` to replace the default API server certificate for the external load balancer. Replacing the certificate might cause some of your managed clusters to move to an offline state. If your clusters are in an offline state after upgrading the certificates, follow the troubleshooting instructions for [Troubleshooting imported clusters offline after certificate change](#) to resolve it.

Note: Adding the named certificate before installing the product helps to avoid your clusters moving to an offline state.

- The named certificate for the external load balancer is expiring and you need to replace it. If both the old and the new certificate share the same root CA certificate, despite the number of intermediate certificates, you can follow the guidance in [Adding API server certificates](#) in the OpenShift Container Platform documentation to create a new secret for the new certificate. Then update the serving certificate reference for host name `api.<cluster_name>.<base_domain>` to the new secret in the `APIServer` custom resource. Otherwise, when the old and new certificates have different root CA certificates, complete the following steps to replace the certificate:

Procedure

1. Locate your `APIServer` custom resource, which resembles the following example:

```
apiVersion: config.openshift.io/v1
kind: APIServer
metadata:
  name: cluster
spec:
  audit:
    profile: Default
  servingCerts:
    namedCertificates:
      - names:
        - api.mycluster.example.com
      servingCertificate:
        name: old-cert-secret
```

2. Create a new secret in the `openshift-config` namespace that contains the content of the existing and new certificates.
 - a. Copy the old certificate into a new certificate by running the following command:

```
cp old.crt combined.crt
```

- b. Add the contents of the new certificate to the copy of the old certificate by running the following command:

```
cat new.crt >> combined.crt
```


c. Apply the combined certificates to create a secret by running the following command:

```
oc create secret tls combined-certs-secret --cert=combined.crt --key=old.key -n openshift-config
```

3. Update your **APIServer** resource to reference the combined certificate as the **servingCertificate**. Your **APIServer** resource might resemble the following file:

```
apiVersion: config.openshift.io/v1
kind: APIServer
metadata:
  name: cluster
spec:
  audit:
    profile: Default
  servingCerts:
    namedCertificates:
      - names:
        - api.mycluster.example.com
      servingCertificate:
        name: combined-cert-secret
```

4. After about 15 minutes, the CA bundle containing both new and old certificates is propagated to the managed clusters.
5. Create another secret named **new-cert-secret** in the **openshift-config** namespace that contains only the new certificate information by entering the following command:

```
oc create secret tls new-cert-secret --cert=new.crt --key=new.key -n openshift-config {code}
```

6. Update the **APIServer** resource by changing the name of **servingCertificate** to reference the **new-cert-secret**. Your resource might resemble the following example:

```
apiVersion: config.openshift.io/v1
kind: APIServer
metadata:
  name: cluster
spec:
  audit:
    profile: Default
  servingCerts:
    namedCertificates:
      - names:
        - api.mycluster.example.com
      servingCertificate:
        name: new-cert-secret
```

After about 15 minutes, the previous certificate is removed from the CA bundle, and the change is automatically propagated to the managed clusters.

Note: Managed clusters must use the host name `api.<cluster_name>.<base_domain>` to access the hub cluster. You cannot use named certificates that are configured with other host names.

Configuring the proxy between hub cluster and managed cluster

To register a managed cluster to your multicluster engine for Kubernetes operator hub cluster, you need to transport the managed cluster to your multicluster engine operator hub cluster. Sometimes your managed cluster cannot directly reach your multicluster engine operator hub cluster. In this instance, configure the proxy settings to allow the communications from the managed cluster to access the multicluster engine operator hub cluster through a HTTP or HTTPS proxy server.

For example, the multicluster engine operator hub cluster is in a public cloud, and the managed cluster is in a private cloud environment within a firewall-protected network. The communications out of the private cloud can only go through a HTTP or HTTPS proxy server.

Prerequisites

- You have an active HTTP or HTTPS proxy server that supports HTTP tunnels. For example, HTTP connect method.
- You have a managed cluster that can reach the HTTP or HTTPS proxy server, and the proxy server can access the multicluster engine operator hub cluster.

Complete the following steps to configure the proxy settings between your hub cluster and managed cluster:

1. Create a `KlusterConfig` resource with either a HTTP or HTTPS proxy configuration.
 - a. Apply the following YAML sample to the resource to use a HTTP proxy configuration:

```
apiVersion: config.open-cluster-management.io/v1alpha1
kind: KlusterletConfig
metadata:
  name: http-proxy
spec:
  hubKubeAPIServerConfig:
    proxyURL: "http://<username>:<password>@<ip>:<port>"
```

- b. A certificate authority (CA) bundle is required for HTTPS proxy. To create a `KlusterConfig` resource for HTTPS proxy where the required certificate authority bundle is defined, run the following command:

```
oc create -n <configmap-namespace> configmap <configmap-name> --from
-file=ca.crt=/path/to/ca/file
```

See the following configuration where the CA bundle refers to a config map that contains

one or multiple CA certificates with HTTPS proxy:

```
apiVersion: config.open-cluster-management.io/v1alpha1
kind: KlusterletConfig
metadata:
  name: https-proxy
spec:
  hubKubeAPIServerConfig:
    proxyURL: "https://<username>:<password>@<ip>:<port>"
    trustedCABundles:
      - name: "proxy-ca-bundle"
        caBundle:
          name: <configmap-name>
          namespace: <configmap-namespace>
```

2. When creating a managed cluster, choose the **KlusterletConfig** resource by adding an annotation that refers to the **KlusterletConfig** resource. See the following example:

```
apiVersion: cluster.open-cluster-management.io/v1
kind: ManagedCluster
metadata:
  annotations:
    agent.open-cluster-management.io/klusterlet-config: <klusterlet-config-name>
  name: <managed-cluster-name>
spec:
  hubAcceptsClient: true
  leaseDurationSeconds: 60
```

Notes:

- You might need to toggle the YAML view to add the annotation to the **ManagedCluster** resource when you operate on the multicluster engine operator console.
- You can use a global **KlusterletConfig** to enable the configuration on every managed cluster without using an annotation for binding.

Disabling the proxy between hub cluster and managed cluster

If your development changes, you might need to disable the HTTP or HTTPS proxy. Complete the following steps:

1. Go to the **ManagedCluster** resource.
2. Remove the **agent.open-cluster-management.io/klusterlet-config** annotation.

Optional: Configuring the klusterlet to run on specific nodes

When you create a cluster using Red Hat Advanced Cluster Management for Kubernetes, you can specify which nodes you want to run the managed cluster klusterlet by configuring the **nodeSelector** and **tolerations** annotation for the managed cluster. Complete the following steps to

configure these settings:

1. Select the managed cluster that you want to update from the Clusters page in the console.
2. Set the **YAML** switch to **On** to view the YAML content.

Note: The YAML editor is only available when importing or creating a cluster. To edit the managed cluster YAML definition after importing or creating, you must use the OpenShift Container Platform command-line interface or the Red Hat Advanced Cluster Management search feature.

3. Add the `nodeSelector` annotation to the managed cluster YAML definition. The key for this annotation is: `open-cluster-management/nodeSelector`. The value of this annotation is a string map with JSON formatting.
4. Add the `tolerations` entry to the managed cluster YAML definition. The key of this annotation is: `open-cluster-management/tolerations`. The value of this annotation represents a `toleration` list with JSON formatting. The resulting YAML might resemble the following example:

```
apiVersion: cluster.open-cluster-management.io/v1
kind: ManagedCluster
metadata:
  annotations:
    open-cluster-management/nodeSelector: '{"dedicated":"acm"}'
    open-cluster-management/tolerations:
      '[{"key":"dedicated","operator":"Equal","value":"acm","effect":"NoSchedule"}]'
```

5. To make sure that you can deploy your content to the correct nodes, see [Configuring klusterlet add-ons](#).

Additional resources

- [Enabling proxy settings for cluster proxy add-ons](#)

Accessing the managed cluster `kube-apiserver` resource to gather, configure, and view data

Use cluster proxy add-ons to access the managed cluster `kube-apiserver` resource or managed cluster services to gather specific resources, get logs, and view data in the console. Access the `kube-apiserver` by using three different methods. See the following topics:

- [Prerequisites](#)
- [Gathering data with a managed cluster user token](#)
- [Gathering data with a hub cluster user token](#)
- [Gathering data with managed cluster services](#)

Required access: Editor

Prerequisites

- You must [enable cluster proxy add-ons](#).

Gathering data with a managed cluster user token

To gather data with a managed cluster user token, list pods in the default namespace of a managed cluster by completing the following steps:

1. Export your cluster proxy URL by running the following command. Replace values where needed:

```
export CLUSTER_PROXY_URL=https://$(oc get route -n multicloud-engine cluster-proxy-addon-user -o=jsonpath='{.spec.host}')/<managed-cluster-name>
```

2. Get the **ConfigMap** by running the following command:

```
oc get configmap kube-root-ca.crt -o=jsonpath='{.data.ca\.crt}' > hub-ca.crt
```

3. Transfer your certificate authority (CA) certificate by running the following command:

```
curl --cacert hub-ca.crt -H "Authorization: Bearer $TOKEN" https://$CLUSTER_PROXY_URL/api/v1/namespaces/default/pods
```

Note: You need a user token that has permission to list pods in the default namespace of the managed cluster.

Gathering data with a hub cluster user token

To gather data with a hub cluster user token, the target hub cluster and target managed cluster both need to use the same external identity provider so that both clusters can recognize the same user.

To list pods in the default namespace by using the hub cluster **developer** user token, you must first create the correct **Role** and **RoleBinding** for the **developer** on the managed cluster.

If you are using Red Hat Advanced Cluster Management for Kubernetes, you can use the **ClusterPermission** API to create **Role** and **RoleBinding** objects from the hub cluster.

See the following example that binds the **get** and **list** permissions for pods to the user **developer**:

```
apiVersion: rbac.open-cluster-management.io/v1alpha1
kind: ClusterPermission
metadata:
  name: <name>
  namespace: <cluster-namespace>
spec:
```

```

roles:
- namespace: default
  rules:
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get","list"]
roleBindings:
- namespace: default
  roleRef:
    kind: Role
  subject:
    apiGroup: rbac.authorization.k8s.io
    kind: User
    name: developer

```

Gathering data with managed cluster services

To gather data by accessing managed cluster services, the managed cluster must be an Red Hat OpenShift Container Platform cluster and the service must use the [services-serving-certificate](#) service to generate server certificates.

Complete the following steps:

1. Get the Prometheus service token from the managed cluster. Run the following command:

```

export PROMETHEUS_TOKEN=$(kubectl get secret -n openshift-monitoring $(kubectl get serviceaccount -n openshift-monitoring prometheus-k8s -o=jsonpath='{.secrets[0].name}') -o=jsonpath='{.data.token}' | base64 -d)

```

2. Get the Prometheus managed cluster metrics from the hub cluster. Run the following commands:

```

export SERVICE_NAMESPACE=openshift-monitoring
export SERVICE_NAME=prometheus-k8s
export SERVICE_PORT=9091
export SERVICE_PATH="api/v1/query?query=machine_cpu_sockets"
curl --cacert hub-ca.crt
$CLUSTER_PROXY_URL/api/v1/namespaces/$SERVICE_NAMESPACE/services/$SERVICE_NAME:$SERVICE_PORT/proxy-service/$SERVICE_PATH -H "Authorization: Bearer $PROMETHEUS_TOKEN"

```

Configuring Ansible Automation Platform tasks to run on managed clusters

multicluster engine operator is integrated with Red Hat Ansible Automation Platform so that you can create prehook and posthook Ansible job instances that occur before or after creating or upgrading your clusters. Configuring prehook and posthook jobs for cluster destroy, and cluster scale actions are not supported.

Required access: Cluster administrator

- [Prerequisites](#)
- [Configuring an Automation template to run on a cluster by using the console](#)
- [Creating an Automation template](#)
- [Viewing the status of an Ansible job](#)
- [Pushing custom labels from the *ClusterCurator* resource to the automation job pod](#)
- [Using the *ClusterCurator* for Extended Update Support \(EUS\) upgrades](#)
- [Upgrading OpenShift Container Platform clusters by using the *ClusterCurator*](#)

Prerequisites

You must meet the following prerequisites to run Automation templates on your clusters:

- Install OpenShift Container Platform.
- Install the Ansible Automation Platform Resource Operator to connect Ansible jobs to the lifecycle of Git subscriptions. For best results when using the Automation template to launch Ansible Automation Platform jobs, the Ansible Automation Platform job template should be idempotent when it is run. You can find the Ansible Automation Platform Resource Operator in the Red Hat OpenShift Software Catalog.
- When installing the Ansible Automation Platform Resource Operator, you must select the ***-cluster-scoped** channel and select the *all namespaces* installation mode.

Configuring an *Automation* template to run on a cluster by using the console

You can specify the Automation template that you want to use for a cluster when you create the cluster, when you import the cluster, or after you create the cluster.

To specify the template when creating or importing a cluster, select the Ansible template that you want to apply to the cluster in the *Automation* step. If there are no Automation templates, click **Add automation template** to create one.

To specify the template after creating a cluster, click **Update automation template** in the action menu of an existing cluster. You can also use the **Update automation template** option to update an existing automation template.

Creating an Automation template

To initiate an Ansible job with a cluster installation or upgrade, you must create an Automation template to specify when you want the jobs to run. They can be configured to run before or after the cluster installs or upgrades.

To specify the details about running the Ansible template while creating a template, complete the steps in the console:

1. Select **Infrastructure > Automation** from the navigation.
2. Select the applicable path for your situation:
 - If you want to create a new template, click **Create Ansible template** and continue with step

3.
 - If you want to modify an existing template, click **Edit template** from the *Options* menu of the template that you want to modify and continue with step 5.
3. Enter a unique name for your template, which contains lowercase alphanumeric characters or a hyphen (-).
4. Select the credential that you want to use for the new template.
5. After you select a credential, you can select an Ansible inventory to use for all the jobs. To link an Ansible credential to an Ansible template, complete the following steps:
 - a. From the navigation, select **Automation**. Any template in the list of templates that is not linked to a credential contains a **Link to credential** icon that you can use to link the template to an existing credential. Only the credentials in the same namespace as the template are displayed.
 - b. If there are no credentials that you can select, or if you do not want to use an existing credential, select **Edit template** from the *Options* menu for the template that you want to link.
 - c. Click **Add credential** and complete the procedure in [Creating a credential for Ansible Automation Platform](#) if you have to create your credential.
 - d. After you create your credential in the same namespace as the template, select the credential in the *Ansible Automation Platform credential* field when you edit the template.
6. If you want to initiate any Ansible jobs before the cluster is installed, select **Add an Automation template** in the *Pre-install Automation templates* section.
7. Select between a **Job template** or a **Workflow job template** in the modal that appears. You can also add **job_tags**, **skip_tags**, and workflow types.
 - Use the **Extra variables** field to pass data to the **AnsibleJob** resource in the form of **key=value** pairs.
 - Special keys **cluster_deployment** and **install_config** are passed automatically as extra variables. They contain general information about the cluster and details about the cluster installation configuration.
8. Select the name of the prehook and posthook Ansible jobs to add to the installation or upgrade of the cluster.
9. Drag the Ansible jobs to change the order, if necessary.
10. Repeat steps 5 - 7 for any Automation templates that you want to initiate after the cluster is installed in the *Post-install Automation templates* section, the *Pre-upgrade Automation templates* section, and the *Post-upgrade Automation templates* section. When upgrading a cluster, you can use the **Extra variables** field to pass data to the **AnsibleJob** resource in the form of **key=value** pairs. In addition to the **cluster_deployment** and **install_config** special keys, the **cluster_info** special key is also passed automatically as an extra variable containing data from the **ManagedClusterInfo** resource.

Your Ansible template is configured to run on clusters that specify this template when the designated actions occur.

Viewing the status of an Ansible job

You can view the status of a running Ansible job to ensure that it started, and is running successfully. To view the current status of a running Ansible job, complete the following steps:

1. In the menu, select **Infrastructure** > **Clusters** to access the *Clusters* page.
2. Select the name of the cluster to view its details.
3. View the status of the last run of the Ansible job on the cluster information. The entry shows one of the following statuses:
 - When an install prehook or posthook job fails, the cluster status shows **Failed**.
 - When an upgrade prehook or posthook job fails, a warning is displayed in the *Distribution* field that the upgrade failed.

Running a failed Ansible job again

You can retry an upgrade from the *Clusters* page if the cluster prehook or posthook failed.

To save time, you can also run only the failed Ansible posthooks that are part of cluster automation templates. Complete the following steps to run only the posthooks again, without retrying the entire upgrade:

1. Add the following content to the root of the **ClusterCurator** resource to run the install posthook again:

```
operation:
  retryPosthook: installPosthook
```

2. Add the following content to the root of the **ClusterCurator** resource to run the upgrade posthook again:

```
operation:
  retryPosthook: upgradePosthook
```

After adding the content, a new job is created to run the Ansible posthook.

Specifying an Ansible inventory to use for all jobs

You can use the **ClusterCurator** resource to specify an Ansible inventory to use for all jobs. See the following example. Replace **channel** and **desiredUpdate** with the correct values for your **ClusterCurator**:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: ClusterCurator
metadata:
  name: test-inno
  namespace: test-inno
```

```
spec:
  desiredCuration: upgrade
  destroy: {}
  install: {}
  scale: {}
  upgrade:
    channel: stable-4.x
    desiredUpdate: 4.x.1
    monitorTimeout: 150
    posthook:
      - extra_vars: {}
        clusterName: test-inno
        type: post_check
        name: ACM Upgrade Checks
    prehook:
      - extra_vars: {}
        clusterName: test-inno
        type: pre_check
        name: ACM Upgrade Checks
  towerAuthSecret: awx
  inventory: Demo Inventory
```

Note: To use the example resource, the inventory must already exist in Ansible.

You can verify that the inventory is created by checking the list of available Ansible inventories from the console.

Pushing custom labels from the *ClusterCurator* resource to the automation job pod

You can use the **ClusterCurator** resource to push custom labels to the automation job pod created by the Cluster Curator. You can push the custom labels on all curation types. See the following example:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: ClusterCurator
metadata:
  name: cluster1
  namespace: cluster1
  labels:
    test1: test1
    test2: test2
spec:
  desiredCuration: install
  install:
    jobMonitorTimeout: 5
    posthook:
      - extra_vars: {}
        name: Demo Job Template
        type: Job
    prehook:
```

```
- extra_vars: {}
  name: Demo Job Template
  type: Job
towerAuthSecret: toweraccess
```

Using the *ClusterCurator* for Extended Update Support (EUS) upgrades

You can use the *ClusterCurator* resource to perform an easier, automatic upgrade between EUS releases.

1. Add *spec.upgrade.intermediateUpdate* to the *ClusterCurator* resource with the intermediate release value. See the following sample, where the intermediate release is *4.17.x*, and the *desiredUpdate* is *4.18.x*:

```
spec:
  desiredCuration: upgrade
  upgrade:
    intermediateUpdate: 4.17.x
    desiredUpdate: 4.18.x
    monitorTimeout: 120
```

2. **Optional:** You can pause the *machineconfigpools* to skip the intermediate release for faster upgrade. Enter *Unpause machinepool* in the *posthook* job, and *pause machinepool* in the *prehook* job. See the following example:

```
posthook:
  - extra_vars: {}
    name: Unpause machinepool
    type: Job
prehook:
  - extra_vars: {}
    name: Pause machinepool
    type: Job
```

See the following full example of the *ClusterCurator* that is configured to upgrade EUS to EUS:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: ClusterCurator
metadata:
  annotations:
    cluster.open-cluster-management.io/upgrade-clusterversion-backoff-limit: "10"
  name: your-name
  namespace: your-namespace
spec:
  desiredCuration: upgrade
  upgrade:
    intermediateUpdate: 4.17.x
```

```

desiredUpdate: 4.18.x
monitorTimeout: 120
posthook:
  - extra_vars: {}
    name: Unpause machinepool
    type: Job
prehook:
  - extra_vars: {}
    name: Pause machinepool
    type: Job

```

Upgrading OpenShift Container Platform clusters by using the *ClusterCurator*

You might want to upgrade your OpenShift Container Platform cluster to a version with known issues to test new features quickly.

Important: Upgrading to an OpenShift Container Platform cluster version with known issues can reduce performance and compatibility.

To upgrade to an OpenShift Container Platform cluster version with known issues, complete the following steps:

1. Create a file named `clustercurator.yaml`.
2. Add the following annotation to the file:

```

apiVersion: cluster.open-cluster-management.io/v1beta1
kind: ClusterCurator
metadata:
  annotations:
    cluster.open-cluster-management.io/upgrade-allow-not-recommended-versions:
'true'
  name: <name> ①
  namespace: <namespace> ②
spec:
  desiredCuration: upgrade
  upgrade:
    desiredUpdate: <4.x.x> ③
    monitorTimeout: 120

```

① Add your cluster name.

② Add your cluster namespace.

③ Add the OpenShift Container Platform cluster version with known issues that you want to upgrade to.

3. Apply the file. Run the following command:

```
oc -f apply clustercurator.yaml
```

The **ClusterCurator** checks the **desiredUpdate** image digest version at startup in the conditional updates list. If the image digest is not found, the **ClusterCurator** uses the image tag instead.

Configuring Ansible Automation Platform jobs to run on hosted clusters

Red Hat Ansible Automation Platform is integrated with multicluster engine operator so that you can create prehook and posthook Ansible Automation Platform job instances that occur before or after you create or update hosted clusters.

Required access: Cluster administrator

- [Running an Ansible Automation Platform job to install a hosted cluster](#)
- [Running an Ansible Automation Platform job to update a hosted cluster](#)
- [Running an Ansible Automation Platform job to delete a hosted cluster](#)

Prerequisites

You must meet the following prerequisites to run Automation templates on your clusters:

- A supported version of OpenShift Container Platform
- Install the Ansible Automation Platform Resource Operator to connect Ansible Automation Platform jobs to the lifecycle of Git subscriptions. When you use the Automation template to start Ansible Automation Platform jobs, ensure that the Ansible Automation Platform job template is idempotent when it is run. You can find the Ansible Automation Platform Resource Operator in the Red Hat OpenShift Software Catalog.

Running an Ansible Automation Platform job to install a hosted cluster

To start an Ansible Automation Platform job that installs a hosted cluster, complete the following steps:

1. Create the **HostedCluster** and **NodePool** resources, including the **pausedUntil: true** field. If you use the **hcp create cluster** command line interface command, you can specify the **--pausedUntil: true** flag.

See the following examples:

```
apiVersion: hypershift.openshift.io/v1beta1
kind: HostedCluster
metadata:
  name: my-cluster
  namespace: clusters
spec:
  pausedUntil: 'true'
```

```
apiVersion: hypershift.openshift.io/v1beta1
kind: NodePool
metadata:
```

```
name: my-cluster-us-east-2
namespace: clusters
spec:
  pausedUntil: 'true'
```

2. Create a **ClusterCurator** resource with the same name as the **HostedCluster** resource and in the same namespace as the **HostedCluster** resource. See the following example:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: ClusterCurator
metadata:
  name: my-cluster
  namespace: clusters
  labels:
    open-cluster-management: curator
spec:
  desiredCuration: install
  install:
    jobMonitorTimeout: 5
    prehook:
      - name: Demo Job Template
        extra_vars:
          variable1: something-interesting
          variable2: 2
      - name: Demo Job Template
    posthook:
      - name: Demo Job Template
  towerAuthSecret: toweraccess
```

3. If your Ansible Automation Platform Tower requires authentication, create a secret resource. See the following example:

```
apiVersion: v1
kind: Secret
metadata:
  name: toweraccess
  namespace: clusters
stringData:
  host: https://my-tower-domain.io
  token: ANSIBLE_TOKEN_FOR_admin
```

Running an Ansible Automation Platform job to update a hosted cluster

To run an Ansible Automation Platform job that updates a hosted cluster, edit the **ClusterCurator** resource of the hosted cluster that you want to update. See the following example:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
```

```

kind: ClusterCurator
metadata:
  name: my-cluster
  namespace: clusters
  labels:
    open-cluster-management: curator
spec:
  desiredCuration: upgrade
  upgrade:
    desiredUpdate: 4.17.1 ①
    monitorTimeout: 120
    prehook:
      - name: Demo Job Template
        extra_vars:
          variable1: something-interesting
          variable2: 2
      - name: Demo Job Template
    posthook:
      - name: Demo Job Template
  towerAuthSecret: toweraccess

```

① For details about supported versions, see *Hosted control planes*.

Running an Ansible Automation Platform job to update the control plane and specific node pools

To run an Ansible Automation Platform job that updates the control plane and specific node pools, edit the `nodePoolNames` field in your `ClusterCurator` resource of the hosted cluster that you want to update. See the following example:

```

apiVersion: cluster.open-cluster-management.io/v1beta1
kind: ClusterCurator
metadata:
  name: my-hosted-cluster
  namespace: clusters
spec:
  desiredCuration: upgrade
  upgrade:
    desiredUpdate: "4.17.1"
    nodePoolNames:
      - my-nodepool-1
      - my-nodepool-2

```

Running an Ansible Automation Platform job to update the control plane

To run an Ansible Automation Platform job that updates the control plane, set the `upgradeType` field to `ControlPlane` in your `ClusterCurator` resource of the hosted cluster that you want to update. See the following example:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: ClusterCurator
metadata:
  name: my-hosted-cluster
  namespace: clusters
spec:
  desiredCuration: upgrade
  upgrade:
    desiredUpdate: "4.17.1"
    upgradeType: ControlPlane
```

Running an Ansible Automation Platform job to update the node pools

To run an Ansible Automation Platform job that updates all node pools associated with your hosted cluster, set the `upgradeType` field to `NodePools` in your `ClusterCurator` resource that you want to update. See the following example:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: ClusterCurator
metadata:
  name: my-hosted-cluster
  namespace: clusters
spec:
  desiredCuration: upgrade
  upgrade:
    desiredUpdate: "4.17.1"
    upgradeType: NodePools
```

Running an Ansible Automation Platform job to update specific node pools

To run an Ansible Automation Platform job that updates only specific node pools, set the `upgradeType` field to `NodePools` in your `ClusterCurator` resource and list the node pools that you want to update in the `nodePoolNames` field. See the following example:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: ClusterCurator
metadata:
  name: my-hosted-cluster
  namespace: clusters
spec:
  desiredCuration: upgrade
  upgrade:
    desiredUpdate: "4.17.1"
    upgradeType: NodePools
    nodePoolNames:
      - my-nodepool-1
      - my-nodepool-2
```


Running an Ansible Automation Platform job to update the cluster channel

To run an Ansible Automation Platform job that updates the cluster channel without changing the version, add the cluster channel to the `channel` field. See the following example:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: ClusterCurator
metadata:
  name: my-hosted-cluster
  namespace: clusters
spec:
  desiredCuration: upgrade
  upgrade:
    channel: "fast-4.17"
```

Running an Ansible Automation Platform job to update the cluster version and channel

To run an Ansible Automation Platform job that updates the cluster version and channel, add the cluster version to the channel to the `desiredUpdate` field and add the cluster channel to the `channel` field. See the following example:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: ClusterCurator
metadata:
  name: my-hosted-cluster
  namespace: clusters
spec:
  desiredCuration: upgrade
  upgrade:
    desiredUpdate: "4.17.1"
    channel: "fast-4.17"
```

Running an Ansible Automation Platform job to delete a hosted cluster

To run an Ansible Automation Platform job that deletes a hosted cluster, edit the `ClusterCurator` resource of the hosted cluster that you want to delete. See the following example:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: ClusterCurator
metadata:
  name: my-cluster
  namespace: clusters
  labels:
    open-cluster-management: curator
spec:
  desiredCuration: destroy
  destroy:
    jobMonitorTimeout: 5
```

```

prehook:
  - name: Demo Job Template
    extra_vars:
      variable1: something-interesting
      variable2: 2
  - name: Demo Job Template
posthook:
  - name: Demo Job Template
towerAuthSecret: toweraccess

```

Note: Deleting a hosted cluster on AWS is not supported.

Additional resources

- For more information about the hosted control plane command line interface, [hcp](#), see [Installing the hosted control planes command-line interface](#).
- For more information about hosted clusters, including supported versions, see [Introduction to hosted control planes](#).

ClusterClaims

A **ClusterClaim** is a cluster-scoped custom resource definition (CRD) on a managed cluster. A **ClusterClaim** represents a piece of information that a managed cluster claims. You can use the **ClusterClaim** to determine the Placement of the resource on the target clusters.

The following example shows a **ClusterClaim** that is identified in the YAML file:

```

apiVersion: cluster.open-cluster-management.io/v1alpha1
kind: ClusterClaim
metadata:
  name: id.openshift.io
spec:
  value: 95f91f25-d7a2-4fc3-9237-2ef633d8451c

```

The following table shows the defined **ClusterClaim** list for a cluster that multicluster engine operator manages:

Claim name	Reserved	Mutable	Description
id.k8s.io	true	false	ClusterID defined in upstream proposal
kubeverison.open-cluster-management.io	true	true	Kubernetes version
platform.open-cluster-management.io	true	false	Platform the managed cluster is running on, such as AWS, GCE, and Equinix Metal

Claim name	Reserved	Mutable	Description
<code>product.open-cluster-management.io</code>	true	false	Product name, such as OpenShift, Anthos, EKS and GKE
<code>id.openshift.io</code>	false	false	OpenShift Container Platform external ID, which is only available for an OpenShift Container Platform cluster
<code>consoleurl.openshift.io</code>	false	true	URL of the management console, which is only available for an OpenShift Container Platform cluster
<code>version.openshift.io</code>	false	true	OpenShift Container Platform version, which is only available for an OpenShift Container Platform cluster

If any of the previous claims are deleted or updated on managed cluster, they are restored or rolled back to a previous version automatically.

After the managed cluster joins the hub cluster, the following two types of `ClusterClaim` resources that are created on the managed cluster are synchronized with the status of the `ManagedCluster` resource on the hub cluster. `ClusterClaim` resources with the `open-cluster-management.io/spoke-only` label are not synchronized.

`ClusterClaim` resources that multicluster engine operator manages

Includes `id.k8s.io`, or labels with the `open-cluster-management.io` or `openshift.io` suffix. These `ClusterClaim` resources are synchronized to the `ManagedCluster` status on the hub cluster.

`ClusterClaim` resources that you manage

Without the `open-cluster-management.io` or `openshift.io` suffix. By default, up to 20 `ClusterClaim` resources are synchronized to the `ManagedCluster` status on the hub cluster. Configure the `KlusterletConfig` API to change the number of `ClusterClaim` resources you manage that are synchronized. Complete the following steps:

If you are managing `ClusterClaim` resources, configure the `KlusterletConfig` API to change the number of `ClusterClaim` resources that are synchronized. Complete the following steps:

1. Create a `KlusterletConfig` and set `clusterClaimConfiguration.maxCustomClusterClaims` to your desired value. The maximum value is `100`. See the following example:

```

kind: KlusterletConfig
apiVersion: config.open-cluster-management.io/v1alpha1
metadata:
  name: klusterlet-cluster-claim-config
spec:
  clusterClaimConfiguration:
    maxCustomClusterClaims: 30

```

2. Add an annotation to refer to the **KlusterletConfig** in the **ManagedCluster** resource. Run the following command:

```

oc annotate managedcluster <cluster-name> agent.open-cluster-
management.io/klusterlet-config="klusterlet-cluster-claim-config"

```

After adding the annotation, your **ManagedCluster** resource resembles the following example:

```

apiVersion: cluster.open-cluster-management.io/v1
kind: ManagedCluster
metadata:
  annotations:
    agent.open-cluster-management.io/klusterlet-config: klusterlet-cluster-claim-
config
  name: cluster1
spec:
  hubAcceptsClient: true
  leaseDurationSeconds: 60

```

Create custom ClusterClaims

You can create a **ClusterClaim** resource with a custom name on a managed cluster, which makes it easier to identify. The custom **ClusterClaim** resource is synchronized with the status of the **ManagedCluster** resource on the hub cluster. The following content shows an example of a definition for a customized **ClusterClaim** resource:

```

apiVersion: cluster.open-cluster-management.io/v1alpha1
kind: ClusterClaim
metadata:
  name: <custom_claim_name>
spec:
  value: <custom_claim_value>

```

The length of **spec.value** field must be 1024 or less. The **create** permission on resource **clusterclaims.cluster.open-cluster-management.io** is required to create a **ClusterClaim** resource.

List existing ClusterClaims

You can use the `kubectl` command to list the ClusterClaims that apply to your managed cluster so that you can compare your ClusterClaim to an error message.

Note: Make sure you have `list` permission on resource `clusterclaims.cluster.open-cluster-management.io`.

Run the following command to list all existing ClusterClaims that are on the managed cluster:

```
kubectl get clusterclaims.cluster.open-cluster-management.io
```

ManagedClusterSets

A `ManagedClusterSet` is a group of managed clusters. A managed cluster set, can help you manage access to all of your managed clusters. You can also create a `ManagedClusterSetBinding` resource to bind a `ManagedClusterSet` resource to a namespace.

Each cluster must be a member of a managed cluster set. When you install the hub cluster, a `ManagedClusterSet` resource is created called `default`. All clusters that are not assigned to a managed cluster set are automatically assigned to the `default` managed cluster set. You cannot delete or update the `default` managed cluster set.

Continue reading to learn more about how to create and manage managed cluster sets:

- [Creating a `ManagedClusterSet`](#)
- [Assigning RBAC permissions to `ManagedClusterSets`](#)
- [Creating a `ManagedClusterSetBinding` resource](#)
- [Removing a cluster from a `ManagedClusterSet`](#)

Creating a `ManagedClusterSet`

You can group managed clusters together in a managed cluster set to limit the user access on managed clusters.

Required access: Cluster administrator

A `ManagedClusterSet` is a cluster-scoped resource, so you must have cluster administration permissions for the cluster where you are creating the `ManagedClusterSet`. A managed cluster cannot be included in more than one `ManagedClusterSet`. You can create a managed cluster set from either the multicluster engine operator console or from the CLI.

Note: Cluster pools that are not added to a managed cluster set are not added to the default `ManagedClusterSet` resource. After a cluster is claimed from the cluster pool, the cluster is added to the default `ManagedClusterSet`.

After installing multicluster engine operator, the following resources are automatically created on the hub cluster to ease management:

- A `ManagedClusterSet` called `global`.
- The namespace called `open-cluster-management-global-set`.
- A `ManagedClusterSetBinding` called `global` to bind the `global` `ManagedClusterSet` to the `open-cluster-management-global-set` namespace.

Managed clusters that you create belong to the `global` `ManagedClusterSet` by default.

Important: You cannot delete the `global` managed cluster set, or update its `spec.clusterSelector`. The `global` managed cluster set includes all managed clusters. See the following example:

```
apiVersion: cluster.open-cluster-management.io/v1beta2
kind: ManagedClusterSetBinding
metadata:
  name: global
  namespace: open-cluster-management-global-set
spec:
  clusterSet: global
```

Creating a `ManagedClusterSet` by using the CLI

Add the following definition of the managed cluster set to your YAML file to create a managed cluster set by using the CLI:

```
apiVersion: cluster.open-cluster-management.io/v1beta2
kind: ManagedClusterSet
metadata:
  name: <cluster_set>
```

Replace `<cluster_set>` with the name of your managed cluster set.

Adding a cluster to a `ManagedClusterSet`

After you create your `ManagedClusterSet`, you can add clusters to your managed cluster set by either following the instructions in the console or by using the CLI.

Adding clusters to a `ManagedClusterSet` by using the CLI

Complete the following steps to add a cluster to a managed cluster set by using the CLI:

1. Ensure that there is an RBAC `ClusterRole` entry that allows you to create on a virtual subresource of `managedclustersets/join`.

Note: Without this permission, you cannot assign a managed cluster to a `ManagedClusterSet`. If this entry does not exist, add it to your YAML file. See the following example:

```
kind: ClusterRole
apiVersion: rbac.authorization.k8s.io/v1
```

```

metadata:
  name: clusterrole1
rules:
  - apiGroups: ["cluster.open-cluster-management.io"]
    resources: ["managedclustersets/join"]
    resourceNames: ["<cluster_set>"]
    verbs: ["create"]

```

Replace `<cluster_set>` with the name of your `ManagedClusterSet`.

Note: If you are moving a managed cluster from one `ManagedClusterSet` to another, you must have that permission available on both managed cluster sets.

- Find the definition of the managed cluster in the YAML file. See the following example definition:

```

apiVersion: cluster.open-cluster-management.io/v1
kind: ManagedCluster
metadata:
  name: <cluster_name>
spec:
  hubAcceptsClient: true

```

- Add the `cluster.open-cluster-management.io/clusterset` parameter and specify the name of the `ManagedClusterSet`. See the following example:

```

apiVersion: cluster.open-cluster-management.io/v1
kind: ManagedCluster
metadata:
  name: <cluster_name>
  labels:
    cluster.open-cluster-management.io/clusterset: <cluster_set>
spec:
  hubAcceptsClient: true

```

Assigning RBAC permissions to a *ManagedClusterSet*

You can assign users or groups to your cluster set that are provided by the configured identity providers on the hub cluster.

Required access: Cluster administrator

Table 12. `ManagedClusterSet` API RBAC permission levels

Cluster set	Access permissions	Create permissions
admin	Full access permission to all of the cluster and cluster pool resources that are assigned to the managed cluster set.	Permission to create clusters, import clusters, and create cluster pools. The permissions must be assigned to the managed cluster set when it is created.
bind	Permission to bind the cluster set to a namespace by creating a ManagedClusterSetBinding . The user or group must also have permission to create the ManagedClusterSetBinding in the target namespace. Read only permissions to all of the cluster and cluster pool resources that are assigned to the managed cluster set.	No permission to create clusters, import clusters, or create cluster pools.
view	Read only permission to all of the cluster and cluster pool resources that are assigned to the managed cluster set.	No permission to create clusters, import clusters, or create cluster pools.

Note: You cannot apply the Cluster set **admin** permission for the global cluster set.

Complete the following steps to assign users or groups to your managed cluster set from the console:

1. From the OpenShift Container Platform console, navigate to **Infrastructure > Clusters**.
2. Select the *Cluster sets* tab.
3. Select your target cluster set.
4. Select the *Access management* tab.
5. Select **Add user or group**.
6. Search for, and select the user or group that you want to provide access.
7. Select the **Cluster set admin** or **Cluster set view** role to give to the selected user or user group. See *Overview of roles* in [multicluster engine operator Role-based access control](#) for more information.
8. Select **Add** to submit the changes.

Your user or group is displayed in the table. It might take a few seconds for the permission assignments for all of the managed cluster set resources to be propagated to your user or group.

See [Filtering ManagedClusters from ManagedClusterSets](#) for placement information.

Creating a *ManagedClusterSetBinding* resource

A *ManagedClusterSetBinding* resource binds a *ManagedClusterSet* resource to a namespace. Applications and policies that are created in the same namespace can only access clusters that are included in the bound managed cluster set resource.

Access permissions to the namespace automatically apply to a managed cluster set that is bound to that namespace. If you have access permissions to that namespace, you automatically have permissions to access any managed cluster set that is bound to that namespace. If you only have permissions to access the managed cluster set, you do not automatically have permissions to access other managed cluster sets on the namespace.

You can create a managed cluster set binding by using the console or the command line.

Creating a *ManagedClusterSetBinding* by using the console

Complete the following steps to create a *ManagedClusterSetBinding* by using the console:

1. From the OpenShift Container Platform console, navigate to **Infrastructure** > **Clusters** and select the *Cluster sets* tab.
2. Select the name of the cluster set that you want to create a binding for.
3. Navigate to **Actions** > **Edit namespace bindings**.
4. On the *Edit namespace bindings* page, select the namespace to which you want to bind the cluster set from the drop-down menu.

Creating a *ManagedClusterSetBinding* by using the CLI

Complete the following steps to create a *ManagedClusterSetBinding* by using the CLI:

1. Create the *ManagedClusterSetBinding* resource in your YAML file.

Note: When you create a managed cluster set binding, the name of the managed cluster set binding must match the name of the managed cluster set to bind. Your *ManagedClusterSetBinding* resource might resemble the following information:

```
apiVersion: cluster.open-cluster-management.io/v1beta2
kind: ManagedClusterSetBinding
metadata:
  namespace: <namespace>
  name: <cluster_set>
spec:
  clusterSet: <cluster_set>
```

2. Ensure that you have the bind permission on the target managed cluster set. View the following example of a *ClusterRole* resource, which contains rules that allow the user to bind to *<cluster_set>*:

```
apiVersion: rbac.authorization.k8s.io/v1
```

```
kind: ClusterRole
metadata:
  name: <clusterrole>
rules:
  - apiGroups: ["cluster.open-cluster-management.io"]
    resources: ["managedclustersets/bind"]
    resourceNames: ["<cluster_set>"]
    verbs: ["create"]
```

Placing managed clusters by using taints and tolerations

You can control the placement of your managed clusters or managed cluster sets by using taints and tolerations. Taints and tolerations provide a way to prevent managed clusters from being selected for certain placements. This control can be helpful if you want to prevent certain managed clusters from being included in some placements. You can add a taint to the managed cluster, and add a toleration to the placement. If the taint and the toleration do not match, then the managed cluster is not selected for that placement.

Adding a taint to a managed cluster

Taints are specified in the properties of a managed cluster and allow a placement to repel a managed cluster or a set of managed clusters.

If the taints section does not exist, you can add a taint to a managed cluster by running a command that resembles the following example:

```
oc patch managedcluster <managed_cluster_name> -p '{"spec":{"taints":[{"key": "key", "value": "value", "effect": "NoSelect"}]}}' --type=merge
```

Alternatively, you can append a taint to existing taints by running a command similar to the following example:

```
oc patch managedcluster <managed_cluster_name> --type=json -p='[{"op": "add", "path": "/spec/taints/-", "value": {"key": "key", "value": "value", "effect": "NoSelect"}}]'
```

The specification of a taint includes the following fields:

- **Required Key** - The taint key that is applied to a cluster. This value must match the value in the toleration for the managed cluster to meet the criteria for being added to that placement. You can determine this value. For example, this value could be `bar` or `foo.example.com/bar`.
- **Optional Value** - The taint value for the taint key. This value must match the value in the toleration for the managed cluster to meet the criteria for being added to that placement. For example, this value could be `value`.
- **Required Effect** - The effect of the taint on placements that do not tolerate the taint, or what occurs when the taint and the toleration of the placement do not match. The value of the effects

must be one of the following values:

- **NoSelect** - Placements are not allowed to select a cluster unless they tolerate this taint. If the cluster was selected by the placement before the taint was set, the cluster is removed from the placement decision.
- **NoSelectIfNew** - The scheduler cannot select the cluster if it is a new cluster. Placements can only select the cluster if they tolerate the taint and already have the cluster in their cluster decisions.
- **Required TimeAdded** - The time when the taint was added. This value is automatically set.

Identifying built-in taints to reflect the status of managed clusters

When a managed cluster is not accessible, you do not want the cluster added to a placement. The following taints are automatically added to managed clusters that are not accessible:

- **cluster.open-cluster-management.io/unavailable** - This taint is added to a managed cluster when the cluster has a condition of **ManagedClusterConditionAvailable** with status of **False**. The taint has the effect of **NoSelect** and an empty value to prevent an unavailable cluster from being scheduled. An example of this taint is provided in the following content:

```
apiVersion: cluster.open-cluster-management.io/v1
kind: ManagedCluster
metadata:
  name: cluster1
spec:
  hubAcceptsClient: true
  taints:
    - effect: NoSelect
      key: cluster.open-cluster-management.io/unavailable
      timeAdded: '2022-02-21T08:11:54Z'
```

- **cluster.open-cluster-management.io/unreachable** - This taint is added to a managed cluster when the status of the condition for **ManagedClusterConditionAvailable** is either **Unknown** or has no condition. The taint has effect of **NoSelect** and an empty value to prevent an unreachable cluster from being scheduled. An example of this taint is provided in the following content:

```
apiVersion: cluster.open-cluster-management.io/v1
kind: ManagedCluster
metadata:
  name: cluster1
spec:
  hubAcceptsClient: true
  taints:
    - effect: NoSelect
      key: cluster.open-cluster-management.io/unreachable
      timeAdded: '2022-02-21T08:11:06Z'
```

Adding a toleration to a placement

Tolerations are applied to placements, and allow the placements to repel managed clusters that do not have taints that match the tolerations of the placement. The specification of a toleration includes the following fields:

- **Optional** Key - The key matches the taint key to allow the placement.
- **Optional** Value - The value in the toleration must match the value of the taint for the toleration to allow the placement.
- **Optional** Operator - The operator represents the relationship between a key and a value. Valid operators are `equal` and `exists`. The default value is `equal`. A toleration matches a taint when the keys are the same, the effects are the same, and the operator is one of the following values:
 - `equal` - The operator is `equal` and the values are the same in the taint and the toleration.
 - `exists` - The wildcard for value, so a placement can tolerate all taints of a particular category.
- **Optional** Effect - The taint effect to match. When left empty, it matches all taint effects. The allowed values when specified are `NoSelect` or `NoSelectIfNew`.
- **Optional** TolerationSeconds - The length of time, in seconds, that the toleration tolerates the taint before moving the managed cluster to a new placement. If the effect value is not `NoSelect` or `PreferNoSelect`, this field is ignored. The default value is `nil`, which indicates that there is no time limit. The starting time of the counting of the `TolerationSeconds` is automatically listed as the `TimeAdded` value in the taint, rather than in the value of the cluster scheduled time or the `TolerationSeconds` added time.

The following example shows how to configure a toleration that tolerates clusters that have taints:

- Taint on the managed cluster for this example:

```
apiVersion: cluster.open-cluster-management.io/v1
kind: ManagedCluster
metadata:
  name: cluster1
spec:
  hubAcceptsClient: true
  taints:
    - effect: NoSelect
      key: gpu
      value: "true"
      timeAdded: '2022-02-21T08:11:06Z'
```

- Toleration on the placement that allows the taint to be tolerated

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: placement1
```

```

namespace: default
spec:
  tolerations:
    - key: gpu
      value: "true"
      operator: Equal

```

With the example tolerations defined, `cluster1` could be selected by the placement because the `key: gpu` and `value: "true"` match.

Note: A managed cluster is not guaranteed to be placed on a placement that contains a toleration for the taint. If other placements contain the same toleration, the managed cluster might be placed on one of those placements.

Specifying a temporary toleration

The value of `TolerationSeconds` specifies the period of time that the toleration tolerates the taint. This temporary toleration can be helpful when a managed cluster is offline and you can transfer applications that are deployed on this cluster to another managed cluster for a tolerated time.

For example, the managed cluster with the following taint becomes unreachable:

```

apiVersion: cluster.open-cluster-management.io/v1
kind: ManagedCluster
metadata:
  name: cluster1
spec:
  hubAcceptsClient: true
  taints:
    - effect: NoSelect
      key: cluster.open-cluster-management.io/unreachable
      timeAdded: '2022-02-21T08:11:06Z'

```

If you define a placement with a value for `TolerationSeconds`, as in the following example, the workload transfers to another available managed cluster after 5 minutes.

```

apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: demo4
  namespace: demo1
spec:
  tolerations:
    - key: cluster.open-cluster-management.io/unreachable
      operator: Exists
      tolerationSeconds: 300

```

The application is moved to another managed cluster after the managed cluster is unreachable for

5 minutes.

Additional resources

- To learn how to use `oc patch`, see [oc patch](#) the OpenShift Container Platform documentation

Setting the hub cluster `KubeAPIServer` verification strategy

Managed clusters communicate with the hub cluster through a mutual connection with the OpenShift Container Platform `KubeAPIServer` external load balancer. An internal OpenShift Container Platform cluster certificate authority (CA) issues the default OpenShift Container Platform `KubeAPIServer` certificate when you install OpenShift Container Platform. The multicluster engine for Kubernetes operator automatically detects and adds the certificate to managed clusters in the `bootstrap-kubeconfig-secret` namespace.

If your automatically detected certificate does not work, you can manually configure a strategy configuration in the `KlusterletConfig` resource. Manually configuring the strategy allows you to control how you verify the hub cluster `KubeAPIServer` certificate.

Required access: Cluster administrator

See the examples in one of the following three strategies to learn how to manually configure a strategy:

Setting the strategy with `UseAutoDetectedCABundle`

The default configuration strategy is `UseAutoDetectedCABundle`. The multicluster engine operator automatically detects the certificate on the hub cluster and merges the certificate configured in the `trustedCABundles` list of config map references to the real CA bundles, if there are any.

The following example merges the automatically detected certificates from the hub cluster and the certificates that you configured in the `new-ocp-ca` config map, and adds both to the managed cluster:

```
apiVersion: config.open-cluster-management.io/v1alpha1
kind: KlusterletConfig
metadata:
  name: ca-strategy
spec:
  hubKubeAPIServerConfig:
    serverVerificationStrategy: UseAutoDetectedCABundle
  trustedCABundles:
    - name: new-ca
      caBundle:
        name: new-ocp-ca
        namespace: default
```

Setting the strategy with `UseSystemTruststore`

When you define `UseSystemTruststore` as a value, multicluster engine operator does not detect any certificate and ignores the certificates configured in the `trustedCABundles` specification. This

configuration does not pass any certificate to the managed clusters. Instead, the managed clusters use certificates from the system trusted store of the managed clusters to verify the hub cluster API server. This applies to situations where a public CA, such as [Let's Encrypt](#), issues the hub cluster certificate. See the following example that uses `UseSystemTruststore` as the parameter value:

```
apiVersion: config.open-cluster-management.io/v1alpha1
kind: KlusterletConfig
metadata:
  name: ca-strategy
spec:
  hubKubeAPIServerConfig:
    serverVerificationStrategy: UseSystemTruststore
```

Setting the strategy with `UseCustomCABundles`

- You can use `UseCustomCABundles` if you know the CA of the hub cluster API server and do not want multicluster engine operator to automatically detect it. For this strategy, multicluster engine operator adds your configured certificates from the `trustedCABundles` parameter to the managed clusters. See the following examples to learn how to use `UseCustomCABundles`:

```
apiVersion: config.open-cluster-management.io/v1alpha1
kind: KlusterletConfig
metadata:
  name: ca-strategy
spec:
  hubKubeAPIServerConfig:
    serverVerificationStrategy: UseCustomCABundles
    trustedCABundles:
      - name: ca
        caBundle:
          name: ocp-ca
          namespace: default
```

- Typically, this policy is the same for each managed cluster. As the hub cluster administrator, you can configure a `KlusterletConfig` named `global` to activate the policy for each managed cluster when you install multicluster engine operator or the hub cluster certificate changes. See the following example:

```
apiVersion: config.open-cluster-management.io/v1alpha1
kind: KlusterletConfig
metadata:
  name: global
spec:
  hubKubeAPIServerConfig:
    serverVerificationStrategy: UseSystemTruststore
```

- When a managed cluster needs to use a different strategy, you can also create a different

`KlusterletConfig` and use the `agent.open-cluster-management.io/klusterlet-config` annotation in the managed clusters to point to a specific strategy. See the following example:

```
apiVersion: config.open-cluster-management.io/v1alpha1
kind: KlusterletConfig
metadata:
  name: test-ca
spec:
  hubKubeAPIServerConfig:
    serverVerificationStrategy: UseCustomCABundles
    trustedCABundles:
      - name: ca
        caBundle:
          name: ocp-ca
          namespace: default
  --
apiVersion: cluster.open-cluster-management.io/v1
kind: ManagedCluster
metadata:
  annotations:
    agent.open-cluster-management.io/klusterlet-config: test-ca
  name: cluster1
spec:
  hubAcceptsClient: true
  leaseDurationSeconds: 60
```

Setting managed namespaces for *ManagedClusterSets*

You can define one or more managed namespaces in a `ManagedClusterSet` to manage and scale clusters efficiently, or to apply configurations consistently across your fleet. If you specify managed namespaces, they are automatically created on every managed cluster that belongs to the set.

Important: Namespaces that you create on managed clusters are not deleted, even in the following situations:

- You remove the namespace from the `ManagedClusterSet` specification
- You move the managed cluster to a different `ManagedClusterSet`
- You detach the managed cluster from the hub cluster

Complete the following steps to create namespaces for your `ManagedClusterSet` resource:

1. Add the name of your managed namespaces. See the following example of a `ManagedClusterSet` resource where the `my-cluster-ns1` and `my-cluster-ns2` managed namespaces are defined:

```
apiVersion: cluster.open-cluster-management.io/v1beta2
kind: ManagedClusterSet
metadata:
  name: my-cluster-set
```



```
spec:
  clusterSelector:
    selectorType: ExclusiveClusterSetLabel
  managedNamespaces:
    - name: my-cluster-set-ns1
    - name: my-cluster-set-ns2
```

1. Check the managed namespaces in the **status** of each **ManagedCluster** that is part of the set.

See the following **ManagedCluster status** with managed namespaces. The **conditions.type** shows if the namespace is available on the managed cluster:

```
apiVersion: cluster.open-cluster-management.io/v1
kind: ManagedCluster
metadata:
  labels:
    cluster.open-cluster-management.io/cluster-set: my-cluster-set
  name: cluster1
status:
  managedNamespaces:
    - clusterSet: my-cluster-set
      conditions:
        - lastTransitionTime: "2025-09-26T03:15:52Z"
          message: Namespace successfully applied and managed
          reason: NamespaceApplied
          status: "True"
          type: NamespaceAvailable
        name: my-cluster-set-ns1
    - clusterSet: my-cluster-set
      conditions:
        - lastTransitionTime: "2025-09-26T03:15:52Z"
          message: Namespace successfully applied and managed
          reason: NamespaceApplied
          status: "True"
          type: NamespaceAvailable
        name: my-cluster-set-ns2
```

Removing a managed cluster from a *ManagedClusterSet*

You might want to remove a managed cluster from a managed cluster set to move it to a different managed cluster set, or remove it from the management settings of the set. You can remove a managed cluster from a managed cluster set by using the console or the CLI.

Notes:

- Every managed cluster must be assigned to a managed cluster set. If you remove a managed cluster from a **ManagedClusterSet** and do not assign it to a different **ManagedClusterSet**, the cluster is automatically added to the **default** managed cluster set.
- If the Submariner add-on is installed on your managed cluster, you must uninstall the add-on

before removing your managed cluster from a **ManagedClusterSet**.

Removing a cluster from a ManagedClusterSet by using the console

Complete the following steps to remove a cluster from a managed cluster set by using the console:

1. Click **Infrastructure > Clusters** and ensure that the *Cluster sets* tab is selected.
2. Select the name of the cluster set that you want to remove from the managed cluster set to view the cluster set details.
3. Select **Actions > Manage resource assignments**.
4. On the *Manage resource assignments* page, remove the checkbox for the resources that you want to remove from the cluster set.

This step removes a resource that is already a member of the cluster set. You can see if the resource is already a member of a cluster set by viewing the details of the managed cluster.

Note: If you are moving a managed cluster from one managed cluster set to another, you must have the required RBAC permissions on both managed cluster sets.

Removing a cluster from a ManagedClusterSet by using the CLI

To remove a cluster from a managed cluster set by using the command line, complete the following steps:

1. Run the following command to display a list of managed clusters in the managed cluster set:

```
oc get managedclusters -l cluster.open-cluster-  
management.io/clusterSet=<cluster_set>
```

Replace **cluster_set** with the name of the managed cluster set.

2. Locate the entry for the cluster that you want to remove.
3. Remove the label from the YAML entry for the cluster that you want to remove. See the following code for an example of the label:

```
labels:  
  cluster.open-cluster-management.io/clusterSet: clusterset1
```

Note: If you are moving a managed cluster from one cluster set to another, you must have the required RBAC permission on both managed cluster sets.

Placement

A placement resource is a namespace-scoped resource that defines a rule to select a set of **ManagedClusters** from the **ManagedClusterSets**, which are bound to the placement namespace.

Required access: Cluster administrator, Cluster set administrator

Continue reading to learn more about how to use placements:

- [Placement overview](#)
- [Filtering with *ManagedCluster* objects](#)
- [Checking selected *ManagedClusters* by using *PlacementDecisions*](#)

Placement overview

See the following information about how placement with managed clusters works:

- Kubernetes clusters are registered with the hub cluster as cluster-scoped *ManagedClusters*.
- The *ManagedClusters* are organized into cluster-scoped *ManagedClusterSets*.
- The *ManagedClusterSets* are bound to workload namespaces.
- The namespace-scoped placements specify a portion of *ManagedClusterSets* that select a working set of the potential *ManagedClusters*.
- Placements filter *ManagedClusters* from *ManagedClusterSets* by using *labelSelector* and *claimSelector*.
- The placement of *ManagedClusters* can be controlled by using taints and tolerations.
- Placements sort the clusters by *Prioritizers* scores and select the top *n* clusters from that group. You can define *n* in *numberOfClusters*.
- Placements do not select managed clusters that you are deleting.

Notes:

- You must bind at least one *ManagedClusterSet* to a namespace by creating a *ManagedClusterSetBinding* in that namespace.
- You must have role-based access to *CREATE* on the virtual sub-resource of *managedclustersets/bind*.

Additional resources

- See [Using taints and tolerations to place managed clusters](#) for more information.
- To learn more about the API and *Prioritizers*, see [Placements API](#).
- Return to [Selecting *ManagedClusters* with placement](#).

Filtering with *ManagedCluster* objects

You can select which *ManagedCluster* objects to filter by using *labelSelector*, *claimSelector*, or *celSelector* fields. See the following examples to learn how to use the filters:

- In the following example, the *labelSelector* field only matches clusters with the *vendor: OpenShift* label:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
```

```

metadata:
  name: placement
  namespace: ns1
spec:
  predicates:
    - requiredClusterSelector:
        labelSelector:
          matchLabels:
            vendor: OpenShift

```

- In the following example, `claimSelector` only matches clusters with `region.open-cluster-management.io` with `us-west-1`:

```

apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: placement
  namespace: ns1
spec:
  predicates:
    - requiredClusterSelector:
        claimSelector:
          matchExpressions:
            - key: region.open-cluster-management.io
              operator: In
              values:
                - us-west-1

```

- You can also filter `ManagedClusters` from particular cluster sets by using the `clusterSets` parameter. In the following example, `claimSelector` only matches the cluster sets `clusterset1` and `clusterset2`:

```

apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: placement
  namespace: ns1
spec:
  clusterSets:
    - clusterset1
    - clusterset2
  predicates:
    - requiredClusterSelector:
        claimSelector:
          matchExpressions:
            - key: region.open-cluster-management.io
              operator: In
              values:

```

- The following example displays five different ways of how you can use the `celSelector` field with Common Expression Language. You can use one or several expressions to filter your `ManagedCluster` objects:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: placement1
  namespace: ns1
spec:
  numberOfClusters: 3
  clusterSets:
    - clusterSet1
  predicates:
    - requiredClusterSelector:
        celSelector:
          celExpressions:
            - managedCluster.status.version.kubernetes == "v1.31.0" ①
            - managedCluster.status.clusterClaims.exists(c, c.name == "kubeversion.open-cluster-management.io" && c.value == "v1.31.0") ②
            - managedCluster.metadata.labels["version"].matches('^1\\.(30|31).d+$') ③
            -
              semver(managedCluster.metadata.labels["version"]).isGreaterThan(semver("1.30.0")) ④
            - managedCluster.scores("resource-usage-score").filter(s, s.name ==
              'memNodeAvailable').all(e, e.value > 0) ⑤
```

- ① Select clusters by Kubernetes version that is listed in the `managedCluster.Status.version.kubernetes` field.
- ② Select clusters by information that is stored in the `clusterClaims` field.
- ③ Use Common Expression Language standard macros and standard functions on the `managedCluster` fields.
- ④ Use Kubernetes `semver` library functions `isLessThan` and `isGreaterThan` to select clusters by version comparison.
- ⑤ Use customized function scores to select clusters by the `AddonPlacementScore` field.

See *Common Expression Language in Kubernetes* in the additional resources section to learn more.

You can also choose how many `ManagedClusters` you want to filter by using the `numberOfClusters` parameter. See the following example:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: placement
  namespace: ns1
```

```
spec:
  numberOfClusters: 3 ①
  predicates:
    - requiredClusterSelector:
        labelSelector:
          matchLabels:
            vendor: OpenShift
        claimSelector:
          matchExpressions:
            - key: region.open-cluster-management.io
              operator: In
              values:
                - us-west-1
```

① Specify how many **ManagedClusters** you want to select. The previous example is set to 3.

Filtering **ManagedClusters** by defining tolerations with placement

To learn how to filter **ManagedClusters** with matching taints, see the following examples:

- By default, the placement cannot select **cluster1** in the following example:

```
apiVersion: cluster.open-cluster-management.io/v1
kind: ManagedCluster
metadata:
  name: cluster1
spec:
  hubAcceptsClient: true
  taints:
    - effect: NoSelect
      key: gpu
      value: "true"
      timeAdded: '2022-02-21T08:11:06Z'
```

To select **cluster1** you must define tolerations. See the following example:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: placement
  namespace: ns1
spec:
  tolerations:
    - key: gpu
      value: "true"
      operator: Equal
```

You can also select **ManagedClusters** with matching taints for a specified amount of time by using the **tolerationSeconds** parameter. **tolerationSeconds** defines how long a toleration stays bound to a

taint. `tolerationSeconds` can automatically transfer applications that are deployed on a cluster that goes offline to another managed cluster after a specified length of time.

Learn how to use `tolerationSeconds` by viewing the following examples:

- In the following example, the managed cluster becomes unreachable:

```
apiVersion: cluster.open-cluster-management.io/v1
kind: ManagedCluster
metadata:
  name: cluster1
spec:
  hubAcceptsClient: true
  taints:
    - effect: NoSelect
      key: cluster.open-cluster-management.io/unreachable
      timeAdded: '2022-02-21T08:11:06Z'
```

If you define a placement with `tolerationSeconds`, the workload is transferred to another available managed cluster. See the following example:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: placement
  namespace: ns1
spec:
  tolerations:
    - key: cluster.open-cluster-management.io/unreachable
      operator: Exists
      tolerationSeconds: 300 ①
```

① Specify after how many seconds you want the workload to be transferred.

Filtering `ManagedClusters` based on add-on status

You might want to select managed clusters for your placements based on the status of the add-ons that are deployed on them. For example, you can select a managed cluster for your placement only if there is a specific add-on that is enabled on the managed cluster.

You can specify the label for the add-on, as well as its status, when you create the placement. A label is automatically created on a `ManagedCluster` resource if an add-on is enabled on the managed cluster. The label is automatically removed if the add-on is disabled.

Each add-on is represented by a label in the format of `feature.open-cluster-management.io/addon-<addon_name>=<status_of_addon>`.

Replace `addon_name` with the name of the add-on that you want to enable on the selected managed cluster.

Replace `status_of_addon` with the status that you want the add-on to have if the managed cluster is selected.

Table 13. Value for `status_of_addon`

Value	Description
<code>available</code>	The add-on is enabled and available.
<code>unhealthy</code>	The add-on is enabled, but the lease is not updated continuously.
<code>unreachable</code>	The add-on is enabled, but there is no lease found for it. This can also be caused when the managed cluster is offline.

For example, an available `application-manager` add-on is represented by a label on the managed cluster that reads the following:

```
feature.open-cluster-management.io/addon-application-manager: available
```

See the following examples to learn how to create placements based on add-ons and their status:

- The following placement example includes all managed clusters that have `application-manager` enabled on them:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: placement1
  namespace: ns1
spec:
  predicates:
    - requiredClusterSelector:
        labelSelector:
          matchExpressions:
            - key: feature.open-cluster-management.io/addon-application-manager
              operator: Exists
```

- The following placement example includes all managed clusters that have `application-manager` enabled with an `available` status:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: placement2
  namespace: ns1
spec:
  predicates:
    - requiredClusterSelector:
```



```
labelSelector:
  matchLabels:
    "feature.open-cluster-management.io/addon-application-manager":
      "available"
```

- The following placement example includes all managed clusters that have **application-manager** disabled:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: placement3
  namespace: ns1
spec:
  predicates:
    - requiredClusterSelector:
        labelSelector:
          matchExpressions:
            - key: feature.open-cluster-management.io/addon-application-manager
              operator: DoesNotExist
```

Prioritizing **ManagedClusters** by defining **prioritizerPolicy** with placement

View the following examples to learn how to prioritize **ManagedClusters** by using the **prioritizerPolicy** parameter with placement.

- The following example selects a cluster with the largest allocatable memory:

Note: Similar to Kubernetes *Node Allocatable*, 'allocatable' is defined as the amount of compute resources that are available for pods on each cluster.

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: placement
  namespace: ns1
spec:
  numberOfClusters: 1
  prioritizerPolicy:
    configurations:
      - scoreCoordinate:
          builtIn: ResourceAllocatableMemory
```

- The following example selects a cluster with the largest allocatable CPU and memory, and makes placement sensitive to resource changes:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
```

```

kind: Placement
metadata:
  name: placement
  namespace: ns1
spec:
  numberOfClusters: 1
  prioritizerPolicy:
    configurations:
      - scoreCoordinate:
          builtIn: ResourceAllocatableCPU
          weight: 2
      - scoreCoordinate:
          builtIn: ResourceAllocatableMemory
          weight: 2

```

- The following example selects two clusters with the largest `addOn` score CPU ratio, and pins the placement decisions:

```

apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: placement
  namespace: ns1
spec:
  numberOfClusters: 2
  prioritizerPolicy:
    mode: Exact
    configurations:
      - scoreCoordinate:
          builtIn: Steady
          weight: 3
      - scoreCoordinate:
          type: AddOn
          addOn:
            resourceName: default
            scoreName: cpuratio

```

Additional resources

- See [Node Allocatable](#) for more details.
- Return to [Selecting ManagedClusters with placement](#) for other topics.
- To learn more about Common Expression Language in Kubernetes, see [Common Expression Language in Kubernetes](#).

Checking selected `ManagedClusters` by using `PlacementDecisions`

One or more `PlacementDecision` kinds with the label `cluster.open-cluster-management.io/placement={placement_name}` are created to represent `ManagedClusters` selected by a

placement.

If you select a **ManagedCluster** and add it to a **PlacementDecision**, the components that consume this placement might apply the workload on this **ManagedCluster**.

When you do not select **ManagedCluster** and you remove it from the **PlacementDecision**, the workload that is applied on this **ManagedCluster** is removed. You can prevent the workload removal by defining tolerations.

To learn more about defining tolerations, see [Filtering ManagedClusters by defining tolerations with placement](#).

See the following **PlacementDecision** example:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: PlacementDecision
metadata:
  labels:
    cluster.open-cluster-management.io/placement: placement1
  name: placement1-kbc7q
  namespace: ns1
  ownerReferences:
    - apiVersion: cluster.open-cluster-management.io/v1beta1
      blockOwnerDeletion: true
      controller: true
      kind: Placement
      name: placement1
      uid: 05441cf6-2543-4ecc-8389-1079b42fe63e
status:
  decisions:
    - clusterName: cluster1
      reason: ''
    - clusterName: cluster2
      reason: ''
    - clusterName: cluster3
      reason: ''
```

Additional resources

- To learn more about the API, see [PlacementDecisions API](#).

Managing cluster pools (Technology Preview)

Cluster pools provide rapid and cost-effective access to configured Red Hat OpenShift Container Platform clusters on-demand and at scale. Cluster pools provision a configurable and scalable number of OpenShift Container Platform clusters on Amazon Web Services, Google Cloud Platform, or Microsoft Azure that can be claimed when they are needed. They are especially useful when providing or replacing cluster environments for development, continuous integration, and production scenarios. You can specify a number of clusters to keep running so that they are

available to be claimed immediately, while the remainder of the clusters will be kept in a hibernating state so that they can be resumed and claimed within a few minutes.

ClusterClaim resources are used to check out clusters from cluster pools. When a cluster claim is created, the pool assigns a running cluster to it. If no running clusters are available, a hibernating cluster is resumed to provide the cluster or a new cluster is provisioned. The cluster pool automatically creates new clusters and resumes hibernating clusters to maintain the specified size and number of available running clusters in the pool.

- [Creating a cluster pool](#)
- [Claiming clusters from cluster pools](#)
- [Updating the cluster pool release image](#)
- [Scaling cluster pools](#)
- [Destroying a cluster pool](#)

The procedure for creating a cluster pool is similar to the procedure for creating a cluster. Clusters in a cluster pool are not created for immediate use.

Creating a cluster pool

The procedure for creating a cluster pool is similar to the procedure for creating a cluster. Clusters in a cluster pool are not created for immediate use.

Required access: Administrator

Prerequisites

See the following prerequisites before creating a cluster pool:

- You need to deploy a multicluster engine operator hub cluster.
- You need Internet access for your multicluster engine operator hub cluster so that it can create the Kubernetes cluster on the provider environment.
- You need an AWS, GCP, or Microsoft Azure provider credential. See [Managing credentials overview](#) for more information.
- You need a configured domain in your provider environment. See your provider documentation for instructions about how to configure a domain.
- You need provider login credentials.
- You need your OpenShift Container Platform image pull secret. See [Using image pull secrets](#).

Note: Adding a cluster pool with this procedure configures it so it automatically imports the cluster for multicluster engine operator management when you claim a cluster from the pool. If you want to create a cluster pool that does not automatically import the claimed cluster for management with the cluster claim, add the following annotation to your **clusterClaim** resource:

```
kind: ClusterClaim
metadata:
```

annotations:

cluster.open-cluster-management.io/createmanageredcluster: "false" ①

① The word "false" must be surrounded by quotation marks to indicate that it is a string.

Create the cluster pool

To create a cluster pool, select **Infrastructure** > **Clusters** in the navigation menu. The *Cluster pools* tab lists the cluster pools that you can access. Select **Create cluster pool** and complete the steps in the console.

If you do not have a infrastructure credential that you want to use for the cluster pool, you can create one by selecting **Add credential**.

You can either select an existing namespace from the list, or type the name of a new one to create one. The cluster pool does not have to be in the same namespace as the clusters.

You can select a cluster set name if you want the RBAC roles for your cluster pool to share the role assignments of an existing cluster set. The cluster set for the clusters in the cluster pool can only be set when you create the cluster pool. You cannot change the cluster set association for the cluster pool or for the clusters in the cluster pool after you create the cluster pool. Any cluster that you claim from the cluster pool is automatically added to the same cluster set as the cluster pool.

Note: If you do not have `cluster admin` permissions, you must select a cluster set. The request to create a cluster set is rejected with a forbidden error if you do not include the cluster set name in this situation. If no cluster sets are available for you to select, contact your cluster administrator to create a cluster set and give you `clusterset admin` permissions to it.

The `cluster pool size` specifies the number of clusters that you want provisioned in your cluster pool, while the cluster pool running count specifies the number of clusters that the pool keeps running and ready to claim for immediate use.

The procedure is very similar to the procedure for creating clusters.

For specific information about the information that is required for your provider, see the following information:

- [Creating a cluster on Amazon Web Services](#)
- [Creating a cluster on Google Cloud Platform](#)
- [Creating a cluster on Microsoft Azure](#)

Claiming clusters from cluster pools

`ClusterClaim` resources are used to check out clusters from cluster pools. A claim is completed when a cluster is running and ready in the cluster pool. The cluster pool automatically creates new running and hibernated clusters in the cluster pool to maintain the requirements that are specified for the cluster pool.

Note: When a cluster that was claimed from the cluster pool is no longer needed and is destroyed, the resources are deleted. The cluster does not return to the cluster pool.

Required access: Administrator

Prerequisites

You must have a cluster pool with or without available clusters. If there are available clusters in the cluster pool, the available clusters are claimed. If there are no available clusters in the cluster pool, a cluster is created to fulfill the claim. See [Creating a cluster pool](#) for information about how to create a cluster pool.

Claim the cluster from the cluster pool

When you create a cluster claim, you request a new cluster from the cluster pool. A cluster is checked out from the pool when a cluster is available. The claimed cluster is automatically imported as one of your managed clusters, unless you disabled automatic import.

Complete the following steps to claim a cluster:

1. From the navigation menu, click **Infrastructure** > **Clusters**, and select the *Cluster pools* tab.
2. Find the name of the cluster pool you want to claim a cluster from and select **Claim cluster**.

If a cluster is available, it is claimed and immediately appears in the *Managed clusters* tab. If there are no available clusters, it might take several minutes to resume a hibernated cluster or provision a new cluster. During this time, the claim status is **pending**. Expand the cluster pool to view or delete pending claims against it.

The claimed cluster remains a member of the cluster set that it was associated with when it was in the cluster pool. You cannot change the cluster set of the claimed cluster when you claim it.

Note: Changes to the pull secret, SSH keys, or base domain of the cloud provider credentials are not reflected for existing clusters that are claimed from a cluster pool, as they have already been provisioned using the original credentials. You cannot edit cluster pool information by using the console, but you can update it by updating its information using the CLI interface. You can also create a new cluster pool with a credential that contains the updated information. The clusters that are created in the new pool use the settings provided in the new credential.

Updating the cluster pool release image

When the clusters in your cluster pool remain in hibernation for some time, the Red Hat OpenShift Container Platform release image of the clusters might become backlevel. If this happens, you can upgrade the version of the release image of the clusters that are in your cluster pool.

Required access: Edit

Complete the following steps to update the OpenShift Container Platform release image for the clusters in your cluster pool:

Note: This procedure does not update clusters from the cluster pool that are already claimed in the cluster pool. After you complete this procedure, the updates to the release images only apply to the following clusters that are related to the cluster pool:

- Clusters that are created by the cluster pool after updating the release image with this

procedure.

- Clusters that are hibernating in the cluster pool. The existing hibernating clusters with the old release image are destroyed, and new clusters with the new release image replace them.

1. From the navigation menu, click **Infrastructure > Clusters**.
2. Select the *Cluster pools* tab.
3. Find the name of the cluster pool that you want to update in the *Cluster pools* table.
4. Click the *Options* menu for the *Cluster pools* in the table, and select **Update release image**.
5. Select a new release image to use for future cluster creations from this cluster pool.

The cluster pool release image is updated.

Tip: You can update the release image for multiple cluster pools with one action by selecting the box for each of the cluster pools and using the *Actions* menu to update the release image for the selected cluster pools.

Scaling cluster pools (Technology Preview)

You can change the number of clusters in the cluster pool by increasing or decreasing the number of clusters in the cluster pool size.

Required access: Cluster administrator

Complete the following steps to change the number of clusters in your cluster pool:

1. From the navigation menu, click **Infrastructure > Clusters**.
2. Select the *Cluster pools* tab.
3. In the *Options* menu for the cluster pool that you want to change, select **Scale cluster pool**.
4. Change the value of the pool size.
5. Optionally, you can update the number of running clusters to increase or decrease the number of clusters that are immediately available when you claim them.

Your cluster pools are scaled to reflect your new values.

Destroying a cluster pool

If you created a cluster pool and determine that you no longer need it, you can destroy the cluster pool.

Important: You can only destroy cluster pools that do not have any cluster claims.

Required access: Cluster administrator

To destroy a cluster pool, complete the following steps:

1. From the navigation menu, click **Infrastructure > Clusters**.
2. Select the *Cluster pools* tab.

3. In the *Options* menu for the cluster pool that you want to delete, type **confirm** in the confirmation box and select **Destroy**.

Notes:

- The **Destroy** button is disabled if the cluster pool has any cluster claims.
- The namespace that contains the cluster pool is not deleted. Deleting the namespace destroys any clusters that have been claimed from the cluster pool, since the cluster claim resources for these clusters are created in the same namespace.

Tip: You can destroy multiple cluster pools with one action by selecting the box for each of the cluster pools and using the *Actions* menu to destroy the selected cluster pools.

Enabling ManagedServiceAccount add-ons

When you install a supported version of multicluster engine operator, the **ManagedServiceAccount** add-on is enabled by default.

The **ManagedServiceAccount** allows you to create or delete a service account on a managed cluster.

Required access: Editor

When a **ManagedServiceAccount** custom resource is created in the `<managed_cluster>` namespace on the hub cluster, a **ServiceAccount** is created on the managed cluster.

A **TokenRequest** is made with the **ServiceAccount** on the managed cluster to the Kubernetes API server on the managed cluster. The token is then stored in a **Secret** in the `<target_managed_cluster>` namespace on the hub cluster.

Note: The token can expire and be rotated. See [TokenRequest](#) for more information about token requests.

Prerequisites

- You need a supported Red Hat OpenShift Container Platform environment.
- You need the multicluster engine operator installed.

Enabling ManagedServiceAccount

To enable a **ManagedServiceAccount** add-on for a hub cluster and a managed cluster, complete the following steps:

1. Enable the **ManagedServiceAccount** add-on on hub cluster. See [Advanced configuration](#) to learn more.
2. Deploy the **ManagedServiceAccount** add-on and apply it to your target managed cluster. Create the following YAML file and replace `target_managed_cluster` with the name of the managed cluster where you are applying the **Managed-ServiceAccount** add-on:

```
apiVersion: addon.open-cluster-management.io/v1alpha1
```



```
kind: ManagedClusterAddOn
metadata:
  name: managed-serviceaccount
  namespace: <target_managed_cluster>
spec:
  installNamespace: open-cluster-management-agent-addon
```

Note: The `spec.installNamespace` field is deprecated. See [Product deprecations and removals](#) to learn more.

3. Run the following command to apply the file:

```
oc apply -f -
```

You have now enabled the `ManagedServiceAccount` plug-in for your managed cluster. See the following steps to configure a `ManagedServiceAccount`.

4. Create a `ManagedServiceAccount` custom resource with the following YAML source:

```
apiVersion: authentication.open-cluster-management.io/v1alpha1
kind: ManagedServiceAccount
metadata:
  name: <managedserviceaccount_name>
  namespace: <target_managed_cluster>
spec:
  rotation: {}
```

- Replace `managed_serviceaccount_name` with the name of your `ManagedServiceAccount`.
- Replace `target_managed_cluster` with the name of the managed cluster to which you are applying the `ManagedServiceAccount`.

5. To verify, view the `tokenSecretRef` attribute in the `ManagedServiceAccount` object status to find the secret name and namespace. Run the following command with your account and cluster name:

```
oc get managedserviceaccount <managed_serviceaccount_name> -n
<target_managed_cluster> -o yaml
```

6. View the `Secret` containing the retrieved token that is connected to the created `ServiceAccount` on the managed cluster. Run the following command:

```
oc get secret <managed_serviceaccount_name> -n <target_managed_cluster> -o yaml
```

Cluster lifecycle advanced configuration

You can configure some cluster settings during or after installation.

Removing a cluster from management

When you remove an OpenShift Container Platform cluster from management that was created with multicluster engine operator, you can either *detach* it or *destroy* it. Detaching a cluster removes it from management, but does not completely delete it. You can import it again if you want to manage it. This is only an option when the cluster is in a *Ready* state.

The following procedures remove a cluster from management in either of the following situations:

- You already deleted the cluster and want to remove the deleted cluster from Red Hat Advanced Cluster Management.
- You want to remove the cluster from management, but have not deleted the cluster.

Important:

- Destroying a cluster removes it from management and deletes the components of the cluster.
- When you detach or destroy a managed cluster, the related namespace is automatically deleted. Do not place custom resources in this namespace.
 - [Removing a cluster by using the console](#)
 - [Removing a cluster by using the command line](#)
 - [Removing remaining resources after removing a cluster](#)
 - [Defragmenting the etcd database after removing a cluster](#)

Removing a cluster by using the console

From the navigation menu, navigate to **Infrastructure** > **Clusters** and select **Destroy cluster** or **Detach cluster** from the options menu beside the cluster that you want to remove from management.

Tip: You can detach or destroy multiple clusters by selecting the check boxes of the clusters that you want to detach or destroy and selecting **Detach** or **Destroy**.

Important: If you attempt to detach the hub cluster, which is named `<your-local-cluster-name>`, be aware that the default setting of `disableHubSelfManagement` is `false` and the hub cluster is the `local-cluster`, which manages itself. This setting causes the hub cluster to reimport itself and manage itself when it is detached, and it reconciles the `MultiClusterHub` controller. It might take hours for the hub cluster to complete the detachment process and reimport.

To reimport the hub cluster without waiting for the processes to finish, you can enter the following command to restart the `multiclusterhub-operator` pod and reimport faster:

```
oc -n open-cluster-management delete -l name=multiclusterhub-operator pods
```

You can change the value of the hub cluster to not import automatically by changing the `disableHubSelfManagement` value to `true`, as described in [Installing while connected online](#).

Removing a cluster by using the command line

To detach a managed cluster by using the command line of the hub cluster, run the following command:

```
oc delete managedcluster $CLUSTER_NAME
```

To destroy the managed cluster after detaching, run the following command:

```
oc delete clusterdeployment <CLUSTER_NAME> -n $CLUSTER_NAME
```

Notes:

- To prevent destroying the managed cluster, set the `spec.preserveOnDelete` parameter to `true` in the `ClusterDeployment` custom resource.
- The default setting of `disableHubSelfManagement` is `false`. The `false` setting causes the hub cluster, which is the `local-cluster`, to reimport and manage itself when it is detached and it reconciles the `MultiClusterHub` controller.

The detachment and reimport process might take hours might take hours for the hub cluster to complete. If you want to reimport the hub cluster without waiting for the processes to finish, you can enter the following command to restart the `multiclusterhub-operator` pod and reimport faster:

```
oc -n open-cluster-management delete -l name=multiclusterhub-operator pods
```

You can change the value of the hub cluster to not import automatically by changing the `disableHubSelfManagement` value to `true`. See [Installing while connected online](#).

Removing remaining resources after removing a cluster

If there are remaining resources on the managed cluster that you removed, there are additional steps that are required to ensure that you remove all of the remaining components. Situations when these extra steps are required include the following examples:

- The managed cluster was detached before it was completely created, and components like the `klusterlet` remain on the managed cluster.
- The hub that was managing the cluster was lost or destroyed before detaching the managed cluster, and there is no way to detach the managed cluster from the hub.
- The managed cluster was not in an online state when it was detached.

If one of these situations apply to your attempted detachment of a managed cluster, there are some resources that cannot be removed from managed cluster. Complete the following steps to detach the managed cluster:

1. Make sure you have the `oc` command line interface configured.

2. Make sure you have **KUBECONFIG** configured on your managed cluster.

If you run `oc get ns | grep open-cluster-management-agent`, you should see two namespaces:

```
open-cluster-management-agent      Active   10m
open-cluster-management-agent-addon Active   10m
```

3. Remove the **klusterlet** custom resource by using the following command:

```
oc get klusterlet | grep klusterlet | awk '{print $1}' | xargs oc patch klusterlet
--type=merge -p '{"metadata":{"finalizers": []}}'
```

4. Run the following command to remove the remaining resources:

```
oc delete namespaces open-cluster-management-agent open-cluster-management-agent-
addon --wait=false
oc get crds | grep open-cluster-management.io | awk '{print $1}' | xargs oc delete
crds --wait=false
oc get crds | grep open-cluster-management.io | awk '{print $1}' | xargs oc patch
crds --type=merge -p '{"metadata":{"finalizers": []}}'
```

5. Run the following command to ensure that both namespaces and all open cluster management **crds** are removed:

```
oc get crds | grep open-cluster-management.io | awk '{print $1}'
oc get ns | grep open-cluster-management-agent
```

Defragmenting the etcd database after removing a cluster

Having many managed clusters can affect the size of the **etcd** database in the hub cluster. In OpenShift Container Platform 4.8, when you delete a managed cluster, the **etcd** database in the hub cluster is not automatically reduced in size. In some scenarios, the **etcd** database can run out of space. An error **etcdserver: mvcc: database space exceeded** is displayed. To correct this error, reduce the size of the **etcd** database by compacting the database history and defragmenting the **etcd** database.

Note: For OpenShift Container Platform version 4.9 and later, the etcd Operator automatically defragments disks and compacts the **etcd** history. No manual intervention is needed. The following procedure is for OpenShift Container Platform version 4.8 and earlier.

Compact the **etcd** history and defragment the **etcd** database in the hub cluster by completing the following procedure.

Prerequisites

- Install the OpenShift CLI (**oc**).

- Log in as a user with `cluster-admin` privileges.

Procedure

1. Compact the `etcd` history.

- a. Open a remote shell session to the `etcd` member, for example:

```
$ oc rsh -n openshift-etcd etcd-control-plane-0.example.com etcdctl endpoint status --cluster -w table
```

- b. Run the following command to compact the `etcd` history:

```
sh-4.4#etcdctl compact $(etcdctl endpoint status --write-out="json" | egrep -o '"revision":[0-9]*' | egrep -o '[0-9]*' -m1)
```

Example output

```
$ compacted revision 158774421
```

2. Defragment the `etcd` database and clear any `NOSPACE` alarms as outlined in [Defragmenting etcd data](#).

Discovery service

You can discover OpenShift Container Platform clusters that are available from [OpenShift Cluster Manager](#). After discovery, you can import your clusters to manage. The Discovery services uses the Discover Operator for back-end and console usage.

You must have an OpenShift Cluster Manager credential. See [Creating a credential for OpenShift Cluster Manager](#) if you need to create a credential.

Required access: Administrator

- [Authenticating service accounts](#)
- [Configure Discovery with the console](#)
- [Configure Discovery using the CLI](#)
- [Enabling a discovered cluster for management](#)

Authenticating service accounts

To discover OpenShift Container Platform clusters that are available from [OpenShift Cluster Manager](#), you need to authenticate with a service account. Note that OpenShift Cluster Manager offline token is deprecated.

Prerequisites

- You must have an OpenShift Cluster Manager credential. See [Creating a credential for OpenShift Cluster Manager](#) if you need to create a credential.
- Learn about creating an OpenShift Cluster Manager service account at [Service accounts](#).

Authenticating

See the following procedure to authenticate with a service account.

1. Create a new credentials secret as a service-account `client-id` and `client-secret` pair in the multicluster engine operator console.
2. Specify a service account `client-id` and `client-secret` pair so that the multicluster engine operator Discovery service can authenticate correctly and discover clusters.
3. Set up the Red Hat OpenShift Service on AWS (classic) `DiscoveredCluster` resource for import as a managed cluster. The Red Hat OpenShift Service on AWS (classic) import flow that is initiated uses the service-account credentials authenticate and import the cluster.

Table 14. List of the deployed components

Role	Access permissions
OpenShift Cluster Manager cluster autoscaler editor	Perform update operations on the cluster autoscaler
OpenShift Cluster Manager cluster editor	Perform update operations on clusters
OpenShift Cluster Manager cluster provisioner	Perform provision operations on clusters
OpenShift Cluster Manager cluster viewer	Perform read operations on clusters
OpenShift Cluster Manager IdP editor	Perform update operations on IdPs
OpenShift Cluster Manager machine pool editor	Perform update operations on machine pools
OpenShift Cluster Manager organization administrator	Perform all available operations on the associated organization's clusters
Subscriptions viewer	Perform read operations on any Subscriptions resource.

Configure Discovery with the console

Configure Discovery in the console to find clusters. When you configure the Discovery feature on your cluster, you must enable a `DiscoveryConfig` resource to connect to the OpenShift Cluster Manager to begin discovering clusters that are a part of your organization. You can create multiple `DiscoveryConfig` resources with separate credentials.

After you discover clusters, you can import clusters that appear in the *Discovered clusters* tab of the console. Use the product console to enable Discovery.

Required access: Access to the namespace where the credential was created.

Prerequisites

- You need a credential. See [Creating a credential for OpenShift Cluster Manager](#) to connect to

OpenShift Cluster Manager.

- You need access to the namespaces that were used to configure Discovery.
 1. Manually import discovered clusters from the console. To import other infrastructure provider discovered clusters, complete the following steps:
 - a. Go to the existing *Clusters* page and click the **Discovered clusters** tab.
 - b. From the *Discovered clusters* table, find the cluster that you want to import.
 - c. From the options menu, choose **Import cluster**.
 - d. For discovered clusters, you can import manually using the documentation, or you can choose Import clusters automatically.
 - e. To import automatically with your credentials or `kubeconfig` file, copy and paste the content.
 - f. Click **Import**.
 2. View and search discovered clusters in the console after you set up your credentials.
 - a. Click **Clusters** > **Discovered clusters**.
 - b. View the populated table with the following information:
 - *Name* is the display name that is designated in OpenShift Cluster Manager. If the cluster does not have a display name, a generated name based on the cluster console URL is displayed. If the console URL is missing or was modified manually in OpenShift Cluster Manager, the cluster external ID is displayed.
 - *Namespace* is the namespace where you created the credential and discovered clusters.
 - *Type* is the discovered cluster OpenShift Container Platform platform type.
 - *Distribution version* is the discovered cluster Red Hat OpenShift version.
 - *Infrastructure provider* is the cloud provider of the discovered cluster.
 - *Last active* is the last time the discovered cluster was active.
 - *Created* when the discovered cluster was created.
 - *Discovered* when the discovered cluster was discovered.
 3. You can search for any information in the table, as well. For example, to show only *Discovered clusters* in a particular namespace, search for that namespace.
 4. You can now click **Import cluster** to create managed clusters.

Enable Discovery using the CLI

Enable discovery using the CLI to find clusters that are available from OpenShift Cluster Manager.

Required access: Administrator

Prerequisites

- Create a credential to connect to OpenShift Cluster Manager.

Discovery set up and process

Note: The `DiscoveryConfig` resource must be named `discovery` and must be created in the same namespace as the selected `credential`. See the following `DiscoveryConfig` sample:

```
apiVersion: discovery.open-cluster-management.io/v1
kind: DiscoveryConfig
metadata:
  name: discovery
  namespace: <NAMESPACE_NAME>
spec:
  credential: <SECRET_NAME>
  filters:
    lastActive: 7
    openshiftVersions:
      - "4.x"
```

1. Replace `SECRET_NAME` with the credential that you previously set up.
2. Replace `NAMESPACE_NAME` with the namespace of `SECRET_NAME`.
3. Enter the maximum time since last activity of your clusters (in days) to discover. For example, with `lastActive: 7`, clusters that active in the last 7 days are discovered.
4. Enter the versions of OpenShift Container Platform clusters to discover as a list of strings. **Note:** Every entry in the `openshiftVersions` list specifies the major and minor version. For example, specifying `4.x` includes all errata releases, such as `4.x.1`, `4.x.2`.

View discovered clusters

View discovered clusters by running `oc get discoveredclusters -n <namespace>` where `namespace` is the namespace where the discovery credential exists. You can use the same command to view the region if you have a region set.

Objects are created by the Discovery controller. These `DiscoveredClusters` represent the clusters that are found in OpenShift Cluster Manager by using the filters and credentials that are specified in the `DiscoveryConfig` `discoveredclusters.discovery.open-cluster-management.io` API. The value for `name` is the cluster external ID:

```
apiVersion: discovery.open-cluster-management.io/v1
kind: DiscoveredCluster
metadata:
  name: <cluster-external-id>
  namespace: <NAMESPACE_NAME>
spec:
  activity_timestamp: "2022-04-19T21:06:14Z"
  cloudProvider: vsphere
  console: https://console-openshift-console.apps.qe1-vmware-pkt.dev02.red-
chesterfield.com
  creation_timestamp: "2022-04-19T16:29:53Z"
  credential:
```



```
apiVersion: v1
kind: Secret
name: <SECRET_NAME>
namespace: <NAMESPACE_NAME>
display_name: qe1-vmware-pkt.dev02.red-chesterfield.com
name: <cluster-external-id>
openshiftVersion: 4.x
status: Stale
```

Enabling a discovered cluster for management

Automatically import supported clusters into your hub cluster with the **Discovery-Operator** for faster cluster management, without manually importing individual clusters.

Required access: Cluster administrator

Prerequisites

- Discovery is enabled by default. If you changed default settings, you need to enable Discovery.
- You must set up the Red Hat OpenShift Service on AWS (classic) command-line interface. See [Getting started with the Red Hat OpenShift Service on AWS \(classic\) CLI](#) documentation.

Importing discovered Red Hat OpenShift Service on AWS (classic) and hosted control plane clusters automatically

The following procedure is an example of how to import your discovered Red Hat OpenShift Service on AWS (classic) and hosted control planes clusters automatically by using the **Discovery-Operator**. You can import from the console or from the command-line interface.

- Import from the console. You must modify the resource and set the **importAsManagedCluster** field to **true** in the console.
 1. Log in to your hub cluster from the console.
 2. Select **Search** from the navigation menu.
 3. From the search bar, enter the following query: "DiscoveredCluster".
 4. The **DiscoveredCluster** resource results appear.
 5. Go to the **DiscoveredCluster** resource and set **importAsManagedCluster** to **true**. See the following example, where **importAsManagedCluster** is set to **true** and **<4.x.z>** is your supported OpenShift Container Platform version:

```
apiVersion: discovery.open-cluster-management.io/v1
kind: DiscoveredCluster
metadata:
  name: 28c17977-fc73-4050-b5cc-a5aa2d1d6892
  namespace: discovery
spec:
  openshiftVersion: <4.x.z>
  isManagedCluster: false
```

```
cloudProvider: aws
name: 28c17977-fc73-4050-b5cc-a5aa2d1d6892
displayName: rosa-dc
status: Active
importAsManagedCluster: true ①
type: <supported-type> ②
```

① By setting the field to `true`, the `Discovery-Operator` imports the `DiscoveredCluster` resource, creates a `ManagedCluster` resource and if the Red Hat Advanced Cluster Management is installed, creates the `KlusterletAddOnConfig` resource. It also creates the `Secret` resources for your automatic import.

② You must use `ROSA` or `MultiClusterEngineHCP` as the parameter value.

6. To verify that the `DiscoveredCluster` resource is imported, go to the **Clusters** page. Check the import status of your cluster from the *Cluster list* tab.
7. If you want to detach managed clusters for Discovery to prevent automatic reimport, select the **Detach cluster** option. The `Discovery-Operator` adds the following annotation, `discovery.open-cluster-management.io/previously-auto-imported: 'true'`.

Your `DiscoveredCluster` resource might resemble the following YAML:

```
apiVersion: discovery.open-cluster-management.io/v1
kind: DiscoveredCluster
metadata:
  annotations:
    discovery.open-cluster-management.io/previously-auto-imported: 'true'
```

8. To verify that the `DiscoveredCluster` resource is not reimported automatically, check for the following message in the `Discovery-Operator` logs, where `"rosa-dc"` is this discovered cluster:

```
2024-06-12T14:11:43.366Z INFO reconcile Skipped automatic import for
DiscoveredCluster due to
existing 'discovery.open-cluster-management.io/previously-auto-imported'
annotation {"Name": "rosa-dc"}
```

Note: If you want to reimport the `DiscoveredCluster` resource automatically, you must remove the previously mentioned annotation.

- Import from the command-line interface.

1. To automatically import the `DiscoveredCluster` resource from the command-line interface. Set the `importAsManagedCluster` paramater to `true` by using the following command after you log in. Replace `<name>` and `<namespace>` with your name and namespace:

```
oc patch discoveredcluster <name> -n <namespace> --type='json' -p='[{"op":
"replace", "path": "/spec/importAsManagedCluster", "value": true}]'
```

2. Run the following command to verify that the cluster was imported as a managed cluster:

```
oc get managedcluster <name>
```

3. To get a description of your Red Hat OpenShift Service on AWS (classic) cluster ID, run the following command from the Red Hat OpenShift Service on AWS (classic) command-line interface:

```
rosa describe cluster --cluster=<cluster-name> | grep -o ^ID:.*
```

Important: For other Kubernetes providers, you must import these infrastructure provider `DiscoveredCluster` resources manually. Directly apply Kubernetes configurations to the other types of `DiscoveredCluster` resources. If you enable the `importAsManagedCluster` field from the `DiscoveredCluster` resource, it is not imported due to the Discovery webhook.

Host inventory introduction

The host inventory management and on-premises cluster installation are available using the multicluster engine operator central infrastructure management feature.

Continue reading to learn more about host inventories and central infrastructure management:

- [Host inventory overview](#)
- [Enabling the central infrastructure management service](#)
- [Enabling central infrastructure management on Amazon Web Services](#)
- [Creating a host inventory by using the console](#)
- [Creating a host inventory by using the command line interface](#)
- [Configuring advanced networking for an infrastructure environment](#)
- [Adding hosts to the host inventory by using the Discovery Image](#)
- [Automatically adding bare metal hosts to the host inventory](#)
- [Managing your host inventory](#)
- [Managing a hub cluster by using central infrastructure management](#)
- [Configuring mirror registries for each cluster](#)
- [Creating a cluster in an on-premises environment](#)
- [Importing an on-premises Red Hat OpenShift Container Platform cluster manually by using central infrastructure management](#)
- [Installing a two nodes with fencing cluster with Assisted Installer \(Technology Preview\)](#) to learn more.

Host inventory overview

The central infrastructure management feature is an Red Hat OpenShift Container Platform install

experience in multicluster engine operator that focuses on managing bare-metal hosts during their lifecycle.

The Assisted Installer is an install method for OpenShift Container Platform that uses agents to run pre-installation validations on the target hosts, and a central service to evaluate and track install progress.

The infrastructure operator for Red Hat OpenShift is a multicluster engine operator component that manages and installs the workloads that run the Assisted Installer service.

You can use the console to create a host inventory, which is a pool of bare-metal or virtual machines that you can use to create on-premises OpenShift Container Platform clusters. These clusters can be standalone, with dedicated machines for the control plane, or [hosted control planes](#), where the control plane runs as pods on a hub cluster.

You can install standalone clusters by using the console, API, or GitOps by using Zero Touch Provisioning (ZTP). See [Installing GitOps ZTP in a disconnected environment](#) in the Red Hat OpenShift Container Platform documentation for more information on ZTP.

A machine joins the host inventory after booting with a Discovery Image. The Discovery Image is a Red Hat CoreOS live image that contains the following:

- An agent that performs discovery, validation, and installation tasks.
- The necessary configuration for reaching the service on the hub cluster, including the endpoint, token, and static network configuration, if applicable.

You have one Discovery Image for each infrastructure environment, which is a set of hosts sharing a common set of properties. The `InfraEnv` custom resource definition represents this infrastructure environment and associated Discovery Image. You can specify the Red Hat Core OS version used for the Discovery Image by setting the `osImageVersion` field in the `InfraEnv` custom resource. If you do not specify a value, the latest Red Hat Core OS version is used.

After the host boots and the agent contacts the service, the service creates a new `Agent` custom resource on the hub cluster representing that host. The `Agent` resources make up the host inventory.

You can install hosts in the inventory as OpenShift nodes later. The agent writes the operating system to the disk, along with the necessary configuration, and reboots the host.

Note: Red Hat Advanced Cluster Management and central infrastructure management supports the Nutanix platform by using the `AgentClusterInstall` resource, which requires additional configuration by creating the Nutanix virtual machines. To learn more, see [Optional: Installing on Nutanix](#) in the Assisted Installer documentation.

Enabling the central infrastructure management service

The central infrastructure management service is provided with the multicluster engine operator and deploys OpenShift Container Platform clusters. The central infrastructure management service is deployed automatically if you installed Red Hat Advanced Cluster Management and enabled the `MultiClusterHub` Operator on the hub cluster, but you have to enable the service manually in this case.

See the following sections:

- [Creating a bare metal host custom resource definition](#)
- [Creating or modifying the *Provisioning* resource](#)
- [Enabling central infrastructure management in disconnected environments](#)
- [Enabling central infrastructure management in connected environments](#)
- [Installing a FIPS-enabled cluster by using the Assisted Installer](#)

Prerequisites

See the following prerequisites before enabling the central infrastructure management service:

- You must have a deployed hub cluster on a supported OpenShift Container Platform version and a supported Red Hat Advanced Cluster Management for Kubernetes version.
- You need internet access for your hub cluster.
- If you are working in a disconnected environment, you need a connection to an internal or mirror registry that has a connection to the internet to retrieve the required images for creating the environment.
- You must open the required ports for bare metal provisioning. See [Ensuring required ports are open](#) in the OpenShift Container Platform documentation.
- You need a bare metal host custom resource definition.
- You need an OpenShift Container Platform [pull secret](#). See *Using image pull secrets* for more information.
- You need a configured default storage class.
- For disconnected environments only, complete the procedure for [Clusters at the network far edge](#) in the OpenShift Container Platform documentation.

Creating a bare metal host custom resource definition

You need a bare metal host custom resource definition before enabling the central infrastructure management service.

1. Check if you already have a bare metal host custom resource definition by running the following command:

```
oc get crd baremetalhosts.metal3.io
```

- If you have a bare metal host custom resource definition, the output shows the date when the resource was created.
- If you do not have the resource, you receive an error that resembles the following:

```
Error from server (NotFound): customresourcedefinitions.apiextensions.k8s.io
"baremetalhosts.metal3.io" not found
```

2. If you do not have a bare metal host custom resource definition, download the [metal3.io_baremetalhosts.yaml](#) file and apply the content by running the following command to create the resource:

```
oc apply -f
```

Creating or modifying the *Provisioning* resource

You need a *Provisioning* resource before enabling the central infrastructure management service.

1. Check if you have the *Provisioning* resource by running the following command:

```
oc get provisioning
```

- If you already have a *Provisioning* resource, modify the *Provisioning* resource.
- If you do not have a *Provisioning* resource, you receive a *No resources found* error. Continue to create the *Provisioning* resource.

2. Modify the *Provisioning* resource.

If you already have a *Provisioning* resource, you must modify the resource if your hub cluster is installed on one of the following platforms:

- Bare metal
- Red Hat OpenStack Platform
- VMware vSphere
- User-provisioned infrastructure (UPI) method and the platform is *None*

If your hub cluster is installed on a different platform, continue at *Enabling central infrastructure management in disconnected environments* or *Enabling central infrastructure management in connected environments*.

Modify the *Provisioning* resource to allow the Bare Metal Operator to watch all namespaces by running the following command:

```
oc patch provisioning provisioning-configuration --type merge -p
'{"spec":{"watchAllNamespaces": true }}'
```

3. Create the *Provisioning* resource if you do not have a *Provisioning* resource.
 - a. Create the following *Provisioning* resource YAML file:

```
apiVersion: metal3.io/v1alpha1
kind: Provisioning
metadata:
  name: provisioning-configuration
```

```
spec:
  provisioningNetwork: "Disabled"
  watchAllNamespaces: true
```

b. Apply the content by running the following command:

```
oc apply -f
```

Enabling central infrastructure management in disconnected environments

To enable central infrastructure management in disconnected environments, complete the following steps:

1. Create a **ConfigMap** in the same namespace as your infrastructure operator to specify the values for **ca-bundle.crt** and **registries.conf** for your mirror registry. Your file **ConfigMap** might resemble the following example:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: <mirror-config>
  namespace: multicluster-engine
  labels:
    app: assisted-service
data:
  ca-bundle.crt: |
    <certificate-content>
  registries.conf: |
    unqualified-search-registries = ["registry.access.redhat.com", "docker.io"]
    [[registry]]
      prefix = ""
      location = "registry.redhat.io/multicluster-engine"
      mirror-by-digest-only = true
      [[registry.mirror]]
        location = "mirror.registry.com:5000/multicluster-engine"
```

Note: You must set **mirror-by-digest-only** to **true** because release images are specified by using a digest.

Registries in the list of **unqualified-search-registries** are automatically added to an authentication ignore list in the **PUBLIC_CONTAINER_REGISTRIES** environment variable. The specified registries do not require authentication when the pull secret of the managed cluster is validated.

2. Write the key pairs representing the headers and query parameters that you want to send with every **osImage** request. If you don't need both parameters, write key pairs for only headers or query parameters.

Important: Headers and query parameters are only encrypted if you use HTTPS. Make sure to use

HTTPS to avoid security issues.

- a. Create a file named `headers` and add content that resembles the following example:

```
{
  "Authorization": "Basic xyz"
}
```

- b. Create a file named `query_params` and add content that resembles the following example:

```
{
  "api_key": "myexampleapikey",
}
```

1. Create a secret from the parameter files that you created by running the following command. If you only created one parameter file, remove the argument for the file that you didn't create:

```
oc create secret generic -n multicluster-engine os-images-http-auth --from
-file=./query_params --from-file=./headers
```

2. If you want to use HTTPS `osImages` with a self-signed or third-party CA certificate, add the certificate to the `image-service-additional-ca ConfigMap`. To create a certificate, run the following command:

```
oc -n multicluster-engine create configmap image-service-additional-ca --from
-file=<file-name>
```

3. Create the `AgentServiceConfig` custom resource by saving the following YAML content in the `agent_service_config.yaml` file:

```
apiVersion: agent-install.openshift.io/v1beta1
kind: AgentServiceConfig
metadata:
  name: agent
spec:
  databaseStorage:
    accessModes:
      - ReadWriteOnce
    resources:
      requests:
        storage: <db_volume_size>
  filesystemStorage:
    accessModes:
      - ReadWriteOnce
```



```

resources:
  requests:
    storage: <fs_volume_size>
mirrorRegistryRef:
  name: <mirror_config> ①
unauthenticatedRegistries:
  - <unauthenticated_registry> ②
imageStorage:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: <img_volume_size> ③
OSImageAdditionalParamsRef:
  name: os-images-http-auth
OSImageCACertRef:
  name: image-service-additional-ca
osImages:
  - openshiftVersion: "<ocp_version>" ④
    version: "<ocp_release_version>" ⑤
    url: "<iso_url>" ⑥
    cpuArchitecture: "x86_64"

```

- ① Replace `mirror_config` with the name of the `ConfigMap` that contains your mirror registry configuration details.
- ② Include the optional `unauthenticated_registry` parameter if you are using a mirror registry that does not require authentication. Entries on this list are not validated or required to have an entry in the pull secret.
- ③ Replace `img_volume_size` with the size of the volume for the `imageStorage` field. This value specifies how much storage is allocated to store base Red Hat Enterprise Linux CoreOS images. The minimum value is `10Gi`, but the value for best practice is at least `50Gi`, especially if you are opting for the default set of images instead of defining a limited set using `osImages`. You need to allow at least `2Gi` of image storage for each instance of Red Hat Enterprise Linux CoreOS that is configured in `osImages`.
- ④ Replace `ocp_version` with the supported OpenShift Container Platform version to install.
- ⑤ Replace `ocp_release_version` with the specific install version, for example, `49.83.202103251640-0`.
- ⑥ Replace `iso_url` with the ISO url. You can find ISO url values at the [rhoc](#).

If you are using HTTPS `osImages` with self-signed or third-party CA certificates, reference the certificate in the `OSImageCACertRef` spec.

Important: If you are using the late binding feature, the OpenShift Container Platform release images that you use when creating your clusters must be the same.

You can verify that your central infrastructure management service is healthy by checking the `assisted-service` and `assisted-image-service` deployments and ensuring that their pods are ready and running.

Enabling central infrastructure management in connected environments

To enable central infrastructure management in connected environments, create the `AgentServiceConfig` custom resource by saving the following YAML content in the `agent_service_config.yaml` file:

```
apiVersion: agent-install.openshift.io/v1beta1
kind: AgentServiceConfig
metadata:
  name: agent
spec:
  databaseStorage:
    accessModes:
      - ReadWriteOnce
    resources:
      requests:
        storage: <db_volume_size> ①
  filesystemStorage:
    accessModes:
      - ReadWriteOnce
    resources:
      requests:
        storage: <fs_volume_size> ②
  imageStorage:
    accessModes:
      - ReadWriteOnce
    resources:
      requests:
        storage: <img_volume_size> ③
```

- ① Replace `db_volume_size` with the volume size for the `databaseStorage` field, for example `10Gi`. This value specifies how much storage is allocated for storing files such as database tables and database views for the clusters. The minimum value that is required is `1Gi`. You might need to use a higher value if there are many clusters.
- ② Replace `fs_volume_size` with the size of the volume for the `filesystemStorage` field, for example `200M` for each cluster and `2-3Gi` for each supported OpenShift Container Platform version. The minimum value that is required is `1Gi`, but the recommended value is at least `100Gi`. This value specifies how much storage is allocated for storing logs, manifests, and `kubeconfig` files for the clusters. You might need to use a higher value if there are many clusters.
- ③ Replace `img_volume_size` with the size of the volume for the `imageStorage` field. This value specifies how much storage is allocated to store base Red Hat Enterprise Linux CoreOS images. The minimum value is `10Gi`, but the value for best practice is at least `50Gi`, especially if you are opting for the default set of images instead of defining a limited set using `osImages`. You need to allow at least `2Gi` of image storage for each instance of Red Hat Enterprise Linux CoreOS that is configured in `osImages`.

See *Enabling central infrastructure management in disconnected environments* for details about how to define custom image set in `osImages`.

Your central infrastructure management service is configured. You can verify that it is healthy by checking the `assisted-service` and `assisted-image-service` deployments and ensuring that their pods are ready and running.

Installing a FIPS-enabled cluster by using the Assisted Installer

When you install a supported OpenShift Container Platform cluster that is in FIPS mode, do not specify any Red Hat Enterprise Linux version for the installers.

Required access: You must have access to edit the `AgentServiceConfig` and `AgentClusterInstall` resources.

Installing a OpenShift Container Platform cluster version 4.15 and earlier

If you install a {ocp-short} cluster version 4.15 and earlier, complete the following steps to update the `'AgentServiceConfig'` resource:

1. Log in to you managed cluster by using the following command:

```
oc login
```

2. Add the `agent-install.openshift.io/service-image-base: el8` annotation in the `AgentServiceConfig` resource.

Your `AgentServiceConfig` resource might resemble the following YAML:

```
apiVersion: agent-install.openshift.io/v1beta1
kind: AgentServiceConfig
metadata:
  annotations:
    agent-install.openshift.io/service-image-base: el8
...
```

Installing a OpenShift Container Platform cluster version 4.16 and later

If you install a OpenShift Container Platform cluster version 4.16 and later, complete the following steps to update the `AgentServiceConfig` resource:

1. Log in to you managed cluster by using the following command:

```
oc login
```

2. If the `agent-install.openshift.io/service-image-base: el8` annotation is present in the `AgentServiceConfig` resource, remove the annotation.

Additional resources

- For additional information about zero touch provisioning, see [Challenges of the network far edge](#) in the OpenShift Container Platform documentation.
- [Using image pull secrets](#)

Enabling central infrastructure management on Amazon Web Services

If you are running your hub cluster on Amazon Web Services and want to enable the central infrastructure management service, complete the following steps after [Enabling the central infrastructure management service](#):

1. Make sure you are logged in at the hub cluster and find the unique domain configured on the **assisted-image-service** by running the following command:

```
oc get routes --all-namespaces | grep assisted-image-service
```

Your domain might resemble the following example: **assisted-image-service-multicluster-engine.apps.<yourdomain>.com**

2. Make sure you are logged in at the hub cluster and create a new **IngressController** with a unique domain using the **NLB type** parameter. See the following example:

```
apiVersion: operator.openshift.io/v1
kind: IngressController
metadata:
  name: ingress-controller-with-nlb
  namespace: openshift-ingress-operator
spec:
  domain: nlb-apps.<domain>.com
  routeSelector:
    matchLabels:
      router-type: nlb
  endpointPublishingStrategy:
    type: LoadBalancerService
  loadBalancer:
    scope: External
  providerParameters:
    type: AWS
    aws:
      type: NLB
```

3. Add **<yourdomain>** to the **domain** parameter in **IngressController** by replacing **<domain>** in **nlb-apps.<domain>.com** with **<yourdomain>**.
4. Apply the new **IngressController** by running the following command:

```
oc apply -f ingresscontroller.yaml
```

- Make sure that the value of the `spec.domain` parameter of the new `IngressController` is not in conflict with an existing `IngressController` by completing the following steps:

- List all `IngressControllers` by running the following command:

```
oc get ingresscontroller -n openshift-ingress-operator
```

- Run the following command on each of the `IngressControllers`, except the `ingress-controller-with-nlb` that you just created:

```
oc edit ingresscontroller <name> -n openshift-ingress-operator
```

If the `spec.domain` report is missing, add a default domain that matches all of the routes that are exposed in the cluster except `nlb-apps.<domain>.com`.

If the `spec.domain` report is provided, make sure that the `nlb-apps.<domain>.com` route is excluded from the specified range.

- Run the following command to edit the `assisted-image-service` route to use the `nlb-apps` location:

```
oc edit route assisted-image-service -n <namespace>
```

The default namespace is where you installed the multicluster engine operator.

- Add the following lines to the `assisted-image-service` route:

```
metadata:
  labels:
    router-type: nlb
  name: assisted-image-service
```

- In the `assisted-image-service` route, find the URL value of `spec.host`. The URL might resemble the following example:

```
assisted-image-service-multicluster-engine.apps.<yourdomain>.com
```

- Replace `apps` in the URL with `nlb-apps` to match the domain configured in the new `IngressController`.
- To verify that the central infrastructure management service is enabled on Amazon Web Services, run the following command to verify that the pods are healthy:

```
oc get pods -n multicluster-engine | grep assist
```

11. Create a new host inventory and ensure that the download URL uses the new **nlb-apps** URL.

Creating a host inventory by using the console

You can create a host inventory (infrastructure environment) to discover physical or virtual machines that you can install your OpenShift Container Platform clusters on.

Prerequisites

- You must enable the central infrastructure management service. See *Enabling the central infrastructure management service* for more information.

Creating a host inventory

Complete the following steps to create a host inventory by using the console:

1. From the console, navigate to **Infrastructure > Host inventory** and click **Create infrastructure environment**.
2. Add the following information to your host inventory settings:

Name

A unique name for your infrastructure environment. Creating an infrastructure environment by using the console also creates a new namespace for the **InfraEnv** resource with the name you chose. If you create **InfraEnv** resources by using the command line interface and want to monitor the resources in the console, use the same name for your namespace and the **InfraEnv**.

Network type

Specifies if the hosts you add to your infrastructure environment use DHCP or static networking. Static networking configuration requires additional steps.

Location

Specifies the geographic location of the hosts. The geographic location can be used to define which data center the hosts are located.

Labels

Optional field where you can add labels to the hosts that are discovered with this infrastructure environment. The specified location is automatically added to the list of labels.

Infrastructure provider credentials

Selecting an infrastructure provider credential automatically populates the pull secret and SSH public key fields with information in the credential. For more information, see *Creating a credential for an on-premises environment*.

Pull secret

Your OpenShift Container Platform [pull secret](#) that enables you to access the OpenShift Container Platform resources. This field is automatically populated if you selected an infrastructure provider credential.

SSH public key

The SSH key that enables the secure communication with the hosts. You can use it to connect to the host for troubleshooting. After installing a cluster, you can no longer connect to the host with the SSH key. The key is generally in your `id_rsa.pub` file. The default file path is `~/.ssh/id_rsa.pub`. This field is automatically populated if you selected an infrastructure provider credential that contains the value of a SSH public key.

If you want to enable proxy settings for your hosts, select the setting to enable it and enter the following information:

HTTP Proxy URL

The URL of the proxy for HTTP requests.

HTTPS Proxy URL

The URL of the proxy for HTTP requests. The URL must start with HTTP. HTTPS is not supported. If you do not provide a value, your HTTP proxy URL is used by default for both HTTP and HTTPS connections.

No Proxy domains

A list of domains separated by commas that you do not want to use the proxy with. Start a domain name with a period (.) to include all of the subdomains that are in that domain. Add an asterisk (*) to bypass the proxy for all destinations. Optionally add your own Network Time Protocol (NTP) sources by providing a comma-separated list of IP or domain names of the NTP pools or servers.

If you need advanced configuration options that are not available in the console, continue to [Creating a host inventory by using the command line interface](#).

If you do not need advanced configuration options, you can continue by configuring static networking, if required, and begin adding hosts to your infrastructure environment.

Accessing a host inventory

To access a host inventory, select **Infrastructure** > **Host inventory** in the console. Select your infrastructure environment from the list to view the details and hosts.

Additional resources

- [Enabling the central infrastructure management service](#)
- [Creating a credential for an on-premises environment](#)

If you created a host inventory as part of the process to configure hosted control planes on bare metal, complete the following procedures:

- [Adding hosts to the host inventory by using the Discovery Image](#)
- [Automatically adding bare metal hosts to the host inventory](#)

Creating a host inventory by using the command line interface

You can create a host inventory (infrastructure environment) to discover physical or virtual machines that you can install your OpenShift Container Platform clusters on. Use the command line interface instead of the console for automated deployments, or for the following advanced configuration options:

- Automatically bind discovered hosts to an existing cluster definition
- Override the ignition configuration of the Discovery Image
- Control the iPXE behavior
- Modify kernel arguments for the Discovery Image
- Pass additional certificates that you want the host to trust during the discovery phase
- Select a Red Hat CoreOS version to boot for testing that is not the default option of the newest version

Prerequisites

- You must enable the central infrastructure management service. See [Enabling the central infrastructure management service](#) for more information.

Creating a host inventory

Complete the following steps to create a host inventory (infrastructure environment) by using the command line interface:

1. Log in to your hub cluster by running the following command:

```
oc login
```

2. Create a namespace for your resource.
 - a. Create a file named, `namespace.yaml`, and add the following content:

```
apiVersion: v1
kind: Namespace
metadata:
  name: <your_namespace> ①
```

- ① Use the same name for your namespace and your infrastructure environment to monitor your inventory in the console.

- b. Apply the YAML content by running the following command:


```
oc apply -f namespace.yaml
```

3. Create a **Secret** custom resource containing your OpenShift Container Platform [pull secret](#).
 - a. Create the **pull-secret.yaml** file and add the following content:

```
apiVersion: v1
kind: Secret
type: kubernetes.io/dockerconfigjson
metadata:
  name: pull-secret ①
  namespace: <your_namespace>
stringData:
  .dockerconfigjson: <your_pull_secret> ②
```

① Add your namespace.

② Add your pull secret.

- b. Apply the YAML content by running the following command:

```
oc apply -f pull-secret.yaml
```

4. Create the infrastructure environment.
 - a. Create the **infra-env.yaml** file and add the following content. Replace values where needed:

```
apiVersion: agent-install.openshift.io/v1beta1
kind: InfraEnv
metadata:
  name: myinfraenv
  namespace: <your_namespace>
spec:
  proxy:
    httpProxy: <http://user:password@ipaddr:port>
    httpsProxy: <http://user:password@ipaddr:port>
    noProxy:
  additionalNTPSources:
  sshAuthorizedKey:
  pullSecretRef:
    name: <name>
  agentLabels:
    <key>: <value>
  nmStateConfigLabelSelector:
    matchLabels:
      <key>: <value>
  clusterRef:
    name: <cluster_name>
    namespace: <project_name>
```

```

ignitionConfigOverride: '{"ignition": {"version": "3.1.0"}, ...}'
cpuArchitecture: x86_64
ipxeScriptType: DiscoveryImageAlways
kernelArguments:
  - operation: append
    value: audit=0
additionalTrustBundle: <bundle>
osImageVersion: <version>
ImageType: <type>

```

See the following field descriptions in the **InfraEnv** table:

Table 15. *InfraEnv* field table

Field	Optional or required	Description
proxy	Optional	Defines the proxy settings for agents and clusters that use the InfraEnv resource. If you do not set the proxy value, agents are not configured to use a proxy.
httpProxy	Optional	The URL of the proxy for HTTP requests. The URL must start with http . HTTPS is not supported..
httpsProxy	Optional	The URL of the proxy for HTTP requests. The URL must start with http . HTTPS is not supported.
noProxy	Optional	A list of domains and CIDRs separated by commas that you do not want to use the proxy with.
additionalNTPSources	Optional	A list of Network Time Protocol (NTP) sources (hostname or IP) to add to all hosts. They are added to NTP sources that are configured by using other options, such as DHCP.
sshAuthorizedKey	Optional	SSH public keys that are added to all hosts for use in debugging during the discovery phase. The discovery phase is when the host boots the Discovery Image.
name	Required	The name of the Kubernetes secret containing your pull secret.

Field	Optional or required	Description
<code>agentLabels</code>	Optional	Labels that are automatically added to the <code>Agent</code> resources representing the hosts that are discovered with your <code>InfraEnv</code> . Make sure to add your key and value.
<code>nmStateConfigLabelSelector</code>	Optional	Consolidates advanced network configuration such as static IPs, bridges, and bonds for the hosts. The host network configuration is specified in one or more <code>NMStateConfig</code> resources with labels you choose. The <code>nmStateConfigLabelSelector</code> property is a Kubernetes label selector that matches your chosen labels. The network configuration for all <code>NMStateConfig</code> labels that match this label selector is included in the Discovery Image. When you boot, each host compares each configuration to its network interfaces and applies the appropriate configuration. To learn more about advanced network configuration, see <i>Configuring advanced networking for an infrastructure environment</i> .

Field	Optional or required	Description
<code>clusterRef</code>	Optional	References an existing <code>ClusterDeployment</code> resource that describes a standalone on-premises cluster. Not set by default. If <code>clusterRef</code> is not set, then the hosts can be bound to one or more clusters later. You can remove the host from one cluster and add it to another. If <code>clusterRef</code> is set, then all hosts discovered with your <code>InfraEnv</code> are automatically bound to the specified cluster. If the cluster is not installed yet, then all discovered hosts are part of its installation. If the cluster is already installed, then all discovered hosts are added.
<code>ignitionConfigOverride</code>	Optional	Modifies the ignition configuration of the Red Hat CoreOS live image, such as adding files. Make sure to only use <code>ignitionConfigOverride</code> if you need it. Must use ignition version 3.1.0, regardless of the cluster version.
<code>cpuArchitecture</code>	Optional	Choose one of the following supported CPU architectures: <code>x86_64</code> , <code>aarch64</code> , <code>ppc64le</code> , or <code>s390x</code> . The default value is <code>x86_64</code> .

Field	Optional or required	Description
<code>ipxeScriptType</code>	Optional	Causes the image service to always serve the iPXE script when set to the default value of <code>DiscoveryImageAlways</code> and when you are using iPXE to boot. As a result, the host boots from the network discovery image. Setting the value to <code>BootOrderControl</code> causes the image service to decide when to return the iPXE script, depending on the host state, which causes the host to boot from the disk when the host is provisioned and is part of a cluster.
<code>kernelArguments</code>	Optional	Allows modifying the kernel arguments for when the Discovery Image boots. Possible values for <code>operation</code> are <code>append</code> , <code>replace</code> , or <code>delete</code> .
<code>additionalTrustBundle</code>	Optional	A PEM-encoded X.509 certificate bundle, usually needed if the hosts are in a network with a re-encrypting man-in-the-middle (MITM) proxy, or if the hosts need to trust certificates for other purposes, such as container image registries. Hosts discovered by your <code>InfraEnv</code> trust the certificates in this bundle. Clusters created from the hosts discovered by your <code>InfraEnv</code> also trust the certificates in this bundle.

Field	Optional or required	Description
<code>osImageVersion</code>	Optional	The Red Hat CoreOS image version to use for your <code>InfraEnv</code> . Make sure the version refers to the OS image specified in either the <code>AgentServiceConfig.spec.osImages</code> or in the default OS images list. Each release has a specific set of Red Hat CoreOS image versions. The <code>OSImageVersion</code> must match an OpenShift Container Platform version in the OS images list. You cannot specify <code>OSImageVersion</code> and <code>ClusterRef</code> at the same time. If you want to use another version of the Red Hat CoreOS image that does not exist by default, then you must manually add the version by specifying it in the <code>AgentServiceConfig.spec.osImages</code> . To learn more about adding versions, see <i>Enabling the central infrastructure management service</i> .
<code>ImageType</code>	Optional	Selects the ISO file type to download. Possible options are <code>minimal-iso</code> or <code>full-iso</code> . If you do not specify an ISO type, the minimal ISO is used by default. The minimal ISO does not contain the root file system, <code>RootFS</code> . If you select a minimal ISO, the <code>RootFS</code> is downloaded later.

- a. Apply the YAML content by running the following command:

```
oc apply -f infra-env.yaml
```

- b. To verify that your host inventory is created, check the status with the following command:

```
oc describe infraenv myinfraenv -n <your_namespace>
```

See the following list of notable properties:

- **conditions**: The standard Kubernetes conditions indicating if the image was created successfully.
- **isoDownloadURL**: The URL to download the Discovery Image.
- **createdTime**: The time at which the image was last created. If you modify the **InfraEnv**, make sure that the timestamp has been updated before downloading a new image.

Note: If you modify the **InfraEnv** resource, make sure that the **InfraEnv** has created a new Discovery Image by looking at the **createdTime** property. If you already booted hosts, boot them again with the latest Discovery Image.

You can continue by configuring static networking, if required, and begin adding hosts to your infrastructure environment.

Additional resources

- [Configuring advanced networking for an infrastructure environment](#)
- [Enabling the central infrastructure management service](#)

Configuring advanced networking for an infrastructure environment

For hosts that require networking beyond DHCP on a single interface, you must configure advanced networking. The required configuration includes creating one or more instances of the **NMStateConfig** resource that describes the networking for one or more hosts.

Each **NMStateConfig** resource must contain a label that matches the **nmStateConfigLabelSelector** on your **InfraEnv** resource. See *Creating a host inventory by using the command line interface* to learn more about the **nmStateConfigLabelSelector**.

The Discovery Image contains the network configurations defined in all referenced **NMStateConfig** resources. After booting, each host compares each configuration to its network interfaces and applies the appropriate configuration.

Prerequisites

- You must enable the central infrastructure management service. See [Enabling the central infrastructure management service](#) for more information.
- You must create a host inventory. See [Creating a host inventory by using the console](#) for more information.

Configuring advanced networking by using the command line interface

To configure advanced networking for your infrastructure environment by using the command line interface, complete the following steps:

1. Create a file named **nmstateconfig.yaml** and add content that is similar to the following template. Replace values where needed:

```

apiVersion: agent-install.openshift.io/v1beta1
kind: NMStateConfig
metadata:
  name: mynmstateconfig
  namespace: <your-infraenv-namespace>
  labels:
    some-key: <some-value>
spec:
  config:
    interfaces:
      - name: eth0
        type: ethernet
        state: up
        mac-address: 02:00:00:80:12:14
        ipv4:
          enabled: true
          address:
            - ip: 192.168.111.30
              prefix-length: 24
          dhcp: false
      - name: eth1
        type: ethernet
        state: up
        mac-address: 02:00:00:80:12:15
        ipv4:
          enabled: true
          address:
            - ip: 192.168.140.30
              prefix-length: 24
          dhcp: false
    dns-resolver:
      config:
        server:
          - 192.168.126.1
    routes:
      config:
        - destination: 0.0.0.0/0
          next-hop-address: 192.168.111.1
          next-hop-interface: eth1
          table-id: 254
        - destination: 0.0.0.0/0
          next-hop-address: 192.168.140.1
          next-hop-interface: eth1
          table-id: 254
    interfaces:
      - name: "eth0"
        macAddress: "02:00:00:80:12:14"
      - name: "eth1"
        macAddress: "02:00:00:80:12:15"

```


Table 16. *NMStateConfig* field table

Field	Optional or required	Description
<code>name</code>	Required	Use a name that is relevant to the host or hosts you are configuring.
<code>namespace</code>	Required	The namespace must match the namespace of your <code>InfraEnv</code> resource.
<code>some-key</code>	Required	Add one or more labels that match the <code>nmStateConfigLabelSelector</code> on your <code>InfraEnv</code> resource.
<code>config</code>	Optional	Describes the network settings in <code>NMstate</code> format. See <i>Declarative Network API</i> for the format specification and additional examples. The configuration can also apply to a single host, where you have one <code>NMStateConfig</code> resource per host, or can describe the interfaces for multiple hosts in a single <code>NMStateConfig</code> resource.

Field	Optional or required	Description
<code>interfaces</code>	Optional	Describes the mapping between interface names found in the specified <code>NMState</code> configuration and MAC addresses found on the hosts. Make sure the mapping uses physical interfaces present on a host. For example, when the <code>NMState</code> configuration defines a bond or VLAN, the mapping only contains an entry for parent interfaces. The mapping has the following purposes: * Allows you to use interface names in the configuration that do not match the interface names on a host. You might find this useful because the operating system chooses the interface names, which might not be predictable. * Tells a host what MAC addresses to look for after booting and applies the correct <code>NMState</code> configuration.

Note: The Image Service automatically creates a new image when you update any `InfraEnv` properties or change the `NMStateConfig` resources that match its label selector. If you add `NMStateConfig` resources after creating the `InfraEnv` resource, make sure that the `InfraEnv` creates a new Discovery Image by checking the `createdTime` property in your `InfraEnv`. If you already booted hosts, boot them again with the latest Discovery Image.

1. Apply the YAML content by running the following command:

```
oc apply -f nmstateconfig.yaml
```

Additional resources

- [Creating a host inventory by using the command line interface](#)
- [Declarative Network API](#)

Adding hosts to the host inventory by using the Discovery Image

After you create your host inventory (infrastructure environment), you can discover your hosts and add them to your inventory.

To add hosts to your inventory, choose a method to download an ISO file and attach it to each

server. For example, you can download ISO files by using a virtual media, or by writing the ISO file to a USB drive.

Important: To prevent the installation from failing, keep the Discovery ISO media connected to the device during the installation process, and set each host to boot from the device one time.

- [Prerequisites](#)
- [Adding hosts by using the console](#)
- [Adding hosts by using the command line interface](#)
- [Hosting iPXE artifacts with HTTP or HTTPS](#)
- [Additional resources](#)

Prerequisites

- You must enable the central infrastructure management service. See [Enabling the central infrastructure management service](#) for more information.
- You must create a host inventory. See [Creating a host inventory by using the console](#) for more information.

Adding hosts by using the console

Download the ISO file by completing the following steps:

1. Select **Infrastructure** > **Host inventory** in the console.
2. Select your infrastructure environment from the list.
3. Click **Add hosts** and select **With Discovery ISO**.
4. Select **Minimal image file** or **Full image file**.
5. Click **Generate Discovery ISO**.

Note: Booted hosts appear in the host inventory table. Hosts might take a few minutes to appear.

6. Approve each host so that you can use it. You can select hosts from the inventory table by clicking **Actions** and selecting **Approve**.

Adding hosts by using the command line interface

The URL to download the ISO file in the `isoDownloadURL` property is in the status of your `InfraEnv` resource. See *Creating a host inventory by using the command line interface* for more information about the `InfraEnv` resource.

Each booted host creates an `Agent` resource in the same namespace.

1. Run the following command to view the download URL in the `InfraEnv` custom resource:

```
oc get infraenv -n <infra env namespace> <infra env name> -o
```

```
jsonpath='{.status.isoDownloadURL}'
```

See the following output:

```
https://assisted-image-service-assisted-installer.apps.example-acm-  
hub.com/byapikey/eyJhbGciOiJIUzI1NiIsInC93XVCJ9.eyJpbmZyYV91bnZfaWQcTA0Y38sWVjYi02M  
TA0LTQ4NDMtODasdkOGIxYTZkZGM5ZTUifQ.3ydTpHaXJmTasd7uDp2NvGUFrKin3Z9Qct3lvDky1N-  
5zj3KsRePhAM48aUccBqmucGt3g/4.17/x86_64/minimal.iso
```

Note: By default, the minimal ISO is provided. You can specify the full ISO by setting **ImageType** to **full-iso** in your **InfraEnv** resource. See *Creating a host inventory by using the command line interface* in the additional resources section to learn more.

2. Use the URL to download the ISO file and boot your hosts with the ISO file.

Next, you need to approve each host. See the following procedure:

3. Run the following command to list all of your **Agents**:

```
oc get agent -n <infra env namespace>
```

You get an output that is similar to the following output:

NAME	CLUSTER	APPROVED	ROLE	STAGE
24a92a6f-ea35-4d6f-9579-8f04c0d3591e		false	auto-assign	

4. Approve any **Agent** from the list with a **false** approval status. Run the following command:

```
oc patch agent -n <infra env namespace> <agent name> -p  
'{"spec":{"approved":true}}' --type merge
```

5. Run the following command to confirm approval status:

```
oc get agent -n <infra env namespace>
```

You get an output that is similar to the following output with a **true** value:

NAME	CLUSTER	APPROVED	ROLE	STAGE
173e3a84-88e2-4fe1-967f-1a9242503bec		true	auto-assign	

Hosting iPXE artifacts with HTTP or HTTPS

You can change how to host iPXE artifacts by editing the **spec.iPXEHTTPRoute** field in the

`AgentServiceConfig` custom resource.

Set the field to `enabled` to use HTTP for iPXE artifacts.

Set the field to `disabled` to use HTTPS for iPXE artifacts.

The default value is `disabled`. See the following example, where the `spec.iPXEHTTPRoute` field is set to `enabled`:

```
apiVersion: agent-install.openshift.io/v1beta1
kind: AgentServiceConfig
metadata:
  name: agent
spec:
  iPXEHTTPRoute: enabled
```

If you set the value to `enabled`, the following endpoints are exposed through HTTP:

- `api/assisted-installer/v2/infra-envs/<id>/downloads/files?file_name=ipxe-script` in `assisted-service`
- `boot-artifacts/` and `images/<infra-env-id>/pxe-initrd` in `assisted-image-service`

Additional resources

- To learn more about the `InfraEnv` resource, see [Creating a host inventory by using the command line interface](#).

Automatically adding bare metal hosts to the host inventory

After creating your infrastructure environment, you can discover your hosts and add them to your host inventory. You can automate booting the Discovery Image of your infrastructure environment by making the bare metal operator communicate with the Baseboard Management Controller (BMC) of each bare metal host. Create a `BareMetalHost` resource and associated BMC secret for each host. The automation is set by a label on the `BareMetalHost` that references your infrastructure environment.

The automation performs the following actions:

- Boots each bare metal host with the Discovery Image represented by the infrastructure environment
- Reboots each host with the latest Discovery Image in case the infrastructure environment or any associated network configurations is updated
- Associates each `Agent` resource with its corresponding `BareMetalHost` resource upon discovery
- Updates `Agent` resource properties based on information from the `BareMetalHost`, such as hostname, role, and installation disk
- Approves the `Agent` for use as a cluster node

Prerequisites

- You must enable the central infrastructure management service. See [Enabling the central infrastructure management service](#) for more information.
- You must create a host inventory. See [Creating a host inventory by using the console](#) for more information.

Adding bare metal hosts by using the console

Complete the following steps to automatically add bare metal hosts to your host inventory by using the console:

1. Select **Infrastructure > Host inventory** in the console.
2. Select your infrastructure environment from the list.
3. Click **Add hosts** and select **With BMC Form**.
4. Add the required information and click **Create**.

To learn more about BMC address formatting, see *BMC addressing* in the additional resources section.

Adding bare metal hosts by using the command line interface

Complete the following steps to automatically add bare metal hosts to your host inventory by using the command line interface.

1. Create a BMC secret by applying the following YAML content and replacing values where needed:

```
apiVersion: v1
kind: Secret
metadata:
  name: <bmc-secret-name>
  namespace: <your_infraenv_namespace> ①
type: Opaque
data:
  username: <username>
  password: <password>
```

① The namespace must be the same as the namespace of your **InfraEnv**.

2. Create a bare metal host by applying the following YAML content and replacing values where needed:

```
apiVersion: metal3.io/v1alpha1
kind: BareMetalHost
metadata:
  name: <bmh-name>
  namespace: <your-infraenv-namespace> ①
```

```

annotations:
  inspect.metal3.io: disabled
  bmac.agent-install.openshift.io/hostname: <hostname> ②
  bmac.agent-install.openshift.io/role: <role> ③
labels:
  infraenvs.agent-install.openshift.io: <your-infraenv> ④
spec:
  online: true
  automatedCleaningMode: disabled ⑤
  bootMACAddress: <your-mac-address> ⑥
  bmc:
    address: <machine-address> ⑦
    credentialsName: <bmc-secret-name> ⑧
  rootDeviceHints:
    deviceName: /dev/sda ⑨

```

- ① The namespace must be the same as the namespace of your **InfraEnv**.
- ② **Optional:** Replace with the name of your host.
- ③ **Optional:** Possible values are **master** or **worker**.
- ④ The name must match the name of your **InfraEnv** and exist in the same namespace.
- ⑤ If you do not set a value, the **metadata** value is automatically used.
- ⑥ Make sure the MAC address matches the MAC address of one of your host interfaces.
- ⑦ Use the address of the BMC. To learn more, see *Port access for the out-of-band management IP address* and *BMC addressing* in the additional resources section.
- ⑧ Make sure that the **credentialsName** value matches the name of the BMC secret you created.
- ⑨ **Optional:** Select the installation disk. See *The BareMetalHost spec* for the available root device hints. After the host is booted with the Discovery Image and the corresponding **Agent** resource is created, the installation disk is set according to this hint.

After turning on the host, the image starts downloading. This might take a few minutes. When the host is discovered, an **Agent** custom resource is created automatically.

Removing managed cluster nodes by using the command line interface

To remove managed cluster nodes from a managed cluster, you need a hub cluster that is running on a supported OpenShift Container Platform version. Any static networking configuration required for the node to boot must be available. Make sure to not delete **NMStateConfig** resources when you delete the agent and bare metal host.

Removing managed cluster nodes with a bare metal host

If you have a bare metal host on your hub cluster and want remove managed cluster nodes from a managed cluster, complete the following steps:

1. Add the following annotation to the **BareMetalHost** resource of the node that you want to delete:

```
bmac.agent-install.openshift.io/remove-agent-and-node-on-delete: true
```

2. Delete the `BareMetalHost` resource by running the following command. Replace `<bmh-name>` with the name of your `BareMetalHost`:

```
oc delete bmh <bmh-name>
```

Removing managed cluster nodes without a bare metal host

If you do not have a bare metal host on your hub cluster and you want to remove managed cluster nodes from a managed cluster, you can unbind the agent by removing the `clusterDeploymentName` field from the `Agent` specification, or delete the `Agent` custom resource that corresponds with the node that you are removing.

If you want to delete an `Agent` resource from the hub cluster, but do not want the node removed from the managed cluster, you can set the annotation `agent.agent-install.openshift.io/skip-spoke-cleanup` to `true` on the `Agent` resource before you remove it.

See the [Deleting nodes](#) instructions in the OpenShift Container Platform documentation.

Binding and unbinding hosts

You can bind hosts to an Red Hat OpenShift Container Platform cluster by setting the `spec.clusterDeploymentName` field in the `Agent` custom resource, or by setting the `{bmac.agent-install.openshift.io/cluster-reference}` bare metal host annotation.

The `{bmac.agent-install.openshift.io/cluster-reference}` bare metal host annotation controls the connection to your OpenShift Container Platform cluster, and binds or unbinds hosts to a specific cluster.

You can use the `{bmac.agent-install.openshift.io/cluster-reference}` annotation in one of the following three ways:

- If you do not set the annotation in the bare metal host, no changes apply to the host.
- If you set the annotation with an empty string value, the host unbinds.
- If you set the annotation and use a string value that follows the `<cluster-namespace>/<cluster-name>` format, the host binds to the cluster that your `ClusterDeployment` custom resource represents.

Note: If the `InfraEnv` that the host belongs to already contains a `cluster-reference` annotation, the `{bmac.agent-install.openshift.io/cluster-reference}` annotation is ignored.

Additional resources

- For additional information about zero touch provisioning, see [Clusters at the network far edge](#) in the OpenShift Container Platform documentation.
- To learn about the required ports for using a bare metal host, see [Port access for the out-of-band](#)

[management IP address](#) in the OpenShift Container Platform documentation.

- To learn about root device hints, see [Postinstallation configuration overview](#) in the OpenShift Container Platform documentation.
- [Using image pull secrets](#)
- [Creating a credential for an on-premises environment](#)
- To learn more about scaling compute machines, see [Manually scaling a compute machine set](#) in the OpenShift Container Platform documentation.
- To learn more about BMC format addressing, see [BMC addressing](#) in the OpenShift Container Platform documentation.

Cancelling host installations

You can cancel host installations to prevent incorrect cluster binding, change host configurations, or fix installation problems.

For example, you can cancel the installation if a production environment host becomes stuck during installation with the `installing-pending-user-action` status.

Prerequisites

- You are using the late binding feature on your managed cluster.

Procedure

Complete the following steps to check if you can cancel the host installation:

1. Check if your managed cluster is using central infrastructure management. Run the following command:

```
oc get agentclusterinstall -n <namespace>
```

If your managed cluster is using central infrastructure management, the following output with the `adding-hosts` STATE appears:

NAME	CLUSTER	STATE
assisted-cluster	assisted-cluster	adding-hosts

2. Check if the host is installing. Run the following command:

```
oc get agent -n <namespace>
```

If the host is installing, the output resembles the following example:

NAME	CLUSTER	APPROVED	ROLE
STAGE			
127877e3-2098-4bf8-9b44-588a78ffcccd	assisted-cluster	true	worker

Rebooting

If your output resembles the previous example, you can continue cancelling the host installation. Complete the following steps to cancel the host installation:

1. Unbind the host from the managed cluster by removing the `clusterDeploymentName` field in the `Agent` resource of the host that you want to stop installing. Run the following command:

```
oc patch agent -n <namespace> <agent-name>
--patch='[{"op":"remove","path":"/spec/clusterDeploymentName"}]' --type=json
```

Confirm that the host is unbound by running the following command:

```
oc get agent -n <namespace>
```

If the host is unbound, the output resembles the following example, where `CLUSTER` is empty and `STAGE` is either empty, or `unbinding-pending-user-action` or similar.

NAME	CLUSTER	APPROVED	ROLE	STAGE
127877e3-2098-4bf8-9b44-588a78ffcccd		true	auto-assign	

2. Return the host to the discovery phase.
 - a. If you are using the Discovery ISO to add a node, download the Discovery ISO again and reboot your host with the Discovery ISO. Run the following command:

```
oc get infraenv -n <namespace> <infraenv-name>
-ojsonpath='{.status.isoDownloadURL}'
```

- b. If you are using the `BareMetalHost` resource to add a node, the host automatically returns to the discovery phase. Run the following command to confirm that the host is in the discovery phase:

```
oc get bmh -n <namespace>
```

If the host is in the discovery phase, the `BareMetalHost` resource status changes from `deprovisioning` to `provisioning`. It might take a few minutes for deprovisioning to complete. After the `BareMetalHost` reaches the `provisioning status`, the host is in discovery mode.

3. Check to see if the `Agent` resource is discovered and validated. Run the following command:

```
oc get agent -n <namespace> <agent-name> -ojsonpath='{ "NAME:
"}{.metadata.name}{ "\n"}{ "VALIDATIONS PASSING: "}{.status.conditions[?(@.type ==
```

```
"Validated")].status}{"\n"}'
```

4. Bind the **Agent** resource to the managed cluster to reinstall the host. Run the following command:

```
oc patch agent -n <namespace> <agent-name>
--patch='[{"op":"add","path":"/spec/clusterDeploymentName","value":{"name":"<cluster name>","namespace":"<cluster namespace>"}}]' --type=json
```

5. Confirm that the **Agent** resource is bound to the managed cluster. Run the following command:

```
oc get agent -n <namespace>
```

If the **Agent** resource is bound to the managed cluster, the output resembles the following example:

NAME STAGE	CLUSTER	APPROVED	ROLE
127877e3-2098-4bf8-9b44-588a78ffcccd	cluster name	true	auto-assign

Managing your host inventory

You can manage your host inventory and edit existing hosts by using the console, or by using the command line interface and editing the **Agent** resource.

Managing your host inventory by using the console

Each host that you successfully boot with the Discovery ISO appears as a row in your host inventory. You can use the console to edit and manage your hosts. If you booted the host manually and are not using the bare metal operator automation, you must approve the host in the console before you can use it. Hosts that are ready to be installed as OpenShift nodes have the **Available** status.

Managing your host inventory by using the command line interface

An **Agent** resource represents each host. You can set the following properties in an **Agent** resource:

- **clusterDeploymentName**

Set this property to the namespace and name of the **ClusterDeployment** you want to use if you want to install the host as a node in a cluster.

- **Optional: role**

Sets the role for the host in the cluster. Possible values are **master**, **worker**, and **auto-assign**. The default value is **auto-assign**.

- **hostname**

Sets the host name for the host. Optional if the host is automatically assigned a valid host name, for example by using DHCP.

- **approved**

Indicates if the host can be installed as an OpenShift node. This property is a boolean with a default value of **False**. If you booted the host manually and are not using the bare metal operator automation, you must set this property to **True** before installing the host.

- **installation_disk_id**

The ID of the installation disk you chose that is visible in the inventory of the host.

- **installerArgs**

A JSON-formatted string containing overrides for the *coreos-installer* arguments of the host. You can use this property to modify kernel arguments. See the following example syntax:

```
[ "--append-karg",  
  "ip=192.0.2.2::192.0.2.254:255.255.255.0:core0.example.com:enp1s0:none", "--save-  
  partindex", "4"]
```

- **ignitionConfigOverrides**

A JSON-formatted string containing overrides for the ignition configuration of the host. You can use this property to add files to the host by using ignition. See the following example syntax:

```
{"ignition": {"version": "3.1.0"}, "storage": {"files": [{"path": "/tmp/example",  
  "contents": {"source":  
    "data:text/plain;base64,aGVscGltdHJhcHB1ZGluYXN3YWdnZXJzcGVj"}}]}}
```

- **nodeLabels**

A list of labels that are applied to the node after the host is installed.

The **status** of an **Agent** resource has the following properties:

- **role**

Sets the role for the host in the cluster. If you previously set a **role** in the **Agent** resource, the value appears in the **status**.

- **inventory**

Contains host properties that the agent running on the host discovers.

- **progress**

The host installation progress.

- `ntpSources`

The configured Network Time Protocol (NTP) sources of the host.

- `conditions`

Contains the following standard Kubernetes conditions with a `True` or `False` value:

- `SpecSynced`: `True` if all specified properties are successfully applied. `False` if some error was encountered.
- `Connected`: `True` if the agent connection to the installation service is not obstructed. `False` if the agent has not contacted the installation service in some time.
- `RequirementsMet`: `True` if the host is ready to begin the installation.
- `Validated`: `True` if all host validations pass.
- `Installed`: `True` if the host is installed as an OpenShift node.
- `Bound`: `True` if the host is bound to a cluster.
- `Cleanup`: `False` if the request to delete the `Agent` resource fails.

- `debugInfo`

Contains URLs for downloading installation logs and events.

- `validationsInfo`

Contains information about validations that the agent runs after the host is discovered to ensure that the installation is successful. Troubleshoot if the value is `False`.

- `installation_disk_id`

The ID of the installation disk you chose that is visible in the inventory of the host.

Additional resources

- [Accessing a host inventory](#)
- [coreos-installer install](#)

Managing a hub cluster by using central infrastructure management

Importing and managing a hub cluster by using central infrastructure management can simplify your cluster management by providing a single point of control.

By default, central infrastructure management does not automatically manage your hub cluster. Enable the feature to add and manage additional hosts on your hub cluster by using central infrastructure management.

Prerequisites

- You must enable the central infrastructure management service. See [Enabling the central](#)

[infrastructure management service](#) for more information.

Enabling hub cluster management by using central infrastructure management

Complete the following step to enable central infrastructure management to manage your hub cluster:

1. Annotate the `AgentServiceConfig` custom resource to enable `local-cluster` import. Run the following command:

```
oc annotate agentserviceconfig agent agent-install.openshift.io/enable-local-cluster-import=true
```

2. Verify that the feature is enabled by checking if an `InfraEnv` custom resource is created for the `local-cluster`. Run the following command:

```
oc get infraenv -n multicluster-engine
```

Additional resources

- See [Adding hosts to the host inventory by using the Discovery Image](#)
- See [Automatically adding bare metal hosts to the host inventory](#)

Configuring mirror registries for each cluster

Use the mirror registry feature to configure and manage custom registries for each cluster by referencing a dedicated mirror registry configuration object from each `AgentClusterInstall` and `InfraEnv` resource.

You might configure mirror registries for each cluster in multicluster environments where you need isolated registry configurations.

- [Prerequisites](#)
- [Mirroring required images](#)
- [Applying custom resource definitions with updated configurations](#)
- [Additional resources](#)

Prerequisites

- You must enable the central infrastructure management service. See [Enabling the central infrastructure management service](#) for more information.
- You must set up your mirror registry.
- The `assisted-service` must run in `KubeAPI` mode.

Mirroring required images

Ensure that the required images are mirrored to the registry. Complete the following steps:

1. Mirror the Red Hat OpenShift Container Platform release images by running the following command. Ensure that the OpenShift Container Platform version in the mirror registry matches the version in the image set configuration:

```
oc adm release mirror \
  -a pull_secret.json \
  --from=quay.io/openshift-release-dev/ocp-release:$OCP_TAG \
  --to=$REGISTRY_IP:$REGISTRY_PORT/openshift-release-dev/ocp-release \
  --to-release-image=$REGISTRY_IP:$REGISTRY_PORT/openshift-release-dev/ocp-release:$OCP_TAG
```

2. Mirror the Assisted Installer images by running the following commands:

```
oc image mirror \
  -a pull_secret.json \
  quay.io/edge-infrastructure/assisted-installer:latest \
  $REGISTRY_IP:$REGISTRY_PORT/edge-infrastructure/assisted-installer:latest

oc image mirror \
  -a pull_secret.json \
  quay.io/edge-infrastructure/assisted-installer-controller:latest \
  $REGISTRY_IP:$REGISTRY_PORT/edge-infrastructure/assisted-installer-controller:latest

oc image mirror \
  -a pull_secret.json \
  quay.io/edge-infrastructure/assisted-installer-agent:latest \
  $REGISTRY_IP:$REGISTRY_PORT/edge-infrastructure/assisted-installer-agent:latest
```

Applying custom resource definitions with updated configurations

After setting up the mirror registry, apply the required custom resource definitions to configure your cluster to use the mirrored images. Complete the following steps:

1. Create a **ConfigMap** file named **mirror_registry.yaml** with the following content. Replace values for **<your-namespace>**, **\$REGISTRY_IP**, and **\$REGISTRY_PORT** with your specific values:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: mirror-registry-config
  namespace: <your-namespace>
data:
  registries.conf: |
```

```
[[registry]]
  prefix = ""
  location = "quay.io/openshift-release-dev/ocp-release"
[[registry.mirror]]
  location = "$REGISTRY_IP:$REGISTRY_PORT/openshift-release-dev/ocp-release"
  pull-from-mirror = "digest-only"

[[registry]]
  prefix = ""
  location = "quay.io/edge-infrastructure"
[[registry.mirror]]
  location = "$REGISTRY_IP:$REGISTRY_PORT/edge-infrastructure"
ca-bundle.crt: |
-----BEGIN CERTIFICATE-----
...
-----END CERTIFICATE-----
```

2. Apply the config map by running the following command:

```
oc apply -f mirror_registry.yaml
```

3. Ensure that your `AgentClusterInstall` and `InfraEnv` custom resource definitions reference the mirror registry configuration. See the following example for both `aci.yaml` and `infraenv.yaml`:

```
...
spec:
  mirrorRegistryRef:
    name: mirror-registry-config
    namespace: <your-namespace>
  ...
```

4. Update and apply the custom resource definitions by running the following commands:

```
oc apply -f aci.yaml
oc apply -f infraenv.yaml
```

Additional resources

- See [Mirroring images for a disconnected installation](#) in the OpenShift Container Platform documentation to learn more about mirroring images.

Installing a TNF cluster with Assisted Installer (Technology Preview)

Technology Preview: A two-node with fencing (TNF) cluster is an Red Hat OpenShift Container Platform cluster that has two control plane nodes. Use TNF clusters to ensure redundancy at the edge and limit hardware costs.

Prerequisites

- You are using the **baremetal** or **none** platform.
- You have access to the baseboard management controller (BMC) of the platform nodes.
- Your OpenShift Container Platform managed cluster version is 4.20.

Procedure

Complete the following steps to install a TNF cluster by using Assisted Installer:

1. Enable the TNF cluster installation feature with Assisted Installer by setting an environment variable on the **assisted-service** pod.
 - a. Create the following **ConfigMap** resource:

```
apiVersion: v1
data:
  TNF_CLUSTERS_SUPPORT: "true"
kind: ConfigMap
metadata:
  name: assisted-service-custom
  namespace: multicluster-engine
```

- b. Reference the **ConfigMap** resource in an annotation on the **AgentServiceConfig** resource. Run the following command:

```
oc annotate agentserviceconfig agent unsupported.agent-
install.openshift.io/assisted-service-configmap=assisted-service-custom
```

2. Configure the TNF cluster object on the hub cluster by creating the required resources.
 - a. Create a namespace. Use the following YAML sample:

```
apiVersion: v1
kind: Namespace
metadata:
  name: tnf-cluster
```

- b. Create a pull secret. Use the following YAML sample. Replace **<your-base64-pull-secret>** with the name of your pull secret:

```
apiVersion: v1
kind: Secret
type: kubernetes.io/dockerconfigjson
metadata:
  name: pull-secret
  namespace: tnf-cluster
data:
```

```
.dockerconfigjson: <your-base64-pull-secret>
```

- c. Create a **AgentClusterInstall** resource. Use the following YAML sample. Replace **<your-public-ssh-key>** with your public SSH key:

```
apiVersion: extensions.hive.openshift.io/v1beta1
kind: AgentClusterInstall
metadata:
  name: tnf-cluster
  namespace: tnf-cluster
spec:
  platformType: BareMetal
  apiVIPs: [192.168.122.10]
  ingressVIPs: [192.168.122.20]
  clusterDeploymentRef:
    name: tnf-cluster
  imageSetRef:
    name: img4.20.0-rc.3-x86-64-appsub
  networking:
    clusterNetwork:
      - cidr: 10.128.0.0/14
        hostPrefix: 23
    machineNetwork:
      - cidr: 192.168.122.0/24
    serviceNetwork:
      - 172.30.0.0/16
  provisionRequirements:
    controlPlaneAgents: 2
  sshPublicKey: <your-public-ssh-key>
```

- d. Create a **ClusterDeployment** resource. Use the following YAML sample. Replace **<yourdomain.com>** with your domain address:

```
apiVersion: hive.openshift.io/v1
kind: ClusterDeployment
metadata:
  name: tnf-cluster
  namespace: tnf-cluster
spec:
  baseDomain: <yourdomain.com>
  clusterInstallRef:
    group: extensions.hive.openshift.io
    kind: AgentClusterInstall
    name: tnf-cluster
    version: v1beta1
  clusterName: tnf-cluster
  controlPlaneConfig:
    servingCertificates: {}
```

```
platform:
  agentBareMetal:
    agentSelector:
      cluster: tnf
pullSecretRef:
  name: pull-secret
```

- e. Create an **InfraEnv** resource. Use the following YAML sample. Replace **<your-public-ssh-key>** with your public SSH key:

```
apiVersion: agent-install.openshift.io/v1beta1
kind: InfraEnv
metadata:
  name: tnf-cluster
  namespace: tnf-cluster
spec:
  clusterRef:
    name: tnf-cluster
    namespace: tnf-cluster
  pullSecretRef:
    name: pull-secret
  sshAuthorizedKey: <your-public-ssh-key>
```

3. **Optional:** Create a fencing credentials secret for each node. Use the following YAML sample. Replace **<username>** and **<password>** with your username and password. The **certificateVerification** field default value is **Enabled**:

```
apiVersion: v1
stringData:
  address: https://bmc.example.com
  certificateVerification: Disabled
  username: <username>
  password: <password>
kind: Secret
metadata:
  name: node-0-fencing-credentials
  namespace: tnf-cluster
type: Opaque
```

Note: Fencing credentials for an Agent are stored as a Kubernetes secret in the Agent namespace that the Agent references in the **agent.spec.fencingCredentialsSecretRef** field.

4. Use one of the following methods to set Agent fencing credentials. The methods are listed in descending order of priority:
 - Manually create the secret and add an annotation on the **BareMetalHost** resource that references the secret. Use the following annotation key:

```
bmac.agent-install.openshift.io/fencing-credentials-secret-name
```

Note: The Assisted Installer sets the Agent `fencingCredentialsSecretRef` field to the value of the annotation.

- Set an annotation on the Agent `BareMetalHost` resource to create the secret from the `BareMetalHost` BMC configuration. Use the following annotation key:

```
bmac.agent-install.openshift.io/create-fencing-credentials-secret
```

Note: You can use any value.

- Manually create the secret. Then, manually set the Agent `fencingCredentialsSecretRef` field to the secret name.

Cluster API

Cluster API offers declarative APIs that simplify the provisioning, upgrading, and management of Kubernetes clusters. multicluster engine operator installs the core Cluster API components along with multiple infrastructure and bootstrap providers to support different use-cases and platforms. Learn how to use Cluster API with either standalone multicluster engine operator or Red Hat Advanced Cluster Management. See the following processes:

- [Installing a managed cluster with Cluster API](#)
- [Enabling automatic import for Cluster API clusters \(Technology Preview\)](#)
- [Switching from AWS credentials to IAM roles on your Cluster API managed cluster \(Technology Preview\)](#)
- [Creating a Red Hat OpenShift Service on AWS with hosted control planes cluster with Cluster API \(Technology Preview\)](#)
- [Deleting a Red Hat OpenShift Service on AWS with hosted control planes cluster with using Cluster API \(Technology Preview\)](#)
- [Preparing to create Microsoft Azure Red Hat OpenShift hosted control plane clusters by using Cluster API Provider for Azure](#)
- [Creating Microsoft Azure Red Hat OpenShift hosted control plane clusters by using Cluster API Provider for Azure](#)
- [Deleting Microsoft Azure Red Hat OpenShift hosted control plane clusters by using Cluster API Provider for Azure](#)

Installing a managed cluster with Cluster API

You can install an OpenShift Container Platform managed cluster with the Cluster API by using the `metal3` infrastructure provider and the OpenShift Container Platform assisted bootstrap and control plane providers.

- A `ControlPlane` resource defines the control plane properties.
- The `ClusterInfrastructure` defines the cluster-level infrastructure details.

Prerequisites

- You need the `MultiClusterEngine` resource either from the Red Hat Advanced Cluster Management installation, or the multicluster engine operator standalone installation.
- The following API base domain must point to the static API VIP: `api.<cluster_name>.<base_domain>`.
- The following application base domain must point to the static IP address for Ingress VIP:

```
*.apps.<cluster_name>.<base_domain>
```

- You need the internal API endpoint for the cluster: `api-int.<baseDomain>`.
- The Cluster API Provider OpenShift Assisted requires the following components:
 - Cluster API: The core components provide the required custom resource definitions and controllers within the management cluster.
 - Assisted service: The infrastructure operator for Red Hat OpenShift in multicluster engine for Kubernetes operator provisions this service. Ensure the service is reachable and active.
 - Infrastructure provider: A provider, such as the `cluster-api-provider-metal3`, manages the physical or virtual resources.

Provisioning a workload with Cluster API

1. Enable the services that you need to provision a workload. By default, `assisted-service` is enabled. By default, `cluster-api-provider-metal3` and `cluster-api-provider-openshift-assisted` are disabled. Ensure all three services are `enabled: true`. Run the following command to edit the resource:

```
oc edit multiclusterengines.multicluster.openshift.io -n multicluster-engine
```

2. Set `enabled: true` in `.spec.overrides` for the `assisted-service` component, the `cluster-api-provider-metal3` component, and the `cluster-api-provider-openshift-assisted` component. See the following `configOverrides` values:

```
- configOverrides: {}
  enabled: true
  name: assisted-service
- configOverrides: {}
  enabled: true
  name: cluster-api
...
- configOverrides: {}
  enabled: true
  name: cluster-api-provider-metal3
```

```
- configOverrides: {}
  enabled: true
  name: cluster-api-provider-openshift-assisted
```

3. Enable the central infrastructure management service. See [Enabling the central infrastructure management service](#) for the procedure.
4. Configure a `Cluster` resource with `clusterNetwork` specifications, `controlPlaneRef` specifications, and `infrastructureRef` specifications. See the following `Cluster` resource:

```
apiVersion: cluster.x-k8s.io/v1beta1
kind: Cluster
metadata:
  name: <cluster-name>
  namespace: <cluster-namespace>
spec:
  clusterNetwork:
    pods:
      cidrBlocks:
        - 172.18.0.0/20
    services:
      cidrBlocks:
        - 10.96.0.0/12
  controlPlaneRef: ①
    apiVersion: controlplane.cluster.x-k8s.io/v1alpha2
    kind: OpenShiftAssistedControlPlane
    name: <cluster-name>
    namespace: <cluster-namespace>
  infrastructureRef: ②
    apiVersion: infrastructure.cluster.x-k8s.io/v1beta1
    kind: Metal3Cluster
    name: <cluster-name>
    namespace: <cluster-namespace>
```

① `OpenShiftAssistedControlPlane` is the control plane.

② `Metal3Cluster` is the infrastructure.

5. Apply the YAML content by running the following command:

```
oc apply -f <filename>.yaml
```

6. Configure the `OpenShiftAssistedControlPlane` resource that includes the `distributionVersion`, `apiVIPs`, and `SSH` keys. Specify your OpenShift Container Platform version in the `distributionVersion` field.

Note: The value for the `distributionVersion` matches the image from the [OpenShift Container Platform Releases](#). See the following YAML resource:

```

apiVersion: controlplane.cluster.x-k8s.io/v1alpha2
kind: OpenshiftAssistedControlPlane
metadata:
  name: <cluster-name>
  namespace: <cluster-namespace>
  annotations: {}
spec: ❶
  openshiftAssistedConfigSpec:
    sshAuthorizedKey: "{{ ssh_authorized_key }}"
    nodeRegistration:
      kubeletExtraLabels:
        - 'metal3.io/uuid="{{ METADATA_UUID }}"'
  distributionVersion: <4.x.0>
  config:
    apiVIPs:
      - 192.168.222.40
    ingressVIPs:
      - 192.168.222.41
    baseDomain: lab.home
    pullSecretRef:
      name: "pull-secret"
    sshAuthorizedKey: "{{ ssh_authorized_key }}" ❷
  machineTemplate:
    infrastructureRef:
      apiVersion: infrastructure.cluster.x-k8s.io/v1beta1
      kind: Metal3MachineTemplate
      name: <cluster-name-control-plane>
      namespace: <cluster-namespace>
  replicas: 3

```

- ❶ The `.spec.openshiftAssistedConfigSpec.sshAuthorizedKey` is used to access nodes in the *boot* phase, also known as *discovery* phase.
- ❷ The `.spec.config.sshAuthorizedKey` is used to access the provisioned OpenShift Container Platform nodes.

7. Apply the YAML file. Run the following command:

```
oc apply -f <filename>.yaml
```

- 8. If you do not have a pull secret, you need to create a pull secret to enable your clusters to pull images from container registries. Complete the following steps to create a pull secret:
- 9. Create a YAML file to pull images. See the following example of a file that is named `pull-secret.yaml`:

```

apiVersion: v1
kind: Secret
metadata: ❶

```

```
name: pull-secret
namespace: <cluster-namespace>
data: ②
  .dockerconfigjson: <encoded_docker_configuration>
type: kubernetes.io/dockerconfigjson
```

① Ensure that the `<cluster-namespace>` value matches the target namespace.

② Specify the base64-encoded configuration file as the value for `<encoded_docker_configuration>`.

10. Run the following command to apply the file:

```
oc apply -f pull-secret.yaml
```

11. Configure the `Metal3Cluster` infrastructure resource, which contains information that is related to the deployment of the cluster on baremetal.

```
apiVersion: infrastructure.cluster.x-k8s.io/v1beta1
kind: Metal3Cluster
metadata:
  name: <cluster-name>
  namespace: <cluster-namespace>
spec:
  controlPlaneEndpoint:
    host: <cluster-name>.lab.home ①
    port: 6443
  noCloudProvider: true
```

① The `host` is the `<clusterName>.<baseDomain>*` value with the `clusterName` from the `Cluster` resource, and the `baseDomain` is defined in the `OpenshiftAssistedControlPlane` resource.

12. Apply the file. Run the following command:

```
oc apply -f <filename>.yaml
```

13. Configure the `Metal3MachineTemplate` resource for the control plane nodes that are referenced from the `OpenshiftAssistedControlPlane` resource. See the following YAML sample:

```
apiVersion: infrastructure.cluster.x-k8s.io/v1beta1
kind: Metal3MachineTemplate
metadata:
  name: <cluster-name>
  namespace: <cluster-namespace>
spec:
  nodeReuse: false
  template:
    spec:
```



```

    automatedCleaningMode: disabled
    dataTemplate:
      name: <cluster-name-template>
      image: ❶
      checksum: https://mirror.openshift.com/pub/openshift-
v4/x86_64/dependencies/rhcos/4.19/4.19.0/sha256sum.txt
      checksumType: sha256
      url: https://mirror.openshift.com/pub/openshift-
v4/x86_64/dependencies/rhcos/4.19/4.19.0/rhcos-4.19.0-x86_64-nutanix.x86_64.qcow2
      format: qcow2
    ---
  apiVersion: infrastructure.cluster.x-k8s.io/v1beta1
  kind: Metal3DataTemplate
  metadata:
    name: <cluster-name-template>
    namespace: <cluster-namespace>
  spec:
    clusterName: <cluster-name> ❷

```

❶ The image matches the `distributionVersion` value and the version from [OpenShift Container Platform Releases](#).

❷ Set the `clusterName` to the same value that it is in the `Cluster` resource

14. Apply the file. Run the following command:

```
oc apply -f <filename>.yaml
```

15. Configure worker nodes with the `MachineDeployment` resource, which refers to a `Metal3MachineTemplate`. See the following YAML example:

```

apiVersion: cluster.x-k8s.io/v1beta1
kind: MachineDeployment
metadata:
  name: <cluster-name-worker>
  namespace: <cluster-namespace>
  labels:
    cluster.x-k8s.io/cluster-name: <cluster-name>
spec:
  clusterName: <cluster-name>
  replicas: 2
  selector:
    matchLabels:
      cluster.x-k8s.io/cluster-name: <cluster-name>
  template:
    metadata:
      labels:
        cluster.x-k8s.io/cluster-name: <cluster-name>
    spec:
      clusterName: <cluster-name>

```

```
bootstrap: ①
  configRef:
    name: <cluster-name-worker>
    apiVersion: bootstrap.cluster.x-k8s.io/v1alpha1
    kind: OpenshiftAssistedConfigTemplate
  infrastructureRef: ②
    name: <cluster-name-workers-2>
    apiVersion: infrastructure.cluster.x-k8s.io/v1alpha3
    kind: Metal3MachineTemplate
```

① The bootstrap configuration is also referenced in the next resource for installation.

② The infrastructure reference points to the resource to provision.

16. Apply the file. Run the following command:

```
oc apply -f <filename>.yaml
```

17. Create the **OpenshiftAssistedConfigTemplate** resource that is referenced in the **MachineDeployment**. The following YAML defines the bootstrap configuration for worker nodes and is used to register the nodes with the Assisted Installer:

```
apiVersion: bootstrap.cluster.x-k8s.io/v1alpha1
kind: OpenshiftAssistedConfigTemplate
metadata:
  name: <cluster-name-worker>
  namespace: <cluster-namespace>
  labels:
    cluster.x-k8s.io/cluster-name: cluster-name
spec:
  template:
    spec:
      nodeRegistration:
        kubeletExtraLabels:
          - 'metal3.io/uuid="${METADATA_UUID}"' ①
        sshAuthorizedKey: "{{ ssh_authorized_key }}" ②
```

① The **kubeletExtraLabels** cannot change from this value.

② Enter your **sshAuthorizedKey**.

18. Create the **Metal3MachineTemplate** that is referenced in the **MachineDeployment** resource. See the following example:

```
apiVersion: infrastructure.cluster.x-k8s.io/v1beta1
kind: Metal3MachineTemplate
metadata:
  name: <cluster-name-workers-2>
  namespace: <cluster-namespace>
```

```
spec:
  nodeReuse: false
  template:
    spec:
      automatedCleaningMode: metadata
      dataTemplate:
        name: <cluster-name-workers-template>
      image:
        checksum: https://mirror.openshift.com/pub/openshift-
v4/x86_64/dependencies/rhcos/4.19/4.19.0/sha256sum.txt
        checksumType: sha256
        url: https://mirror.openshift.com/pub/openshift-
v4/x86_64/dependencies/rhcos/4.19/4.19.0/rhcos-4.19.0-x86_64-nutanix.x86_64.qcow2
        format: qcow2
    ---
  apiVersion: infrastructure.cluster.x-k8s.io/v1beta1
  kind: Metal3DataTemplate
  metadata:
    name: <cluster-name-workers-template>
    namespace: <cluster-namespace>
  spec:
    clusterName: <cluster-name>
```

19. Run the following command to save the YAML file and provision a cluster:

```
oc apply -f <filename>.yaml
```

20. Verify your cluster provisioning status.

- a. Run `oc get cluster --namespace <cluster-namespace> <cluster-name> -o yaml` to check your **Cluster** resource status.

See the following output and status:

```
status:
  conditions:
  - lastTransitionTime: "2025-06-25T11:23:38Z"
    status: "True"
    type: Ready
  - lastTransitionTime: "2025-06-25T11:23:38Z"
    status: "True"
    type: ControlPlaneReady
  - lastTransitionTime: "2025-06-25T10:37:03Z"
    status: "True"
    type: InfrastructureReady
```

- b. Run the `oc get metal3cluster --namespace <cluster-namespace> <cluster-name> -o yaml` command to check your cluster infrastructure status.

See the following output and status:

```
status:
  conditions:
  - lastTransitionTime: "2025-06-25T10:37:03Z"
    status: "True"
    type: Ready
  - lastTransitionTime: "2025-06-25T10:37:03Z"
    status: "True"
    type: BaremetalInfrastructureReady
```

- c. Run the `oc get openshiftassistedcontrolplane --namespace <cluster-namespace> <cluster-name> -o yaml` command to check your control plane status.

See the following output and status:

```
status:
  conditions:
  - lastTransitionTime: "2025-06-25T11:23:38Z"
    status: "True"
    type: Ready
  - lastTransitionTime: "2025-06-25T11:23:38Z"
    status: "True"
    type: ControlPlaneReady
  - lastTransitionTime: "2025-06-25T10:45:48Z"
    status: "True"
    type: KubeconfigAvailable
  - lastTransitionTime: "2025-06-25T10:38:25Z"
    status: "True"
    type: MachinesCreated
  - lastTransitionTime: "2025-06-25T11:10:54Z"
    status: "True"
    type: MachinesReady
  - lastTransitionTime: "2025-06-25T11:23:38Z"
    status: "True"
    type: UpgradeCompleted
```

- d. Run the `oc get machinedeployment --namespace <cluster-namespace> <cluster-name> -o yaml` command to check your machine deployments status.

See the following output and status:

```
status:
  conditions:
  - lastTransitionTime: "2025-06-25T11:10:29Z"
    status: "True"
    type: Ready
  - lastTransitionTime: "2025-06-25T11:10:29Z"
```

```
status: "True"
type: Available
- lastTransitionTime: "2025-06-25T11:10:29Z"
status: "True"
type: MachineSetReady
```

- e. Run the `kubectl get machine -l cluster.x-k8s.io/cluster-name=cluster-name -n test-capi -o yaml` command to check your machines.

See the following output and status:

```
status:
  conditions:
    - lastTransitionTime: "2025-06-25T11:09:57Z"
      status: "True"
      type: Ready
    - lastTransitionTime: "2025-06-25T10:38:20Z"
      status: "True"
      type: BootstrapReady
    - lastTransitionTime: "2025-06-25T11:09:57Z"
      status: "True"
      type: InfrastructureReady
    - lastTransitionTime: "2025-06-25T11:10:29Z"
      status: "True"
      type: NodeHealthy
```

21. Access the cluster.

- a. Run the following command to get the `kubeconfig` file:

```
oc get secret -n test-capi cluster-name-admin-kubeconfig -o json | jq -r
.data.kubeconfig | base64 --decode > kubeconfig
```

- b. Run the following command to use the `kubeconfig` file to access the cluster:

```
export KUBECONFIG=$(realpath kubeconfig)
oc get nodes
```

Enabling automatic import for Cluster API clusters (Technology Preview)

You can save time by automatically importing clusters that you provision by using the Cluster API.

Required access: Cluster administrator

Prerequisites

- You need the `MultiClusterEngine` resource either from the Red Hat Advanced Cluster Management installation, or the multicluster engine operator standalone installation.

Preparing your hub cluster

Before you can automatically import provisioned clusters with the Cluster API, you must complete the following steps:

1. Enable the `ClusterImporter` feature gate in the `ClusterManager` resource by adding the following YAML file sample:

```
apiVersion: operator.open-cluster-management.io/v1
kind: ClusterManager
metadata:
  name: cluster-manager
spec:
  registrationConfiguration:
    featureGates:
      - feature: ClusterImporter
        mode: Enable
```

2. Configure the import controller to create cluster import configuration secrets by setting `clusterImportConfig` to `true`. Your `ConfigMap` resource might resemble the following YAML file sample:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: import-controller-config
  namespace: multicloud-engine
data:
  clusterImportConfig: "true"
```

- a. If you do not have a `ConfigMap` resource yet, run the following command to apply the changes. Replace `<filename>` with your file name:

```
oc apply -f <filename>.yaml
```

- b. If you already have a `ConfigMap` resource, run the following command to apply the changes:

```
oc patch configmap import-controller-config -n multicloud-engine --type merge -p '{"data":{"clusterImportConfig":"true"}}'
```

3. Bind the Cluster API manager permissions to the import controller service account. Add the following YAML file sample:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
```

```
name: cluster-manager-registration-capi
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: capi-manager-role
subjects:
- kind: ServiceAccount
  name: registration-controller-sa
  namespace: open-cluster-management-hub
```

4. Run the following command to apply the changes. Replace `<filename>` with your file name:

```
oc apply -f <filename>.yaml
```

Creating the *ManagedCluster* resource

To enable automatic import, create a **ManagedCluster** resource with the same name and namespace as your Cluster API cluster. Complete the following steps:

1. Create a **ManagedCluster** resource by adding the following YAML sample. Add the name of your Cluster API cluster to the `name` parameter:

```
apiVersion: cluster.open-cluster-management.io/v1
kind: ManagedCluster
metadata:
  name: <clusterapi_cluster_name>
spec:
  hubAcceptsClient: true
```

2. Run the following command to apply the changes. Replace `<filename>` with your file name:

```
oc apply -f <filename>.yaml
```

Switching from AWS credentials to IAM roles on your managed cluster

You can switch to using Identity and Access Management (IAM) roles when deploying a managed cluster with Cluster API. If you created a managed cluster with Amazon Web Services (AWS) credentials, you can switch from AWS credentials to IAM roles to use `cluster-api-provider-aws` to create Red Hat OpenShift Service on AWS with hosted control planes clusters without storing AWS credentials on your managed cluster.

Required access: Cluster administrator

Prerequisites

- You need a Red Hat OpenShift Container Platform or Red Hat OpenShift Service on AWS with hosted control planes bootstrap cluster that you created with AWS credentials. You can use

temporary credentials. See [Request temporary security credentials](#) in the AWS documentation to learn more.

- You need Red Hat Advanced Cluster Management for Kubernetes installed on the bootstrap cluster.
- You need to disable HyperShift components before enabling Cluster API and Cluster API Provider AWS.

Procedure

Complete the following steps:

1. Edit the **MultiClusterEngine** resource. Run the following command:

```
oc edit multiclusterengine engine
```

2. Disable the **hypershift** and **hypershift-local-hosting** components, and enable the **cluster-api** and **cluster-api-provider-aws** components. See the following example:

```
- configOverrides: {}
  enabled: false
  name: hypershift
- configOverrides: {}
  enabled: false
  name: hypershift-local-hosting
- configOverrides: {}
  enabled: true
  name: cluster-api
- configOverrides: {}
  enabled: true
  name: cluster-api-provider-aws
```

3. Verify that the Cluster API and Cluster API Provider AWS deployments are running. Run the following command:

```
oc get deploy -n multicluster-engine
```

The output might resemble the following example:

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
capa-controller-manager	1/1	1	1	12d
capi-controller-manager	1/1	1	1	12d

4. Pause the **MultiClusterEngine** custom resource so that the Cluster API Provider AWS controller service account can update. Run the following command:


```
oc annotate mce multiclusterengine installer.multicluster.openshift.io/pause=true
```

5. Retrieve the OpenID Connect (OIDC) provider details. Run the following command:

```
export OIDC_PROVIDER=$(oc get authentication.config.openshift.io cluster -ojson |  
jq -r .spec.serviceAccountIssuer | sed 's/https:\\/\\/'')
```

6. Set your AWS account ID. Run the following command. Change the account ID value:

```
export AWS_ACCOUNT_ID={YOUR_AWS_ACCOUNT_ID}
```

7. Create the trust policy file for the `capa-controller-manager` IAM role. Run the following command:

```
cat ./trust.json
```

8. Define who is allowed to assume an associated IAM role by adding the following content to the file:

```
{  
  "Version": "2012-10-17",  
  "Statement": [  
    {  
      "Effect": "Allow",  
      "Principal": {  
        "Federated": "arn:aws:iam::${AWS_ACCOUNT_ID}:oidc-  
provider/${OIDC_PROVIDER}"  
      },  
      "Action": "sts:AssumeRoleWithWebIdentity",  
      "Condition": {  
        "StringEquals": {  
          "${OIDC_PROVIDER}:sub": "system:serviceaccount:multicluster-engine:capa-  
controller-manager"  
        }  
      }  
    }  
  ]  
}
```

9. Create the IAM role and attach the required AWS policies. Run the following commands:

```
aws iam create-role --role-name "capa-manager-role" --assume-role-policy-document  
file://trust.json --description "IAM role for CAPA to assume"
```

```
aws iam attach-role-policy --role-name capa-manager-role --policy-arn
arn:aws:iam::aws:policy/AWSCloudFormationFullAccess
```

```
aws iam attach-role-policy --role-name capa-manager-role --policy-arn
arn:aws:iam::aws:policy/AmazonVPCFullAccess
```

10. Retrieve the IAM role Amazon Resource Name (ARN). Run the following commands:

```
export APP_IAM_ROLE_ARN=$(aws iam get-role --role-name=capa-manager-role --query
Role.Arn --output text)
```

```
export IRSA_ROLE_ARN=eks.amazonaws.com/role-arn=$APP_IAM_ROLE_ARN
```

11. Annotate the service account with the IAM role ARN. Run the following command:

```
oc annotate serviceaccount -n multicloud-engine capa-controller-manager
$IRSA_ROLE_ARN
```

12. Restart the Cluster API Provider AWS deployment. Run the following command:

```
oc rollout restart deployment capa-controller-manager -n multicloud-engine
```

Creating a Red Hat OpenShift Service on AWS with hosted control planes cluster with Cluster API

A Red Hat OpenShift Service on AWS with hosted control planes cluster is a deployment model for Red Hat OpenShift Service on AWS (classic) where the cluster control plane is hosted and managed in a Red Hat Amazon Web Services (AWS) account. Complete the steps in the following topics to create a Red Hat OpenShift Service on AWS with hosted control planes cluster by using Cluster API.

Required access: Cluster administrator

Prerequisites

- You need an AWS access key. See [Manage access keys for IAM users](#) in the AWS documentation to learn more.
- The AWS Red Hat OpenShift Service on AWS (classic) managed policies are attached to your AWS user permissions. See [AWS managed policies for ROSA](#) in the AWS documentation to learn more.
- You need the OpenShift CLI (`oc`). See [OpenShift CLI \(oc\)](#) in the OpenShift Container Platform documentation to learn more.
- You need the ROSA CLI (`rosa`). See [ROSA CLI](#) in the Red Hat OpenShift Service on AWS (classic) documentation to learn more.

Creating a service account

You need a service account before creating a Red Hat OpenShift Service on AWS with hosted control planes cluster. If you have a service account, you do not need to complete this section.

Procedure

Complete the following steps:

1. Create a service account by navigating to **Identity & Access Management > Service Accounts** in the console.
2. Click the **Create service account** button.
3. For every new service account you create, activate the account by using the ROSA CLI.
 - a. Log in to your new service account. Run the following command. Replace values where needed:

```
rosa login --client-id <your-client-id> --client-secret <your-client-secret>
```

- b. Activate your service account. Run the following command:

```
rosa whoami
```

Setting AWS credentials

You need to set your AWS credentials before creating a Red Hat OpenShift Service on AWS with hosted control planes cluster. Complete the following steps:

1. If you are not using multi-factor authentication with AWS, set your AWS credentials by using the AWS access key you created. Run the following command. Replace values where needed:

```
echo '[default]
aws_access_key_id = <your-access-key>
aws_secret_access_key = <your-secret-access-key>
region = us-east-1
' | base64 -w 0
```

2. If you are using multi-factor authentication with AWS, run the following command. Replace values where needed:

```
echo '[default]
aws_access_key_id = <your-access-key>
aws_secret_access_key = <your-secret-access-key>
aws_session_token = <your-aws-session-token>
region = us-east-1
' | base64 -w 0
```

3. Update the `capa-manager-bootstrap-credentials` secret.
 - a. Copy the output of the previous command and add the output to the `capa-manager-bootstrap-credentials` secret. Run the following command to edit the secret:

```
oc edit secret -n multicluster-engine capa-manager-bootstrap-credentials
```

- b. Add your output to the `credentials` field. See the following example:

```
apiVersion: v1
data:
  credentials: <your-aws-credentials>
kind: Secret
metadata:
  name: capa-manager-bootstrap-credentials
  namespace: multicluster-engine
```

Optional: If you want to use AWS IAM roles with your service account to authenticate the `capa-controller-manager`, see the Additional resources section and complete the steps in *Switching from AWS credentials to IAM roles on your Cluster API managed cluster (Technology Preview)*.

4. Restart the `capa-controller-manager` deployment. Run the following command:

```
oc rollout restart deployment capa-controller-manager -n multicluster-engine
```

Authenticating OpenShift Cluster Manager

The Cluster API Provider for AWS controller requires OpenShift Cluster Manager credentials to provision Red Hat OpenShift Service on AWS with hosted control planes. Complete the following steps:

1. Create a Kubernetes secret in the target namespace with the service account credentials you created. The `ROSAControlPlane` resource references this secret during provisioning. Run the following command. Replace values where needed:

```
oc create namespace <your-rosa-hcp-namespace>
oc -n <your-rosa-hcp-namespace> create secret generic rosa-creds-secret \
  --from-literal=ocmClientID='....' \
  --from-literal=ocmClientSecret='eyJhbGciOiJIUzI1NiIsI....' \
  --from-literal=ocmApiUrl='https://api.openshift.com'
```

2. **Optional:** You can consume the secret without referencing it from your `ROSAControlPlane` resource by calling the secret `rosa-creds-secret` and creating it in the `multicluster-engine` namespace. Run the following command:

```
oc -n multicloud-engine create secret generic rosa-creds-secret \
  --from-literal=ocmClientID='....' \
  --from-literal=ocmClientSecret='eyJhbGciOiJIUzI1NiIsI....' \
  --from-literal=ocmApiUrl='https://api.openshift.com'
```

Creating a Red Hat OpenShift Service on AWS with hosted control planes cluster

After creating a service account, setting your AWS credentials, and authenticating your OpenShift Cluster Manager credentials, complete the following steps to create a Red Hat OpenShift Service on AWS with hosted control planes cluster:

1. Create the **AWSClusterControllerIdentity** resource. See the following YAML file sample:

```
apiVersion: infrastructure.cluster.x-k8s.io/v1beta2
kind: AWSClusterControllerIdentity
metadata:
  name: "default"
spec:
  allowedNamespaces: {}
```

2. Create the **ROSARoleConfig** resource . See the following YAML file sample:

```
apiVersion: infrastructure.cluster.x-k8s.io/v1beta2
kind: ROSARoleConfig
metadata:
  name: "role-config"
  namespace: "ns-rosa-hcp"
spec:
  accountRoleConfig:
    prefix: "rosa"
    version: "4.20.0"
  operatorRoleConfig:
    prefix: "rosa"
  credentialsSecretRef:
    name: rosa-creds-secret
  oidcProviderType: Managed
```

3. Create the **ROSANetwork** resource. See the following YAML file sample:

```
apiVersion: infrastructure.cluster.x-k8s.io/v1beta2
kind: ROSANetwork
metadata:
  name: "rosa-vpc"
  namespace: "ns-rosa-hcp"
spec:
  region: "us-west-2"
  stackName: "rosa-hcp-net"
```

```
availabilityZones:
- "us-west-2a"
- "us-west-2b"
- "us-west-2c"
cidrBlock: 10.0.0.0/16
identityRef:
  kind: AWSClusterControllerIdentity
  name: default
```

4. Verify that the `ROSARoleConfig` resource is created. Check that the `ROSARoleConfig` status contains the `accountRolesRef`, `oidcID`, `oidcProviderARN`, and `operatorRolesRef`. Run the following command:

```
oc get rosaroleconfig -n ns-rosa-hcp role-config -o yaml
```

5. Verify that the `ROSANetwork` resource is created. The `ROSANetwork` status contains the created subnets. Run the following command:

```
oc get rosanetwork -n ns-rosa-hcp rosa-vpc -o yaml
```

6. Create the required custom resources for the `ROSAControlPlane` resource.
 - a. Create the `ManagedCluster` resource with the following YAML file content:

```
apiVersion: cluster.open-cluster-management.io/v1
kind: ManagedCluster
metadata:
  name: rosa-hcp-1
  spec:
    hubAcceptsClient: true
```

- b. Create the `Cluster` resource with the following YAML file content:

```
apiVersion: cluster.x-k8s.io/v1beta1
kind: Cluster
metadata:
  name: "rosa-hcp-1"
  namespace: "ns-rosa-hcp"
  spec:
    clusterNetwork:
      pods:
        cidrBlocks: ["192.168.0.0/16"]
    infrastructureRef:
      apiVersion: infrastructure.cluster.x-k8s.io/v1beta2
      kind: ROSACluster
      name: "rosa-hcp-1"
      namespace: "ns-rosa-hcp"
    controlPlaneRef:
```

```
apiVersion: controlplane.cluster.x-k8s.io/v1beta2
kind: ROSAControlPlane
name: "rosa-cp-1"
namespace: "ns-rosa-hcp"
```

7. Create the **ROSACluster** resource with the following YAML file content:

```
apiVersion: infrastructure.cluster.x-k8s.io/v1beta2
kind: ROSACluster
metadata:
  name: "rosa-hcp-1"
  namespace: "ns-rosa-hcp"
spec: {}
```

8. Create the **ROSAControlPlane** resource with the following YAML file content. Make sure the **region** matches the AWS region you used to create the **ROSANetwork** resource:

```
apiVersion: controlplane.cluster.x-k8s.io/v1beta2
kind: ROSAControlPlane
metadata:
  name: "rosa-cp-1"
  namespace: "ns-rosa-hcp"
spec:
  credentialsSecretRef:
    name: rosa-creds-secret
  rosaClusterName: rosa-hcp-1
  domainPrefix: rosa-hcp
  rosaRoleConfigRef:
    name: role-config
  version: "4.20.0"
  region: "us-west-2"
  rosaNetworkRef:
    name: "rosa-vpc"
  network:
    machineCIDR: "10.0.0.0/16"
    podCIDR: "10.128.0.0/14"
    serviceCIDR: "172.30.0.0/16"
  defaultMachinePoolSpec:
    instanceType: "m5.xlarge"
    autoscaling:
      maxReplicas: 6
      minReplicas: 3
  additionalTags:
    env: "demo"
    profile: "hcp"
```

9. Check the **ROSAControlPlane** status. Run the following command. Make sure the **READY** column says **true**:

```
oc get ROSAControlPlane rosa-cp-1 -n ns-rosa-hcp
```

Note: The Red Hat OpenShift Service on AWS with hosted control planes cluster might need up to 40 minutes to fully provision.

10. After the `ROSAControlPlane` resource provisioning is completed, verify the `ROSAMachinePool` is created. Run the following command:

```
oc get ROSAMachinePool -n ns-rosa-hcp
```

The following output might be displayed:

NAMESPACE	NAME	READY	REPLICAS
ns-rosa-hcp	workers-0	true	1
ns-rosa-hcp	workers-1	true	1
ns-rosa-hcp	workers-2	true	1

Note: The default available `ROSAMachinePools` count is based on the assigned availability zones.

Additional resources

- If you want to use AWS IAM roles with your service account to authenticate the `capa-controller-manager`, complete the steps in [Switching from AWS credentials to IAM roles on your Cluster API managed cluster \(Technology Preview\)](#).

Deleting a Red Hat OpenShift Service on AWS with hosted control planes cluster with Cluster API

Deleting the `ROSAControlPlane` deprovisions the Red Hat OpenShift Service on AWS with hosted control planes cluster. The process takes 30 to 50 minutes to complete. The associated `ROSAMachinePool` resources are automatically deleted.

Required access: Cluster administrator

Prerequisites

- You have a Red Hat OpenShift Service on AWS with hosted control planes cluster with Cluster API.

Procedure

1. Delete the `ROSAControlPlane` custom resource and the matching cluster custom resource. Run the following command. Replace values where needed:

```
oc delete -n ns-rosa-hcp cluster/rosa-hcp-1 rosacontrolplane/rosa-cp-1
```

2. After the `ROSAControlPlane` deletion completes, delete the `ROSARoleConfig` and `ROSANetwork`

resources.

Preparing to create Microsoft Azure Red Hat OpenShift hosted control plane clusters by using Cluster API Provider for Azure

Before creating an Azure Red Hat OpenShift hosted control plane managed cluster by using Cluster API Provider for Azure, you must complete the following preparation procedure.

Required access: Cluster administrator

Prerequisites

- You need the **MultiClusterEngine** resource either from the Red Hat Advanced Cluster Management installation, or the multicluster engine operator standalone installation.
- You need a valid Microsoft Azure subscription with permissions to create Azure Red Hat OpenShift hosted control plane resources.
- You need the OpenShift CLI. See [OpenShift CLI \(oc\)](#) in the OpenShift Container Platform documentation to learn more.
- You need the **jq** tool. See [./jq](#) to learn more.
- You need the Azure Service Operator v2. See [Azure Service Operator v2](#) to learn more.
- Your hub cluster must have internet access.

Procedure

1. Check if the **hypershift** and **hypershift-local-hosting** components are disabled. Run the following command:

```
oc get multiclusterengine multiclusterengine -o json | \
jq -r '.spec.overrides.components[] | [.name, .enabled] | @tsv' | \
column -t -s $'\t' -N "COMPONENT,ENABLED"
```

The output might look similar to the following example:

COMPONENT	ENABLED
hypershift	false
hypershift-local-hosting	false

2. If the components are enabled, disable both components.
 - a. Run the following command to disable the **hypershift** component:

```
oc patch mce multiclusterengine --type=merge -p
"{\"spec\":{\"overrides\":{\"components\":$(oc get mce multiclusterengine -o
json | jq -c '.spec.overrides.components | map(if .name == \"hypershift\" then
.enabled = false else . end)')}}}"
```

b. Run the following command to disable the **hypershift-local-hosting** component:

```
oc patch mce multiclusterengine --type=merge -p
"{\"spec\":{\"overrides\":{\"components\":{\"(oc get mce multiclusterengine -o json |
jq -c '.spec.overrides.components | map(if .name == \"hypershift-local-hosting\" then
.enabled = false else . end)')}}}"
```

3. Verify that the components are disabled. Run the following command:

```
oc get deployment -n hypershift
```

Make sure the output matches the following sample:

```
No resources found in hypershift namespace.
```

4. Enable the Cluster API component. Run the following command:

```
oc patch mce multiclusterengine --type=merge -p
"{\"spec\":{\"overrides\":{\"components\":{\"(oc get mce multiclusterengine -o json |
jq -c '.spec.overrides.components | map(if .name == \"cluster-api\" then .enabled =
true else . end)')}}}"
```

5. Verify that the Cluster API component is enabled. Run the following command:

```
oc get mce multiclusterengine -o json | \
jq -r '.spec.overrides.components[] | select(.name == \"cluster-api\") | [.name,
.enabled] | @tsv'
```

Make sure the output matches the following sample:

```
cluster-api true
```

6. Enable the Cluster API Provider for Azure component. Run the following command:

```
oc patch mce multiclusterengine --type=merge -p
"{\"spec\":{\"overrides\":{\"components\":{\"(oc get mce multiclusterengine -o json |
jq -c '.spec.overrides.components | map(if .name == \"cluster-api-provider-azure-
preview\" then .enabled = true else . end)')}}}"
```

7. Verify that the Cluster API Provider for Azure component is enabled. Run the following command:

```
oc get mce multiclusterengine -o json | \
jq -r '.spec.overrides.components[] | select(.name == "cluster-api-provider-
azure-preview") | [.name, .enabled] | @tsv'
```

Make sure the output matches the following sample:

```
cluster-api-provider-azure-preview true
```

8. Verify that the operators are ready. Run the following command:

```
oc get deployment -n multicluster-engine | grep -E
"(capi|capz|azureserviceoperator)"
```

Make sure the output shows **1** in the **AVAILABLE** column for each operator. See the following example:

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
azureserviceoperator-controller-manager	1/1	1	1	2m
capz-controller-manager	1/1	1	1	2m
capi-controller-manager	1/1	1	1	2m

If the operators are not ready, wait until they are ready.

9. Configure your Microsoft Azure credentials.

- a. Create a service principal with permissions to manage Azure Red Hat OpenShift hosted control plane resources. Run the following command. Replace **<subscription_id>** with your subscription ID:

```
az ad sp create-for-rbac \
--name "aro-hcp-capz-sp" \
--role "Contributor" \
--scopes "/subscriptions/<subscription_id>"
```

Save the following information from the output for the next step:

- **tenant**
- **appId**
- **password**

- b. Create a namespace-scoped **Secret** resource with your Microsoft Azure credentials for Azure Service Operator. Use the following YAML sample:

```
apiVersion: v1
```

```
kind: Secret
metadata:
  name: aso-credential
  namespace: default
stringData:
  AZURE_SUBSCRIPTION_ID: "<subscription_id>"
  AZURE_TENANT_ID: "<tenant_id>"
  AZURE_CLIENT_ID: "<client_id>"
  AZURE_CLIENT_SECRET: "<client_secret>"
```

- Replace `<subscription_id>` with your subscription ID.
- Replace `<tenant_id>` with the `tenant` value from the previous step.
- Replace `<client_id>` with the `appId` value from the previous step.
- Replace `<client_secret>` with the `password` value from the previous step.

Note: The namespace-scoped `Secret` resource overrides global settings for all resources in that namespace. The namespace-scoped `Secret` resource is used for objects in the default namespace. It is not aligned with provisioning Azure Red Hat OpenShift hosted control plane clusters by using Cluster API Provider for Azure. Provisioning uses the `aro-clusters` namespace.

c. Apply the YAML file. Run the following command:

```
oc apply -f
```

Additional resources

- To learn more about Azure Red Hat OpenShift, see the [Azure Red Hat OpenShift documentation](#).

Creating Microsoft Azure Red Hat OpenShift hosted control plane clusters by using Cluster API Provider for Azure

Create an Azure Red Hat OpenShift hosted control plane managed cluster by using Cluster API Provider for Azure to reduce costs and setup time by swapping dedicated main virtual machines for a managed, containerized control plane. Use a GitOps workflow to provision and scale clusters by using the same declarative manifests that you use for your applications.

Required access: Cluster administrator

Prerequisites

- You must complete the preparation procedure. See [Preparing to create Azure Red Hat OpenShift hosted control plane clusters by using Cluster API Provider for Azure](#) to learn more.
- You need the `clusterctl` tool. See [Overview of clusterctl](#) to learn more.

Procedure

1. Create a cluster namespace. Run the following command:

```
oc create namespace aro-clusters
```

2. Create a YAML file named `aro-hcp-cluster.yaml` to define your cluster resources. Replace `<subscription_id>` with your subscription ID. Your resource might resemble the following example:

```
apiVersion: cluster.x-k8s.io/v1beta2
kind: Cluster
metadata:
  name: aro-hcp-cluster
  namespace: aro-clusters
  labels:
    cluster.x-k8s.io/cluster-name: aro-hcp-cluster
spec:
  clusterNetwork:
    pods:
      cidrBlocks:
        - 10.128.0.0/14
    services:
      cidrBlocks:
        - 172.30.0.0/16
  controlPlaneRef:
    apiVersion: controlplane.cluster.x-k8s.io/v1beta2
    kind: AROControlPlane
    name: aro-hcp-cluster
  infrastructureRef:
    apiVersion: infrastructure.cluster.x-k8s.io/v1beta2
    kind: AROCluster
    name: aro-hcp-cluster
```

3. Create an `AROCluster` resource and define your infrastructure resources in `spec.resources`. Replace `<subscription_id>` with your subscription ID. Your resource might resemble the following example:

```
apiVersion: infrastructure.cluster.x-k8s.io/v1beta2
kind: AROCluster
metadata:
  name: aro-hcp-cluster
  namespace: aro-clusters
spec:
  location: eastus
  resourceGroupName: aro-hcp-cluster-rg
  subscriptionID: "<subscription_id>"
  resources:
```

4. Create an `AROControlPlane` resource and define your control plane resources in `spec.resources` field. Your resource might resemble the following example:

```

apiVersion: controlplane.cluster.x-k8s.io/v1beta2
kind: AROControlPlane
metadata:
  name: aro-hcp-cluster
  namespace: aro-clusters
spec:
  location: eastus
  subscriptionID: "<subscription_id>"
  version: "4.21.0"
  resources:

```

5. Create a **MachinePool** resource and define your node pool resources in **spec.resources** field. Your resource might resemble the following example:

```

apiVersion: cluster.x-k8s.io/v1beta2
kind: MachinePool
metadata:
  name: aro-hcp-cluster-workers
  namespace: aro-clusters
spec:
  clusterName: aro-hcp-cluster
  replicas: 3
  template:
    spec:
      bootstrap:
        dataSecretName: ""
      clusterName: aro-hcp-cluster
      infrastructureRef:
        apiVersion: infrastructure.cluster.x-k8s.io/v1beta2
        kind: AROMachinePool
        name: aro-hcp-cluster-workers

```

6. Create an **AROMachinePool** resource and define your node pool resources in **spec.resources** field. Your resource might resemble the following example:

```

apiVersion: infrastructure.cluster.x-k8s.io/v1beta2
kind: AROMachinePool
metadata:
  name: aro-hcp-cluster-workers
  namespace: aro-clusters
spec:
  resources:

```

7. Run the following command to apply the file:

```
oc apply -f aro-hcp-cluster.yaml
```

Note: It might take 15 to 20 minutes for the managed cluster to provision.

Verification

1. Check the status of the Cluster API resources by running the following command:

```
clusterctl describe cluster aro-hcp-cluster -n aro-clusters --show-conditions=all
```

2. Check the status of the Azure Service Operator resources by running the following command:

```
oc get HcpOpenShiftCluster,HcpOpenShiftClustersNodePool -n aro-clusters
```

3. Check the Cluster API Provider for Azure controller logs by running the following command:

```
oc logs -n multicloud-engine deployment/capz-controller-manager -f
```

4. Check your managed cluster status.

- a. Check the Cluster API status. Run the following command:

```
oc get cluster aro-hcp-cluster -n aro-clusters
```

- b. **Optional:** Check the detailed cluster status and conditions. Run the following command:

```
clusterctl describe cluster aro-hcp-cluster -n aro-clusters --show-conditions=all
```

If your cluster is ready, the output resembles the following example:

NAME	CLUSTERCLASS	AVAILABLE	CP AVAILABLE	AGE
aro-hcp-cluster		True	True	20m

5. Verify your infrastructure resources.

- a. Check the **AROCluster** status. Run the following command:

```
oc get arocluster aro-hcp-cluster -n aro-clusters
```

- b. Check the **AROControlPlane** status. Run the following command:

```
oc get arocontrolplane aro-hcp-cluster -n aro-clusters
```

c. Check the **AROMachinePool** status. Run the following command:

```
oc get aromachinepool -n aro-clusters
```

6. Verify your Azure Service Operator resources.

a. Check the hosted control plane cluster status in Microsoft Azure. Run the following command:

```
oc get HcpOpenShiftCluster aro-hcp-cluster -n aro-clusters
```

b. Check the node pool status. Run the following command:

```
oc get HcpOpenShiftClustersNodePool -n aro-clusters
```

7. Get your managed cluster **kubeconfig** file. The **kubeconfig** is automatically created when the managed cluster is ready. Run the following command:

```
oc get secret aro-hcp-cluster-kubeconfig -n aro-clusters -o  
jsonpath='{.data.value}' | base64 -d > aro-hcp-cluster-kubeconfig
```

8. Test the connection to your managed cluster. Run the following command:

```
export KUBECONFIG=aro-hcp-cluster-kubeconfig  
oc get nodes
```

9. Get the console URL from the control plane status. Run the following command:

```
oc get arocontrolplane aro-hcp-cluster -n aro-clusters -o  
jsonpath='{.status.consoleURL}'
```

Deleting Azure Red Hat OpenShift hosted control plane clusters by using Cluster API Provider for Azure

Deleting an Azure Red Hat OpenShift hosted control plane managed cluster deprovisions the cluster and deletes the associated resources. The process takes 10 to 15 minutes to complete.

Required access: Cluster administrator

Prerequisites

- You have an Azure Red Hat OpenShift hosted control plane managed cluster.

Procedure

1. Delete your cluster by running the following command:


```
oc delete cluster <cluster_name> -n <cluster_namespace>
```

Verification

1. Check if the Cluster API resources are removed. Run the following command:

```
oc get cluster,arocontrolplane,arocluster,aromachinepool -n <cluster_namespace>
```

2. Check if the Azure Service Operator resources are removed. Run the following command:

```
oc get HcpOpenShiftCluster,HcpOpenShiftClustersNodePool -n <cluster_namespace>
```

3. Check if the Microsoft Azure resources are removed. Run the following commands:

```
az resource list \
  --resource-group <cluster_namespace>-rg \
  --output table
```

```
az resource show \
  --resource-type Microsoft.RedHatOpenShift/HCP0penShiftClusters \
  --name <cluster_namespace> \
  --resource-group <cluster_namespace>-rg \
  --output json | jq '.properties'
```

APIs

Deprecated: The documentation for the APIs is deprecated. Use the *API Explorer* in the console, or an **oc** command, to view current and supported APIs.

You can access the following APIs for cluster lifecycle management with the multicluster engine operator. **User required access:** You can only perform actions that your role is assigned.

Notes:

- You can also access all APIs from the integrated console. Select the **Administrator** perspective, then navigate to **Home > API Explorer** to explore API groups.
- For Red Hat OpenShift Container Platform versions earlier than version 4.20, select **local-cluster** from the cluster switcher. Then select **Home > API Explorer** to explore API groups.

For more information, review the API documentation for each of the following resources:

- [Clusters API](#)
- [ClusterSets API \(v1beta2\)](#)
- [Clusterview API](#)

- [ClusterSetBindings API \(v1beta2\)](#)
- [MultiClusterEngine API](#)
- [Placements API \(v1beta1\)](#)
- [PlacementDecisions API \(v1beta1\)](#)
- [ManagedServiceAccount API](#)
- [KlusterletConfig API \(v1alpha1\)](#)

Clusters API

Deprecated: The documentation for the APIs is deprecated. Use the *API Explorer* in the console, or an **oc** command, to view current and supported APIs.

Overview

This documentation is for the cluster resource for multicluster engine for Kubernetes operator. Cluster resource has four possible requests: create, query, delete and update.

URI scheme

BasePath : /kubernetes/apis

Schemes : HTTPS

Tags

- cluster.open-cluster-management.io : Create and manage clusters

Paths

Query all clusters

```
GET /cluster.open-cluster-management.io/v1/managedclusters
```

Description

Query your clusters for more details.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Consumes

- `cluster/yaml`

Tags

- `cluster.open-cluster-management.io`

Create a cluster

POST `/cluster.open-cluster-management.io/v1/managedclusters`

Description

Create a cluster

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string
Body	body <i>required</i>	Parameters describing the cluster to be created.	Cluster

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Consumes

- `cluster/yaml`

Tags

- `cluster.open-cluster-management.io`

Example HTTP request body

```
{
  "apiVersion" : "cluster.open-cluster-management.io/v1",
  "kind" : "ManagedCluster",
  "metadata" : {
    "labels" : {
      "vendor" : "OpenShift"
    },
    "name" : "cluster1"
  },
  "spec": {
    "hubAcceptsClient": true,
    "managedClusterClientConfigs": [
      {
        "caBundle": "test",
        "url": "https://test.com"
      }
    ]
  },
  "status" : { }
}
```

Query a single cluster

```
GET /cluster.open-cluster-management.io/v1/managedclusters/{cluster_name}
```

Description

Query a single cluster for more details.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string
Path	cluster_name <i>required</i>	Name of the cluster that you want to query.	string

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Tags

- cluster.open-cluster-management.io

Delete a cluster

```
DELETE /cluster.open-cluster-management.io/v1/managedclusters/{cluster_name}
```

Description

Delete a single cluster

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string
Path	cluster_name <i>required</i>	Name of the cluster that you want to delete.	string

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Tags

- cluster.open-cluster-management.io

Definitions

Cluster

Name	Schema
apiVersion <i>required</i>	string
kind <i>required</i>	string
metadata <i>required</i>	object
spec <i>required</i>	spec

spec

Name	Schema
hubAcceptsClient <i>required</i>	bool
managedClusterClientConfigs <i>optional</i>	< managedClusterClientConfigs > array
leaseDurationSeconds <i>optional</i>	integer (int32)

managedClusterClientConfigs

Name	Description	Schema
URL <i>required</i>		string
CABundle <i>optional</i>	Pattern : "^(?:[A-Za-z0-9+/]{4})*(?:[A-Za-z0-9+/]{2}= [A-Za-z0-9+/]{3}=)?\$"	string (byte)

Clustersets API (v1beta2)

Deprecated: The documentation for the APIs is deprecated. Use the *API Explorer* in the console, or an **oc** command, to view current and supported APIs.

Overview

This documentation is for the Clusterset resource for multicluster engine for Kubernetes operator. Clusterset resource has four possible requests: create, query, delete and update.

URI scheme

BasePath : /kubernetes/apis

Schemes : HTTPS

Tags

- cluster.open-cluster-management.io : Create and manage Clustersets

Paths

Query all clustersets

```
GET /cluster.open-cluster-management.io/v1beta2/managedclustersets
```

Description

Query your Clustersets for more details.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Consumes

- `clusterset/yaml`

Tags

- cluster.open-cluster-management.io

Create a clusterset

POST /cluster.open-cluster-management.io/v1beta2/managedclustersets

Description

Create a Clusterset.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string
Body	body <i>required</i>	Parameters describing the clusterset to be created.	Clusterset

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Consumes

- `clusterset/yaml`

Tags

- cluster.open-cluster-management.io

Example HTTP request body

```
{
  "apiVersion" : "cluster.open-cluster-management.io/v1beta2",
  "kind" : "ManagedClusterSet",
  "metadata" : {
    "name" : "clusterset1"
  },
  "spec": { },
  "status" : { }
}
```


Query a single clusterset

```
GET /cluster.open-cluster-management.io/v1beta2/managedclustersets/{clusterset_name}
```

Description

Query a single clusterset for more details.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string
Path	clusterset_name <i>required</i>	Name of the clusterset that you want to query.	string

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Tags

- cluster.open-cluster-management.io

Delete a clusterset

```
DELETE /cluster.open-cluster-management.io/v1beta2/managedclustersets/{clusterset_name}
```

Description

Delete a single clusterset.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string
Path	clusterset_name <i>required</i>	Name of the clusterset that you want to delete.	string

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Tags

- cluster.open-cluster-management.io

Definitions

Clusterset

Name	Schema
apiVersion <i>required</i>	string
kind <i>required</i>	string
metadata <i>required</i>	object

Clustersetbindings API (v1beta2)

Deprecated: The documentation for the APIs is deprecated. Use the *API Explorer* in the console, or an **oc** command, to view current and supported APIs.

Overview

This documentation is for the clustersetbinding resource for multicluster engine for Kubernetes. Clustersetbinding resource has four possible requests: create, query, delete and update.

URI scheme

BasePath : /kubernetes/apis

Schemes : HTTPS

Tags

- cluster.open-cluster-management.io : Create and manage clustersetbindings

Paths

Query all clustersetbindings

```
GET /cluster.open-cluster-management.io/v1beta2/namespaces/{namespace}/managedclustersetbindings
```

Description

Query your clustersetbindings for more details.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string
Path	namespace <i>required</i>	Namespace that you want to use, for example, default.	string

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Consumes

- `clustersetbinding/yaml`

Tags

- cluster.open-cluster-management.io

Create a clustersetbinding

```
POST /cluster.open-cluster-  
management.io/v1beta2/namespaces/{namespace}/managedclustersetbindings
```

Description

Create a clustersetbinding.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string
Path	namespace <i>required</i>	Namespace that you want to use, for example, default.	string
Body	body <i>required</i>	Parameters describing the clustersetbinding to be created.	Clustersetbinding

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Consumes

- clustersetbinding/yaml

Tags

- cluster.open-cluster-management.io

Example HTTP request body

```
{  
  "apiVersion" : "cluster.open-cluster-management.io/v1",  
  "kind" : "ManagedClusterSetBinding",  
  "metadata" : {  
    "name" : "clusterset1",  
    "namespace" : "ns1"
```

```
  },
  "spec": {
    "clusterSet": "clusterSet1"
  },
  "status" : { }
}
```

Query a single clustersetbinding

```
GET /cluster.open-cluster-
management.io/v1beta2/namespaces/{namespace}/managedclustersetbindings/{clustersetbind
ing_name}
```

Description

Query a single clustersetbinding for more details.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string
Path	namespace <i>required</i>	Namespace that you want to use, for example, default.	string
Path	clustersetbind ing_name <i>required</i>	Name of the clustersetbinding that you want to query.	string

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Tags

- cluster.open-cluster-management.io

Delete a clustersetbinding

```
DELETE /cluster.open-cluster-  
management.io/v1beta2/managedclustersetbindings/{clustersetbinding_name}
```

Description

Delete a single clustersetbinding.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string
Path	namespace <i>required</i>	Namespace that you want to use, for example, default.	string
Path	clustersetbinding_name <i>required</i>	Name of the clustersetbinding that you want to delete.	string

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Tags

- cluster.open-cluster-management.io

Definitions

Clustersetbinding

Name	Schema
apiVersion <i>required</i>	string
kind <i>required</i>	string

Name	Schema
metadata <i>required</i>	object
spec <i>required</i>	spec

spec

Name	Schema
clusterSet <i>required</i>	string

Clusterview API (v1alpha1)

Deprecated: The documentation for the APIs is deprecated. Use the *API Explorer* in the console, or an **oc** command, to view current and supported APIs.

Overview

This documentation is for the **clusterview** resource for multicluster engine for Kubernetes. The **clusterview** resource provides a CLI command that enables you to view a list of the managed clusters and managed cluster sets that that you can access. The three possible requests are: list, get, and watch.

URI scheme

BasePath : /kubernetes/apis

Schemes : HTTPS

Tags

- **clusterview.open-cluster-management.io** : View a list of managed clusters that your ID can access.

Paths

Get managed clusters

```
GET /managedclusters.clusterview.open-cluster-management.io
```

Description

View a list of the managed clusters that you can access.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Consumes

- `managedcluster/yaml`

Tags

- `clusterview.open-cluster-management.io`

List managed clusters

```
LIST /managedclusters.clusterview.open-cluster-management.io
```

Description

View a list of the managed clusters that you can access.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string
Body	body <i>optional</i>	Name of the user ID for which you want to list the managed clusters.	string

Responses

HTTP Code	Description	Schema
200	Success	No Content

HTTP Code	Description	Schema
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Consumes

- `managedcluster/yaml`

Tags

- `clusterview.open-cluster-management.io`

Example HTTP request body

```
{
  "apiVersion" : "clusterview.open-cluster-management.io/v1alpha1",
  "kind" : "ClusterView",
  "metadata" : {
    "name" : "<user_ID>"
  },
  "spec": { },
  "status" : { }
}
```

Watch the managed cluster sets

```
WATCH /managedclusters.clusterview.open-cluster-management.io
```

Description

Watch the managed clusters that you can access.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string
Path	clusterview_name <i>optional</i>	Name of the user ID that you want to watch.	string

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

List the managed cluster sets.

```
GET /managedclustersets.clusterview.open-cluster-management.io
```

Description

List the managed clusters that you can access.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string
Path	clusterview_name <i>optional</i>	Name of the user ID that you want to watch.	string

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

List the managed cluster sets.

```
LIST /managedclustersets.clusterview.open-cluster-management.io
```

Description

List the managed clusters that you can access.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string
Path	clusterview_name <i>optional</i>	Name of the user ID that you want to watch.	string

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Watch the managed cluster sets.

```
WATCH /managedclustersets.clusterview.open-cluster-management.io
```

Description

Watch the managed clusters that you can access.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string
Path	clusterview_name <i>optional</i>	Name of the user ID that you want to watch.	string

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

ManagedServiceAccount API (v1alpha1) (Deprecated)

Deprecated: The documentation for the APIs is deprecated. Use the *API Explorer* in the console, or an `oc` command, to view current and supported APIs.

Overview

This documentation is for the `ManagedServiceAccount` resource for the multicluster engine operator. The `ManagedServiceAccount` resource has four possible requests: create, query, delete, and update.

Deprecated: The `v1alpha1` API is deprecated. For best results, use `v1beta1` instead.

URI scheme

BasePath : /kubernetes/apis

Schemes : HTTPS

Tags

- `managedserviceaccounts.authentication.open-cluster-management.io``: Create and manage `ManagedServiceAccounts`

Paths

Create a ManagedServiceAccount

```
POST /authentication.open-cluster-management.io/v1beta1/managedserviceaccounts
```

Description

Create a `ManagedServiceAccount`.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string

Type	Name	Description	Schema
Body	body <i>required</i>	Parameters describing the ManagedServiceAccount to be created.	ManagedServiceAccount

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Consumes

- `managedserviceaccount/yaml`

Tags

- `managedserviceaccounts.authentication.open-cluster-management.io`

Request body

```

apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  annotations:
    controller-gen.kubebuilder.io/version: v0.14.0
  name: managedserviceaccounts.authentication.open-cluster-management.io
spec:
  group: authentication.open-cluster-management.io
  names:
    kind: ManagedServiceAccount
    listKind: ManagedServiceAccountList
    plural: managedserviceaccounts
    singular: managedserviceaccount
  scope: Namespaced
  versions:
    - deprecated: true
      deprecationWarning: authentication.open-cluster-management.io/v1alpha1
      ManagedServiceAccount
        is deprecated; use authentication.open-cluster-management.io/v1beta1
      ManagedServiceAccount;
        version v1alpha1 will be removed in the next release
    name: v1alpha1
    schema:

```

```

openAPIV3Schema:
  description: ManagedServiceAccount is the Schema for the
managedserviceaccounts
  API
  properties:
    apiVersion:
      description: |-
        APIVersion defines the versioned schema of this representation of an
object.
        Servers should convert recognized schemas to the latest internal value,
and
        may reject unrecognized values.
        More info: https://git.k8s.io/community/contributors/devel/sig-
architecture/api-conventions.md#resources
      type: string
    kind:
      description: |-
        Kind is a string value representing the REST resource this object
represents.
        Servers may infer this from the endpoint the client submits requests to.
        Cannot be updated.
        In CamelCase.
        More info: https://git.k8s.io/community/contributors/devel/sig-
architecture/api-conventions.md#types-kinds
      type: string
    metadata:
      type: object
    spec:
      description: ManagedServiceAccountSpec defines the desired state of
ManagedServiceAccount
      properties:
        rotation:
          description: Rotation is the policy for rotation the credentials.
          properties:
            enabled:
              default: true
              description: |-
                Enabled prescribes whether the ServiceAccount token will
                be rotated from the upstream
              type: boolean
            validity:
              default: 8640h0m0s
              description: Validity is the duration for which the signed
ServiceAccount
                token is valid.
              type: string
            type: object
          ttlSecondsAfterCreation:
            description: |-
              ttlSecondsAfterCreation limits the lifetime of a
ManagedServiceAccount.

```



```

description: |-
    message is a human readable message indicating details about
the transition.
    This may be an empty string.
maxLength: 32768
type: string
observedGeneration:
description: |-
    observedGeneration represents the .metadata.generation that
the condition was set based upon.
    For instance, if .metadata.generation is currently 12, but the
.status.conditions[x].observedGeneration is 9, the condition is out of date
    with respect to the current state of the instance.
format: int64
minimum: 0
type: integer
reason:
description: |-
    reason contains a programmatic identifier indicating the
reason for the condition's last transition.
    Producers of specific condition types may define expected
values and meanings for this field,
    and whether the values are considered a guaranteed API.
    The value should be a CamelCase string.
    This field may not be empty.
maxLength: 1024
minLength: 1
pattern: ^[A-Za-z]([A-Za-z0-9_,:]*[A-Za-z0-9_])?$
type: string
status:
description: status of the condition, one of True, False,
Unknown.
enum:
- "True"
- "False"
- Unknown
type: string
type:
description: |-
    type of condition in CamelCase or in
foo.example.com/CamelCase.
---
    Many .condition.type values are consistent across resources
like Available, but because arbitrary conditions can be
    useful (see .node.status.conditions), the ability to
deconflict is important.
    The regex it matches is
(dns1123SubdomainFmt/)?(qualifiedNameFmt)
maxLength: 316
pattern: ^([a-z0-9]([-a-z0-9]*[a-z0-9])?(\.[a-z0-9]([-a-z0-9]*[a-z0-9])?)*\/)?((([A-Za-z0-9]([-A-Za-z0-9_\.]*)?[A-Za-z0-9])$

```



```

        type: string
        required:
        - lastTransitionTime
        - message
        - reason
        - status
        - type
        type: object
    type: array
expirationTimestamp:
    description: ExpirationTimestamp is the time when the token will
expire.
    format: date-time
    type: string
tokenSecretRef:
    description: |-
        TokenSecretRef is a reference to the corresponding ServiceAccount's
Secret, which stores
        the CA certificate and token from the managed cluster.
    properties:
        lastRefreshTimestamp:
            description: |-
                LastRefreshTimestamp is the timestamp indicating when the token
in the Secret
                is refreshed.
            format: date-time
            type: string
        name:
            description: Name is the name of the referenced secret.
            type: string
        required:
        - lastRefreshTimestamp
        - name
        type: object
    type: object
type: object
served: true
storage: false
subresources:
    status: {}
- name: v1beta1
  schema:
    openAPIV3Schema:
        description: ManagedServiceAccount is the Schema for the
managedserviceaccounts
        API
        properties:
            apiVersion:
                description: |-
                    APIVersion defines the versioned schema of this representation of an
object.

```

Servers should convert recognized schemas to the latest internal value,
and
may reject unrecognized values.
More info: <https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#resources>

```

type: string
kind:
  description: |-
    Kind is a string value representing the REST resource this object
    represents.
    Servers may infer this from the endpoint the client submits requests to.
    Cannot be updated.
    In CamelCase.
    More info: https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#types-kinds
type: string
metadata:
  type: object
spec:
  description: ManagedServiceAccountSpec defines the desired state of
  ManagedServiceAccount
  properties:
    rotation:
      description: Rotation is the policy for rotation the credentials.
      properties:
        enabled:
          default: true
          description: |-
            Enabled prescribes whether the ServiceAccount token will be
            rotated before it expires.
            Deprecated: All ServiceAccount tokens will be rotated before
            they expire regardless of this field.
          type: boolean
        validity:
          default: 8640h0m0s
          description: Validity is the duration of validity for requesting
            the signed ServiceAccount token.
          type: string
      type: object
    ttlSecondsAfterCreation:
      description: |-
        ttlSecondsAfterCreation limits the lifetime of a
        ManagedServiceAccount.
        If the ttlSecondsAfterCreation field is set, the
        ManagedServiceAccount will be
        automatically deleted regardless of the ManagedServiceAccount's
        status.
        When the ManagedServiceAccount is deleted, its lifecycle guarantees
        (e.g. finalizers) will be honored. If this field is unset, the
        ManagedServiceAccount
        won't be automatically deleted. If this field is set to zero, the

```


observedGeneration represents the .metadata.generation that the condition was set based upon.

For instance, if .metadata.generation is currently 12, but the .status.conditions[x].observedGeneration is 9, the condition is out of date with respect to the current state of the instance.

format: int64

minimum: 0

type: integer

reason:

description: |-

reason contains a programmatic identifier indicating the reason for the condition's last transition.

Producers of specific condition types may define expected values and meanings for this field,

and whether the values are considered a guaranteed API.

The value should be a CamelCase string.

This field may not be empty.

maxLength: 1024

minLength: 1

pattern: ^[A-Za-z]([A-Za-z0-9_,:]*[A-Za-z0-9_])?\$

type: string

status:

description: status of the condition, one of True, False,

Unknown.

enum:

- "True"

- "False"

- Unknown

type: string

type:

description: |-

type of condition in CamelCase or in

foo.example.com/CamelCase.

Many .condition.type values are consistent across resources like Available, but because arbitrary conditions can be

useful (see .node.status.conditions), the ability to deconflict is important.

The regex it matches is

(dns1123SubdomainFmt/)?(qualifiedNameFmt)

maxLength: 316

pattern: ^([a-z0-9]([-a-z0-9]*[a-z0-9])?(\.[a-z0-9]([-a-z0-9]*[a-z0-9])?)*\/)?(([A-Za-z0-9]([-A-Za-z0-9_\.]*)?[A-Za-z0-9])\$

type: string

required:

- lastTransitionTime

- message

- reason

- status

- type

type: object

```

    type: array
    expirationTimestamp:
      description: ExpirationTimestamp is the time when the token will
expire.
      format: date-time
      type: string
    tokenSecretRef:
      description: |-
        TokenSecretRef is a reference to the corresponding ServiceAccount's
Secret, which stores
        the CA certificate and token from the managed cluster.
      properties:
        lastRefreshTimestamp:
          description: |-
            LastRefreshTimestamp is the timestamp indicating when the token
in the Secret
            is refreshed.
          format: date-time
          type: string
        name:
          description: Name is the name of the referenced secret.
          type: string
      required:
      - lastRefreshTimestamp
      - name
      type: object
    type: object
  type: object
served: true
storage: true
subresources:
  status: {}

```

Query a single ManagedServiceAccount

```

GET /authentication.open-cluster-
management.io/v1beta1/namespaces/{namespace}/managedserviceaccounts/{managedserviceacc
ount_name}

```

Description

Query a single [ManagedServiceAccount](#) for more details.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string

Type	Name	Description	Schema
Path	managedserviceaccount_name <i>required</i>	Name of the ManagedServiceAccount that you want to query.	string

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Tags

- managedserviceaccounts.authentication.open-cluster-management.io

Delete a **ManagedServiceAccount**

```
DELETE /authentication.open-cluster-management.io/v1beta1/namespaces/{namespace}/managedserviceaccounts/{managedserviceaccount_name}
```

Description

Delete a single **ManagedServiceAccount**.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string
Path	managedserviceaccount_name <i>required</i>	Name of the ManagedServiceAccount that you want to delete.	string

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Tags

- managedserviceaccounts.authentication.open-cluster-management.io

Definitions

ManagedServiceAccount

Name	Description	Schema
apiVersion <i>required</i>	The versioned schema of the ManagedServiceAccount .	string
kind <i>required</i>	String value that represents the REST resource.	string
metadata <i>required</i>	The meta data of the ManagedServiceAccount .	object
spec <i>required</i>	The specification of the ManagedServiceAccount .	

MultiClusterEngine API (v1alpha1)

Deprecated: The documentation for the APIs is deprecated. Use the *API Explorer* in the console, or an **oc** command, to view current and supported APIs.

Overview

This documentation is for the MultiClusterEngine resource for multicluster engine for Kubernetes. The **MultiClusterEngine** resource has four possible requests: create, query, delete, and update.

URI scheme

BasePath : /kubernetes/apis

Schemes : HTTPS

Tags

- multiclusterengines.multicluster.openshift.io : Create and manage MultiClusterEngines

Paths

Create a MultiClusterEngine

POST /apis/multicluster.openshift.io/v1alpha1/multiclusterengines

Description

Create a MultiClusterEngine.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string
Body	body <i>required</i>	Parameters describing the MultiClusterEngine to be created.	MultiClusterEngine

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Consumes

- [MultiClusterEngines/yaml](#)

Tags

- multiclusterengines.multicluster.openshift.io

Request body

```
{
  "apiVersion": "apiextensions.k8s.io/v1",
  "kind": "CustomResourceDefinition",
  "metadata": {
    "annotations": {
      "controller-gen.kubebuilder.io/version": "v0.4.1"
    },
    "creationTimestamp": null,
    "name": "multiclusterengines.multicluster.openshift.io"
  }
}
```



```

},
"spec": {
  "group": "multicluster.openshift.io",
  "names": {
    "kind": "MultiClusterEngine",
    "listKind": "MultiClusterEngineList",
    "plural": "multiclusterengines",
    "shortNames": [
      "mce"
    ],
    "singular": "multiclusterengine"
  },
  "scope": "Cluster",
  "versions": [
    {
      "additionalPrinterColumns": [
        {
          "description": "The overall state of the MultiClusterEngine",
          "jsonPath": ".status.phase",
          "name": "Status",
          "type": "string"
        },
        {
          "jsonPath": ".metadata.creationTimestamp",
          "name": "Age",
          "type": "date"
        }
      ],
      "name": "v1alpha1",
      "schema": {
        "openAPIV3Schema": {
          "description": "MultiClusterEngine is the Schema for the multiclusterengines\nAPI",
          "properties": {
            "apiVersion": {
              "description": "APIVersion defines the versioned schema of this representation\nof an object. Servers should convert recognized schemas to the latest\ninternal value, and may reject unrecognized values. More info: https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#resources",
              "type": "string"
            },
            "kind": {
              "description": "Kind is a string value representing the REST resource this\nobject represents. Servers may infer this from the endpoint the client\nsubmits requests to. Cannot be updated. In CamelCase. More info: https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#types-kinds",
              "type": "string"
            },
            "metadata": {

```

```

        "type": "object"
    },
    "spec": {
        "description": "MultiClusterEngineSpec defines the desired state of
MultiClusterEngine",
        "properties": {
            "imagePullSecret": {
                "description": "Override pull secret for accessing
MultiClusterEngine\noperand and endpoint images",
                "type": "string"
            },
            "nodeSelector": {
                "additionalProperties": {
                    "type": "string"
                },
                "description": "Set the nodeselectors",
                "type": "object"
            },
            "targetNamespace": {
                "description": "Location where MCE resources will be placed",
                "type": "string"
            },
            "tolerations": {
                "description": "Tolerations causes all components to tolerate any
taints.",
                "items": {
                    "description": "The pod this Toleration is attached to tolerates
any\ntaint that matches the triple <key,value,effect> using the matching\noperator
<operator>.",
                    "properties": {
                        "effect": {
                            "description": "Effect indicates the taint effect to match.
Empty\nmeans match all taint effects. When specified, allowed values\nare NoSchedule,
PreferNoSchedule and NoExecute.",
                            "type": "string"
                        },
                        "key": {
                            "description": "Key is the taint key that the toleration
applies\nto. Empty means match all taint keys. If the key is empty,\noperator must be
Exists; this combination means to match all\nvalues and all keys.",
                            "type": "string"
                        },
                        "operator": {
                            "description": "Operator represents a key's relationship to
the\nvalue. Valid operators are Exists and Equal. Defaults to Equal.\nExists is
equivalent to wildcard for value, so that a pod\ncan tolerate all taints of a
particular category.",
                            "type": "string"
                        },
                        "tolerationSeconds": {
                            "description": "TolerationSeconds represents the period of

```

time\nthe toleration (which must be of effect NoExecute, otherwise\nthis field is ignored) tolerates the taint. By default, it\nis not set, which means tolerate the taint forever (do not\nevict). Zero and negative values will be treated as 0 (evict\nimmediately) by the system.",

```

        "format": "int64",
        "type": "integer"
    },
    "value": {
        "description": "Value is the taint value the toleration
matches\nto. If the operator is Exists, the value should be empty,\notherwise just a
regular string.",
        "type": "string"
    }
},
"type": "object"
},
"type": "array"
}
},
"type": "object"
},
"status": {
    "description": "MultiClusterEngineStatus defines the observed state of
MultiClusterEngine",
    "properties": {
        "components": {
            "items": {
                "description": "ComponentCondition contains condition
information for\ntracked components",
                "properties": {
                    "kind": {
                        "description": "The resource kind this condition
represents",
                        "type": "string"
                    },
                    "lastTransitionTime": {
                        "description": "LastTransitionTime is the last time the
condition\nchanged from one status to another.",
                        "format": "date-time",
                        "type": "string"
                    },
                    "message": {
                        "description": "Message is a human-readable message
indicating\ndetails about the last status change.",
                        "type": "string"
                    },
                    "name": {
                        "description": "The component name",
                        "type": "string"
                    },
                    "reason": {

```

```

        "description": "Reason is a (brief) reason for the
condition's\nlast status change.",
        "type": "string"
    },
    "status": {
        "description": "Status is the status of the condition. One
of True,\nFalse, Unknown.",
        "type": "string"
    },
    "type": {
        "description": "Type is the type of the cluster condition.",
        "type": "string"
    }
},
"type": "object"
},
"type": "array"
},
"conditions": {
    "items": {
        "properties": {
            "lastTransitionTime": {
                "description": "LastTransitionTime is the last time the
condition\nchanged from one status to another.",
                "format": "date-time",
                "type": "string"
            },
            "lastUpdateTime": {
                "description": "The last time this condition was updated.",
                "format": "date-time",
                "type": "string"
            },
            "message": {
                "description": "Message is a human-readable message
indicating\ndetails about the last status change.",
                "type": "string"
            },
            "reason": {
                "description": "Reason is a (brief) reason for the
condition's\nlast status change.",
                "type": "string"
            },
            "status": {
                "description": "Status is the status of the condition. One
of True,\nFalse, Unknown.",
                "type": "string"
            },
            "type": {
                "description": "Type is the type of the cluster condition.",
                "type": "string"
            }
        }
    }
}

```

```

        },
        "type": "object"
      },
      "type": "array"
    },
    "phase": {
      "description": "Latest observed overall state",
      "type": "string"
    }
  },
  "type": "object"
}
},
"type": "object"
}
},
"served": true,
"storage": true,
"subresources": {
  "status": {}
}
}
]
},
"status": {
  "acceptedNames": {
    "kind": "",
    "plural": ""
  },
  "conditions": [],
  "storedVersions": []
}
}

```

Query all MultiClusterEngines

```
GET /apis/multicluster.openshift.io/v1alpha1/multiclusterengines
```

Description

Query your multicluster engine for more details.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Consumes

- `operator/yaml`

Tags

- `multiclusterengines.multicluster.openshift.io`

Delete a MultiClusterEngine operator

```
DELETE /apis/multicluster.openshift.io/v1alpha1/multiclusterengines/{name}
```

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string
Path	name <i>required</i>	Name of the multiclusterengine that you want to delete.	string

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Tags

- `multiclusterengines.multicluster.openshift.io`

Definitions

MultiClusterEngine

Name	Description	Schema
apiVersion <i>required</i>	The versioned schema of the MultiClusterEngines.	string
kind <i>required</i>	String value that represents the REST resource.	string
metadata <i>required</i>	Describes rules that define the resource.	object
spec <i>required</i>	MultiClusterEngineSpec defines the desired state of MultiClusterEngine.	See <i>List of specs</i>

List of specs

Name	Description	Schema
nodeSelector <i>optional</i>	Set the nodeselectors.	map[string]string
imagePullSecret <i>optional</i>	Override pull secret for accessing MultiClusterEngine operand and endpoint images.	string
tolerations <i>optional</i>	Tolerations causes all components to tolerate any taints.	[]corev1.Toleration
targetNamespace <i>optional</i>	Location where MCE resources will be placed.	string

Placements API (v1beta1)

Deprecated: The documentation for the APIs is deprecated. Use the *API Explorer* in the console, or an **oc** command, to view current and supported APIs.

Overview

This documentation is for the Placement resource for multicluster engine for Kubernetes. Placement resource has four possible requests: create, query, delete and update.

URI scheme

BasePath : /kubernetes/apis

Schemes : HTTPS

Tags

- cluster.open-cluster-management.io : Create and manage Placements

Paths

Query all Placements

```
GET /cluster.open-cluster-management.io/v1beta1/namespaces/{namespace}/placements
```

Description

Query your Placements for more details.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Consumes

- placement/yaml

Tags

- cluster.open-cluster-management.io

Create a Placement

```
POST /cluster.open-cluster-management.io/v1beta1/namespaces/{namespace}/placements
```

Description

Create a Placement.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string
Body	body <i>required</i>	Parameters describing the placement to be created.	Placement

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Consumes

- `placement/yaml`

Tags

- `cluster.open-cluster-management.io`

Example HTTP request body

```
{
  "apiVersion": "cluster.open-cluster-management.io/v1beta1",
  "kind": "Placement",
  "metadata": {
    "name": "placement1",
    "namespace": "ns1"
  },
  "spec": {
    "predicates": [
      {
        "requiredClusterSelector": {
          "labelSelector": {
            "matchLabels": {
              "vendor": "OpenShift"
            }
          }
        }
      }
    ]
  }
}
```

```
},  
  "status" : { }  
}
```

Query a single Placement

```
GET /cluster.open-cluster-  
management.io/v1beta1/namespaces/{namespace}/placements/{placement_name}
```

Description

Query a single Placement for more details.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string
Path	placement_name <i>required</i>	Name of the Placement that you want to query.	string

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Tags

- cluster.open-cluster-management.io

Delete a Placement

```
DELETE /cluster.open-cluster-  
management.io/v1beta1/namespaces/{namespace}/placements/{placement_name}
```

Description

Delete a single Placement.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string
Path	placement_name <i>required</i>	Name of the Placement that you want to delete.	string

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Tags

- cluster.open-cluster-management.io

Definitions

Placement

Name	Description	Schema
apiVersion <i>required</i>	The versioned schema of the Placement.	string
kind <i>required</i>	String value that represents the REST resource.	string
metadata <i>required</i>	The meta data of the Placement.	object
spec <i>required</i>	The specification of the Placement.	spec

spec

Name	Description	Schema
ClusterSets <i>optional</i>	A subset of ManagedClusterSets from which the ManagedClusters are selected. If it is empty, ManagedClusters is selected from the ManagedClusterSets that are bound to the Placement namespace. Otherwise, ManagedClusters are selected from the intersection of this subset and the ManagedClusterSets are bound to the placement namespace.	string array
numberOfClusters <i>optional</i>	The desired number of ManagedClusters to be selected.	integer (int32)
predicates <i>optional</i>	A subset of cluster predicates to select ManagedClusters. The conditional logic is <i>OR</i> .	clusterPredicate array

clusterPredicate

Name	Description	Schema
requiredClusterSelector <i>optional</i>	A cluster selector to select ManagedClusters with a label and cluster claim.	clusterSelector

clusterSelector

Name	Description	Schema
labelSelector <i>optional</i>	A selector of ManagedClusters by label.	object
claimSelector <i>optional</i>	A selector of ManagedClusters by claim.	clusterClaimSelector

clusterClaimSelector

Name	Description	Schema
matchExpressions <i>optional</i>	A subset of the cluster claim selector requirements. The conditional logic is <i>AND</i> .	< object > array

PlacementDecisions API (v1beta1)

Deprecated: The documentation for the APIs is deprecated. Use the *API Explorer* in the console, or

an **oc** command, to view current and supported APIs.

Overview

This documentation is for the PlacementDecision resource for multicluster engine for Kubernetes. PlacementDecision resource has four possible requests: create, query, delete and update.

URI scheme

BasePath : /kubernetes/apis

Schemes : HTTPS

Tags

- cluster.open-cluster-management.io : Create and manage PlacementDecisions.

Paths

Query all PlacementDecisions

```
GET /cluster.open-cluster-  
management.io/v1beta1/namespaces/{namespace}/placementdecisions
```

Description

Query your PlacementDecisions for more details.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Consumes

- placementdecision/yaml

Tags

- cluster.open-cluster-management.io

Create a PlacementDecision

```
POST /cluster.open-cluster-management.io/v1beta1/namespaces/{namespace}/placementdecisions
```

Description

Create a PlacementDecision.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string
Body	body <i>required</i>	Parameters describing the PlacementDecision to be created.	PlacementDecision

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Consumes

- [placementdecision/yaml](#)

Tags

- cluster.open-cluster-management.io

Example HTTP request body

```
{
  "apiVersion" : "cluster.open-cluster-management.io/v1beta1",
  "kind" : "PlacementDecision",
  "metadata" : {
```

```
"labels" : {  
  "cluster.open-cluster-management.io/placement" : "placement1"  
},  
"name" : "placement1-decision1",  
"namespace": "ns1"  
},  
"status" : { }  
}
```

Query a single PlacementDecision

```
GET /cluster.open-cluster-  
management.io/v1beta1/namespaces/{namespace}/placementdecisions/{placementdecision_name}
```

Description

Query a single PlacementDecision for more details.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string
Path	placementdecision_name <i>required</i>	Name of the PlacementDecision that you want to query.	string

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Tags

- cluster.open-cluster-management.io

Delete a PlacementDecision

```
DELETE /cluster.open-cluster-  
management.io/v1beta1/namespaces/{namespace}/placementdecisions/{placementdecision_name}
```

Description

Delete a single PlacementDecision.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string
Path	placementdecision_name <i>required</i>	Name of the PlacementDecision that you want to delete.	string

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Tags

- cluster.open-cluster-management.io

Definitions

PlacementDecision

Name	Description	Schema
apiVersion <i>required</i>	The versioned schema of PlacementDecision.	string
kind <i>required</i>	String value that represents the REST resource.	string
metadata <i>required</i>	The meta data of PlacementDecision.	object

KlusterletConfig API (v1alpha1)

Deprecated: The documentation for the APIs is deprecated. Use the *API Explorer* in the console, or an **oc** command, to view current and supported APIs.

Overview

This documentation is for the **KlusterletConfig** resource for the multicluster engine for Kubernetes operator. The **KlusterletConfig** resource is used to configure the Klusterlet installation. The four possible requests are: create, query, delete, and update.

URI scheme

BasePath : /kubernetes/apis

Schemes : HTTPS

Tags

- klusterletconfigs.config.open-cluster-management.io : Create and manage klusterletconfigs

Paths

Query all KlusterletConfig

```
GET /config.open-cluster-management.io/v1alpha1/klusterletconfigs
```

Description

Query all **KlusterletConfig** for more details.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string

Responses

HTTP Code	Description	Schema
200	Success	KlusterletConfig yaml
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Consumes

- `klusterletconfig/yaml`

Tags

- `klusterletconfigs.config.open-cluster-management.io`

Create a KlusterletConfig

POST /config.open-cluster-management.io/v1alpha1/klusterletconfigs

Description

Create a `KlusterletConfig`.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string
Body	body <i>required</i>	Parameters describing the KlusterletConfig you want to create.	klusterletconfig

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Consumes

- `klusterletconfig/yaml`

Tags

- `klusterletconfigs.config.open-cluster-management.io`

Example HTTP request body

```
{
```

```

"apiVersion": "apiextensions.k8s.io/v1",
"kind": "CustomResourceDefinition",
"metadata": {
  "annotations": {
    "controller-gen.kubebuilder.io/version": "v0.7.0"
  },
  "creationTimestamp": null,
  "name": "klusterletconfigs.config.open-cluster-management.io"
},
"spec": {
  "group": "config.open-cluster-management.io",
  "names": {
    "kind": "KlusterletConfig",
    "listKind": "KlusterletConfigList",
    "plural": "klusterletconfigs",
    "singular": "klusterletconfig"
  },
  "preserveUnknownFields": false,
  "scope": "Cluster",
  "versions": [
    {
      "name": "v1alpha1",
      "schema": {
        "openAPIV3Schema": {
          "description": "KlusterletConfig contains the configuration of a
klusterlet including the upgrade strategy, config overrides, proxy configurations
etc.",
          "properties": {
            "apiVersion": {
              "description": "APIVersion defines the versioned schema of this
representation of an object. Servers should convert recognized schemas to the latest
internal value, and may reject unrecognized values. More info:
https://git.k8s.io/community/contributors/devel/sig-architecture/api-
conventions.md#resources",
              "type": "string"
            },
            "kind": {
              "description": "Kind is a string value representing the REST resource
this object represents. Servers may infer this from the endpoint the client submits
requests to. Cannot be updated. In CamelCase. More info:
https://git.k8s.io/community/contributors/devel/sig-architecture/api-
conventions.md#types-kinds",
              "type": "string"
            },
            "metadata": {
              "type": "object"
            },
            "spec": {
              "description": "Spec defines the desired state of KlusterletConfig",
              "properties": {
                "appliedManifestWorkEvictionGracePeriod": {

```

```

        "description": "AppliedManifestWorkEvictionGracePeriod is the
eviction grace period the work agent will wait before evicting the
AppliedManifestWorks, whose corresponding ManifestWorks are missing on the hub
cluster, from the managed cluster. If not present, the default value of the work agent
will be used. If its value is set to \"INFINITE\", it means the AppliedManifestWorks
will never been evicted from the managed cluster.",
        "pattern": "^[0-9]+(s|m|h))+$|^INFINITE$",
        "type": "string"
    },
    "bootstrapKubeConfigs": {
        "description": "BootstrapKubeConfigSecrets is the list of secrets
that reflects the Klusterlet.Spec.RegistrationConfiguration.BootstrapKubeConfigs.",
        "properties": {
            "localSecretsConfig": {
                "description": "LocalSecretsConfig include a list of secrets
that contains the kubeconfigs for ordered bootstrap kubeconifigs. The secrets must be
in the same namespace where the agent controller runs.",
                "properties": {
                    "hubConnectionTimeoutSeconds": {
                        "default": 600,
                        "description": "HubConnectionTimeoutSeconds is used to set
the timeout of connecting to the hub cluster. When agent loses the connection to the
hub over the timeout seconds, the agent do a rebootstrap. By default is 10 mins.",
                        "format": "int32",
                        "minimum": 180,
                        "type": "integer"
                    },
                },
            },
            "kubeConfigSecrets": {
                "description": "KubeConfigSecrets is a list of secret
names. The secrets are in the same namespace where the agent controller runs.",
                "items": {
                    "properties": {
                        "name": {
                            "description": "Name is the name of the secret.",
                            "type": "string"
                        },
                    },
                },
                "type": "object"
            },
        },
        "type": "array"
    },
    "type": {
        "default": "None",
        "description": "Type specifies the type of priority bootstrap
kubeconfigs. By default, it is set to None, representing no priority bootstrap
kubeconfigs are set.",
        "enum": [
            "None",

```

```

        "LocalSecrets"
      ],
      "type": "string"
    }
  },
  "type": "object"
},
"hubKubeAPIServerCABundle": {
  "description": "HubKubeAPIServerCABundle is the CA bundle to
verify the server certificate of the hub kube API against. If not present, CA bundle
will be determined with the logic below: 1). Use the certificate of the named
certificate configured in APIServer/cluster if FQDN matches; 2). Otherwise use the CA
certificates from kube-root-ca.crt ConfigMap in the cluster namespace; \n Deprecated
and maintained for backward compatibility, use
HubKubeAPIServerConfig.ServerVarificationStrategy and
HubKubeAPIServerConfig.TrustedCABundles instead",
  "format": "byte",
  "type": "string"
},
"hubKubeAPIServerConfig": {
  "description": "HubKubeAPIServerConfig specifies the settings
required for connecting to the hub Kube API server. If this field is present, the
below deprecated fields will be ignored: - HubKubeAPIServerProxyConfig -
HubKubeAPIServerURL - HubKubeAPIServerCABundle",
  "properties": {
    "proxyURL": {
      "description": "ProxyURL is the URL to the proxy to be used
for all requests made by client If an HTTPS proxy server is configured, you may also
need to add the necessary CA certificates to TrustedCABundles.",
      "type": "string"
    },
    "serverVerificationStrategy": {
      "description": "ServerVerificationStrategy is the strategy
used for verifying the server certification; The value could be
\"UseSystemTruststore\", \"UseAutoDetectedCABundle\", \"UseCustomCABundles\", empty.
\n When this strategy is not set or value is empty; if there is only one
klusterletConfig configured for a cluster, the strategy is eaval to
\"UseAutoDetectedCABundle\", if there are more than one klusterletConfigs, the empty
strategy will be overridden by other non-empty strategies.",
      "enum": [
        "UseSystemTruststore",
        "UseAutoDetectedCABundle",
        "UseCustomCABundles"
      ],
      "type": "string"
    },
    "trustedCABundles": {
      "description": "TrustedCABundles refers to a collection of
user-provided CA bundles used for verifying the server certificate of the hub
Kubernetes API If the ServerVerificationStrategy is set to \"UseSystemTruststore\",
this field will be ignored. Otherwise, the CA certificates from the configured bundles

```

```

will be appended to the klusterlet CA bundle.",
    "items": {
      "description": "CABundle is a user-provided CA bundle",
      "properties": {
        "caBundle": {
          "description": "CABundle refers to a ConfigMap with
label \"import.open-cluster-management.io/ca-bundle\" containing the user-provided CA
bundle The key of the CA data could be \"ca-bundle.crt\", \"ca.crt\", or
\"tls.crt\".",
          "properties": {
            "name": {
              "description": "name is the metadata.name of the
referenced config map",
              "type": "string"
            },
            "namespace": {
              "description": "name is the metadata.namespace of
the referenced config map",
              "type": "string"
            }
          },
          "required": [
            "name",
            "namespace"
          ],
          "type": "object"
        },
        "name": {
          "description": "Name is the identifier used to reference
the CA bundle; Do not use \"auto-detected\" as the name since it is the reserved name
for the auto-detected CA bundle.",
          "type": "string"
        }
      },
      "required": [
        "caBundle",
        "name"
      ],
      "type": "object"
    },
    "type": "array",
    "x-kubernetes-list-map-keys": [
      "name"
    ],
    "x-kubernetes-list-type": "map"
  },
  "url": {
    "description": "URL is the endpoint of the hub Kube API
server. If not present, the .status.apiServerURL of Infrastructure/cluster will be
used as the default value. e.g. `oc get infrastructure cluster -o
jsonpath='{.status.apiServerURL}'`",

```

```

        "type": "string"
      },
    },
    "type": "object"
  },
  "hubKubeAPIServerProxyConfig": {
    "description": "HubKubeAPIServerProxyConfig holds proxy settings
for connections between klusterlet/add-on agents on the managed cluster and the kube-
apiserver on the hub cluster. Empty means no proxy settings is available. \n
Deprecated and maintained for backward compatibility, use
HubKubeAPIServerConfig.ProxyURL instead",
    "properties": {
      "caBundle": {
        "description": "CABundle is a CA certificate bundle to verify
the proxy server. It will be ignored if only HTTPProxy is set; And it is required when
HTTPSProxy is set and self signed CA certificate is used by the proxy server.",
        "format": "byte",
        "type": "string"
      },
    },
    "httpProxy": {
      "description": "HTTPProxy is the URL of the proxy for HTTP
requests",
      "type": "string"
    },
    "httpsProxy": {
      "description": "HTTPSProxy is the URL of the proxy for HTTPS
requests HTTPSProxy will be chosen if both HTTPProxy and HTTPSProxy are set.",
      "type": "string"
    }
  },
  "type": "object"
},
"hubKubeAPIServerURL": {
  "description": "HubKubeAPIServerURL is the URL of the hub Kube API
server. If not present, the .status.apiServerURL of Infrastructure/cluster will be
used as the default value. e.g. `oc get infrastructure cluster -o
jsonpath='{.status.apiServerURL}'` \n Deprecated and maintained for backward
compatibility, use HubKubeAPIServerConfig.URL instead",
  "type": "string"
},
"installMode": {
  "description": "InstallMode is the mode to install the
klusterlet",
  "properties": {
    "noOperator": {
      "description": "NoOperator is the setting of klusterlet
installation when install type is noOperator.",
      "properties": {
        "postfix": {
          "description": "Postfix is the postfix of the klusterlet
name. The name of the klusterlet is \"klusterlet\" if it is not set, and \"klusterlet-

```

```

{Postfix}\". The install namespace is \"open-cluster-management-agent\" if it is not
set, and \"open-cluster-management-{Postfix}\".",
    "maxLength": 33,
    "pattern": "^[-a-z0-9]*[a-z0-9]$",
    "type": "string"
  },
  "type": "object"
},
"type": {
  "default": "default",
  "description": "InstallModeType is the type of install mode.",
  "enum": [
    "default",
    "noOperator"
  ],
  "type": "string"
}
},
"type": "object"
},
"nodePlacement": {
  "description": "NodePlacement enables explicit control over the
scheduling of the agent components. If the placement is nil, the placement is not
specified, it will be omitted. If the placement is an empty object, the placement will
match all nodes and tolerate nothing.",
  "properties": {
    "nodeSelector": {
      "additionalProperties": {
        "type": "string"
      },
      "description": "NodeSelector defines which Nodes the Pods are
scheduled on. The default is an empty list.",
      "type": "object"
    },
    "tolerations": {
      "description": "Tolerations are attached by pods to tolerate
any taint that matches the triple <key,value,effect> using the matching operator
<operator>. The default is an empty list.",
      "items": {
        "description": "The pod this Toleration is attached to
tolerates any taint that matches the triple <key,value,effect> using the matching
operator <operator>.",
        "properties": {
          "effect": {
            "description": "Effect indicates the taint effect to
match. Empty means match all taint effects. When specified, allowed values are
NoSchedule, PreferNoSchedule and NoExecute.",
            "type": "string"
          },
          "key": {

```



```

        "description": "Key is the taint key that the toleration
applies to. Empty means match all taint keys. If the key is empty, operator must be
Exists; this combination means to match all values and all keys.",
        "type": "string"
    },
    "operator": {
        "description": "Operator represents a key's relationship
to the value. Valid operators are Exists and Equal. Defaults to Equal. Exists is
equivalent to wildcard for value, so that a pod can tolerate all taints of a
particular category.",
        "type": "string"
    },
    "tolerationSeconds": {
        "description": "TolerationSeconds represents the period
of time the toleration (which must be of effect NoExecute, otherwise this field is
ignored) tolerates the taint. By default, it is not set, which means tolerate the
taint forever (do not evict). Zero and negative values will be treated as 0 (evict
immediately) by the system.",
        "format": "int64",
        "type": "integer"
    },
    "value": {
        "description": "Value is the taint value the toleration
matches to. If the operator is Exists, the value should be empty, otherwise just a
regular string.",
        "type": "string"
    }
},
"type": "object"
},
"type": "array"
}
},
"type": "object"
},
"pullSecret": {
    "description": "PullSecret is the name of image pull secret.",
    "properties": {
        "apiVersion": {
            "description": "API version of the referent.",
            "type": "string"
        },
        "fieldPath": {
            "description": "If referring to a piece of an object instead
of an entire object, this string should contain a valid JSON/Go field access
statement, such as desiredState.manifest.containers[2]. For example, if the object
reference is to a container within a pod, this would take on a value like:
\"spec.containers{name}\" (where \"name\" refers to the name of the container that
triggered the event) or if no container name is specified \"spec.containers[2]\"
(container with index 2 in this pod). This syntax is chosen only to have some well-
defined way of referencing a part of an object. TODO: this design is not final and

```

```

this field is subject to change in the future.",
    "type": "string"
  },
  "kind": {
    "description": "Kind of the referent. More info:
https://git.k8s.io/community/contributors/devel/sig-architecture/api-
conventions.md#types-kinds",
    "type": "string"
  },
  "name": {
    "description": "Name of the referent. More info:
https://kubernetes.io/docs/concepts/overview/working-with-objects/names/#names",
    "type": "string"
  },
  "namespace": {
    "description": "Namespace of the referent. More info:
https://kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/",
    "type": "string"
  },
  "resourceVersion": {
    "description": "Specific resourceVersion to which this
reference is made, if any. More info:
https://git.k8s.io/community/contributors/devel/sig-architecture/api-
conventions.md#concurrency-control-and-consistency",
    "type": "string"
  },
  "uid": {
    "description": "UID of the referent. More info:
https://kubernetes.io/docs/concepts/overview/working-with-objects/names/#uids",
    "type": "string"
  }
},
"type": "object",
"x-kubernetes-map-type": "atomic"
},
"registries": {
  "description": "Registries includes the mirror and source
registries. The source registry will be replaced by the Mirror.",
  "items": {
    "properties": {
      "mirror": {
        "description": "Mirror is the mirrored registry of the
Source. Will be ignored if Mirror is empty.",
        "type": "string"
      },
      "source": {
        "description": "Source is the source registry. All image
registries will be replaced by Mirror if Source is empty.",
        "type": "string"
      }
    }
  }
},

```

```

        "required": [
            "mirror"
        ],
        "type": "object"
    },
    "type": "array"
}
},
"type": "object"
},
"status": {
    "description": "Status defines the observed state of
KubernetesCluster",
    "type": "object"
}
},
"type": "object"
}
},
"served": true,
"storage": true,
"subresources": {
    "status": {}
}
}
]
},
"status": {
    "acceptedNames": {
        "kind": "",
        "plural": ""
    },
    "conditions": [],
    "storedVersions": []
}
}

```

Query a single `klusterletconfig`

```
GET /config.open-cluster-
management.io/v1alpha1/klusterletconfigs/{klusterletconfig_name}
```

Description

Query a single `KlusterletConfig` for more details.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string
Path	klusterletconfig_name <i>required</i>	Name of the klusterletconfig that you want to query.	string

Responses

HTTP Code	Description	Schema
200	Success	KlusterletConfig yaml
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Tags

- klusterletconfigs.config.open-cluster-management.io

Delete a klusterletconfig

```
DELETE /config.open-cluster-management.io/v1alpha1/klusterletconfigs/{klusterletconfig_name}
```

Description

Delete a single **KlusterletConfig**.

Parameters

Type	Name	Description	Schema
Header	COOKIE <i>required</i>	Authorization: Bearer {ACCESS_TOKEN} ; ACCESS_TOKEN is the user access token.	string
Path	klusterletconfig_name <i>required</i>	Name of the klusterletconfig that you want to delete.	string

Responses

HTTP Code	Description	Schema
200	Success	No Content
403	Access forbidden	No Content
404	Resource not found	No Content
500	Internal service error	No Content
503	Service unavailable	No Content

Tags

- `klusterletconfig.authentication.open-cluster-management.io`

Definitions

klusterletconfig

Name	Description	Schema
apiVersion <i>required</i>	The versioned schema of the klusterletconfig.	string
kind <i>required</i>	String value that represents the REST resource.	string
metadata <i>required</i>	The meta data of the <code>KlusterletConfig</code> .	object
spec <i>required</i>	The specification of the <code>KlusterletConfig</code> .	

Troubleshooting

Before using the Troubleshooting guide, you can run the `oc adm must-gather` command to gather details, logs, and take steps in debugging issues. For more details, see [Running the must-gather command to troubleshoot](#).

Additionally, check your role-based access. See [multicluster engine operator Role-based access control](#) for details.

Documented troubleshooting

View the list of troubleshooting topics for the multicluster engine operator:

Installation:

To view the main documentation for the installing tasks, see [Installing and upgrading multicluster engine operator](#).

- [Troubleshooting installation status stuck in installing or pending](#)

- [Troubleshooting reinstallation failure](#)

Cluster management:

To view the main documentation about managing your clusters, see [Cluster lifecycle introduction](#).

- [Troubleshooting adding day-two nodes to an existing cluster fails with pending user action](#)
- [Troubleshooting an offline cluster](#)
- [Troubleshooting a managed cluster import failure](#)
- [Reimporting cluster fails with unknown authority error](#)
- [Troubleshooting cluster with pending import status](#)
- [Troubleshooting imported clusters offline after certificate change](#)
- [Troubleshooting cluster status changing from offline to available](#)
- [Troubleshooting cluster creation on VMware vSphere](#)
- [Troubleshooting cluster in console with pending or failed status](#)
- [Troubleshooting Klusterlet with degraded conditions](#)
- [Troubleshooting namespace that remains after deleting a cluster](#)
- [Auto-import-secret-exists error when importing a cluster](#)
- [Troubleshooting missing PlacementDecision after creating Placement](#)
- [Troubleshooting a discovery failure of bare metal hosts on Dell hardware](#)
- [Troubleshooting Minimal ISO boot failures](#)
- [Troubleshooting managed clusters *Unknown* on Red Hat OpenShift Service on AWS \(classic\) with hosted control planes cluster](#)
- [Troubleshooting an attempt to upgrade managed cluster with missing OpenShift Container Platform version](#)
- [Troubleshooting certificate authority update error on Hive and Assisted Installer clusters](#)
- [Troubleshooting: Missing `open-cluster-management-global-set` namespace](#)
- [Troubleshooting issues when creating Microsoft Azure Red Hat OpenShift hosted control plane clusters by using Cluster API Provider for Azure](#)
- [Troubleshooting Microsoft Azure Red Hat OpenShift hosted control plane cluster with Cluster API Provider for Azure becomes stuck during provisioning](#)

Running the must-gather command to troubleshoot

To get started with troubleshooting, learn about the troubleshooting scenarios for users to run the `must-gather` command to debug the issues, then see the procedures to start using the command.

Required access: Cluster administrator

Must-gather scenarios

- **Scenario one:** Use the *Documented troubleshooting* section to see if a solution to your problem is documented. The guide is organized by the major functions of the product.

With this scenario, you check the guide to see if your solution is in the documentation.

- **Scenario two:** If your problem is not documented with steps to resolve, run the `must-gather` command and use the output to debug the issue.
- **Scenario three:** If you cannot debug the issue using your output from the `must-gather` command, then share your output with Red Hat Support.

Must-gather procedure

See the following procedure to start using the `must-gather` command:

1. Learn about the `must-gather` command and install the prerequisites that you need at [Gathering data about your cluster](#) in the OpenShift Container Platform documentation.
2. Log in to your cluster. For the usual use-case, you should run the `must-gather` while you are logged into your *engine* cluster.

Note: If you want to check your managed clusters, find the `gather-managed.log` file that is located in the `cluster-scoped-resources` directory:

```
<your-directory>/cluster-scoped-resources/gather-managed.log
```

Check for managed clusters that are not set `True` for the JOINED and AVAILABLE column. You can run the `must-gather` command on those clusters that are not connected with `True` status.

3. Add the multicluster engine for Kubernetes image that is used for gathering data and the directory. Run the following command:

```
oc adm must-gather --image=registry.redhat.io/multicluster-engine/must-gather-rhel9:v2.11 --dest-dir=<directory>
```

1. Go to your specified directory to see your output, which is organized in the following levels:
 - Two peer levels: `cluster-scoped-resources` and `namespace` resources.
 - Sub-level for each: API group for the custom resource definitions for both cluster-scope and namespace-scoped resources.
 - Next level for each: YAML file sorted by `kind`.

Must-gather in a disconnected environment

Complete the following steps to run the `must-gather` command in a disconnected environment:

1. In a disconnected environment, mirror the Red Hat operator catalog images into their mirror registry. For more information, see [Install on disconnected networks](#).

2. Run the following command to extract logs, which reference the image from their mirror registry. Replace `sha256` with the current image:

```
REGISTRY=registry.example.com:5000
IMAGE=$REGISTRY/multicluster-engine/must-gather-
rhel9@sha256:ff9f37eb400dc1f7d07a9b6f2da9064992934b69847d17f59e385783c071b9d8>

oc adm must-gather --image=$IMAGE --dest-dir=./data
```

You can open a Jira bug for the product team [here](#).

Troubleshooting: Adding day-two nodes to an existing cluster fails with pending user action

Adding a node, or scaling out, to your existing cluster that is created by the multicluster engine for Kubernetes operator with Zero Touch Provisioning or Host inventory create methods fails during installation. The installation process works correctly during the Discovery phase, but fails on the installation phase.

The configuration of the network is failing. From the hub cluster in the integrated console, you see a **Pending** user action. In the description, you can see it failing on the rebooting step.

The error message about failing is not very accurate, since the agent that is running in the installing host cannot report information.

Symptom: Installation for day two workers fails

After the Discover phase, the host reboots to continue the installation, but it cannot configure the network. Check for the following symptoms and messages:

- From the hub cluster in the integrated console, check for **Pending** user action on the adding node, with the **Rebooting** indicator:

```
This host is pending user action. Host timed out when pulling ignition. Check the
host console... Rebooting
```

- From the Red Hat OpenShift Container Platform configuration managed cluster, check the **MachineConfigs** of the existing cluster. Check if any of the **MachineConfigs** create any file on the following directories:
 - `/sysroot/etc/NetworkManager/system-connections/`
 - `/sysroot/etc/sysconfig/network-scripts/`
- From the terminal of the installing host, check the failing host for the following messages. You can use `journalctl` to see the log messages:

```
info: networking config is defined in the real root
```


info: will not attempt to propagate initramfs networking

If you get the last message in the log, the networking configuration is not propagated because it already found an existing network configuration on the folders previously listed in the *Symptom*.

Resolving the problem: Recreate the node merging network configuration

Perform the following task to use a proper network configuration during the installation:

1. Delete the node from your hub cluster.
2. Repeat your previous process to install the node in the same way.
3. Create the `BareMetalHost` object of the node with the following annotation:

```
"bmac.agent-install.openshift.io/installer-args": "[\"--append-karg\",  
\"coreos.force_persist_ip\"]"
```

The node starts the installation. After the Discovery phase, the node merges the network configuration between the changes on the existing cluster and the initial configuration.

Troubleshooting deletion failure of a hosted control plane cluster on the Agent platform

When you destroy a hosted control plane cluster on the Agent platform, all the back-end resources are normally deleted. If the machine resources are not deleted properly, a cluster deletion fails. In that case, you must manually remove the remaining machine resources.

Symptom: An error occurs when destroying a hosted control plane cluster

After you attempt to destroy the hosted control plane cluster on the Agent platform, the `hcp destroy` command fails with the following error:

+

```
2024-02-22T09:56:19-05:00    ERROR    HostedCluster deletion failed    {"namespace":  
"clusters", "name": "hosted-0", "error": "context deadline exceeded"}  
2024-02-22T09:56:19-05:00    ERROR    Failed to destroy cluster    {"error": "context  
deadline exceeded"}
```

Resolving the problem: Remove the remaining machine resources manually

Complete the following steps to destroy a hosted control plane cluster successfully on the Agent platform:

1. Run the following command to see the list of remaining machine resources by replacing `<hosted_cluster_namespace>` with the name of hosted cluster namespace:

```
oc get machine -n <hosted_cluster_namespace>
```

See the following example output:

NAMESPACE	NAME	CLUSTER	NODENAME	PROVIDERID	PHASE
AGE	VERSION				
clusters-hosted-0	hosted-0-9gg8b	hosted-0-nhdbp			
Deleting	10h	<4.x.0>-rc.8			

2. Run the following command to remove the `machine.cluster.x-k8s.io` finalizer attached to machine resources:

```
oc edit machines -n <hosted_cluster_namespace>
```

3. Run the following command to verify you receive the `No resources found` message on your terminal:

```
oc get agentmachine -n <hosted_cluster_namespace>
```

4. Run the following command to destroy a hosted control plane cluster on the Agent platform:

```
hcp destroy cluster agent --name <cluster_name>
```

Replace `<cluster_name>` with the name of your cluster.

Troubleshooting installation status stuck in installing or pending

When installing the multicluster engine operator, the `MultiClusterEngine` remains in `Installing` phase, or multiple pods maintain a `Pending` status.

Symptom: Stuck in Pending status

More than ten minutes passed since you installed `MultiClusterEngine` and one or more components from the `status.components` field of the `MultiClusterEngine` resource report `ProgressDeadlineExceeded`. Resource constraints on the cluster might be the issue.

Check the pods in the namespace where `MultiClusterEngine` was installed. You might see `Pending` with a status similar to the following:

```
reason: Unschedulable
message: '0/6 nodes are available: 3 Insufficient cpu, 3 node(s) had taint {node-
role.kubernetes.io/master:
    }, that the pod didn't tolerate.'
```

In this case, the worker nodes resources are not sufficient in the cluster to run the product.

Resolving the problem: Adjust worker node sizing

If you have this problem, then your cluster needs to be updated with either larger or more worker nodes. See [Sizing your cluster](#) for guidelines on sizing your cluster.

Troubleshooting reinstallation failure

When reinstalling multicluster engine operator, the pods do not start.

Symptom: Reinstallation failure

If your pods do not start after you install the multicluster engine operator, it is often because items from a previous installation of multicluster engine operator were not removed correctly when it was uninstalled.

In this case, the pods do not start after completing the installation process.

Resolving the problem: Reinstallation failure

If you have this problem, complete the following steps:

1. Run the uninstallation process to remove the current components by following the steps in [Uninstalling](#).
2. Ensure that your Red Hat OpenShift Container Platform CLI is configured to run **oc** commands. See [Getting started with the OpenShift CLI](#) in the OpenShift Container Platform documentation for more information about how to configure the **oc** commands.
3. Copy the following script into a file, replacing the **<namespace>** value in the script with the name of the namespace where you previously installed multicluster engine operator.

Important: Ensure that you specify the correct namespace because the namespace is also cleaned out and deleted when you run the script.

```
MCE_NAMESPACE=<namespace>
oc delete multiclusterengine --all
oc delete apiservice v1.admission.cluster.open-cluster-management.io
v1.admission.work.open-cluster-management.io
oc delete crd discoveredclusters.discovery.open-cluster-management.io
discoveryconfigs.discovery.open-cluster-management.io
oc delete mutatingwebhookconfiguration ocm-mutating-webhook
managedclustermutators.admission.cluster.open-cluster-management.io
oc delete validatingwebhookconfiguration ocm-validating-webhook
oc delete ns $MCE_NAMESPACE
```

4. Run the script. When you receive a message that no resources were found, then you can proceed with the installation.
5. Run the installation. See [Installing while connected online](#).

Troubleshooting an offline cluster

There are a few common causes for a cluster showing an offline status.

Symptom: Cluster status is offline

After you complete the procedure for creating a cluster, you cannot access it from the Red Hat Advanced Cluster Management console, and it shows a status of **offline**.

Resolving the problem: Cluster status is offline

1. Determine if the managed cluster is available. You can check this in the *Clusters* area of the Red Hat Advanced Cluster Management console.

If it is not available, try restarting the managed cluster.

2. If the managed cluster status is still offline, complete the following steps:
 - a. Run the `oc get managedcluster <cluster_name> -o yaml` command on the hub cluster. Replace `<cluster_name>` with the name of your cluster.
 - b. Find the `status.conditions` section.
 - c. Check the messages for `type: ManagedClusterConditionAvailable` and resolve any problems.

Troubleshooting a managed cluster import failure

If your cluster import fails, there are a few steps that you can take to determine why the cluster import failed.

Symptom: Imported cluster not available

After you complete the procedure for importing a cluster, you cannot access it from the console.

Resolving the problem: Imported cluster not available

There can be a few reasons why an imported cluster is not available after an attempt to import it. If the cluster import fails, complete the following steps, until you find the reason for the failed import:

1. On the hub cluster, run the following command to ensure that the import controller is running.

```
kubectl -n multicloud-engine get pods -l app=managedcluster-import-controller-v2
```

You should see two pods that are running. If either of the pods is not running, run the following command to view the log to determine the reason:

```
kubectl -n multicloud-engine logs -l app=managedcluster-import-controller-v2  
--tail=-1
```

2. On the hub cluster, run the following command to determine if the managed cluster import

secret was generated successfully by the import controller:

```
kubectl -n <managed_cluster_name> get secrets <managed_cluster_name>-import
```

If the import secret does not exist, run the following command to view the log entries for the import controller and determine why it was not created:

```
kubectl -n multicluster-engine logs -l app=managedcluster-import-controller-v2  
--tail=-1 | grep importconfig-controller
```

3. On the hub cluster, if your managed cluster is the **local-cluster**, provisioned by Hive, or has an auto-import secret, run the following command to check the import status of the managed cluster.

```
kubectl get managedcluster <managed_cluster_name> -o=jsonpath='{range  
.status.conditions[*]}{.type}{"\t"}{.status}{"\t"}{.message}{"\n"}{end}'} | grep  
ManagedClusterImportSucceeded
```

If the condition **ManagedClusterImportSucceeded** is not **true**, the result of the command indicates the reason for the failure.

4. Check the Klusterlet status of the managed cluster for a degraded condition. See [Troubleshooting Klusterlet with degraded conditions](#) to find the reason that the Klusterlet is degraded.

Reimporting cluster fails with unknown authority error

If you experience a problem when reimporting a managed cluster to your multicluster engine operator hub cluster, follow the procedure to troubleshoot the problem.

Symptom: Reimporting cluster fails with unknown authority error

After you provision an OpenShift Container Platform cluster with multicluster engine operator, reimporting the cluster might fail with a **x509: certificate signed by unknown authority** error when you change or add API server certificates to your OpenShift Container Platform cluster.

Identifying the problem: Reimporting cluster fails with unknown authority error

After failing to reimport your managed cluster, run the following command to get the import controller log on your multicluster engine operator hub cluster:

```
kubectl -n multicluster-engine logs -l app=managedcluster-import-controller-v2 -f
```

If the following error log appears, your managed cluster API server certificates might have changed:

```
ERROR Reconciler error {"controller": "clusterdeployment-controller", "object": {"name": "awscluster1", "namespace": "awscluster1"}, "namespace": "awscluster1", "name": "awscluster1", "reconcileID": "a2cccf24-2547-4e26-95fb-f258a6710d80", "error": "Get \\"https://api.awscluster1.dev04.red-chesterfield.com:6443/api?timeout=32s\\": x509: certificate signed by unknown authority"}
```

To determine if your managed cluster API server certificates have changed, complete the following steps:

1. Run the following command to specify your managed cluster name by replacing **your-managed-cluster-name** with the name of your managed cluster:

```
cluster_name=<your-managed-cluster-name>
```

2. Get your managed cluster **kubeconfig** secret name by running the following command:

```
kubeconfig_secret_name=$(oc -n ${cluster_name} get clusterdeployments  
${cluster_name} -ojsonpath='{.spec.clusterMetadata.adminKubeconfigSecretRef.name}')
```

3. Export **kubeconfig** to a new file by running the following commands:

```
oc -n ${cluster_name} get secret ${kubeconfig_secret_name}  
-ojsonpath='{.data.kubeconfig}' | base64 -d > kubeconfig.old
```

```
export KUBECONFIG=kubeconfig.old
```

4. Get the namespace from your managed cluster with **kubeconfig** by running the following command:

```
oc get ns
```

If you receive an error that resembles the following message, your cluster API server certificates have been changed and your **kubeconfig** file is invalid.

Unable to connect to the server: x509: certificate signed by unknown authority

Resolving the problem: Reimporting cluster fails with unknown authority error

The managed cluster administrator must create a new valid **kubeconfig** file for your managed cluster.

After creating a new **kubeconfig**, complete the following steps to update the new **kubeconfig** for your managed cluster:

1. Run the following commands to set your **kubeconfig** file path and cluster name. Replace **<path_to_kubeconfig>** with the path to your new **kubeconfig** file. Replace **<managed_cluster_name>**

with the name of your managed cluster:

```
cluster_name=<managed_cluster_name>  
kubeconfig_file=<path_to_kubeconfig>
```

2. Run the following command to encode your new **kubeconfig**:

```
kubeconfig=$(cat ${kubeconfig_file} | base64 -w0)
```

Note: On macOS, run the following command instead:

```
kubeconfig=$(cat ${kubeconfig_file} | base64)
```

3. Run the following command to define the **kubeconfig** json patch:

```
kubeconfig_patch="[{\\"op\\":\\"replace\\", \\"path\\":\\"/data/kubeconfig\\",  
  \\"value\\":\\"${kubeconfig}\\", \\"op\\":\\"replace\\", \\"path\\":\\"/data/raw-  
  kubeconfig\\", \\"value\\":\\"${kubeconfig}\\"}]"
```

4. Retrieve your administrator **kubeconfig** secret name from your managed cluster by running the following command:

```
kubeconfig_secret_name=$(oc -n ${cluster_name} get clusterdeployments  
  ${cluster_name} -ojsonpath='{.spec.clusterMetadata.adminKubeconfigSecretRef.name}')
```

5. Patch your administrator **kubeconfig** secret with your new **kubeconfig** by running the following command:

```
oc -n ${cluster_name} patch secrets ${kubeconfig_secret_name} --type='json'  
  -p="${kubeconfig_patch}"
```

Troubleshooting cluster with pending import status

If you receive *Pending import* continually on the console of your cluster, follow the procedure to troubleshoot the problem.

Symptom: Cluster with pending import status

After importing a cluster by using the Red Hat Advanced Cluster Management console, the cluster appears in the console with a status of *Pending import*.

Identifying the problem: Cluster with pending import status

1. Run the following command on the managed cluster to view the Kubernetes pod names that are having the issue:

```
kubectl get pod -n open-cluster-management-agent | grep klusterlet-agent
```

2. Run the following command on the managed cluster to find the log entry for the error:

```
kubectl logs <klusterlet_agent_pod> -n open-cluster-management-agent
```

Replace *registration_agent_pod* with the pod name that you identified in step 1.

3. Search the returned results for text that indicates there was a networking connectivity problem. Example includes: **no such host**.

Resolving the problem: Cluster with pending import status

1. Retrieve the port number that is having the problem by entering the following command on the hub cluster:

```
oc get infrastructure cluster -o yaml | grep apiServerURL
```

2. Ensure that the hostname from the managed cluster can be resolved, and that outbound connectivity to the host and port is occurring.

If the communication cannot be established by the managed cluster, the cluster import is not complete. The cluster status for the managed cluster is *Pending import*.

Troubleshooting imported clusters offline after certificate change

Installing a custom **apiserver** certificate is supported, but one or more clusters that were imported before you changed the certificate information can have an **offline** status.

Symptom: Clusters offline after certificate change

After you complete the procedure for updating a certificate secret, one or more of your clusters that were online are now displaying an **offline** status in the console.

Identifying the problem: Clusters offline after certificate change

After updating the information for a custom API server certificate, clusters that were imported and running before the new certificate are now in an **offline** state.

The errors that indicate that the certificate is the problem are found in the logs for the pods in the **open-cluster-management-agent** namespace of the offline managed cluster. The following examples are similar to the errors that are displayed in the logs:

See the following `klusterlet-agent` log:

```
E0917 02:58:26.315984      1 reflector.go:127] k8s.io/client-
go@v0.19.0/tools/cache/reflector.go:156: Failed to watch
*v1beta1.CertificateSigningRequest: Get "https://api.aaa-
ocp.dev02.location.com:6443/apis/cluster.management.io/v1/managedclusters?allowWatchBo
okmarks=true&fieldSelector=metadata.name%3Dtest-
1&resourceVersion=607408&timeout=9m33s&timeoutSeconds=573&watch=true": x509:
certificate signed by unknown authority
E0917 02:58:26.598343      1 reflector.go:127] k8s.io/client-
go@v0.19.0/tools/cache/reflector.go:156: Failed to watch *v1.ManagedCluster: Get
"https://api.aaa-
ocp.dev02.location.com:6443/apis/cluster.management.io/v1/managedclusters?allowWatchBo
okmarks=true&fieldSelector=metadata.name%3Dtest-
1&resourceVersion=607408&timeout=9m33s&timeoutSeconds=573&watch=true": x509:
certificate signed by unknown authority
E0917 02:58:27.613963      1 reflector.go:127] k8s.io/client-
go@v0.19.0/tools/cache/reflector.go:156: Failed to watch *v1.ManagedCluster: failed to
list *v1.ManagedCluster: Get "https://api.aaa-
ocp.dev02.location.com:6443/apis/cluster.management.io/v1/managedclusters?allowWatchBo
okmarks=true&fieldSelector=metadata.name%3Dtest-
1&resourceVersion=607408&timeout=9m33s&timeoutSeconds=573&watch=true": x509:
certificate signed by unknown authority
E0917 03:04:05.874759      1 manifestwork_controller.go:179] Reconcile work test-1-
klusterlet-addon-workmgr fails with err: Failed to update work status with err Get
"https://api.aaa-
ocp.dev02.location.com:6443/apis/cluster.management.io/v1/namespaces/test-
1/manifestworks/test-1-klusterlet-addon-workmgr": x509: certificate signed by unknown
authority
E0917 03:04:05.874887      1 base_controller.go:231] "ManifestWorkAgent" controller
failed to sync "test-1-klusterlet-addon-workmgr", err: Failed to update work status
with err Get "api.aaa-
ocp.dev02.location.com:6443/apis/cluster.management.io/v1/namespaces/test-
1/manifestworks/test-1-klusterlet-addon-workmgr": x509: certificate signed by unknown
authority
E0917 03:04:37.245859      1 reflector.go:127] k8s.io/client-
go@v0.19.0/tools/cache/reflector.go:156: Failed to watch *v1.ManifestWork: failed to
list *v1.ManifestWork: Get "api.aaa-
ocp.dev02.location.com:6443/apis/cluster.management.io/v1/namespaces/test-
1/manifestworks?resourceVersion=607424": x509: certificate signed by unknown authority
```

Resolving the problem: Clusters offline after certificate change

If you want to recover a managed cluster, complete the following steps to import the managed cluster again:

1. On the hub cluster, recreate the managed cluster import secret by running the following command:

```
oc delete secret -n <cluster_name> <cluster_name>-import
```

Replace **<cluster_name>** with the name of the managed cluster that you want to import.

2. On the hub cluster, expose the managed cluster import secret to a YAML file by running the following command:

```
oc get secret -n <cluster_name> <cluster_name>-import  
-ojsonpath='{.data.import\.yaml}' | base64 --decode > import.yaml
```

Replace **<cluster_name>** with the name of the managed cluster that you want to import.

3. On the managed cluster, apply the **import.yaml** file by running the following command:

```
oc apply -f import.yaml
```

Note: The previous steps do not detach the managed cluster from the hub cluster. The steps update the required manifests with current settings on the managed cluster, including the new certificate information.

Troubleshooting cluster status changing from offline to available

The status of the managed cluster alternates between **offline** and **available** without any manual change to the environment or cluster.

Symptom: Cluster status changing from offline to available

When the network that connects the managed cluster to the hub cluster is unstable, the status of the managed cluster that is reported by the hub cluster cycles between **offline** and **available**.

Resolving the problem: Cluster status changing from offline to available

To attempt to resolve this issue, complete the following steps:

1. Edit your **ManagedCluster** specification on the hub cluster by entering the following command:

```
oc edit managedcluster <cluster-name>
```

Replace *cluster-name* with the name of your managed cluster.

2. Increase the value of **leaseDurationSeconds** in your **ManagedCluster** specification. The default value is 60 seconds, and the offline grace period for the managed cluster is 5 minutes, which is 5 times the **leaseDurationSeconds**. This might not be enough time to maintain the connection with the network issues. Specify a greater amount of time for the lease. For example, setting **leaseDurationSeconds** to 240 seconds provides a 20 minute offline grace period.

Troubleshooting cluster creation on VMware vSphere

If you experience a problem when creating a Red Hat OpenShift Container Platform cluster on VMware vSphere, see the following troubleshooting information to see if one of them addresses your problem.

Note: Sometimes when the cluster creation process fails on VMware vSphere, the link is not enabled for you to view the logs. If this happens, you can identify the problem by viewing the log of the `hive-controllers` pod. The `hive-controllers` log is in the `hive` namespace.

Managed cluster creation fails with certificate IP SAN error

Symptom: Managed cluster creation fails with certificate IP SAN error

After creating a new Red Hat OpenShift Container Platform cluster on VMware vSphere, the cluster fails with an error message that indicates a certificate IP SAN error.

Identifying the problem: Managed cluster creation fails with certificate IP SAN error

The deployment of the managed cluster fails and returns the following errors in the deployment log:

```
time="2020-08-07T15:27:55Z" level=error msg="Error: error setting up new vSphere SOAP client: Post https://147.1.1.1/sdk: x509: cannot validate certificate for xx.xx.xx.xx because it doesn't contain any IP SANs"
time="2020-08-07T15:27:55Z" level=error
```

Resolving the problem: Managed cluster creation fails with certificate IP SAN error

Use the VMware vCenter server fully-qualified host name instead of the IP address in the credential. You can also update the VMware vCenter CA certificate to contain the IP SAN.

Managed cluster creation fails with unknown certificate authority

Symptom: Managed cluster creation fails with unknown certificate authority

After creating a new Red Hat OpenShift Container Platform cluster on VMware vSphere, the cluster fails because the certificate is signed by an unknown authority.

Identifying the problem: Managed cluster creation fails with unknown certificate authority

The deployment of the managed cluster fails and returns the following errors in the deployment log:

```
Error: error setting up new vSphere SOAP client: Post https://vspherehost.com/sdk: x509: certificate signed by unknown authority"
```

Resolving the problem: Managed cluster creation fails with unknown certificate authority

Ensure you entered the correct certificate from the certificate authority when creating the credential.

Managed cluster creation fails with expired certificate

Symptom: Managed cluster creation fails with expired certificate

After creating a new Red Hat OpenShift Container Platform cluster on VMware vSphere, the cluster fails because the certificate is expired or is not yet valid.

Identifying the problem: Managed cluster creation fails with expired certificate

The deployment of the managed cluster fails and returns the following errors in the deployment log:

```
x509: certificate has expired or is not yet valid
```

Resolving the problem: Managed cluster creation fails with expired certificate

Ensure that the time on your ESXi hosts is synchronized.

Managed cluster creation fails with insufficient privilege for tagging

Symptom: Managed cluster creation fails with insufficient privilege for tagging

After creating a new Red Hat OpenShift Container Platform cluster on VMware vSphere, the cluster fails because there is insufficient privilege to use tagging.

Identifying the problem: Managed cluster creation fails with insufficient privilege for tagging

The deployment of the managed cluster fails and returns the following errors in the deployment log:

```
time="2020-08-07T19:41:58Z" level=debug msg="vsphere_tag_category.category:
Creating..."
time="2020-08-07T19:41:58Z" level=error
time="2020-08-07T19:41:58Z" level=error msg="Error: could not create category: POST
https://vspherehost.com/rest/com/vmware/cis/tagging/category: 403 Forbidden"
time="2020-08-07T19:41:58Z" level=error
time="2020-08-07T19:41:58Z" level=error msg=" on ../tmp/openshift-install-
436877649/main.tf line 54, in resource \"vsphere_tag_category\" \"category\":"
time="2020-08-07T19:41:58Z" level=error msg=" 54: resource \"vsphere_tag_category\"
\"category\" {"
```

Resolving the problem: Managed cluster creation fails with insufficient privilege for tagging

Ensure that your VMware vCenter required account privileges are correct. See [Image registry removed during installation](#) for more information.

Managed cluster creation fails with invalid dnsVIP

Symptom: Managed cluster creation fails with invalid dnsVIP

After creating a new Red Hat OpenShift Container Platform cluster on VMware vSphere, the cluster fails because there is an invalid dnsVIP.

Identifying the problem: Managed cluster creation fails with invalid dnsVIP

If you see the following message when trying to deploy a new managed cluster with VMware vSphere, it is because you have an older OpenShift Container Platform release image that does not support VMware Installer Provisioned Infrastructure (IPI):

```
failed to fetch Master Machines: failed to load asset \\\"Install Config\\\": invalid  
\\\"install-config.yaml\\\" file: platform.vsphere.dnsVIP: Invalid value: \\\"\\\":  
\\\"\\\" is not a valid IP
```

Resolving the problem: Managed cluster creation fails with invalid dnsVIP

Select a release image from a later version of OpenShift Container Platform that supports VMware Installer Provisioned Infrastructure.

Managed cluster creation fails with incorrect network type

Symptom: Managed cluster creation fails with incorrect network type

After creating a new Red Hat OpenShift Container Platform cluster on VMware vSphere, the cluster fails because there is an incorrect network type specified.

Identifying the problem: Managed cluster creation fails with incorrect network type

If you see the following message when trying to deploy a new managed cluster with VMware vSphere, it is because you have an older OpenShift Container Platform image that does not support VMware Installer Provisioned Infrastructure (IPI):

```
time="2020-08-11T14:31:38-04:00" level=debug msg="vsphereprivate_import_ova.import:  
Creating..."  
time="2020-08-11T14:31:39-04:00" level=error  
time="2020-08-11T14:31:39-04:00" level=error msg="Error: rpc error: code = Unavailable  
desc = transport is closing"  
time="2020-08-11T14:31:39-04:00" level=error  
time="2020-08-11T14:31:39-04:00" level=error  
time="2020-08-11T14:31:39-04:00" level=fatal msg="failed to fetch Cluster: failed to  
generate asset \"Cluster\": failed to create cluster: failed to apply Terraform:  
failed to complete the change"
```

Resolving the problem: Managed cluster creation fails with incorrect network type

Select a valid VMware vSphere network type for the specified VMware cluster.

Managed cluster creation fails with an error processing disk changes

Symptom: Adding the VMware vSphere managed cluster fails due to an error processing disk changes

After creating a new Red Hat OpenShift Container Platform cluster on VMware vSphere, the cluster fails because there is an error when processing disk changes.

Identifying the problem: Adding the VMware vSphere managed cluster fails due to an error processing disk changes

A message similar to the following is displayed in the logs:

```
ERROR
ERROR Error: error reconfiguring virtual machine: error processing disk changes post-clone: disk.0: ServerFaultCode: NoPermission: RESOURCE (vm-71:2000), ACTION (queryAssociatedProfile): RESOURCE (vm-71), ACTION (PolicyIDByVirtualDisk)
```

Resolving the problem: Adding the VMware vSphere managed cluster fails due to an error processing disk changes

Use the VMware vSphere client to give the user **All privileges** for *Profile-driven Storage Privileges*.

Troubleshooting cluster in console with pending or failed status

If you observe *Pending* status or *Failed* status in the console for a cluster you created, follow the procedure to troubleshoot the problem.

Symptom: Cluster in console with pending or failed status

After creating a new cluster by using the console, the cluster does not progress beyond the status of *Pending* or displays *Failed* status.

Identifying the problem: Cluster in console with pending or failed status

If the cluster displays *Failed* status, navigate to the details page for the cluster and follow the link to the logs provided. If no logs are found or the cluster displays *Pending* status, continue with the following procedure to check for logs:

- Procedure 1
 1. Run the following command on the hub cluster to view the names of the Kubernetes pods that were created in the namespace for the new cluster:

```
oc get pod -n <new_cluster_name>
```

Replace `new_cluster_name` with the name of the cluster that you created.

2. If no pod that contains the string `provision` in the name is listed, continue with Procedure 2. If there is a pod with `provision` in the title, run the following command on the hub cluster to view the logs of that pod:

```
oc logs <new_cluster_name_provision_pod_name> -n <new_cluster_name> -c hive
```

Replace `new_cluster_name_provision_pod_name` with the name of the cluster that you created, followed by the pod name that contains `provision`.

3. Search for errors in the logs that might explain the cause of the problem.

- Procedure 2

If there is not a pod with `provision` in its name, the problem occurred earlier in the process. Complete the following procedure to view the logs:

1. Run the following command on the hub cluster:

```
oc describe clusterdeployments -n <new_cluster_name>
```

Replace `new_cluster_name` with the name of the cluster that you created. For more information about cluster installation logs, see [Gathering installation logs](#) in the Red Hat OpenShift documentation.

2. See if there is additional information about the problem in the *Status.Conditions.Message* and *Status.Conditions.Reason* entries of the resource.

Resolving the problem: Cluster in console with pending or failed status

After you identify the errors in the logs, determine how to resolve the errors before you destroy the cluster and create it again.

The following example provides a possible log error of selecting an unsupported zone, and the actions that are required to resolve it:

```
No subnets provided for zones
```

When you created your cluster, you selected one or more zones within a region that are not supported. Complete one of the following actions when you recreate your cluster to resolve the issue:

- Select a different zone within the region.
- Omit the zone that does not provide the support, if you have other zones listed.
- Select a different region for your cluster.

After determining the issues from the log, destroy the cluster and recreate it.

See [Creating clusters](#) for more information about creating a cluster.

Troubleshooting Klusterlet with degraded conditions

The Klusterlet degraded conditions can help to diagnose the status of the klusterlet agent on managed cluster. If a Klusterlet is in the degraded condition, the klusterlet agent on managed cluster might have errors that need to be troubleshooted. See the following information for Klusterlet degraded conditions that are set to **True**.

Symptom: Klusterlet is in the degraded condition

After deploying a Klusterlet on managed cluster, the **RegistrationDesiredDegraded** or **WorkDesiredDegraded** condition displays a status of **True**.

Identifying the problem: Klusterlet is in the degraded condition

1. Run the following command on the managed cluster to view the Klusterlet status:

```
kubectl get klusterlets klusterlet -oyaml
```

2. Check **RegistrationDesiredDegraded** or **WorkDesiredDegraded** to see if the condition is set to **True**. Proceed to *Resolving the problem* for any degraded conditions that are listed.

Resolving the problem: Klusterlet is in the degraded condition

See the following list of degraded statuses and how you can attempt to resolve those issues:

- If the **RegistrationDesiredDegraded** condition with a status of **True** and the condition reason is: *BootstrapSecretMissing*, you need create a bootstrap secret on **open-cluster-management-agent** namespace.
- If the **RegistrationDesiredDegraded** condition displays **True** and the condition reason is a *BootstrapSecretError*, or *BootstrapSecretUnauthorized*, then the current bootstrap secret is invalid. Delete the current bootstrap secret and recreate a valid bootstrap secret on **open-cluster-management-agent** namespace.
- If the **RegistrationDesiredDegraded** and **WorkDesiredDegraded** displays **True** and the condition reason is *HubKubeConfigSecretMissing*, delete the Klusterlet and recreate it.
- If the **RegistrationDesiredDegraded** and **WorkDesiredDegraded** displays **True** and the condition reason is: *ClusterNameMissing*, *KubeConfigMissing*, *HubConfigSecretError*, or *HubConfigSecretUnauthorized*, delete the hub cluster kubeconfig secret from **open-cluster-management-agent** namespace. The klusterlet agent creates a new hub cluster kubeconfig secret.
- If the **RegistrationDesiredDegraded** displays **True** and the condition reason is *GetRegistrationDeploymentFailed* or *UnavailableRegistrationPod*, you can check the condition message to get the problem details and attempt to resolve.
- If the **WorkDesiredDegraded** displays **True** and the condition reason is *GetWorkDeploymentFailed* or *UnavailableWorkPod*, you can check the condition message to get the problem details and attempt to resolve.

Troubleshooting namespace that remains after deleting a cluster

When you remove a managed cluster, the namespace is normally removed as part of the cluster removal process. In rare cases, the namespace remains with some artifacts, so you must manually remove the namespace.

Symptom: Namespace remains after deleting a cluster

After removing a managed cluster, the namespace is not removed.

Resolving the problem: Namespace remains after deleting a cluster

Complete the following steps to remove the namespace manually:

1. Run the following command to produce a list of the resources that remain in the `<cluster_name>` namespace:

```
oc api-resources --verbs=list --namespaced -o name | grep -E
'^secrets|^serviceaccounts|^managedclusteraddons|^roles|^rolebindings|^manifestwork
s|^leases|^managedclusterinfo|^appliedmanifestworks|^clusteroauths'| xargs -n 1 oc
get --show-kind --ignore-not-found -n <cluster_name>
```

Replace `<cluster_name>` with the name of the namespace for the cluster that you attempted to remove.

2. Delete each identified resource on the list that does not have a status of `Delete` by entering the following command to edit the list:

```
oc edit <resource_kind> <resource_name> -n <namespace>
```

Replace `<resource_kind>` with the kind of the resource. Replace `<resource_name>` with the name of the resource. Replace `<namespace>` with the name of the namespace of the resource.

3. Locate the `finalizer` attribute in the in the metadata.
4. Delete the non-Kubernetes finalizers by using the `vi` editor `dd` command.
5. Save the list and exit the `vi` editor by entering the `:wq` command.
6. Delete the namespace by entering the following command:

```
oc delete ns <cluster-name>
```

Replace `<cluster-name>` with the name of the namespace that you are trying to delete.

Auto-import-secret-exists error when importing a cluster

Your cluster import fails with an error message that reads: auto import secret exists.

Symptom: Auto import secret exists error when importing a cluster

When importing a hive cluster for management, an `auto-import-secret already exists` error is displayed.

Resolving the problem: Auto-import-secret-exists error when importing a cluster

This problem occurs when you attempt to import a cluster that was previously managed. When this happens, the secrets conflict when you try to reimport the cluster.

To work around this problem, complete the following steps:

1. To manually delete the existing `auto-import-secret`, run the following command on the hub cluster:

```
oc delete secret auto-import-secret -n <cluster-namespace>
```

Replace `cluster-namespace` with the namespace of your cluster.

2. Import your cluster again by using the procedure in [Cluster import introduction](#).

Troubleshooting missing PlacementDecision after creating Placement

If no `PlacementDescision` is generated after creating a `Placement`, follow the procedure to troubleshoot the problem.

Symptom: Missing PlacementDecision after creating Placement

After creating a `Placement`, a `PlacementDescision` is not automatically generated.

Resolving the problem: Missing PlacementDecision after creating Placement

To resolve the issue, complete the following steps:

1. Check the `Placement` conditions by running the following command:

```
kubectl describe placement <placement-name>
```

Replace `placement-name` with the name of the `Placement`.

The output might resemble the following example:

```
Name:          demo-placement
Namespace:     default
Labels:        <none>
Annotations:   <none>
API Version:   cluster.open-cluster-management.io/v1beta1
Kind:          Placement
Status:
```

Conditions:

```

Last Transition Time:    2022-09-30T07:39:45Z
Message:                Placement configurations check pass
Reason:                 Succeededconfigured
Status:                 False
Type:                   PlacementMisconfigured
Last Transition Time:    2022-09-30T07:39:45Z
Message:                No valid ManagedClusterSetBindings found in
placement namespace
Reason:                 NoManagedClusterSetBindings
Status:                 False
Type:                   PlacementSatisfied
Number Of Selected Clusters: 0

```

2. Check the output for the **Status** of **PlacementMisconfigured** and **PlacementSatisfied**:
 - If the **PlacementMisconfigured Status** is true, your **Placement** has configuration errors. Check the included message for more details on the configuration errors and how to resolve them.
 - If the **PlacementSatisfied Status** is false, no managed cluster satisfies your **Placement**. Check the included message for more details and how to resolve the error. In the previous example, no **ManagedClusterSetBindings** were found in the placement namespace.
3. You can check the score of each cluster in **Events** to find out why some clusters with lower scores are not selected. The output might resemble the following example:

```

Name:      demo-placement
Namespace: default
Labels:    <none>
Annotations: <none>
API Version: cluster.open-cluster-management.io/v1beta1
Kind:      Placement
Events:
  Type    Reason          Age    From                      Message
  ----    -
  Normal  DecisionCreate  2m10s  placementController      Decision demo-placement-
decision-1 is created with placement demo-placement in namespace default
  Normal  DecisionUpdate  2m10s  placementController      Decision demo-placement-
decision-1 is updated with placement demo-placement in namespace default
  Normal  ScoreUpdate     2m10s  placementController      cluster1:0 cluster2:100
cluster3:200
  Normal  DecisionUpdate  3s     placementController      Decision demo-placement-
decision-1 is updated with placement demo-placement in namespace default
  Normal  ScoreUpdate     3s     placementController      cluster1:200 cluster2:145
cluster3:189 cluster4:200

```

Note: The placement controller assigns a score and generates an event for each filtered **ManagedCluster**. The placement controller generates a new event when the cluster score changes.

Troubleshooting a discovery failure of bare metal hosts on Dell hardware

If the discovery of bare metal hosts fails on Dell hardware, the Integrated Dell Remote Access Controller (iDRAC) is likely configured to not allow certificates from unknown certificate authorities.

Symptom: Discovery failure of bare metal hosts on Dell hardware

After you complete the procedure for discovering bare metal hosts by using the baseboard management controller, an error message similar to the following is displayed:

```
ProvisioningError 51s metal3-baremetal-controller Image provisioning failed: Deploy
step deploy.deploy failed with BadRequestError: HTTP POST
https://<bmc_address>/redfish/v1/Managers/iDRAC.Embedded.1/VirtualMedia/CD/Actions/Vir
tualMedia.InsertMedia returned code 400. Base.1.8.GeneralError: A general error has
occurred. See ExtendedInfo for more information Extended information: [
{"Message": "Unable to mount remote share https://<ironic_address>/redfish/boot-
<uuid>.iso.", 'MessageArgs': ["https://<ironic_address>/redfish/boot-<uuid>.iso"],
"MessageArgs@odata.count": 1, "MessageId": "IDRAC.2.5.RAC0720", "RelatedProperties":
["#/Image"], "RelatedProperties@odata.count": 1, "Resolution": "Retry the operation.",
"Severity": "Informational"}
]
```

Resolving the problem: Discovery failure of bare metal hosts on Dell hardware

The iDRAC is configured not to accept certificates from unknown certificate authorities.

To bypass the problem, disable the certificate verification on the baseboard management controller of the host iDRAC by completing the following steps:

1. In the iDRAC console, navigate to **Configuration > Virtual media > Remote file share**.
2. Change the value of **Expired or invalid certificate action** to **Yes**.

Troubleshooting Minimal ISO boot failures

You might encounter issues when trying to boot a minimal ISO.

Symptom: Minimal ISO boot failures

The boot screen shows that the host has failed to download the root file system image.

Resolving the problem: Minimal ISO boot failures

See [Troubleshooting minimal discovery ISO issues](#) in the Assisted Installer for OpenShift Container Platform} documentation to learn how to troubleshoot the issue.

Troubleshooting the RHCOS image mirroring

For hosted control planes on Red Hat OpenShift Virtualization in a disconnected environment, **oc-**

mirror fails to automatically mirror the Red Hat Enterprise Linux CoreOS (RHCOS) image to the internal registry. When you create your first hosted cluster, the Kubevirt virtual machine does not boot, because the boot image is not available in the internal registry.

Symptom: *oc-mirror* fails to attempt the RHCOS image mirroring

The **oc-mirror** plugin does not mirror the {op-system-first} image from the release payload to the internal registry.

Resolving the problem: *oc-mirror* fails to attempt the RHCOS image mirroring

To resolve this issue, manually mirror the RHCOS image to the internal registry. Complete the following steps:

1. Get the internal registry name by running the following command:

```
oc get imagecontentsourcepolicy -o json | jq -r  
' .items[].spec.repositoryDigestMirrors[0].mirrors[0] '
```

2. Get a payload image by running the following command:

```
oc get clusterversion version -ojsonpath='{.status.desired.image}'
```

3. Extract the **0000_50_installer_coreos-bootimages.yaml** file that contains boot images from your payload image on the hosted cluster. Replace **<payload_image>** with the name of your payload image. Run the following command:

```
oc image extract --file /release-manifests/0000_50_installer_coreos-bootimages.yaml  
<payload_image> --confirm
```

4. Get the RHCOS image by running the following command:

```
cat 0000_50_installer_coreos-bootimages.yaml | yq -r .data.stream | jq -r  
' .architectures.x86_64.images.kubevirt."digest-ref" '
```

5. Mirror the RHCOS image to your internal registry. Replace **<rhcos_image>** with your RHCOS image for example, **quay.io/openshift-release-dev/ocp-v4.0-art-dev@sha256:d9643ead36b1c026be664c9c65c11433c6cdf71bfd93ba229141d134a4a6dd94**. Replace **<internal_registry>** with the name of your internal registry, for example, **virthost.ostest.test.metalkube.org:5000/localimages/ocp-v4.0-art-dev**. Run the following command:

```
oc image mirror <rhcos_image> <internal_registry>
```

6. Create a YAML file named `rhcos-boot-kubvirt.yaml` that defines the `ImageDigestMirrorSet` object. See the following example configuration:

```
apiVersion: config.openshift.io/v1
kind: ImageDigestMirrorSet
metadata:
  name: rhcos-boot-kubvirt
spec:
  repositoryDigestMirrors:
    - mirrors:
        - <rhcos_image_no_digest> ①
      source: virthost.ostest.test.metalkube.org:5000/localimages/ocp-v4.0-art-dev
  ②
```

① Specify your RHCOS image without its digest, for example, `quay.io/openshift-release-dev/ocp-v4.0-art-dev`.

② Specify the name of your internal registry, for example, `virthost.ostest.test.metalkube.org:5000/localimages/ocp-v4.0-art-dev`.

7. Apply the `rhcos-boot-kubvirt.yaml` file to create the `ImageDigestMirrorSet` object by running the following command:

```
oc apply -f rhcos-boot-kubvirt.yaml
```

Troubleshooting: Returning non bare metal clusters to the late binding pool

If you are using late binding managed clusters without `BareMetalHosts`, you must complete additional manual steps to destroy a late binding cluster and return the nodes back to the Discovery ISO.

Symptom: Returning non bare metal clusters to the late binding pool

For late binding managed clusters without `BareMetalHosts`, removing cluster information does not automatically return all nodes to the Discovery ISO.

Resolving the problem: Returning non bare metal clusters to the late binding pool

To unbind the non bare metal nodes with late binding, complete the following steps:

1. Remove the cluster information. See [Removing a cluster from management](#) to learn more.
2. Clean the root disks.
3. Reboot manually with the Discovery ISO.

Troubleshooting managed clusters *Unknown* on Red Hat OpenShift Service on AWS (classic) with hosted control planes cluster

The status of all managed clusters on a Red Hat OpenShift Service on AWS (classic) hosted clusters

suddenly becomes **Unknown**.

Symptom: All managed clusters are in **Unknown** status on Red Hat OpenShift Service on AWS (classic) with hosted control planes cluster

When you check the **klusterlet-agent** pod log in the **open-cluster-management-agent** namespace on your managed cluster, you see an error that resembles the following:

```
E0809 18:45:29.450874      1 reflector.go:147] k8s.io/client-go@v0.29.4/tools/cache/reflector.go:229: Failed to watch *v1.CertificateSigningRequest: failed to list *v1.CertificateSigningRequest: Get "https://api.xxx.openshiftapps.com:443/apis/certificates.k8s.io/v1/certificatesigningrequests?limit=500&resourceVersion=0": tls: failed to verify certificate: x509: certificate signed by unknown authority
```

Resolving the problem: All managed clusters are in **Unknown** status on Red Hat OpenShift Service on AWS (classic) with hosted control planes cluster

1. Create a **KlusterletConfig** resource with name **global** if it does not exist.
2. Set the **spec.hubKubeAPIServerConfig.serverVerificationStrategy** to **UseSystemTruststore**. See the following example:

```
apiVersion: config.open-cluster-management.io/v1alpha1
kind: KlusterletConfig
metadata:
  name: global
spec:
  hubKubeAPIServerConfig:
    serverVerificationStrategy: UseSystemTruststore
```

3. Apply the resource by running the following command on the hub cluster. Replace **<filename>** with the name of your file:

```
oc apply -f <filename>
```

The state of some managed clusters might recover. Continue with the process for managed clusters that remain in the **Unknown** status.

4. Export and decode the **import.yaml** file from the hub cluster by running the following command on the hub cluster. Replace **<cluster_name>** with the name of your cluster.

```
oc get secret <cluster_name>-import -n <cluster_name> -o jsonpath={.data.import\.yaml} | base64 --decode > <cluster_name>-import.yaml
```

5. Apply the file by running the following command on the managed cluster.

```
oc apply -f <cluster_name>-import.yaml
```

Troubleshooting an attempt to upgrade managed cluster with missing OpenShift Container Platform version

You do not see the OpenShift Container Platform version that you want in the console when you attempt to upgrade your managed cluster in the console.

Symptom: Attempt to upgrade managed cluster with missing OpenShift Container Platform version

When you attempt to upgrade a managed cluster from the console and click **Upgrade available** in the *Cluster details* view to choose the OpenShift Container Platform version from the dropdown list, the version is missing.

Resolving the problem: Attempt to upgrade managed cluster with missing OpenShift Container Platform version

See the following procedure:

1. Ensure the version you want is included in the status of the **ClusterVersion** resource on the managed cluster. Run the following command:

```
oc get clusterversion version -o jsonpath='{.status.availableUpdates[*].version}'
```

If your expected version is not displayed, then the version is not applicable for this managed cluster.

2. Check if the **ManagedClusterInfo** resource includes the version on the hub cluster. Run the following command:

```
oc -n <cluster_name> get managedclusterinfo <cluster_name> -o  
jsonpath='{.status.distributionInfo.ocp.availableUpdates[*]}'
```

3. If the version is included, check to see if there is a **ClusterCurator** resource with a failure on the hub cluster. Run the following command:

```
oc -n <cluster_name> get ClusterCurator <cluster_name> -o yaml
```

4. If the **ClusterCurator** resource exists and the status of its **clustercurator-job** condition is **False**, delete the **ClusterCurator** resource from the hub cluster. Run the following command:

```
oc -n <cluster_name> delete ClusterCurator <cluster_name>
```


5. If the `ManagedClusterInfo` resource does not include the version, check the `work-manager` add-on log on the managed cluster and fix errors that are reported. Run the following command and replace the pod name with the real name in your environment:

```
oc -n open-cluster-management-agent-addon logs klusterlet-addon-workmgr-  
<your_pod_name>
```

Troubleshooting certificate authority update error on Hive and Assisted Installer clusters

Troubleshoot a connection error message that appears after you update the cluster certificate authority (CA) on a Hive-provisioned or Assisted Installer cluster.

Symptom: Certificate authority update error on Hive and Assisted Installer clusters

When you update the cluster CA with your enterprise CA on the Hive-provisioned or Assisted Installer cluster, the corresponding cluster deployment on the hub cluster displays the following connection error message:

```
x509: certificate signed by unknown authority
```

If you back up and restore the cluster in this state, you cannot automatically import the managed cluster to the restored hub cluster.

Resolving the problem: Certificate authority update error on Hive and Assisted Installer clusters

Use the following procedure to resolve the error:

1. Define your custom CA with the global variable in your system. Run the following command:

```
export CA=<your_customized_CA>
```

2. Run the following command to generate a new secret with your customized CA:

```
oc create secret generic additional-ca \  
  --from-literal=ca.crt="$CA" \  
  --namespace hive
```

3. Update your cluster with the new secret. Run the following command with the additional CA patch:

```
oc patch hiveconfig hive --type=merge -p '  
{  
  "spec": {
```

```
"additionalCertificateAuthoritiesSecretRef": [
  {
    "name": "additional-ca"
  }
]
}
```

Troubleshooting: Missing `open-cluster-management-global-set` namespace

The `open-cluster-management-global-set` namespace contains the `global` Placement, which is required to deploy add-ons such as `cluster-proxy`, `managed-serviceaccount`, and `work-manager` to managed clusters.

If the `open-cluster-management-global-set` namespace is missing or deleted, add-ons are not installed on managed clusters. If you previously installed the add-ons, they are removed if the namespace is missing or deleted.

Symptom: Missing `open-cluster-management-global-set` namespace

After installing Red Hat Advanced Cluster Management, the `open-cluster-management-global-set` namespace is missing on the hub cluster. As a result, no `ManagedClusterAddOn` resources are created for the `cluster-proxy`, `managed-serviceaccount`, and `work-manager` add-ons.

Identifying the problem: Missing `open-cluster-management-global-set` namespace

The `open-cluster-management-global-set` namespace is missing and the `ManagedClusterSet` called `global` includes the `open-cluster-management.io/ns-create: "true"` annotation. You can also find the following message in the 'ocm-controller' log in the `multicluster-engine` namespace:

```
I0904 04:50:09.628951      1 globalset_controller.go:202] GlobalSet Namespace open-
cluster-management-global-set does not exist, skip applying binding and placement
```

Resolving the problem: Missing `open-cluster-management-global-set` namespace

To resolve the issue, remove the annotation from the `ManagedClusterSet` called `global`. Run the following command to recreate the namespace and the `global` Placement:

```
oc annotate managedclusterset global open-cluster-management.io/ns-create- --overwrite
```

Troubleshooting cluster creation failure when creating Microsoft Azure Red Hat OpenShift hosted control plane clusters by using Cluster API Provider for Azure

When using Cluster API Provider for Azure to create Azure Red Hat OpenShift hosted control plane clusters, you might encounter problems that prevent cluster creation, or the process might require more information.

Symptom: Cluster creation failure

After creating an Azure Red Hat OpenShift hosted control plane cluster by using Cluster API Provider for Azure, your cluster does not appear when you check the cluster status, or the cluster `AVAILABLE` status is `False`.

Identifying the problem: Cluster creation failure

To start troubleshooting the cluster creation failure on an Azure Red Hat OpenShift hosted control plane cluster by using Cluster API Provider for Azure, get additional information that helps you identify the problem. Complete the following steps.

1. Verify that multicluster engine for Kubernetes operator components are enabled. Run the following command to get the full component status:

```
oc get mce multiclusterengine -o yaml | yq '.spec.overrides.components'
```

2. Check if specific components are enabled. Run the following command:

```
oc get mce multiclusterengine -o json | \
jq -r '.spec.overrides.components[] | select(.name == "cluster-api")'
```

3. If cluster creation still fails, check the controller logs for the Cluster API, Cluster API Provider for Azure, and Azure Service Operator for troubleshooting details. Complete the following steps:
4. View Cluster API controller logs. Run the following command:

```
oc logs -n multicluster-engine deployment/capi-controller-manager --tail=100
```

5. View Cluster API Provider for Azure controller logs. Run the following command. Replace `<cluster_name>` with the name of your cluster:

```
oc logs -n multicluster-engine deployment/capz-controller-manager --tail=100 | grep <cluster_name>
```

6. View Azure Service Operator controller logs. Run the following command:

```
oc logs -n multicluster-engine deployment/azureserviceoperator-controller-manager --tail=100
```

Troubleshooting Microsoft Azure Red Hat OpenShift hosted control plane cluster with Cluster API Provider for Azure becomes stuck during provisioning

When you create an Azure Red Hat OpenShift hosted control plane cluster by using Cluster API

Provider for Azure, the cluster status might become stuck during provisioning.

Symptom: Cluster becomes stuck during provisioning

Azure Red Hat OpenShift hosted control plane clusters with Cluster API Provider for Azure become stuck during provisioning. The cluster status might be *Provisioning* or *Provisioned*, but the cluster is not available for use.

Identifying the problem: Cluster becomes stuck during provisioning

To start troubleshooting, get additional information about the cluster and infrastructure status, Azure Service Operator resource status, and events. Complete the following steps to get more information:

1. Check the cluster status and conditions. Run the following command. Replace `<cluster-name>` and `<cluster-namespace>` with the name and namespace of your cluster:

```
clusterctl describe cluster <cluster-name> -n <cluster-namespace> --show
-conditions=all
```

2. Check the infrastructure status. Run the following command. Replace `<cluster-name>` and `<cluster-namespace>` with the name and namespace of your cluster:

```
oc get arocluster <cluster-name> -n <cluster-namespace> -o yaml | yq '.status'
```

3. Check the Azure Service Operator resource status. Run the following command. Replace `<cluster-name>` and `<cluster-namespace>` with the name and namespace of your cluster:

```
oc get HcpOpenShiftCluster <cluster-name> -n <cluster-namespace> -o yaml | yq
'.status.conditions'
```

4. Check for errors in events. Run the following command. Replace `<cluster-namespace>` and `<cluster-name>` and with the namespace and name of your cluster:

```
oc get events -n <cluster-namespace> --sort-by='.lastTimestamp' | grep <cluster-
name>
```